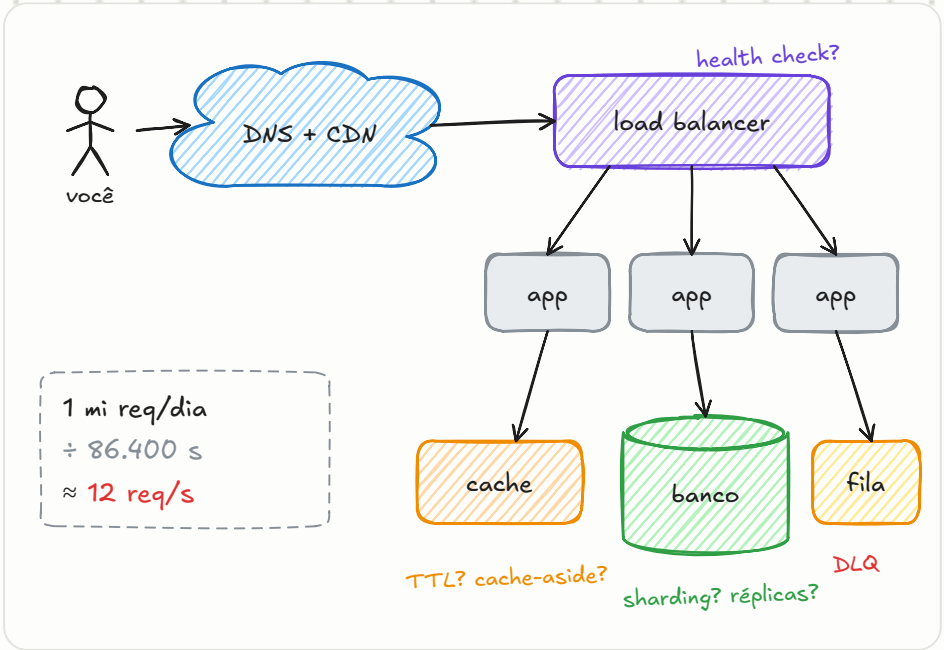
EDIÇÃO DE ESTUDO EM PORTUGUÊS

System Design

~~50~~ 51 conceitos essenciais

Tradução fiel e expansão do artigo *50 System Design Concepts for Beginners*, de Arslan Ahmad (DesignGurus). Com diagramas de lousa, dicas de entrevista, flashcards e simulado.



51 CONCEITOS **6** PARTES **49** DIAGRAMAS **153** PERGUNTAS DE REVISÃO

40 QUESTÕES DE SIMULADO

Sumário

Sobre este ebook 5

Como estudar com este ebook 8

PARTE 1 **Princípios de arquitetura** 11

1 Escalabilidade vertical vs. horizontal 12
2 Teorema CAP 18
3 Teorema PACELC 23
4 ACID vs. BASE 28
5 Throughput vs. latência 33
6 Lei de Amdahl 38
7 Consistência forte vs. eventual 43
8 Arquitetura stateful vs. stateless 48
9 Microsserviços vs. monólito 53
10 Arquitetura serverless 59

PARTE 2 **Redes e comunicação** 64

11 Balanceamento de carga 65
12 Algoritmos de balanceamento 70
13 Proxy reverso vs. proxy direto 75
14 API Gateway 80
15 CDN 85
16 DNS 90
17 TCP vs. UDP 95
18 HTTP/2 e HTTP/3 (QUIC) 100
19 gRPC vs. REST 105
20 WebSocket e Server-Sent Events 111
21 Long polling 116
22 Protocolo gossip 121

PARTE 3 **Bancos de dados e armazenamento** 126

23 Sharding 127
24 Padrões de replicação 132
25 Hashing consistente 137
26 Índices: B-Tree e LSM-Tree 142

27 Write-Ahead Logging (WAL) 147
28 Normalização vs. desnormalização 152
29 Persistência poliglota 157
30 Bloom filter 162
31 Bancos de dados vetoriais 167

PARTE 4 **Confiabilidade e tolerância a falhas** 172

32 Rate limiting 173
33 Circuit breaker 178
34 Bulkhead 183
35 Retries e backoff exponencial 188
36 Idempotência 193
37 Heartbeat 199
38 Eleição de líder (Paxos e Raft) 204
39 Transações distribuídas: o padrão SAGA 209
40 Two-Phase Commit (2PC) 214

PARTE 5 **Cache e mensageria** 219

41 Cache 220
42 Estratégias de cache 226
43 Políticas de remoção (LRU, LFU) 232
44 Filas de mensagens 237
45 Pub/Sub 242
46 Dead letter queue 247

PARTE 6 **Observabilidade e segurança** 252

47 Tracing distribuído 253
48 SLA vs. SLO vs. SLI 258
49 OAuth 2.0 e OIDC 263
50 Handshake TLS 268
51 Zero Trust 273

Para fechar 279

Cola: os conceitos em uma linha 281

APÊNDICE A **Roteiro para a entrevista de system design** 288

APÊNDICE B **Números e estimativas** 294

APÊNDICE C **Estudo de caso: um encurtador de links** 299

APÊNDICE D **Glossário** 308

APÊNDICE E **Simulado** 318

ABERTURA

Sobre este ebook

De onde ele vem

Este ebook nasceu do artigo **"50 System Design Concepts for Beginners in 90 Minutes"** (edição 2026), de **Arslan Ahmad**, publicado na newsletter da DesignGurus em 6 de dezembro de 2025. Nas palavras do autor:

> *Quando você começa a aprender system design, a parte mais difícil não são os conceitos em si. É encontrar explicações claras num lugar só. Por isso, ter um único guia que cubra todo o essencial muda o jogo.*
>
> *Por isso, montei este guia para cobrir 50 dos conceitos mais importantes de system design. Pense nele como a sua referência completa para entender como sistemas reais escalam, continuam confiáveis, se comunicam e lidam com dados.*
>
> *Meu objetivo é passar com você por cinquenta ideias importantes, com explicações curtas e exemplos simples, para que tudo se encaixe rápido. Se você está se preparando para uma entrevista de system design, este guia é o seu recurso principal.*

O artigo original está em designgurus.substack.com. Todo o crédito pela seleção dos conceitos e pelas explicações originais é do autor.

O que esta edição acrescenta

O artigo é um mapa rápido. Este ebook transforma o mapa num material de estudo:

- **A tradução fiel de cada conceito**, na seção "O essencial" de cada capítulo.
- **Tudo o que o artigo não tinha espaço para dizer**: como o conceito funciona por dentro, uma analogia, exemplos de sistemas reais, os trade-offs, o que falar numa entrevista, as pegadinhas mais comuns e três perguntas de revisão.
- **Diagramas no estilo lousa**, desenhados à mão como num quadro branco de entrevista.
- **Notas de atualização** nos pontos em que o original simplificou demais ou envelheceu.

- **Ferramentas de estudo**: uma cola com os conceitos em uma linha, flashcards, um simulado de 40 questões, um roteiro de entrevista, números de referência para estimativas, um estudo de caso completo e um glossário.

O título fala em 50 conceitos, mas o artigo traz 51. Estão todos aqui.

Como cada capítulo funciona

Os 51 capítulos seguem a mesma estrutura, para você saber onde procurar cada coisa:

Seção	Para que serve
Em uma frase	A ideia central, para guardar na memória
O essencial	A tradução fiel do trecho do artigo original
Aprofundando	Como o conceito funciona por dentro, com exemplo e diagrama
Analogia	Uma imagem do dia a dia para fixar a ideia
No mundo real	Sistemas e tecnologias que usam o conceito
Trade-offs	O que você ganha e o que você paga
Na entrevista	Como e quando trazer o conceito na conversa
Cuidado com	Confusões comuns e atualizações do original
Teste-se	Três perguntas com resposta, para conferir se ficou

No fim de cada capítulo, “Conecta com” aponta os capítulos relacionados. System design é uma rede de ideias, e boa parte do aprendizado está nas ligações entre elas.

Créditos

- **Conteúdo original:** Arslan Ahmad, DesignGurus, “50 System Design Concepts for Beginners in 90 Minutes [2026 Edition]”, 6 de dezembro de 2025.
- **Esta edição:** tradução livre, expansão, diagramas e material de estudo, feitos para estudo pessoal e sem fins comerciais.

- **Referências citadas pelo autor:** Complete Roadmap to Learn System Design, Grokking the System Design Interview, System Design Crash Course e How I Would Prepare for a System Design Interview in 2026.

ABERTURA

Como estudar com este ebook

Quatro hábitos que fazem a diferença

1. **Leia com uma pergunta na cabeça.** Antes de cada capítulo, leia o "Em uma frase" e pergunte: *que problema isso resolve*? Todo conceito de system design existe porque alguma coisa quebrou em escala.
2. **Feche o livro e explique.** Ao terminar um capítulo, explique o conceito em voz alta, como se fosse para alguém que nunca ouviu falar. Onde você travar, é ali que precisa reler. Tentar lembrar fixa muito mais do que reler.
3. **Desenhe.** Entrevista de system design acontece numa lousa. Refaça o diagrama do capítulo numa folha, de memória, com caixas e setas. Se você consegue desenhar, você entendeu.
4. **Revise espaçado.** Volte aos flashcards depois de 1, 3, 7 e 14 dias. Revisar quando você está quase esquecendo é o que transforma leitura em memória de longo prazo.

Plano de 14 dias

Cerca de uma hora por dia. Se um dia ficar pesado, divida em dois; o importante é não pular as revisões.

Dia	Leitura	Prática
1	Capítulos 1 a 4	Teste-se de cada capítulo; desenhe o CAP de memória
2	Capítulos 5 a 8	Explique stateless vs. stateful em voz alta
3	Capítulos 9 a 12	Flashcards da Parte 1
4	Capítulos 13 a 16	Desenhe o caminho de uma requisição: DNS, CDN, load balancer

Dia	Leitura	Prática
5	Capítulos 17 a 20	Monte uma tabela TCP, UDP, HTTP/2, HTTP/3
6	Capítulos 21 a 24	Flashcards da Parte 2
7	Revisão	Simulado, questões 1 a 17; releia os capítulos dos erros
8	Capítulos 25 a 28	Desenhe o anel de hashing consistente
9	Capítulos 29 a 32	Flashcards da Parte 3
10	Capítulos 33 a 36	Explique circuit breaker, retry e idempotência juntos
11	Capítulos 37 a 40	Compare SAGA e 2PC numa folha
12	Capítulos 41 a 46	Flashcards das Partes 4 e 5
13	Capítulos 47 a 51 e apêndices A e B	Faça a estimativa do apêndice B sem olhar
14	Apêndice C (estudo de caso)	Simulado completo; revise tudo o que errou

Tem uma entrevista em três dias? Leia a cola e o roteiro de entrevista (apêndice A) no primeiro dia, os números de referência (apêndice B) e o estudo de caso (apêndice C) no segundo, e faça o simulado e os flashcards no terceiro. Releia só os capítulos dos erros.

As ferramentas do livro

- **Cola:** os 51 conceitos em uma linha cada. Ótima para a véspera.
- **Flashcards:** todas as perguntas do "Teste-se" e o "Em uma frase" de cada capítulo. Na versão HTML, marque "Sabia" ou "Não sabia": as que você erra voltam antes. No PDF, cubra a resposta com a mão.
- **Simulado:** 40 questões de múltipla escolha, muitas com cenários de entrevista. Faça sem consultar e anote o capítulo de cada erro.
- **Glossário:** os termos técnicos com tradução e o capítulo onde aparecem.

Quando você domina um conceito

Use esta lista como critério. Você domina um conceito quando consegue:

1. explicá-lo em uma frase;
2. desenhar o diagrama dele;
3. dar um exemplo de sistema real que o usa;
4. dizer o que ele custa (o trade-off);
5. dizer quando **não** usá-lo.

O quinto item é o que separa quem decorou de quem entendeu. É também o que os entrevistadores mais gostam de ouvir.

Parte 1

Princípios de arquitetura

Core Architecture Principles

Antes de falar de load balancers, bancos e filas, você precisa do vocabulário que aparece em toda conversa de system design. Esta parte reúne dez princípios que sustentam o resto do livro: como um sistema cresce (cap. 1), o que ele sacrifica quando a rede falha (cap. 2) e no dia a dia (cap. 3), que garantias os dados oferecem (caps. 4 e 7) e como medir e limitar o desempenho (caps. 5 e 6).

Os últimos capítulos tratam de como organizar código e infraestrutura: serviços stateful ou stateless (cap. 8), monólito ou microsserviços (cap. 9) e serverless (cap. 10). As ideias se encadeiam: escalar na horizontal exige serviços stateless; réplicas trazem o dilema entre consistência, disponibilidade e latência; e cada máquina a mais esbarra no gargalo que a Lei de Amdahl prevê. Quase todos os capítulos seguintes voltam a algum destes.

1	Escalabilidade vertical vs. horizontal	2	Teorema CAP
3	Teorema PACELC	4	ACID vs. BASE
5	Throughput vs. latência	6	Lei de Amdahl
7	Consistência forte vs. eventual	8	Arquitetura stateful vs. stateless
9	Microsserviços vs. monólito	10	Arquitetura serverless

Escalabilidade vertical vs. horizontal

Vertical vs Horizontal Scaling

EM UMA FRASE

Escalar na vertical é turbinar uma única máquina; escalar na horizontal é somar máquinas e dividir o trabalho entre elas, o que exige load balancer, serviços stateless e dados compartilhados.

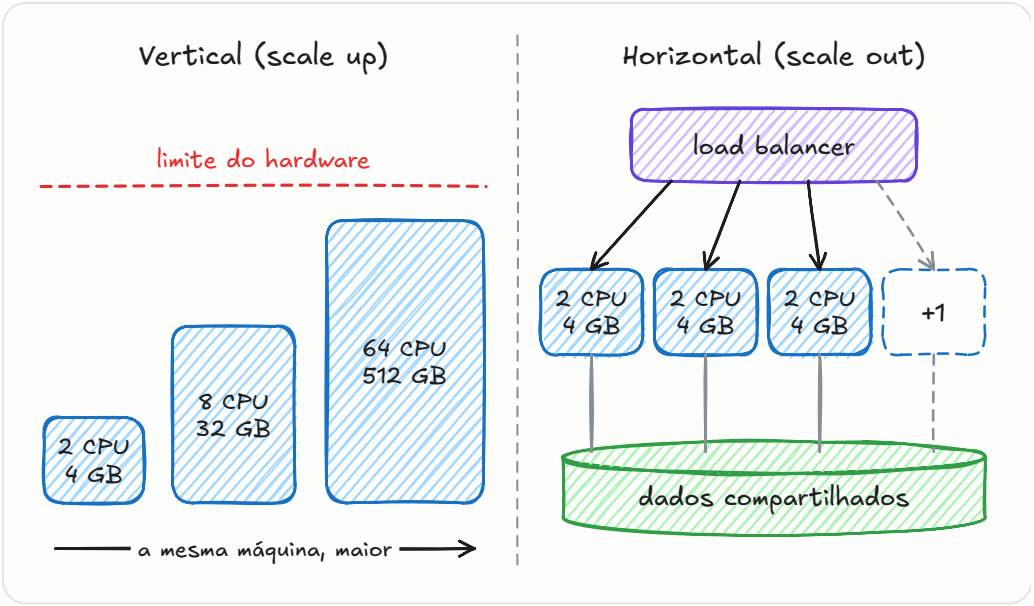
O ESSENCIAL · tradução do artigo original

- **Escalar verticalmente** significa fazer upgrade de uma única máquina, por exemplo adicionando mais CPU, mais RAM ou um armazenamento mais rápido.
- **Escalar horizontalmente** significa adicionar mais máquinas e espalhar o trabalho entre elas.

A escala vertical é mais fácil, mas esbarra nos limites do hardware e fica cara. A horizontal é mais difícil, porque você precisa de balanceamento de carga, serviços stateless e armazenamento compartilhado. Pense assim: a vertical é um super-herói ficando mais forte; a horizontal é montar uma equipe.

APROFUNDANDO

Em inglês, a escala vertical também é chamada de scale up, e a horizontal, de scale out. As duas resolvem o mesmo problema, mais carga do que a máquina atual aguenta, de jeitos opostos.



Na vertical, a mesma máquina cresce até encostar no limite do hardware. Na horizontal, um load balancer reparte as requisições entre máquinas iguais, que guardam os dados num lugar compartilhado.

Vertical: simples até bater no teto

Trocar uma máquina de 8 núcleos por uma de 64 não exige mudar uma linha de código, e tudo continua num lugar só: sem rede entre nós, sem dados divididos. Por isso é o primeiro passo de quase todo sistema, principalmente do banco de dados. Os problemas aparecem depois:

- **Teto físico.** Não dá para crescer para sempre: as maiores instâncias de nuvem têm centenas de vCPUs e terabytes de RAM, e acima disso não há para onde subir.
- **Ponto único de falha.** Se a máquina cai, cai tudo. E trocar o tamanho de uma máquina virtual costuma exigir desligá-la e religá-la.
- **Retorno decrescente.** Um software que não aproveita vários núcleos não fica mais rápido com mais núcleos (cap. 6).

Horizontal: quase sem teto, mas com pré-requisitos

Com várias máquinas atrás de um load balancer (cap. 11), a capacidade cresce somando peças comuns. Um exemplo: se cada servidor aguenta 2.000 requisições por segundo e o pico é de 9.000, você precisa de 5 servidores e coloca 6. Com essa folga,

chamada de N+1, a queda de um deles ainda deixa 10.000 req/s de capacidade. A falha de uma máquina vira perda de capacidade, e não queda do sistema. E um autoscaler pode ligar e desligar máquinas conforme a demanda.

O preço são os três pré-requisitos citados no começo do capítulo:

1. **Balanceamento de carga**, para distribuir as requisições.
2. **Serviços stateless** (cap. 8): se a sessão do usuário mora na memória do servidor 1, o servidor 2 não consegue atendê-lo.
3. **Armazenamento compartilhado**: os dados precisam estar acessíveis de qualquer máquina, num banco, num cache ou num storage de objetos.

O gargalo migra para os dados

Servidores de aplicação stateless escalam na horizontal com facilidade. O banco de dados, que guarda o estado, é o último a escalar. O caminho típico: crescer o banco na vertical, depois adicionar réplicas de leitura (cap. 24) e cache, e só então dividir os dados em shards (cap. 23). Na prática, os dois tipos se combinam: você escolhe um tamanho de máquina que faça sentido e multiplica.

> *Analogia*
>
> Pense numa linha de ônibus lotada. A prefeitura pode trocar o ônibus comum por um biarticulado (escala vertical): resolve por um tempo, mas não existe ônibus maior que caiba nas ruas e, se ele quebrar, a linha para. Ou pode colocar mais ônibus na mesma linha (escala horizontal): a capacidade cresce quase sem limite e um ônibus quebrado não para a linha, mas agora alguém precisa coordenar os horários e distribuir os passageiros.

NO MUNDO REAL

- **Bancos relacionais** como PostgreSQL e MySQL costumam crescer primeiro na vertical: todas as escritas passam por um primário, e uma máquina maior é o jeito mais simples de dar fôlego a ele.
- **O Stack Overflow** ficou conhecido por atender um tráfego enorme com poucos servidores robustos, inclusive os de banco (SQL Server), crescendo mais na vertical do que na horizontal.

- **Kubernetes e nuvem:** o Horizontal Pod Autoscaler cria e remove réplicas de um serviço conforme métricas como uso de CPU; na AWS, os Auto Scaling groups fazem o mesmo com máquinas virtuais.
- **Cassandra** foi desenhado para escalar na horizontal: mais capacidade é mais nós, com os dados repartidos entre eles (cap. 23).

TRADE-OFFS

	Vertical (scale up)	Horizontal (scale out)
Mudança no código	Nenhuma	Serviços stateless e dados compartilhados
Limite	A maior máquina disponível	Quase ilimitado na camada stateless
Falha de uma máquina	O sistema cai	Perde-se um pedaço da capacidade
Custo	Dispara no topo da linha	Cresce aos poucos, máquina a máquina
Complexidade	Baixa	Load balancer, rede, deploy em várias máquinas
Elasticidade	Redimensionar costuma exigir reinício	Autoscaling liga e desliga máquinas

NA ENTREVISTA

- Comece simples e mostre o caminho: "*Começo com algumas instâncias stateless atrás de um load balancer e um banco primário bem dimensionado. Quando o banco virar gargalo, adiciono réplicas de leitura e cache; se as escritas não couberem numa máquina, faço sharding.*"
- Faça a conta: estime o pico em req/s, a capacidade de uma máquina e quantas você precisa com folga (N+1).
- Mostre que sabe onde a horizontal dói: estado local (sessões, uploads em disco) e dados.
- Não descarte a vertical: para o banco, uma máquina maior costuma ser a solução mais barata no curto prazo.

CUIDADO COM

- ATUALIZAÇÃO: o “fica cara” vale menos na nuvem. Dentro de uma mesma família de instâncias, o preço costuma crescer na proporção do tamanho: o dobro de vCPUs custa cerca do dobro. Hoje, os argumentos fortes contra a vertical são o teto e o ponto único de falha.
- **Horizontal não é linear para sempre.** Coordenação, banco compartilhado e partes sequenciais limitam o ganho (cap. 6).
- **Mais máquinas com estado local não é escala horizontal.** Se cada servidor guarda sessões ou arquivos no próprio disco, adicionar servidores só espalha o problema.

TESTE-SE

1 **Qual a diferença entre escalar na vertical e escalar na horizontal?**

Na vertical, você aumenta a capacidade de uma única máquina (CPU, RAM, disco). Na horizontal, você adiciona máquinas e divide o trabalho entre elas.

2 **Cada servidor da sua API aguenta 1.500 req/s e o pico esperado é de 5.000 req/s. Quantos servidores você coloca?**

Cinco. Quatro cobrem o pico (6.000 req/s de capacidade para 5.000 de demanda) e o quinto é a folga para que a queda de um servidor não derrube o serviço (N+1).

3 **Você colocou 10 servidores atrás de um load balancer e o sistema continua lento. Adicionar mais 10 resolve?**

Não necessariamente. Se o gargalo é um banco único ou outro recurso compartilhado, mais servidores de aplicação só aumentam a fila na frente dele. É preciso achar e escalar o gargalo (cap. 6).

CONECTA COM

- 6 Lei de Amdahl
- 8 Arquitetura stateful vs. stateless
- 11 Balanceamento de carga
- 23 Sharding
- 24 Padrões de replicação

Teorema CAP

CAP Theorem

EM UMA FRASE

Quando a rede entre os nós falha, um sistema distribuído precisa escolher entre responder com dados talvez velhos (disponibilidade) ou recusar para não responder errado (consistência).

O ESSENCIAL · tradução do artigo original

O Teorema CAP diz que, na presença de uma partição de rede, um sistema distribuído precisa escolher entre **Consistência** e **Disponibilidade**. Consistência significa que todos os usuários veem os mesmos dados ao mesmo tempo. Disponibilidade significa que o sistema sempre responde, mesmo que os dados possam estar um pouco desatualizados.

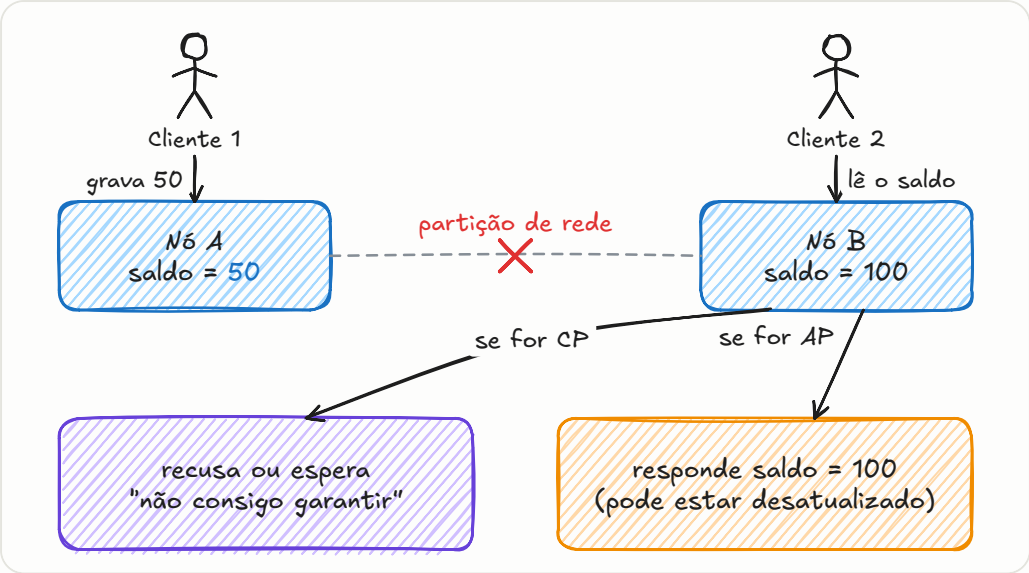
Você não consegue ter consistência perfeita e disponibilidade perfeita quando a sua rede está quebrada. Por isso, você decide qual das duas sacrificar de acordo com o seu caso de uso.

APROFUNDANDO

CAP é a sigla de três propriedades:

- **C de Consistency (consistência):** toda leitura recebe a escrita mais recente ou um erro. O nome técnico é *linearizabilidade*: o sistema se comporta como se existisse uma única cópia dos dados.
- **A de Availability (disponibilidade):** toda requisição que chega a um nó saudável recebe uma resposta que não é erro, sem garantia de que seja a mais recente.
- **P de Partition tolerance (tolerância a partições):** o sistema continua funcionando mesmo quando mensagens entre os nós se perdem ou atrasam.

O ponto que muita gente erra: **num sistema distribuído, P não é opcional.** Redes falham. Um cabo é cortado, um switch trava, um datacenter perde contato com o outro. Então a escolha real nunca foi "escolha 2 de 3". A pergunta é: *quando acontecer uma partição, o que o sistema faz?*



Durante a partição, o nó B não sabe da escrita feita no nó A. Ele pode recusar (CP) ou responder com o que tem (AP).

Imagine dois nós, A e B, que guardam cópias de um saldo de R$ 100. A rede entre eles cai. Um cliente grava "saldo = 50" no nó A e, logo depois, outro cliente lê o saldo no nó B:

- **Sistema CP:** o nó B percebe que não consegue confirmar com A e **recusa** a leitura (ou espera a rede voltar). Você perde disponibilidade, mas nunca devolve um dado errado.
- **Sistema AP:** o nó B **responde** com o último valor que conhece, R$ 100. Você continua no ar, mas pode servir um dado velho. Quando a rede volta, os nós reconciliam as diferenças.

Fora das partições, um sistema pode, e deve, ser consistente e disponível ao mesmo tempo. O CAP só fala do momento da falha. O que acontece no dia a dia, sem partição, é assunto do PACELC (cap. 3).

Analogia

Duas agências de um banco anotam os saldos dos clientes em cadernos e se atualizam por telefone. A linha cai. Um cliente chega na agência B querendo sacar. A agência pode **recusar o saque** até a linha voltar (escolheu consistência) ou **liberar o saque** com base no caderno, correndo o risco de o cliente já ter sacado na agência A (escolheu disponibilidade).

NO MUNDO REAL

- **Tendência CP:** sistemas de coordenação como **etcd** e **ZooKeeper** (que guardam configuração e locks do Kubernetes e do Kafka antigo, por exemplo) preferem ficar indisponíveis do lado minoritário de uma partição a devolver dados inconsistentes.
- **Tendência AP:** o **Cassandra** e o **DynamoDB** (com leituras eventualmente consistentes, que são o padrão dele) priorizam continuar respondendo e reconciliam as réplicas depois.
- **Ajustável por consulta:** no Cassandra você escolhe o nível de consistência de cada leitura e escrita (`ONE`, `QUORUM`, `ALL`), movendo o ponteiro entre C e A conforme a operação.

TRADE-OFFS

	Priorizar consistência (CP)	Priorizar disponibilidade (AP)
Durante a partição	Recusa ou atrasa parte das requisições	Responde sempre, talvez com dado velho
Bom para	Saldo, estoque, reserva de assento, locks	Feed, curtidas, carrinho, contadores
Risco	Erros e timeouts para o usuário	Conflitos que precisam ser resolvidos depois
Custo de engenharia	Consenso e quórum entre réplicas	Reconciliação (last-write-wins, CRDTs)

NA ENTREVISTA

- Nunca diga “escolhi CA”. Diga: “*Como partições são inevitáveis, a pergunta é o que fazemos quando uma acontece.*”
- Amarre a escolha ao **requisito de negócio**: “Para o saldo da carteira, prefiro CP: melhor um erro temporário do que gastar o mesmo dinheiro duas vezes. Para a contagem de curtidas, AP: ninguém se importa se o número atrasar alguns segundos.”
- Mostre que a escolha pode ser **por funcionalidade**, e não uma só para o sistema inteiro.

CUIDADO COM

- **“Escolha 2 de 3”** é uma simplificação enganosa. Em sistemas distribuídos, P não é negociável.
- **O C do CAP não é o C do ACID** (cap. 4). No ACID, consistência é respeitar as regras de integridade do banco. No CAP, é linearizabilidade.
- **“Disponível” no CAP é uma definição formal** (todo nó saudável responde). Não é o mesmo que os “99,9% de uptime” de um SLA (cap. 48).

TESTE-SE

1 **Por que “escolher CA” não faz sentido num sistema distribuído?**

Porque partições de rede acontecem de qualquer jeito. O sistema precisa tolerá-las; a escolha real é entre C e A *durante* a partição.

2 **Um sistema de reserva de assentos de avião deve tender a CP ou AP?**

CP. Vender o mesmo assento para duas pessoas é muito pior do que recusar uma reserva por alguns segundos durante uma falha de rede.

3 Qual a diferença entre o “C” do CAP e o “C” do ACID?

No CAP, é linearizabilidade: toda leitura vê a última escrita. No ACID, é manter as regras de integridade do banco (constraints, invariantes) depois de cada transação.

CONECTA COM

3 Teorema PACELC
4 ACID vs. BASE
7 Consistência forte vs. eventual
24 Padrões de replicação
38 Eleição de líder (Paxos e Raft)

Teorema PACELC

PACELC Theorem

EM UMA FRASE

Se há partição de rede (P), o sistema escolhe entre disponibilidade (A) e consistência (C); senão (E), escolhe entre latência baixa (L) e consistência (C), porque manter réplicas em sincronia custa tempo.

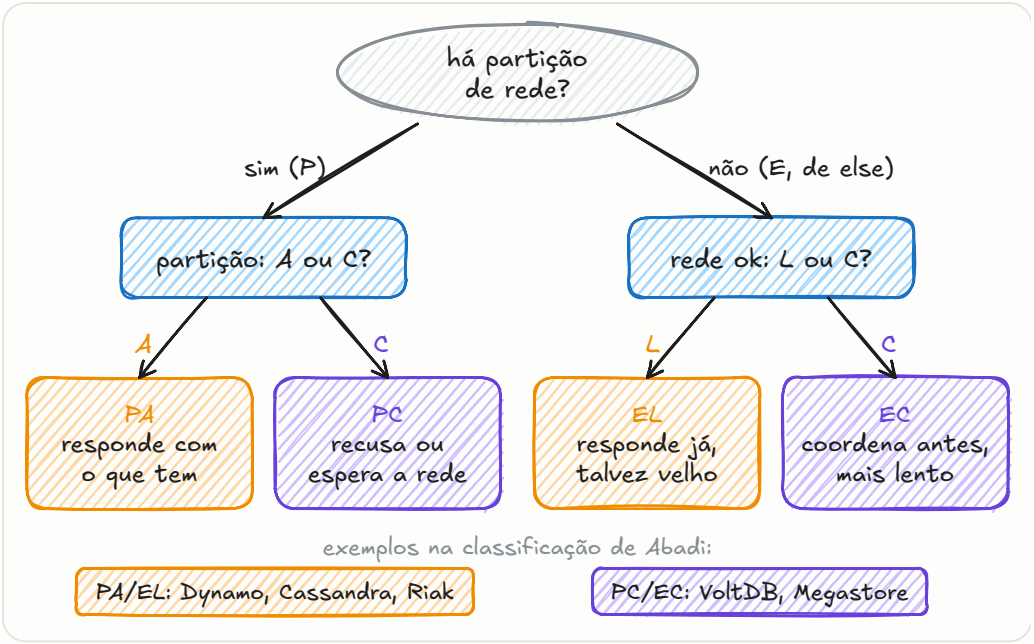
O ESSENCIAL · tradução do artigo original

O PACELC estende o CAP e diz: se houver uma **P**artição, escolha entre disponibilidade (**A**vailability) e **C**onsistência; senão (**E**lse), escolha entre **L**atência e **C**onsistência. Mesmo quando a rede está funcionando bem, você continua tendo de escolher entre leituras lentas, mas consistentes, e leituras rápidas, mas eventualmente consistentes.

Sistemas que sincronizam dados entre regiões muitas vezes pagam em latência para manter a consistência forte. Isso explica por que alguns bancos de dados são rápidos, mas um pouco desatualizados, enquanto outros são mais lentos, mas sempre precisos.

APROFUNDANDO

A sigla foi proposta por Daniel Abadi em 2010 e detalhada num artigo dele de 2012. A crítica ao CAP (cap. 2): partições são raras, mas o sistema paga um preço pela consistência o tempo todo, mesmo com a rede perfeita. Leia a sigla como um if/else de programação: **se P, então A ou C; senão (E), L ou C.**



Leia o PACELC como um if/else: com partição, o sistema escolhe entre A e C; sem partição, entre L e C. Os exemplos vêm da classificação original de Abadi e valem para as configurações padrão.

De onde vem a latência

O custo do lado E (else) vem da replicação (cap. 24). Para uma escrita ser consistente, a réplica que a recebe precisa ouvir as outras antes de confirmar. Para uma leitura ser consistente, ela precisa ir ao líder ou consultar várias réplicas. Cada consulta custa pelo menos um round trip pela rede:

Onde estão as réplicas	Round trip típico
Mesmo datacenter	menos de 1 ms
Zonas diferentes da mesma região	poucos ms
São Paulo e costa leste dos EUA	por volta de 120 ms

Com réplicas no mesmo datacenter, a consistência sai quase de graça. Com réplicas em continentes diferentes, cada operação consistente fica uma viagem intercontinental mais lenta.

O botão do quórum

Bancos no estilo Dynamo deixam você girar esse botão. Com 3 réplicas (N = 3), você define quantas precisam confirmar cada escrita (W) e quantas respondem cada leitura (R):

- **W = 1, R = 1:** responde a réplica mais rápida. Latência mínima, mas a leitura pode cair numa réplica que ainda não recebeu a escrita. É o EL.
- **W = 2, R = 2:** como R + W > N, todo conjunto de leitura cruza com o de escrita, e a leitura sempre encontra pelo menos uma réplica com a escrita mais recente. Em troca, cada operação espera a segunda resposta, e não só a mais rápida. É o EC.

Na partição, o mesmo botão decide o PAC: se o cliente só alcança uma réplica, exigir W = 2 significa recusar a escrita (PC); aceitar com W = 1 mantém o sistema no ar, ao custo de conflitos para resolver depois (PA).

Analogia

Você precisa confirmar quantas pessoas da família vão ao churrasco de domingo. Num dia normal, pode responder na hora, pelo que acha (rápido, mas talvez errado), ou perguntar no grupo da família e esperar todo mundo responder (certo, mas demorado): latência ou consistência. Se o celular de metade da família estiver sem sinal (a partição), a escolha muda: chutar o número (disponibilidade) ou não confirmar até falar com todos (consistência).

NO MUNDO REAL

- **DynamoDB:** as leituras são eventualmente consistentes por padrão (EL). Com `ConsistentRead`, a leitura é fortemente consistente, consome o dobro de capacidade, pode demorar mais e pode falhar se houver problema de rede: EC e PC na mesma tabela.
- **Cassandra:** o nível de consistência é escolhido a cada consulta. Com vários datacenters, `QUORUM` conta réplicas de todos eles e pode esperar por outra região; `LOCAL_QUORUM` conta só as do datacenter local e responde mais rápido.
- **Azure Cosmos DB:** oferece cinco níveis de consistência, de Strong a Eventual, com Session como padrão, e a documentação apresenta cada nível como um trade-off entre consistência, latência e disponibilidade.

TRADE-OFFS

Sem partição	Priorizar latência (EL)	Priorizar consistência (EC)
Leitura	Da réplica mais próxima ou mais rápida	Do líder ou de um quórum
Escrita	Confirma depois de gravar numa réplica	Confirma depois de várias réplicas gravarem
Tempo de resposta	Mínimo, mesmo com réplicas distantes	Soma round trips; cresce com a distância
Dado lido	Pode estar atrasado	O mais recente já confirmado
Bom para	Feed, catálogo, contadores, recomendações	Saldo, estoque, configuração, locks

NA ENTREVISTA

- Traga o PACELC quando o CAP aparecer: "O CAP *só fala da falha. No dia a dia, a pergunta é quanta latência eu aceito pagar por consistência.*"
- Se o design tem réplicas em várias regiões, faça a conta: uma escrita consistente entre São Paulo e a Virgínia custa pelo menos uns 120 ms a mais.
- Escolha por operação: "Saldo: EC, leitura no líder. Feed: EL, leitura na réplica local."
- Cite os botões concretos: quórum (R + W > N), nível de consistência por consulta, leitura forte opcional.

CUIDADO COM

- `ATUALIZAÇÃO:` o artigo fala em bancos "rápidos" e bancos "precisos", mas hoje muitos deixam escolher por operação: `ConsistentRead` no DynamoDB, nível por consulta no Cassandra, cinco níveis no Cosmos DB. Rótulos como PA/EL descrevem o comportamento padrão, não uma lei do produto.
- **O L vale para escritas também.** Replicação síncrona deixa a escrita mais lenta, não só a leitura.

- **EL não é "inconsistente para sempre".** As réplicas convergem; o risco é ler um valor atrasado por algum tempo (cap. 7).

TESTE-SE

1 **O que a parte "ELC" do PACELC acrescenta ao CAP?**

Que, mesmo sem partição, existe um trade-off entre latência e consistência: manter réplicas sincronizadas exige esperar por elas pela rede, o que deixa cada operação mais lenta.

2 **Uma rede social com réplicas em três continentes quer carregar o feed em menos de 100 ms. Como o feed deve se comportar no dia a dia?**

Como EL: ler da réplica do próprio continente e aceitar alguns segundos de atraso. Coordenar com outros continentes custaria mais de 100 ms só em round trips.

3 **O DynamoDB é PA/EL ou PC/EC?**

Depende da operação. Leituras eventualmente consistentes, o padrão, se comportam como EL; leituras com `ConsistentRead` são fortes, podem demorar mais e podem falhar durante problemas de rede, um comportamento EC/PC.

CONECTA COM

2 Teorema CAP · 4 ACID vs. BASE · 7 Consistência forte vs. eventual · 24 Padrões de replicação

ACID vs. BASE

ACID vs BASE

EM UMA FRASE

ACID garante transações tudo-ou-nada, isoladas e duráveis, ideais para dinheiro e estoque; BASE aceita inconsistências temporárias em troca de disponibilidade e escala, com as réplicas convergindo depois.

O ESSENCIAL · tradução do artigo original

ACID trata de transações rígidas e confiáveis: **A**tomicidade, **C**onsistência, **I**solamento e **D**urabilidade. Combina com sistemas financeiros, controle de estoque e qualquer coisa em que erros custem muito caro.

BASE significa Basically Available (basicamente disponível), Soft state (estado flexível) e Eventual consistency (consistência eventual), e é usado em grandes sistemas distribuídos que precisam ficar no ar e responder rápido. Sistemas BASE podem mostrar inconsistências temporárias, mas se corrigem com o tempo.

Na prática, muitas arquiteturas combinam os dois: ACID para os fluxos centrais de dinheiro e BASE para coisas como feeds e analytics.

APROFUNDANDO

ACID, letra por letra

Pense numa transferência de R$ 200 da Ana para o Bruno:

```
BEGIN;
UPDATE contas SET saldo = saldo - 200 WHERE id = 'ana';
UPDATE contas SET saldo = saldo + 200 WHERE id = 'bruno';
COMMIT;
```

- **Atomicidade:** os dois `UPDATE` acontecem juntos ou nenhum acontece. Se o servidor cair entre um e outro, o banco desfaz o primeiro (rollback).

- **Consistência:** a transação leva o banco de um estado válido a outro. Se existe a regra `CHECK (saldo ⩾ 0)` e a Ana só tem R$ 150, a transação inteira é recusada.
- **Isolamento:** transações simultâneas não enxergam o trabalho pela metade umas das outras. Ninguém vê o dinheiro já fora da conta da Ana e ainda não na do Bruno. O rigor depende do nível de isolamento, de read committed a serializable.
- **Durabilidade:** depois que o `COMMIT` é confirmado, a transferência sobrevive a uma queda de energia, porque o banco gravou a mudança num log em disco antes de responder (cap. 27).

BASE, letra por letra

A sigla surgiu no fim dos anos 1990, no grupo de Eric Brewer (o mesmo do CAP, cap. 2), como contraponto ao ACID:

- **Basically Available:** o sistema responde mesmo com partes falhando, ainda que com dados parciais ou atrasados.
- **Soft state:** o estado pode mudar sem nenhuma escrita nova, enquanto as réplicas trocam atualizações entre si.
- **Eventual consistency:** se as escritas param, todas as réplicas acabam com o mesmo valor (cap. 7).

Exemplo: um post recebe curtidas em São Paulo e em Lisboa ao mesmo tempo. Cada região soma localmente e responde na hora; por alguns segundos, uma mostra 1.204 curtidas e a outra, 1.206. Depois as réplicas trocam os incrementos e as duas chegam ao mesmo número.

Os dois no mesmo sistema

Num e-commerce, criar o pedido e registrar o pagamento ficam numa transação ACID, num banco relacional. Atualizar o índice de busca, as recomendações e o painel de vendas pode ser BASE: eventos publicados depois do commit são processados com alguns segundos de atraso, e ninguém perde dinheiro com isso.

Analogia

Comprar um apartamento é ACID: a escritura passa no cartório inteira ou não passa, ninguém registra meio imóvel e, depois de registrada, ela não some. Já a notícia de que a padaria vai fechar no feriado se espalha pelo bairro como BASE: de boca em boca, uns ficam sabendo antes dos outros, ninguém precisou parar a rua para anunciar e, no fim do dia, todo mundo sabe.

NO MUNDO REAL

- **PostgreSQL, MySQL (InnoDB), Oracle e SQL Server** oferecem transações ACID completas; é onde mora o dinheiro da maioria das empresas.
- **Cassandra** segue a linha BASE: escritas rápidas, réplicas que convergem depois. Para operações do tipo "grave só se ainda não existir", oferece lightweight transactions, baseadas em Paxos e bem mais lentas.
- **DynamoDB** é BASE no uso comum, mas tem transações ACID desde 2018 (`TransactWriteItems`); o **MongoDB** tem transações com vários documentos desde a versão 4.0.
- **Spanner** e **CockroachDB** entregam ACID distribuído entre regiões, pagando com latência de coordenação.

TRADE-OFFS

	ACID	BASE
Prioridade	Correção de cada transação	Disponibilidade e escala
Leitura logo após a escrita	Vê o valor confirmado	Pode ver um valor antigo
Falha no meio da operação	Rollback: tudo ou nada	Reconciliação depois
Escala	Mais difícil de distribuir	Feita para muitos nós
Bom para	Pagamentos, estoque, reservas	Feeds, curtidas, analytics, logs

NA ENTREVISTA

- Divida o sistema por tipo de dado: "*Pedido e pagamento num banco relacional com transações; feed e contadores num armazenamento eventualmente consistente.*"
- Justifique pelo custo do erro: vender o mesmo item duas vezes ou cobrar em dobro exige ACID; um contador de visualizações atrasado, não.
- Se o entrevistador pedir uma transação entre serviços, lembre que ACID não atravessa a rede de graça: fale de sagas (cap. 39) ou de two-phase commit (cap. 40).

CUIDADO COM

- `ATUALIZAÇÃO:` a divisão "SQL é ACID, NoSQL é BASE" envelheceu. MongoDB e DynamoDB têm transações ACID, e bancos como Spanner e CockroachDB oferecem ACID distribuído. Hoje a pergunta é que garantia cada operação precisa.
- **O C do ACID depende de você.** O banco só protege as regras que você declarou (`CHECK`, `UNIQUE`, chaves estrangeiras). Ele não sabe que o estoque não pode ficar negativo se ninguém disse isso a ele.
- **Isolamento padrão não é serializável.** O PostgreSQL usa read committed por padrão e o MySQL (InnoDB), repeatable read. Ler um valor, calcular na aplicação e gravar o resultado pode perder atualizações concorrentes.

TESTE-SE

1 **O que garante o D, de durabilidade?**

Que uma transação confirmada sobrevive a quedas e reinícios. Em geral, o banco grava a mudança num log em disco antes de responder ao `COMMIT`.

2 **Num app de delivery, o que você deixaria em ACID e o que poderia ser BASE?**

Criação do pedido e pagamento em ACID, porque erro ali custa dinheiro. Posição do entregador no mapa, contagem de avaliações e recomendações podem ser BASE, porque alguns segundos de atraso não prejudicam ninguém.

3 **Escolher um banco NoSQL significa abrir mão de transações ACID?**

Não necessariamente. MongoDB e DynamoDB oferecem transações ACID, com limites e custo extra. A escolha é por operação, não pelo rótulo do banco.

CONECTA COM

2 Teorema CAP

7 Consistência forte vs. eventual

27 Write-Ahead Logging (WAL)

39 Transações distribuídas: o padrão SAGA

40 Two-Phase Commit (2PC)

Throughput vs. latência

Throughput vs Latency

EM UMA FRASE

Throughput é quantas requisições o sistema atende por segundo; latência é quanto tempo cada uma leva. Paralelismo e lotes aumentam a vazão, mas filas perto do limite fazem a latência disparar.

O ESSENCIAL · tradução do artigo original

- **Throughput** (vazão) é quantas requisições o seu sistema consegue atender por segundo.
- **Latência** é quanto tempo uma única requisição leva, do início ao fim.

Muitas vezes você consegue aumentar o throughput fazendo mais trabalho em paralelo, mas isso pode aumentar a latência se filas se formarem. Pense num restaurante que aceita muitos pedidos ao mesmo tempo, mas faz os clientes esperarem mais. Um bom design de sistema tenta equilibrar os dois: throughput suficiente para o pico de carga, mas latência baixa para uma experiência de uso fluida.

APROFUNDANDO

Como medir

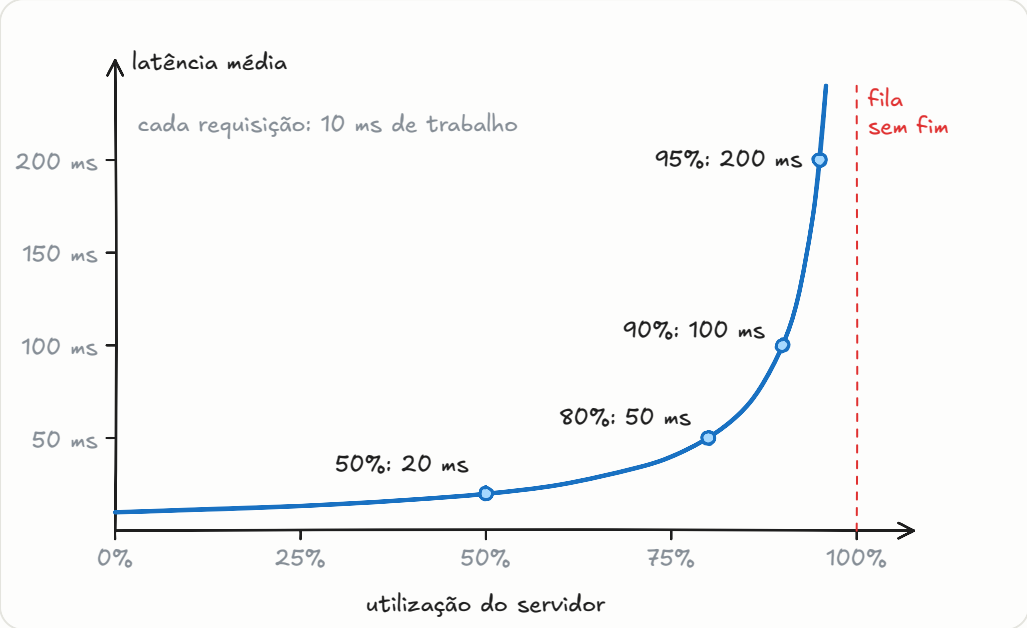
Throughput se mede em requisições por segundo (RPS, ou QPS quando são consultas) ou em MB/s. Latência se mede em milissegundos, e nunca só pela média. Use percentis: p50 é a mediana; um p99 de 800 ms significa que 1 em cada 100 requisições leva 800 ms ou mais. A cauda pesa mais do que parece: no artigo *The Tail at Scale* (Google, 2013), Jeff Dean e Luiz André Barroso mostram que, se uma requisição depende de 100 servidores em paralelo e cada um é lento em 1% das vezes, 63% das requisições ficam lentas ($1 - 0{,}99^{100} \approx 0{,}63$).

A ponte entre os dois: a Lei de Little

Quantas requisições estão em andamento ao mesmo tempo? A Lei de Little responde: **concorrência = throughput × latência**. Para atender 1.000 req/s com 200 ms cada, você precisa sustentar 1.000 × 0,2 = 200 requisições simultâneas, em threads, conexões ou workers. Se cada requisição segura uma conexão com o banco durante esses 200 ms e o banco aceita no máximo 50 conexões, o teto é 50 ÷ 0,2 = 250 req/s, não importa quantos servidores de aplicação você ligue.

Por que a latência dispara perto do limite

Quando as requisições chegam mais rápido do que o servidor dá conta, elas esperam na fila. Num modelo clássico de fila (M/M/1), a latência média é o tempo de serviço dividido por (1 – utilização):



A latência média dobra a 50% de utilização, quintuplica a 80% e dispara perto de 100%: a fila cresce mais rápido do que o servidor consegue esvaziá-la.

Com 10 ms de trabalho por requisição, o servidor a 50% de utilização responde em 20 ms, em média; a 90%, em 100 ms. Subir de 50% para 90% rende 80% mais vazão e multiplica a latência por cinco. Por isso sistemas saudáveis operam com folga, longe dos 100%.

Lotes: vazão em troca de latência

Juntar trabalho em lotes (batching) aumenta a vazão porque divide um custo fixo, como uma ida ao disco ou um pacote de rede, entre muitos itens; em troca, cada item espera o lote encher. O produtor do Kafka faz isso com a configuração `linger.ms`, que espera alguns milissegundos para agrupar mensagens, e o algoritmo de Nagle, no TCP, faz o mesmo com pacotes pequenos.

> *Analogia*
>
> A balsa que cruza o rio leva 60 carros por viagem, mas cada travessia, com embarque e desembarque, dura meia hora: vazão alta, latência alta. Uma lancha leva seis pessoas em cinco minutos: latência baixa, vazão pequena. Pôr uma segunda balsa dobra quantos carros cruzam por hora, mas não encurta nenhuma travessia. E no feriado, quando a fila cresce no cais, a espera passa a ser maior do que a própria viagem.

NO MUNDO REAL

- **Kafka** é o exemplo clássico de throughput: agrupa mensagens em lotes e grava em disco de forma sequencial para sustentar volumes enormes, aceitando alguns milissegundos a mais por mensagem.
- **Redis** e outros caches em memória (cap. 41) atacam a latência: uma leitura com ida e volta na rede local costuma levar menos de 1 ms, contra vários milissegundos de uma consulta que precisa ir ao disco.
- **CDNs** (cap. 15) reduzem latência encurtando a distância: servir um arquivo de um servidor na mesma cidade evita a viagem até outro continente.
- **Metas em percentis:** SLOs costumam ser escritos como "p99 abaixo de 300 ms", e não como média (cap. 48).

TRADE-OFFS

	Otimizar throughput	Otimizar latência
Técnicas típicas	Lotes, filas, paralelismo	Cache, proximidade, menos round trips

	Otimizar throughput	Otimizar latência
Utilização das máquinas	Alta, sempre ocupadas	Com folga, para manter filas curtas
Custo por requisição	Menor	Maior (capacidade ociosa)
Bom para	Ingestão de logs, ETL, jobs em lote	Checkout, busca, jogos, APIs interativas
Métrica principal	Itens por segundo	p95 e p99

NA ENTREVISTA

- Pergunte cedo os dois números: "*Quantas requisições por segundo no pico? Qual latência é aceitável no p99?*" Eles guiam o resto do design.
- Use a Lei de Little para dimensionar: "1.000 req/s × 200 ms = 200 requisições simultâneas; preciso de pelo menos isso em conexões e workers."
- Separe o caminho síncrono do assíncrono: o que o usuário espera precisa de latência baixa; o resto vai para uma fila (cap. 44) e é otimizado para throughput.
- Diga que planeja com folga, longe dos 100% de utilização.

CUIDADO COM

- **Média esconde a cauda.** Uma média de 50 ms convive com um p99 de 2 s: basta 1% de requisições muito lentas. Acompanhe p95 e p99.
- **Mais servidores não encurtam uma requisição.** Eles aumentam a vazão e reduzem a fila, mas, se a requisição faz dez consultas em série, continua lenta. Para cortar latência, corte trabalho ou paralelize dentro da requisição.
- **A latência do servidor não é a que o usuário sente.** O usuário soma DNS, conexão TLS, rede e renderização. Meça também do lado do cliente.

TESTE-SE

1 Qual a diferença entre throughput e latência?

Throughput é quantas requisições o sistema completa por unidade de tempo; latência é quanto tempo uma requisição leva, do início ao fim.

2 Seu serviço precisa atender 2.000 req/s, e cada requisição segura uma conexão com o banco por 50 ms. Quantas conexões o pool precisa ter, no mínimo?

Pela Lei de Little, 2.000 × 0,05 = 100 conexões ocupadas em média. Na prática, um pouco mais, para absorver variações.

3 Se cada requisição leva 100 ms, o servidor atende no máximo 10 por segundo?

Só se atender uma de cada vez. Com 50 requisições em paralelo, ele pode chegar a 500 por segundo: a latência limita cada requisição, não a vazão total.

CONECTA COM

1 Escalabilidade vertical vs. horizontal
6 Lei de Amdahl
41 Cache
44 Filas de mensagens
48 SLA vs. SLO vs. SLI

6 PARTE 1 · PRINCÍPIOS DE ARQUITETURA

Lei de Amdahl

Amdahl's Law

EM UMA FRASE

O ganho de velocidade com paralelismo é limitado pela parte do trabalho que continua sequencial; se 20% não se paraleliza, nem infinitas máquinas deixam o sistema mais de 5 vezes mais rápido.

O ESSENCIAL · tradução do artigo original

A Lei de Amdahl diz que o ganho de velocidade obtido com paralelização é limitado pela parte que não pode ser paralelizada. Se 20% do seu sistema é sempre sequencial, nenhuma quantidade de máquinas extras vai resolver esse gargalo.

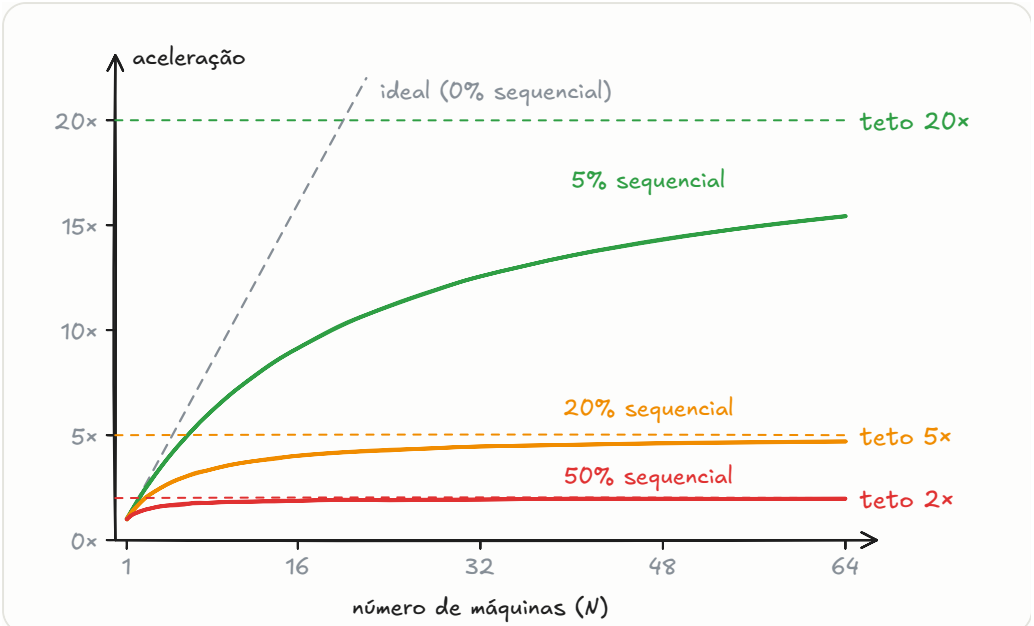
Explicando melhor: se toda requisição sempre precisa passar por um único banco de dados primário (o master), esse primário vai limitar o seu desempenho. Essa lei lembra você de caçar gargalos, em vez de simplesmente adicionar mais servidores.

APROFUNDANDO

Gene Amdahl apresentou o argumento em 1967. Se **s** é a fração do trabalho que só roda em sequência e **N** é o número de processadores ou máquinas, a aceleração (speedup) é:

```
aceleração(N) = 1 / (s + (1 - s) / N)
```

A parte paralela encolhe à medida que N cresce; a sequencial, não. Com N tendendo ao infinito, (1 − s)/N vai a zero e sobra o teto: **1/s.** Com s = 20%, o máximo é 5×; com 5%, 20×; com 50%, 2×.



Cada curva achata ao se aproximar do seu teto, 1/s: com 20% sequencial, 16 máquinas dão 4× e 64 máquinas, só 4,7×.

Um exemplo com números

Imagine um job noturno que leva 100 minutos numa máquina: 80 minutos de processamento que dá para dividir e 20 minutos de uma etapa que só roda em sequência, como consolidar tudo num arquivo único.

Máquinas	Tempo	Aceleração
1	20 + 80 = 100 min	1×
4	20 + 20 = 40 min	2,5×
16	20 + 5 = 25 min	4×
64	20 + 1,25 = 21,25 min	4,7×

Ir de 16 para 64 máquinas multiplica o custo por quatro e economiza menos de 4 minutos. Os 20 minutos sequenciais nunca somem.

Na arquitetura

O exemplo do primário único aplica a mesma ideia à vazão: se toda escrita passa por ele e ele aguenta 5.000 escritas por segundo, o sistema nunca passa disso, com 10 ou com 100 servidores de aplicação. As saídas atacam o s:

- **Medir antes de escalar:** tracing (cap. 47) e profiling mostram onde o tempo realmente vai.
- **Dividir o recurso único:** sharding (cap. 23), réplicas de leitura (cap. 24), cache (cap. 41).
- **Tirar trabalho do caminho crítico:** o que não precisa acontecer antes da resposta vai para uma fila (cap. 44).
- **Evitar locks globais e contadores centrais**, que fazem esperar em fila quem poderia correr em paralelo.

> *Analogia*
>
> Numa mudança, chamar mais amigos ajuda a encaixotar e a levar as caixas até o corredor. Mas o prédio tem um elevador de serviço só, e cada viagem dele demora o mesmo, com 2 ou com 20 amigos. Se metade do dia é esperar o elevador, nem um time inteiro faz a mudança terminar em menos da metade do tempo. Para ganhar de verdade, você precisa de um segundo elevador, ou de menos caixas.

NO MUNDO REAL

- **Processadores com muitos núcleos:** um programa que passa 10% do tempo em trechos sequenciais nunca fica mais de 10× mais rápido, tenha a máquina 64 ou 128 núcleos. Depois de certo ponto, núcleos extras rendem pouco.
- **MapReduce e Spark:** o processamento se divide entre centenas de máquinas, mas cada etapa só termina quando a tarefa mais lenta termina, e juntar o resultado final num lugar só é trabalho sequencial. O artigo original do MapReduce (Google, 2004) já descrevia tarefas de reserva para não ficar refém dessas retardatárias.
- **Bancos com um único primário:** no PostgreSQL e no MySQL com replicação tradicional, todas as escritas passam pelo primário; réplicas aumentam a capacidade de leitura, mas não a de escrita.

TRADE-OFFS

	Adicionar máquinas	Reduzir a parte sequencial
Esforço	Baixo: mudar um número no autoscaler	Alto: redesenhar dados e fluxo
Ganho	Decrescente, com teto em 1/s	Sobe o próprio teto
Custo	Cresce com cada máquina	Engenharia, uma vez
Risco	Pagar por máquinas quase ociosas	Complexidade: shards, filas, consistência
Quando faz sentido	Longe do teto, com s pequeno	Curva já achatada

NA ENTREVISTA

- Quando propuser "mais servidores", diga o que continua único: banco primário, lock global, contador central, fila com um só consumidor.
- Faça a conta de cabeça: "*Se 10% do trabalho de cada requisição é sequencial no primário, o teto é 10×; para passar disso, preciso particionar as escritas.*"
- Mostre que mede antes: "Primeiro eu olharia o tracing para ver onde a latência está."
- Mencione que a curva achata: dobrar as máquinas perto do teto quase não muda nada.

CUIDADO COM

- ATUALIZAÇÃO: a lei é otimista para sistemas distribuídos, porque ignora o custo de coordenar as máquinas (rede, locks, consenso). A Lei Universal de Escalabilidade, de Neil Gunther, inclui esse custo e mostra que, a partir de certo ponto, mais máquinas podem deixar o sistema **mais lento**.
- **Amdahl supõe um problema de tamanho fixo.** Se o volume de dados cresce junto com as máquinas, a parte paralela cresce e o ganho total é maior; essa é a Lei de Gustafson, de 1988.
- **O exemplo do primário é sobre vazão**, não sobre o tempo de uma tarefa. O raciocínio é o mesmo: o recurso que todos disputam define o teto.

TESTE-SE

1 O que diz a Lei de Amdahl?

Que a aceleração obtida com paralelismo é limitada pela fração sequencial do trabalho: com fração s, o máximo é 1/s, não importa quantas máquinas você adicione.

2 Um processo tem 10% de trabalho sequencial. Qual a aceleração com 9 máquinas? E com infinitas?

Com 9: 1 / (0,1 + 0,9/9) = 1 / 0,2 = 5×. Com infinitas, o teto é 1 / 0,1 = 10×.

3 Com 20% de trabalho sequencial, dobrar as máquinas de 32 para 64 dobra a velocidade?

Não. A aceleração passa de cerca de 4,4× para 4,7×: a parte paralela já ficou pequena perto dos 20% sequenciais, que não encolhem.

CONECTA COM

1 Escalabilidade vertical vs. horizontal
5 Throughput vs. latência
23 Sharding
24 Padrões de replicação
47 Tracing distribuído

7 PARTE 1 · PRINCÍPIOS DE ARQUITETURA

Consistência forte vs. eventual

Strong vs Eventual Consistency

EM UMA FRASE

Com consistência forte, toda leitura vê a última escrita confirmada; com consistência eventual, as réplicas podem divergir por um tempo, mas convergem para o mesmo valor quando as escritas param.

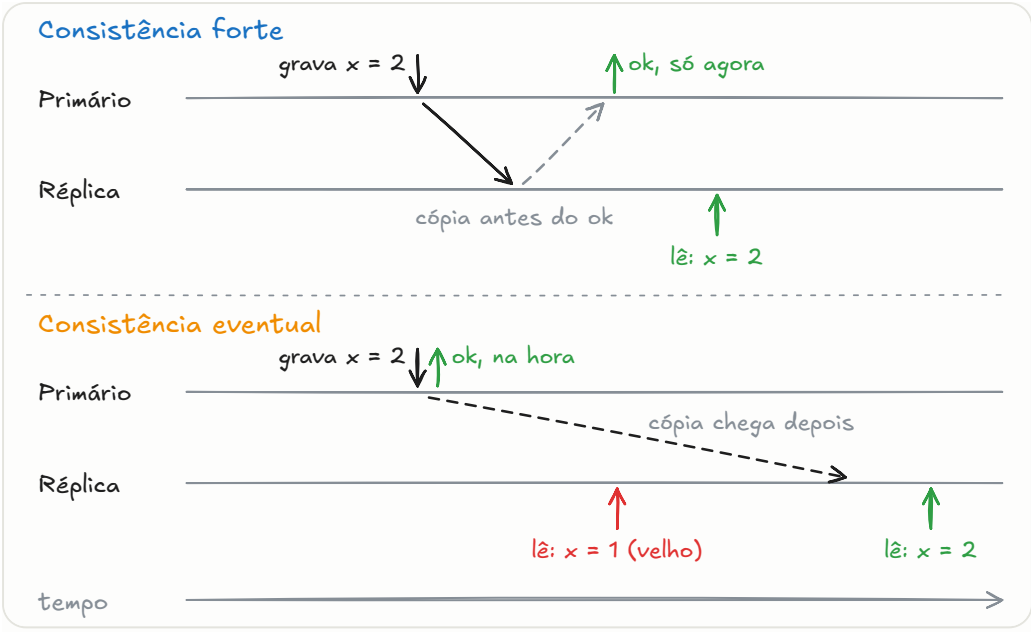
O ESSENCIAL · tradução do artigo original

- **Consistência forte** significa que todos os usuários veem os mesmos dados imediatamente depois de uma escrita.
- **Consistência eventual** significa que as atualizações se espalham ao longo do tempo e os nós podem discordar por um breve período.

A consistência forte é mais fácil de entender, mas costuma ser mais lenta e menos disponível quando há falhas. A consistência eventual é ótima para sistemas de grande escala, como timelines ou contadores, em que o frescor perfeito não é crítico. O segredo é escolher o modelo que combina com a experiência de usuário de que você precisa.

APROFUNDANDO

Os dois modelos aparecem assim que você replica dados (cap. 24): um primário recebe as escritas e réplicas copiam o que ele grava. A diferença está em **quando o primário confirma a escrita** e **de onde vêm as leituras**.



Com consistência forte, o primário só confirma depois de copiar para a réplica, e a leitura vê o valor novo. Com consistência eventual, a confirmação vem na hora, e uma leitura logo depois ainda encontra o valor velho.

- **Forte:** o primário só confirma depois que as réplicas também aplicaram a mudança (replicação síncrona), ou as leituras vão sempre ao primário ou a um quórum. Depois do "ok", qualquer leitura vê o valor novo. O nome técnico é linearizabilidade (cap. 2).
- **Eventual:** o primário confirma na hora e as réplicas recebem a mudança depois (replicação assíncrona). A distância entre o primário e uma réplica é o atraso de replicação (replication lag): em geral, milissegundos; sob carga pesada ou falha, segundos ou minutos.

A promessa eventual é modesta: *se as escritas pararem, um dia todas as réplicas terão o mesmo valor*. Ela não diz quando, nem o que você vê no meio do caminho.

O meio-termo

Entre os dois extremos, há garantias que resolvem os bugs mais visíveis sem pagar o preço inteiro:

- **Ler o que você escreveu (read-your-writes):** você sempre vê as próprias escritas. Resolve o clássico "postei um comentário, recarreguei a página e ele sumiu".

- **Leituras monotônicas:** você nunca volta no tempo. Sem essa garantia, um contador pode mostrar 105 e, ao recarregar, 103, porque a segunda leitura caiu numa réplica mais atrasada.
- **Consistência causal:** se uma escrita depende de outra, todos veem as duas na ordem certa. A resposta nunca aparece antes da pergunta.

Para ter read-your-writes com réplicas assíncronas, um truque comum é mandar as leituras do próprio usuário para o primário por alguns segundos depois de ele escrever. Outro é guardar a versão da escrita e só ler de uma réplica que já chegou nela.

Analogia

Quando o supermercado muda um preço, o caixa passa a cobrar o valor novo na hora, mas as etiquetas das gôndolas são trocadas por um funcionário ao longo da manhã. Até ele passar pelo corredor, quem olha a prateleira vê o preço antigo: consistência eventual. Consistência forte seria só cobrar o preço novo depois que a última etiqueta fosse trocada: nunca haveria divergência, mas a mudança demoraria bem mais para valer.

NO MUNDO REAL

- **Amazon S3:** até 2020, sobrescrever um objeto e lê-lo logo depois podia devolver a versão antiga. Desde dezembro de 2020, o S3 garante consistência forte de leitura após escrita para objetos (gravações, sobrescritas, exclusões e listagens), sem custo extra.
- **PostgreSQL:** a replicação por streaming é assíncrona por padrão, e uma réplica pode estar atrasada. Com `synchronous_commit = remote_apply`, o commit só retorna depois que a réplica síncrona aplicou a mudança e já a mostra nas consultas.
- **DNS** é eventual por desenho: uma mudança de registro só chega a todos quando os caches expiram, conforme o TTL (cap. 16).
- **Google Spanner** oferece consistência forte entre regiões, com relógios sincronizados (TrueTime) e consenso, ao custo de escritas mais lentas.

TRADE-OFFS

	Consistência forte	Consistência eventual
Leitura logo após a escrita	Sempre vê o valor novo	Pode ver o valor antigo por um tempo
Latência de escrita	Maior: espera réplicas ou quórum	Menor: confirma na hora
Durante falhas	Pode recusar ou atrasar	Continua respondendo
Código da aplicação	Simples de raciocinar	Precisa tolerar dados atrasados e conflitos
Bom para	Saldo, estoque, senha, permissões	Timeline, curtidas, contadores, recomendações

NA ENTREVISTA

- Escolha por funcionalidade, não para o sistema inteiro: "*Saldo e estoque com consistência forte; timeline e curtidas, eventual.*"
- Mostre que conhece o atraso de replicação e o efeito dele no usuário, e ofereça a correção: "No perfil, garanto read-your-writes: logo depois de editar, as leituras do próprio usuário vão para o primário."
- Se o entrevistador perguntar "eventual quanto?", fale em medir o replication lag e alertar quando ele passar de um limite.

CUIDADO COM

- **"Eventual" não tem prazo.** A definição só promete convergência se as escritas pararem. Na prática, o atraso costuma ser de milissegundos, mas sob carga pode chegar a minutos; monitore.
- **"Imediatamente" conta a partir da confirmação.** Mesmo com consistência forte, uma leitura simultânea à escrita pode ver o valor velho ou o novo; a garantia vale para leituras que começam depois do "ok".
- ATUALIZAÇÃO: o artigo apresenta só os dois extremos. Na prática, os bancos oferecem níveis intermediários (read-your-writes, leituras monotônicas, causal, atraso máximo garantido) e deixam escolher por operação (cap. 3).

TESTE-SE

1 **O que a consistência eventual garante, e o que ela não garante?**

Garante que, se as escritas pararem, as réplicas convergem para o mesmo valor. Não garante quando isso acontece, nem que uma leitura logo depois da escrita veja o valor novo.

2 **Um usuário troca a foto de perfil, recarrega a página e vê a foto antiga. Que garantia está faltando, e como você corrige?**

Read-your-writes. Mande as leituras desse usuário para o primário por alguns segundos depois da escrita, ou leia só de réplicas que já alcançaram a versão que ele gravou.

3 **Num sistema eventualmente consistente, um contador de curtidas pode mostrar 105 e, ao recarregar, 103?**

Pode. Sem a garantia de leituras monotônicas, a segunda leitura pode cair numa réplica mais atrasada que a primeira.

CONECTA COM

2 Teorema CAP · 3 Teorema PACELC · 4 ACID vs. BASE · 24 Padrões de replicação

Arquitetura stateful vs. stateless

Stateful vs Stateless Architecture

EM UMA FRASE

Um serviço stateful guarda o contexto do usuário na própria memória entre requisições; um stateless trata cada requisição como nova e guarda o estado fora, então qualquer instância pode atender qualquer requisição.

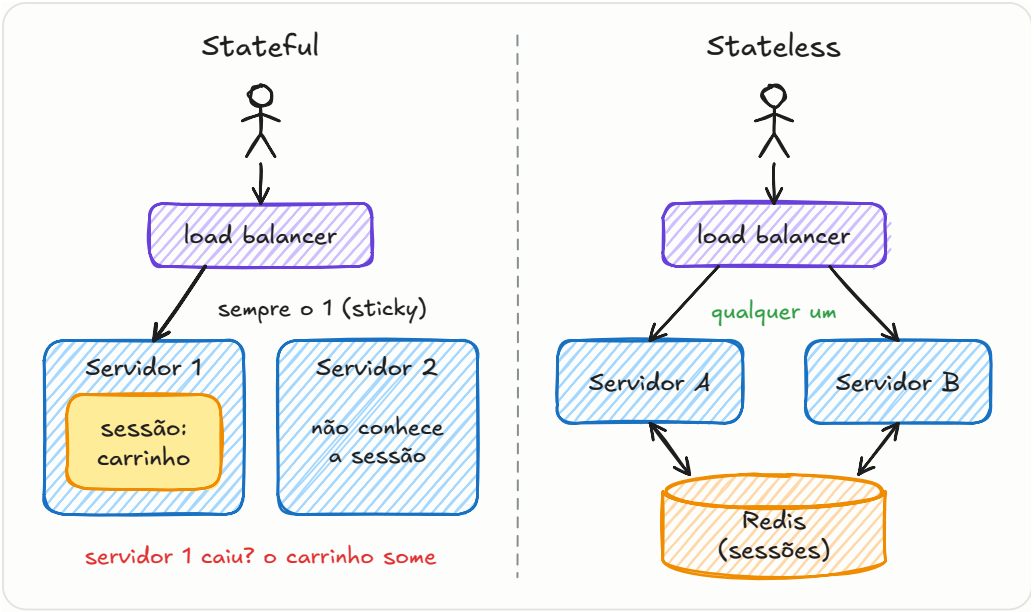
O ESSENCIAL · tradução do artigo original

- Um serviço **stateful** (com estado) lembra o contexto do usuário entre uma requisição e outra, muitas vezes guardando os dados de sessão localmente.
- Um serviço **stateless** (sem estado) trata cada requisição como nova e depende de armazenamentos externos, como caches ou bancos de dados, para qualquer estado.

Serviços stateless são mais fáceis de escalar horizontalmente, porque qualquer instância pode atender qualquer requisição. Sistemas stateful podem ser mais simples de programar, mas são mais difíceis de balancear e de recuperar quando um servidor falha (failover). Nos sistemas modernos em nuvem, tentamos empurrar o estado para os bancos de dados e manter os serviços o mais stateless possível.

APROFUNDANDO

Imagine uma loja virtual com dois servidores atrás de um load balancer (cap. 11). A usuária põe um tênis no carrinho; a requisição cai no servidor 1, que guarda o carrinho num dicionário em memória. A próxima requisição cai no servidor 2, e o carrinho está vazio.



Com o estado na memória, a usuária fica presa ao servidor 1. Sem estado nos servidores, qualquer um atende, porque a sessão mora no Redis.

Há três jeitos de resolver:

1. **Sticky sessions (afinidade de sessão):** o load balancer marca a usuária, em geral com um cookie, e manda todas as requisições dela para o servidor 1. Funciona, mas o estado continua preso à máquina: se o servidor 1 cair ou for reiniciado num deploy, o carrinho some; a carga fica desigual; e desligar servidores no autoscaling derruba sessões.
2. **Estado num armazenamento externo:** o carrinho vai para um Redis ou um banco, indexado por um ID de sessão que viaja no cookie. Qualquer servidor busca a sessão pelo ID. Custa uma ida à rede por requisição (menos de 1 ms no mesmo datacenter), mas os servidores viram peças descartáveis.
3. **Estado com o cliente:** a sessão viaja na própria requisição, num token assinado, como um JWT. O servidor só confere a assinatura, sem consultar nada. O preço: é difícil revogar um token antes de ele expirar, e um cookie comporta só uns 4 KB.

Stateless não quer dizer "sem estado nenhum"

O estado não desaparece: ele sai da memória de cada servidor e vai para uma camada feita para guardá-lo, com replicação e backup. Algumas coisas são stateful por natureza: bancos de dados, conexões WebSocket abertas (cap. 20), servidores de jogo com

a partida em memória. Para elas, a saída não é fingir que o estado não existe, e sim tratá-lo com técnicas próprias, como réplicas, particionamento e checkpoints.

Um cache local em memória é aceitável num serviço stateless, desde que seja só otimização: se a instância morrer, nada se perde além de algumas consultas mais lentas.

Analogia

Ligar para um call center onde só a atendente Márcia conhece o seu caso é stateful: se a ligação cai e você é atendido por outra pessoa, precisa explicar tudo de novo; se a Márcia sai de férias, ninguém sabe do seu problema. O atendimento com número de protocolo é stateless: qualquer atendente digita o protocolo, puxa o histórico do sistema e continua de onde parou.

NO MUNDO REAL

- **The Twelve-Factor App**, guia de boas práticas para aplicações em nuvem, manda executar a aplicação como processos stateless, chama sticky sessions de violação e sugere guardar sessões num armazenamento com expiração, como Redis ou Memcached.
- **Kubernetes** separa os dois mundos: Deployments para serviços stateless, cujos pods são intercambiáveis, e StatefulSets para bancos e afins, que precisam de identidade estável e disco próprio por pod.
- **HTTP** é stateless por definição: cada requisição pode ser entendida sozinha. Os cookies foram criados justamente para carregar estado entre requisições.

TRADE-OFFS

	Stateful	Stateless
Onde fica a sessão	Na memória do servidor	Em Redis, banco ou token do cliente
Balanceamento	Precisa de afinidade (sticky)	Qualquer algoritmo serve
Falha de um servidor	As sessões dele se perdem	Outro servidor assume sem perda

	Stateful	Stateless
Escala e deploy	Desligar máquinas derruba usuários	Liga e desliga instâncias à vontade
Latência	O estado já está na memória	Uma consulta extra ao armazenamento
Código	Mais simples no começo	Exige decidir onde o estado mora

NA ENTREVISTA

- Ao desenhar a camada de aplicação, diga logo: "*Os servidores são stateless; a sessão fica no Redis com TTL. Assim, o autoscaling e os deploys não derrubam ninguém.*"
- Se surgir algo stateful (WebSocket, jogo, upload em andamento), mostre como lida com isso: um registro de qual servidor tem a conexão de cada usuário e pub/sub (cap. 45) para entregar as mensagens.
- Se citar JWT, mencione o trade-off: dispensa a consulta, mas complica a revogação; tokens de vida curta ajudam.

CUIDADO COM

- **Sticky session não torna o serviço stateless.** Ela só esconde o estado atrás do load balancer; a sessão continua morrendo com o servidor.
- **Disco local também é estado.** Um upload salvo em `/tmp` no servidor 1 não existe no servidor 2; use um storage de objetos.
- **JWT não é cofre.** O conteúdo de um token assinado só está codificado em base64, e qualquer um que tenha o token consegue lê-lo. Não coloque nele nada sigiloso.

TESTE-SE

1 **O que torna um serviço stateless?**

Ele não guarda, entre uma requisição e outra, nada que só ele saiba. O estado necessário vem na própria requisição (um token) ou é buscado num armazenamento externo, então qualquer instância pode atender.

2 **Você tem quatro servidores com sessões em memória e precisa fazer deploy sem deslogar ninguém. O que muda na arquitetura?**

As sessões saem da memória: vão para um Redis com TTL ou viram tokens assinados. Aí dá para reiniciar os servidores um a um sem perder nenhuma sessão.

3 **Ligar sticky sessions no load balancer resolve o problema de um serviço stateful?**

Não. O usuário volta sempre ao mesmo servidor, mas o estado continua preso a ele: se o servidor cair ou for reiniciado, a sessão se perde, e a carga pode ficar desigual.

CONECTA COM

1 Escalabilidade vertical vs. horizontal
10 Arquitetura serverless
11 Balanceamento de carga
20 WebSocket e Server-Sent Events
41 Cache

Microsserviços vs. monólito

Microservices vs Monoliths

EM UMA FRASE

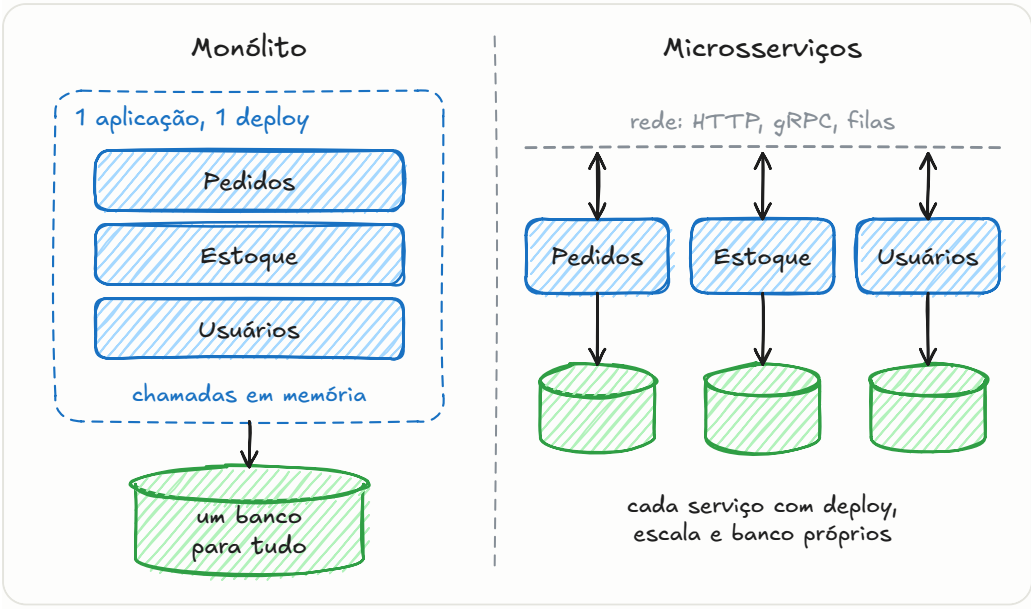
Um monólito reúne todas as funcionalidades numa única unidade de deploy; microsserviços as separam em serviços independentes que conversam pela rede, trocando simplicidade por autonomia dos times e escala separada.

O ESSENCIAL · tradução do artigo original

Um monólito é uma única aplicação que reúne muitas funcionalidades numa só unidade de deploy. Microsserviços dividem as funcionalidades em serviços separados, que se comunicam pela rede.

Microsserviços ajudam os times a trabalhar de forma independente e a escalar partes diferentes separadamente, mas trazem complexidade na comunicação, na depuração e na consistência dos dados. Monólitos são mais simples para começar e muitas vezes dão conta até uma certa escala.

E aqui mora o detalhe: muitos sistemas excelentes começam como monólitos e evoluem aos poucos para microsserviços quando a dor é real.



No monólito, os módulos se chamam em memória e dividem um banco. Nos microsserviços, cada serviço tem deploy e banco próprios e fala com os outros pela rede.

Quando uma função vira chamada de rede

No monólito, o checkout chama `estoque.reservar()` e `pagamento.cobrar()` como funções: leva nanossegundos, não falha pela metade e tudo cabe numa transação do banco. Separe em serviços e cada chamada passa a:

- **Demorar mais:** ida e volta na rede, mais serialização em JSON ou Protobuf. Dez chamadas em sequência somam milissegundos.
- **Falhar parcialmente:** o pagamento pode ter sido cobrado e a resposta, se perdido no caminho. Você passa a precisar de timeouts, retries com backoff (cap. 35), idempotência (cap. 36) e circuit breakers (cap. 33).
- **Perder a transação única:** cada serviço tem o próprio banco, então "reservar e cobrar" vira uma saga, com ações de compensação (cap. 39).
- **Dificultar a depuração:** um erro atravessa cinco serviços; sem tracing distribuído (cap. 47), ninguém acha a origem.

A disponibilidade também se multiplica. Se uma página depende de 10 serviços, cada um com 99,9% de disponibilidade, ela fica em $0{,}999^{10} \approx 99\%$: de uns 43 minutos fora do ar por mês para mais de 7 horas.

O que você ganha em troca

- **Deploy independente:** o time de pagamentos publica várias vezes por dia sem esperar os outros.
- **Escala separada:** se a busca recebe 50 vezes mais tráfego que o cadastro, só ela ganha mais instâncias.
- **Isolamento de falhas:** com timeouts e fallback, um vazamento de memória nas recomendações não derruba o checkout.

O caminho do meio

O **monólito modular** tem um único deploy, mas módulos com fronteiras claras: cada um com sua API interna e suas tabelas, sem mexer nas dos outros. Quando um módulo precisa mesmo escalar ou ser publicado sozinho, ele é extraído aos poucos, com o padrão strangler fig (figueira-estranguladora): o serviço novo assume uma rota de cada vez, até o código antigo poder ser removido.

E vale lembrar a Lei de Conway: sistemas tendem a copiar a estrutura de comunicação da organização que os constrói. Microsserviços fazem mais sentido quando há vários times que precisam andar sem pisar uns nos outros.

> *Analogia*
>
> Um restaurante tradicional tem uma cozinha só: todos os pratos saem dela, a equipe se fala por cima do balcão e, se o gás acaba, para tudo. Uma praça de alimentação é como microsserviços: cada quiosque tem cozinha, estoque e equipe próprios e muda o cardápio sem pedir licença aos vizinhos. Mas montar uma bandeja com o hambúrguer de um, o suco de outro e a sobremesa de um terceiro exige enfrentar três filas.

NO MUNDO REAL

- **Netflix:** depois de uma corrupção de banco de dados que, em 2008, parou o envio de DVDs por três dias, trocou o datacenter próprio e a aplicação monolítica por centenas de microsserviços na AWS, migração concluída em 2016.
- **Shopify:** mantém o núcleo como um grande monólito em Ruby on Rails, dividido em componentes com fronteiras verificadas por uma ferramenta própria, o Packwerk, em vez de quebrá-lo em serviços.

- **Amazon Prime Video:** em 2023, a equipe de monitoramento de qualidade de vídeo trocou uma arquitetura distribuída, com funções serverless, por um único processo e relatou corte de mais de 90% no custo de infraestrutura daquele serviço.

TRADE-OFFS

	Monólito	Microsserviços
Para começar	Rápido: um repositório, um deploy	Lento: infraestrutura, pipelines, rede
Chamadas internas	Funções em memória	Rede, com latência e falhas parciais
Dados	Um banco, transações ACID	Banco por serviço, sagas, consistência eventual
Deploy	Tudo junto, coordenado entre times	Por serviço, times independentes
Escala	O sistema inteiro de uma vez	Cada parte conforme a demanda
Depuração	Um log, uma stack trace	Tracing distribuído obrigatório

NA ENTREVISTA

- Comece pelo porquê, não pela moda: "*Com um time pequeno, começo com um monólito modular. Extraio um serviço quando houver um motivo concreto: escala muito diferente, ritmo de deploy próprio ou um time dono dele.*"
- Se desenhar microsserviços, mostre que sabe o custo: API Gateway (cap. 14), comunicação síncrona ou por eventos, sagas para consistência, tracing para depurar.
- Diga explicitamente que cada serviço tem o próprio banco.

CUIDADO COM

- **Monólito distribuído:** serviços que só podem ser publicados juntos, ou que leem as tabelas uns dos outros, somam os custos dos dois mundos sem os benefícios.
- **Microsserviço não se define pelo tamanho**, e sim por deploy independente, dados próprios e uma API clara.
- ATUALIZAÇÃO: o artigo descreve um caminho de mão única, do monólito para microsserviços. Hoje o monólito modular é tratado como destino legítimo, e há casos documentados de volta, como o do Prime Video em 2023.

TESTE-SE

1 **O que caracteriza um microsserviço, além de ser "pequeno"?**

Deploy independente, dados próprios e comunicação com os outros serviços apenas por uma API, pela rede. O tamanho não é o critério.

2 **Uma startup com quatro desenvolvedores vai lançar um MVP. Monólito ou microsserviços?**

Monólito, de preferência modular: um deploy, um banco, transações simples e menos infraestrutura. Microsserviços resolvem problemas de times grandes e escala desigual, que ainda não existem.

3 **Dez serviços separados que leem e escrevem no mesmo banco de dados formam uma arquitetura de microsserviços?**

Não de verdade. É um monólito distribuído: uma mudança no esquema afeta todos, e nenhum serviço consegue evoluir ou escalar os próprios dados de forma independente.

CONECTA COM

- 8 Arquitetura stateful vs. stateless
- 10 Arquitetura serverless
- 14 API Gateway
- 39 Transações distribuídas: o padrão SAGA
- 47 Tracing distribuído

Arquitetura serverless

Serverless Architecture

EM UMA FRASE

No serverless, você entrega funções que a nuvem executa sob demanda, cuidando dos servidores e da escala; paga por execução e, em troca, aceita cold starts, limite de duração e custo alto com tráfego constante.

O ESSENCIAL · tradução do artigo original

Serverless permite rodar pequenas funções na nuvem sem gerenciar servidores diretamente. Você só paga quando o seu código roda, e a plataforma cuida da escala e da infraestrutura por você.

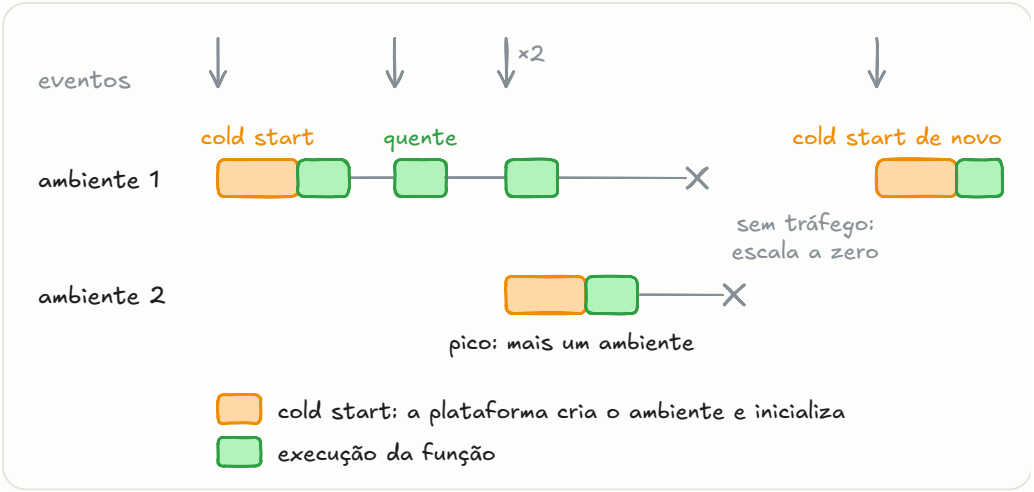
É ideal para cargas orientadas a eventos, como webhooks, jobs em segundo plano ou APIs leves com tráfego em picos. O preço a pagar é menos controle sobre tarefas longas, cold starts (partidas a frio) e, às vezes, um custo maior em volumes muito altos.

Pense em serverless como "funções como serviço" (FaaS, de functions as a service), perfeito para código de cola (glue code) e serviços leves.

APROFUNDANDO

Você não aluga uma máquina: entrega uma função e diz qual evento a dispara. Pode ser uma requisição HTTP vinda de um API Gateway (cap. 14), um arquivo novo no storage, uma mensagem numa fila (cap. 44) ou um horário agendado. O AWS Lambda, lançado em 2014, popularizou o modelo.

O ciclo de vida de uma função



O primeiro evento paga o cold start; os seguintes reaproveitam o ambiente quente. Um pico cria mais ambientes, e, sem tráfego, a escala volta a zero.

1. **Chega um evento.** Se não há ambiente de execução livre, a plataforma cria um: aloca um contêiner ou uma microVM isolada, carrega o seu código, inicia o runtime e roda a inicialização (imports, conexões). Esse é o **cold start**, que vai de dezenas de milissegundos a alguns segundos, conforme a linguagem, o tamanho do pacote e o que o código faz ao iniciar.
2. **A função executa** e devolve o resultado.
3. **O ambiente fica quente** por um tempo e é reaproveitado nas chamadas seguintes, sem novo cold start.
4. **Num pico**, a plataforma cria mais ambientes em paralelo. No modelo padrão do Lambda, cada ambiente atende uma invocação por vez: 300 requisições simultâneas pedem 300 ambientes, até o limite de concorrência da conta.
5. **Sem tráfego**, os ambientes são descartados e a escala volta a zero.

A conta

No Lambda, você paga por invocação e pelo tempo de execução multiplicado pela memória configurada (GB-segundo), medido em milissegundos. Com tráfego esparso, como um webhook que dispara algumas vezes por hora, a conta é quase zero, porque nada fica ligado esperando. Com tráfego alto e constante, ela se inverte: você

paga cada milissegundo de cada execução, inclusive o tempo em que a função só espera o banco responder, enquanto um servidor sempre ligado atenderia muitas requisições em paralelo pelo mesmo preço por hora.

Limites que moldam o design

- **Duração:** o Lambda interrompe qualquer execução em 15 minutos. Trabalho longo precisa ser dividido em pedaços ou ir para contêineres.
- **Sem estado entre chamadas:** o ambiente pode ou não ser reaproveitado, então tudo que importa vai para fora (cap. 8).
- **Picos que derrubam vizinhos:** mil funções abrindo mil conexões esgotam um banco relacional. Para isso existem proxies de conexão, como o RDS Proxy.

> *Analogia*
>
> Ter carro próprio é como ter servidor: você paga IPVA, seguro e garagem mesmo com ele parado, mas ele está sempre na porta. Usar carro de aplicativo é serverless: você paga só as corridas, não cuida de manutenção e nunca falta carro no dia em que a família inteira precisa sair. Em compensação, às vezes espera o motorista chegar (o cold start) e, se rodar o dia todo, todo dia, o aplicativo sai bem mais caro que o carro.

NO MUNDO REAL

- **AWS Lambda**, **Azure Functions** e **Google Cloud Run functions** são as plataformas de funções dos três grandes provedores. O **Cloudflare Workers** roda funções na borda da rede usando isolates do V8, com cold starts bem menores que os de contêineres.
- **Processamento de arquivos:** gerar miniaturas quando uma imagem chega num bucket do S3 é o exemplo clássico da documentação do Lambda.
- **Contra o cold start:** o Lambda oferece provisioned concurrency (ambientes pré-aquecidos, cobrados mesmo parados) e o SnapStart, que restaura o ambiente a partir de um snapshot já inicializado.

TRADE-OFFS

	Serverless (FaaS)	Servidores sempre ligados
Operação	O provedor cuida de máquinas e patches	Você provisiona, atualiza e monitora
Escala	Automática, de zero a milhares	Autoscaling que você configura
Custo parado	Zero	Paga as máquinas ligadas
Custo com tráfego alto e constante	Tende a ser maior	Tende a ser menor
Latência	Cold starts ocasionais	Estável, sem partida a frio
Tarefas longas	Limitadas (15 minutos no Lambda)	Sem limite de duração

NA ENTREVISTA

- Use serverless para o que é orientado a eventos e irregular: "*O processamento das fotos enviadas roda numa função disparada pelo upload; escala sozinho na Black Friday e custa quase nada de madrugada.*"
- Evite no caminho crítico com p99 apertado ou com tráfego alto e constante, e diga por quê: cold start e custo.
- Não esqueça o banco: muitas funções simultâneas exigem um pool ou proxy de conexões, ou um banco que aceite muitas conexões.

CUIDADO COM

- **"Serverless" tem servidor**, só que você não o gerencia.
- ATUALIZAÇÃO: serverless hoje é mais que FaaS. O termo também cobre contêineres que escalam a zero, como o Cloud Run, e bancos e filas cobrados por uso, como o DynamoDB sob demanda e o SQS. E os cold starts encolheram com SnapStart e ambientes pré-aquecidos.
- **Escala sem freio pode derrubar dependências.** A função escala em segundos; o banco atrás dela, não. Configure limites de concorrência.

TESTE-SE

1 O que é um cold start?

É o atraso da primeira execução num ambiente novo: a plataforma precisa criar o ambiente, carregar o código e rodar a inicialização antes de executar a função.

2 Você precisa transcodificar vídeos que levam 40 minutos cada. Uma função Lambda resolve?

Não diretamente, porque o Lambda tem limite de 15 minutos por execução. Use contêineres ou um serviço de processamento em lote, ou divida o vídeo em trechos processados em paralelo por várias funções.

3 Serverless é sempre mais barato, já que você só paga pelo que usa?

Não. Com tráfego alto e constante, pagar cada milissegundo de cada execução costuma sair mais caro do que manter ligados servidores bem aproveitados.

CONECTA COM

8 Arquitetura stateful vs. stateless
9 Microsserviços vs. monólito
14 API Gateway
44 Filas de mensagens
45 Pub/Sub

Parte 2

Redes e comunicação

Networking and Communication

Na Parte 1, o sistema ganhou forma: várias instâncias, serviços separados, estado fora dos servidores. Agora a pergunta é como essas peças conversam. Toda requisição faz uma viagem. O DNS (cap. 16) traduz o nome em endereço, uma CDN (cap. 15) responde o que puder perto do usuário, e o resto passa por proxies reversos (cap. 13), API gateways (cap. 14) e load balancers (cap. 11), que escolhem com um algoritmo (cap. 12) qual servidor atende.

A segunda metade desce até os protocolos que carregam as mensagens: TCP e UDP (cap. 17), HTTP/2 e HTTP/3 (cap. 18), gRPC e REST (cap. 19). Depois vêm as formas de o servidor avisar o cliente sem esperar a pergunta, com WebSocket e SSE (cap. 20) e long polling (cap. 21), e o protocolo gossip (cap. 22), que espalha informação num cluster sem coordenador central. Numa entrevista, é esta parte que vira as caixas e setas do seu desenho.

11 Balanceamento de carga
12 Algoritmos de balanceamento
13 Proxy reverso vs. proxy direto
14 API Gateway
15 CDN
16 DNS
17 TCP vs. UDP
18 HTTP/2 e HTTP/3 (QUIC)
19 gRPC vs. REST
20 WebSocket e Server-Sent Events
21 Long polling
22 Protocolo gossip

11 PARTE 2 · REDES E COMUNICAÇÃO

Balanceamento de carga

Load Balancing

EM UMA FRASE

Um load balancer recebe o tráfego num endereço único e o reparte entre vários servidores, tirando da rotação os que falham no health check; é a primeira peça de qualquer escala horizontal.

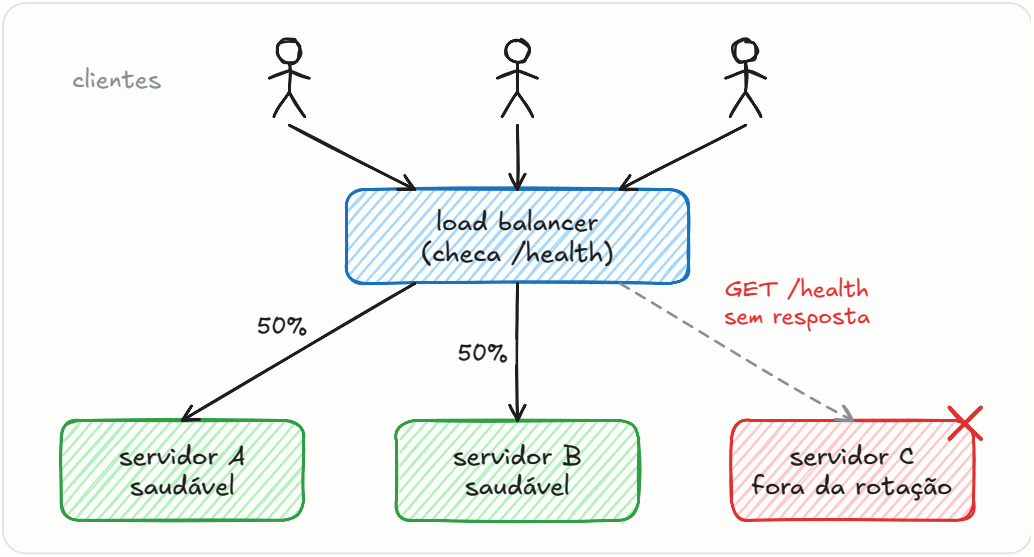
O ESSENCIAL · tradução do artigo original

O balanceamento de carga (load balancing) distribui o tráfego que chega entre vários servidores, para que nenhum deles fique sobrecarregado. Ele melhora tanto a confiabilidade quanto o desempenho, já que a falha de um único servidor não derruba o sistema inteiro.

Os balanceadores de carga (load balancers) podem ser equipamentos de hardware ou serviços de software. Eles costumam oferecer **health checks** (verificações de saúde), para parar de mandar tráfego para instâncias que não estão saudáveis. Do ponto de vista da entrevista, eles são o seu primeiro bloco de construção quando você escala horizontalmente.

APROFUNDANDO

O load balancer (LB) fica entre os clientes e um grupo de servidores iguais, o pool. Os clientes conhecem um único endereço, o do LB. Quantos servidores existem atrás dele, e quais, é problema do LB. A cada nova conexão ou requisição, ele escolhe um servidor saudável segundo um algoritmo (cap. 12) e encaminha o tráfego para lá.



O load balancer reparte o tráfego entre A e B. O servidor C falhou no health check: saiu da rotação e só continua sendo checado.

Health check

O LB pergunta a cada servidor, de tempos em tempos, se ele está bem. Uma configuração típica: a cada 5 segundos, um `GET /health` com timeout de 2 segundos; três falhas seguidas tiram o servidor da rotação, e duas respostas `200` seguidas o trazem de volta. Exigir falhas seguidas evita tirar alguém do ar por um soluço da rede. O preço é uma janela cega: nesse exemplo, um servidor que morre ainda recebe requisições por uns 15 segundos. Para encurtá-la, muitos LBs também observam o tráfego real e afastam quem começa a devolver erros (checagem passiva).

Camada 4 ou camada 7

Um LB de camada 4 (transporte) enxerga só IPs e portas e repassa conexões TCP ou UDP inteiras, sem abrir o conteúdo. É rápido e barato. Um LB de camada 7 (aplicação) entende HTTP: lê caminho, cabeçalhos e cookies, então consegue mandar `/api` para um pool e `/imagens` para outro. Em troca, gasta mais CPU por requisição, inclusive para decifrar o TLS.

Folga e redundância

Balancear não cria capacidade. Se cada servidor aguenta 2.000 req/s e o pico é de 9.000 req/s, cinco servidores dão conta, com 1.800 cada. Com seis, um pode cair sem que os outros passem do limite: é o dimensionamento N+1. E o próprio LB não pode

virar o novo ponto único de falha. Rode dois, em par ativo-passivo com um IP flutuante (o keepalived faz isso com o protocolo VRRP), ou use o LB gerenciado do provedor de nuvem, que já é distribuído.

Analogia

Pense na fila única de um supermercado grande. Você não escolhe o caixa: o painel chama "caixa 7" e você vai para lá. O painel é o load balancer, e os caixas são os servidores. Quando o sistema do caixa 3 trava, a luzinha em cima dele apaga e o painel para de chamá-lo, sem que ninguém na fila precise saber o motivo. A fila continua andando, só que com um caixa a menos.

NO MUNDO REAL

- **NGINX** e **HAProxy** estão entre os load balancers de software mais usados. Os dois trabalham tanto na camada 4 quanto na 7.
- Na AWS, o **Application Load Balancer** atua na camada 7, roteando por caminho e por host, e o **Network Load Balancer** atua na camada 4 (TCP, UDP e TLS). Os dois tiram da rotação os alvos que falham no health check.
- No **Kubernetes**, um Service distribui as conexões entre os pods, e a readiness probe faz o papel de health check: o pod que falha sai da lista de endpoints do Service.
- Do lado do hardware, appliances como o **F5 BIG-IP** ainda aparecem em datacenters próprios.

TRADE-OFFS

	Camada 4 (transporte)	Camada 7 (aplicação)
Enxerga	IP, porta e protocolo	URL, cabeçalhos, cookies e método HTTP
Decide por	conexão	requisição
Pode fazer	repassar qualquer protocolo sobre TCP ou UDP	rotear por caminho, terminar TLS, reescrever cabeçalhos, afinidade por cookie

	Camada 4 (transporte)	Camada 7 (aplicação)
Custo	pouca CPU, latência mínima	mais CPU por requisição (decifra o TLS e interpreta o HTTP)
Exemplos	AWS NLB, HAProxy em modo TCP	AWS ALB, NGINX, Envoy

NA ENTREVISTA

- Ponha o LB assim que o desenho tiver mais de um servidor de aplicação e diga o que ele checa: "*LB de camada 7 com health check em /health; como os servidores são stateless (cap. 8), qualquer um atende qualquer requisição.*"
- Antecipe o "e se o LB cair?": dois LBs com IP flutuante, ou o LB gerenciado da nuvem, em mais de uma zona.
- Fale de deploy sem queda: o LB tira o servidor da rotação, espera as requisições em andamento terminarem (connection draining) e só então o servidor é desligado.

CUIDADO COM

- `ATUALIZAÇÃO:` o artigo diz que a falha de um servidor não derruba o sistema. Só vale se os outros tiverem folga para absorver a parte dele; sem folga, a carga extra derruba o próximo, num efeito dominó.
- **Health check raso ou fundo demais.** Um `/health` que sempre devolve 200 não percebe que o app perdeu o banco. O oposto também machuca: se o check depende do banco e ele oscila, todos os servidores reprovam juntos. Por isso o Envoy tem um "limiar de pânico": abaixo de 50% de servidores saudáveis, ignora o health check e distribui entre todos.
- **Sticky session é muleta.** Prender cada usuário a um servidor desequilibra a carga e, quando ele cai, leva as sessões junto. Guarde o estado fora do servidor (cap. 8).

TESTE-SE

1 **O que acontece com um servidor que falha no health check?**

Depois de um número combinado de falhas seguidas, o load balancer para de mandar tráfego novo para ele, mas continua checando. Quando o servidor volta a responder bem algumas vezes seguidas, retorna à rotação.

2 **Você precisa mandar `/api` para um grupo de servidores e `/imagens` para outro. LB de camada 4 ou de camada 7?**

Camada 7. Só ele interpreta o HTTP e enxerga o caminho da URL; um LB de camada 4 vê apenas IP e porta.

3 **Três servidores rodam a 70% de CPU no pico, atrás de um load balancer. Um cai. O LB resolve?**

Não. Os dois restantes passam a receber metade do tráfego cada um, cerca de 105% da capacidade, e tendem a cair também. O LB só protege se houver folga para absorver a falha (N+1).

CONECTA COM

1 Escalabilidade vertical vs. horizontal

8 Arquitetura stateful vs. stateless

12 Algoritmos de balanceamento

13 Proxy reverso vs. proxy direto

37 Heartbeat

Algoritmos de balanceamento

Load Balancing Algorithms

EM UMA FRASE

Round robin reveza os servidores em ordem, least connections escolhe o que tem menos conexões abertas e IP hash prende cada cliente ao mesmo servidor; a escolha define quão justa fica a distribuição.

O ESSENCIAL · tradução do artigo original

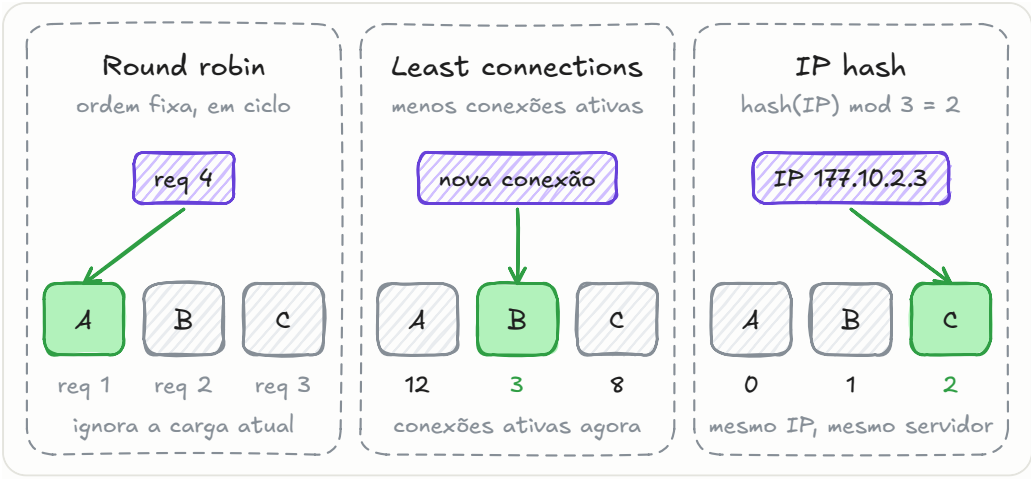
Os algoritmos de balanceamento de carga mais comuns são Round Robin, Least Connections e IP Hash.

- **Round Robin** percorre os servidores em ordem, em ciclo, e é simples de implementar.
- **Least Connections** (menos conexões) envia o tráfego para o servidor com menos conexões ativas, o que ajuda quando as requisições têm durações diferentes.
- **IP Hash** usa um hash do IP do cliente para que o mesmo usuário geralmente caia no mesmo servidor, o que ajuda a manter uma afinidade de sessão (session stickiness) simples.

Escolher o algoritmo certo afeta a justiça da distribuição, o uso de recursos e a experiência do usuário.

APROFUNDANDO

Os três respondem à mesma pergunta, a cada requisição ou conexão nova: para qual servidor mando esta? O que muda é a informação usada para decidir: nenhuma, a carga do momento ou a identidade do cliente.



Cada algoritmo decide com uma informação diferente: o round robin manda para A pela ordem, o least connections manda para B pela carga, o IP hash manda para C pelo IP.

Round robin: só a ordem

O balanceador guarda um ponteiro e o avança a cada requisição: A, B, C, A, B, C... Não pergunta nada a ninguém. Funciona bem quando os servidores são iguais e as requisições custam parecido. Se as máquinas são diferentes, entra o round robin ponderado (weighted round robin): com pesos 2, 1 e 1, o servidor A recebe metade das requisições.

O ponto fraco é que ele distribui requisições, não trabalho. Se A está processando três relatórios pesados enquanto B e C estão ociosos, A continua recebendo uma em cada três requisições.

Least connections: a carga do momento

O balanceador mantém um contador de conexões abertas por servidor e escolhe o menor. Com A em 12, B em 3 e C em 8, a próxima vai para B. Um servidor travado em requisições lentas acumula conexões e deixa de ser escolhido até os outros o alcançarem: o algoritmo se corrige sozinho. É a escolha natural para conexões longas e de duração desigual, como WebSocket (cap. 20), uploads e streaming. Também existe a versão ponderada, que divide as conexões pelo peso de cada servidor.

IP hash: quem está pedindo

A conta é `servidor = hash(IP do cliente) mod N`. Se o hash do IP dá 1.234.568 e há 3 servidores, 1.234.568 mod 3 = 2: o cliente vai sempre para o servidor de índice 2, o C. Sem guardar estado nenhum, o mesmo cliente volta ao mesmo servidor, o que preserva uma sessão em memória ou um cache local.

O preço aparece quando N muda. Com um quarto servidor, 1.234.568 mod 4 = 0, e o cliente pula para A, perdendo a sessão. Na média, passar de 3 para 4 servidores muda o destino de 75% dos clientes. O hashing consistente (cap. 25) reduz isso para cerca de 25%.

> *Analogia*
>
> Num salão de beleza movimentado, a recepcionista pode distribuir as clientes de três jeitos. Pela vez, em rodízio: cada cabeleireira pega a próxima que chegar, na ordem combinada (round robin). Olhando quem tem menos clientes na cadeira e na espera (least connections). Ou mandando cada cliente sempre para a mesma profissional, que já conhece o cabelo dela (IP hash). Esse último funciona muito bem, até o dia em que a cabeleireira falta.

NO MUNDO REAL

- **NGINX:** o padrão é round robin ponderado; `least_conn` e `ip_hash` são diretivas do bloco `upstream`. O `ip_hash` usa só os três primeiros octetos do IPv4 como chave, então uma sub-rede /24 inteira cai no mesmo servidor.
- **HAProxy:** `balance roundrobin`, `leastconn` ou `source` (hash do IP de origem). A documentação recomenda o `leastconn` para sessões muito longas, como LDAP e conexões de banco de dados, e não para protocolos de sessões curtas, como HTTP.
- **Envoy**: o algoritmo de least request sorteia dois servidores e manda para o que tem menos requisições ativas, a técnica conhecida como “power of two choices”. Equilibra quase tão bem quanto comparar todos, a um custo fixo por decisão.

TRADE-OFFS

	Round robin	Least connections	IP hash
Decide com base em	a ordem	as conexões abertas agora	o IP do cliente
Estado no balanceador	um ponteiro	um contador por servidor	nenhum, só a conta
Bom para	requisições curtas e parecidas	conexões longas e desiguais	afinidade simples, cache local por cliente
Ponto fraco	ignora a carga real	conexão aberta nem sempre mede trabalho	NAT concentra usuários; mudar N remaneja quase todos

NA ENTREVISTA

- Comece pelo simples: round robin, ou ponderado, para serviços stateless com requisições curtas. Troque com motivo: *"as conexões do chat duram horas, então uso least connections."*
- Se o assunto for sessão, prefira tirar o estado do servidor (cap. 8) a depender de IP hash. Se a afinidade for mesmo necessária, um cookie no LB de camada 7 é mais preciso que o IP.
- Em caches distribuídos, onde a mesma chave precisa cair sempre no mesmo nó, cite o hashing consistente (cap. 25) e explique por que `hash mod N` sofre quando N muda.

CUIDADO COM

- **Afinidade não é garantia.** O "geralmente" do IP hash tem motivo: o celular troca de IP ao sair do Wi-Fi para o 4G, servidores entram e saem, e o CGNAT das operadoras põe muitos clientes atrás de um mesmo IP.
- **Conexão longa engana o balanceamento de camada 4.** Com HTTP/2 (cap. 18) e gRPC (cap. 19), o cliente abre uma única conexão e multiplexa milhares de requisições nela. Quem escolhe o servidor por conexão manda tudo para o mesmo. A saída é balancear por requisição, na camada 7.

- ATUALIZAÇÃO: os três algoritmos do artigo são o básico. Proxies modernos oferecem least request com duas escolhas aleatórias, balanceamento pela latência observada (o Linkerd usa uma média móvel exponencial) e hash consistente (ring hash e Maglev, no Envoy).

TESTE-SE

1 O que o least connections leva em conta que o round robin ignora?

A carga do momento: quantas conexões cada servidor tem abertas. O round robin só segue a ordem, mesmo que um servidor esteja atolado.

2 Um serviço de chat mantém conexões WebSocket abertas por horas. Qual algoritmo você escolhe?

Least connections. As conexões duram muito e têm durações diferentes; contar as conexões abertas evita que alguns servidores acumulem muito mais que outros, e manda as novas para quem acabou de reiniciar e está vazio.

3 Uma empresa com 2.000 funcionários acessa seu sistema por um único IP de saída (NAT). Com IP hash, como essa carga se distribui entre seus 4 servidores?

Não se distribui: os 2.000 têm o mesmo IP, então o hash manda todos para o mesmo servidor. IP hash só equilibra bem quando há muitos IPs diferentes.

CONECTA COM

8 Arquitetura stateful vs. stateless
11 Balanceamento de carga
19 gRPC vs. REST
20 WebSocket e Server-Sent Events
25 Hashing consistente

Proxy reverso vs. proxy direto

Reverse Proxy vs Forward Proxy

EM UMA FRASE

O proxy direto fica do lado dos clientes e fala por eles com a internet; o proxy reverso fica do lado dos servidores e fala por eles, cuidando de TLS, cache e roteamento.

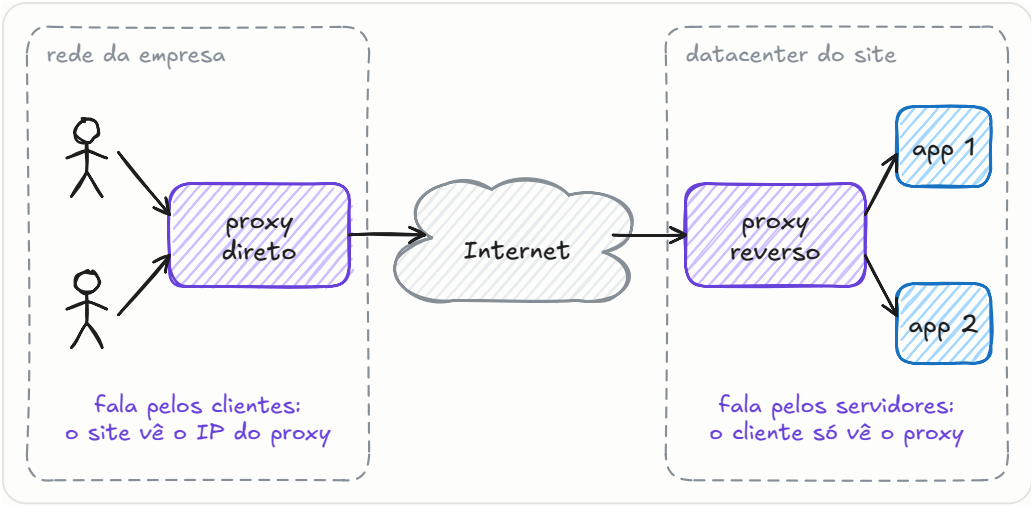
O ESSENCIAL · tradução do artigo original

- Um **proxy reverso** (reverse proxy) fica na frente dos servidores e os representa perante os clientes. Ele esconde a topologia interna e pode fazer terminação TLS, cache, compressão e roteamento.
- Um **proxy direto** (forward proxy) fica na frente dos clientes e os representa perante o mundo externo, muitas vezes por segurança, cache ou filtragem de conteúdo.

Pense no proxy reverso como a recepção de uma empresa, que esconde todas as salas internas, e no proxy direto como um portão pelo qual o seu notebook precisa passar para chegar à internet. Saber a diferença ajuda quando você fala de API gateways e de proxies corporativos.

APROFUNDANDO

Os dois são intermediários: recebem a conexão de um lado, abrem outra do outro lado e repassam as mensagens. O que muda é para quem trabalham, e isso define quem os configura e quem fica escondido.



O proxy direto fala pelos clientes da empresa; o proxy reverso fala pelos servidores do site. A internet fica no meio.

Proxy direto: escolhido pelo cliente

Quem configura é o lado do cliente: o navegador, o sistema operacional ou a rede da empresa, que bloqueia qualquer saída que não passe pelo proxy. O site de destino vê o IP do proxy, não o seu. Com HTTPS, o navegador pede ao proxy que abra um túnel:

```
CONNECT www.loja.com.br:443 HTTP/1.1
Host: www.loja.com.br:443
```

O proxy responde `200` e passa a copiar bytes cifrados de um lado para o outro. Ele sabe *aonde* você foi, mas não o *que* trafegou. Isso basta para bloquear domínios e registrar acessos.

Proxy reverso: escolhido pelo dono do site

Quem configura é o dono dos servidores. O DNS do site aponta para o proxy, e o cliente nem sabe que ele existe. Exemplo: uma requisição a `https://www.loja.com.br/api/pedidos` chega ao NGINX. Ele termina o TLS com o certificado da loja, vê que o caminho começa com `/api` e repassa a requisição para `10.0.1.12:8080` pela rede interna. No caminho, acrescenta o cabeçalho `X-Forwarded-For: 177.20.4.9`, para a aplicação saber quem é o cliente de verdade. Res-

postas grandes saem comprimidas, e arquivos estáticos podem sair direto do cache do proxy, sem incomodar a aplicação. O cliente nunca fica sabendo do endereço `10.0.1.12`.

Load balancer (cap. 11), API gateway (cap. 14) e CDN (cap. 15) são, todos, proxies reversos especializados.

Analogia

Um despachante trabalha para você: vai ao Detran no seu lugar, e o Detran lida com ele, não com você. Esse é o proxy direto. O garçom trabalha para o restaurante: você faz o pedido a ele e nunca vê qual cozinheiro preparou o prato nem como a cozinha se organiza. Se você pede só uma água, ele pega na geladeira do salão sem incomodar a cozinha. Esse é o proxy reverso, com cache. A pergunta que separa os dois: para quem o intermediário trabalha?

NO MUNDO REAL

- **NGINX** e **HAProxy** são proxies reversos clássicos: terminam TLS, servem arquivos estáticos e repassam o resto para os servidores de aplicação.
- A **Cloudflare** funciona como proxy reverso dos sites que a usam: o DNS do domínio aponta para ela, que filtra ataques, guarda cache e repassa o restante para a origem.
- O **Squid** é o proxy direto clássico de empresas e escolas, usado para cache e controle de acesso. O mesmo software também pode trabalhar como proxy reverso, prova de que a diferença está na posição, não no programa.

TRADE-OFFS

	Proxy direto (forward)	Proxy reverso (reverse)
Trabalha para	os clientes	os servidores
Quem configura	o cliente ou a rede dele	o dono do site
Quem fica escondido	o cliente: o site vê o IP do proxy	os servidores: o cliente só vê o proxy

	Proxy direto (forward)	Proxy reverso (reverse)
Usos típicos	filtrar e auditar acessos, controlar a saída de uma rede	terminar TLS, cache, compressão, balanceamento, roteamento
Exemplo	Squid na rede de uma empresa	NGINX na frente de uma aplicação

NA ENTREVISTA

- Num desenho de sistema, quase todo proxy é reverso. Diga isso com todas as letras: "*LB, API gateway e CDN são proxies reversos especializados.*"
- O proxy direto aparece em perguntas de segurança: servidores internos que só saem para a internet por um proxy de saída (egress) com uma lista de domínios liberados, o que dificulta vazar dados para o servidor de um atacante.
- Lembre do IP real do cliente. Atrás de um proxy reverso, a aplicação vê o IP do proxy; rate limit e logs precisam ler o `X-Forwarded-For` ou o cabeçalho padronizado `Forwarded`.

CUIDADO COM

- `X-Forwarded-For` **pode ser forjado.** O cliente pode mandar o cabeçalho já preenchido. Confie só na parte acrescentada pelos seus próprios proxies.
- **Esconder não é proteger.** Se o servidor de origem também tem IP público, um atacante que o descubra contorna o proxy reverso e todas as defesas dele. Feche a origem para aceitar só o tráfego que vem do proxy.
- `ATUALIZAÇÃO:` o cache em proxy direto perdeu quase todo o valor com a web em HTTPS. Num túnel `CONNECT`, o proxy vê o domínio, mas não o conteúdo, então não tem o que guardar. Filtrar por domínio continua possível; inspecionar o conteúdo exige interceptar o TLS com um certificado da empresa instalado nas máquinas.

TESTE-SE

1 **Qual a diferença essencial entre um proxy direto e um proxy reverso?**

Para quem eles trabalham. O direto age em nome dos clientes e esconde quem eles são; o reverso age em nome dos servidores e esconde como eles estão organizados.

2 **Você quer tirar dos servidores de aplicação o trabalho com certificados TLS e compressão. O que você põe na frente deles?**

Um proxy reverso, como o NGINX. Ele termina o TLS, comprime as respostas e repassa as requisições aos servidores pela rede interna.

3 **Atrás de um proxy reverso, o seu app registra que todos os acessos vêm do mesmo IP. Por quê, e como corrigir?**

Porque quem abre a conexão com o app é o proxy, então o IP de origem é o dele. O proxy deve repassar o IP do cliente no `X-Forwarded-For` (ou `Forwarded`), e o app deve ler esse cabeçalho, confiando só no valor escrito pelo próprio proxy.

CONECTA COM

11 Balanceamento de carga · 14 API Gateway · 15 CDN · 41 Cache · 50 Handshake TLS

API Gateway

API Gateway

EM UMA FRASE

Um API gateway é o proxy reverso que serve de porta única para as APIs de um sistema de microsserviços, concentrando roteamento, autenticação, rate limit e logs; precisa ficar enxuto para não virar gargalo.

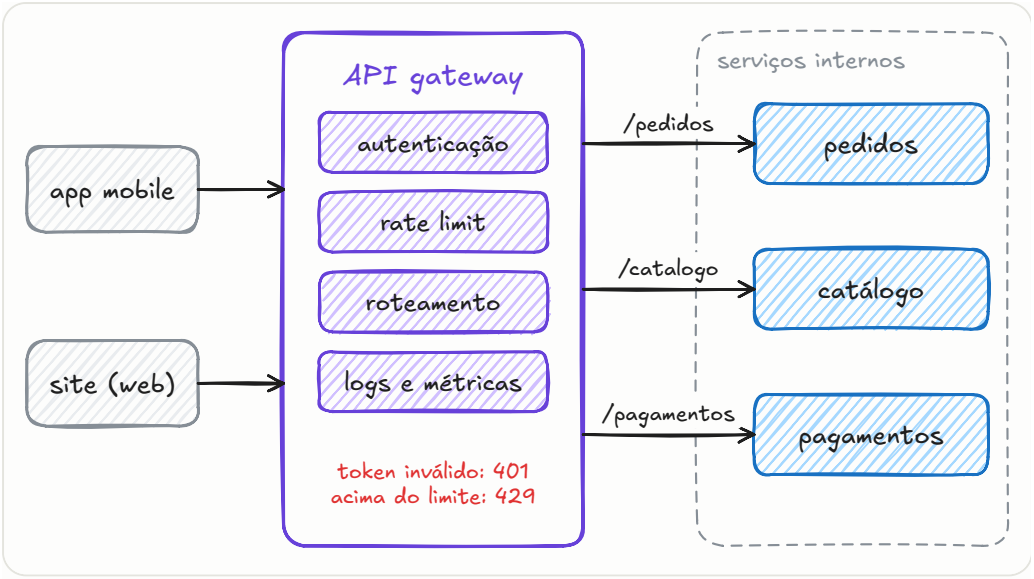
O ESSENCIAL · tradução do artigo original

Um API gateway é um proxy reverso especial que funciona como ponto único de entrada para todas as chamadas de API num sistema de microsserviços. Ele cuida do roteamento para o serviço certo, do rate limiting (limite de requisições), da autenticação, dos logs e, às vezes, da adaptação do formato das respostas. Isso reduz a complexidade do lado do cliente, já que os clientes só conversam com um único endpoint.

Se você puser lógica demais no gateway, ele pode virar um gargalo ou um pequeno monólito por si só. Bons designs o mantêm focado e enxuto.

APROFUNDANDO

Sem gateway, o app precisaria saber o endereço de cada microsserviço, e cada serviço repetiria a mesma checagem de token, o mesmo limite de requisições e o mesmo formato de log. O gateway puxa essas preocupações transversais, as que valem para todas as rotas, para um lugar só.



O app mobile e o site falam só com o gateway. Cada chamada passa por autenticação, rate limit, roteamento e logs antes de chegar ao serviço certo.

Acompanhe uma chamada do app, `GET /pedidos/123`, com um token no cabeçalho `Authorization`:

1. **TLS:** o gateway termina a conexão segura.
2. **Autenticação:** valida a assinatura e o prazo de validade do token (um JWT, por exemplo) com a chave pública do provedor de identidade, sem consultar outro serviço a cada chamada. Token inválido volta com `401` ali mesmo.
3. **Rate limit:** confere a cota do cliente (cap. 32). Passou de 100 chamadas por minuto? Volta `429 Too Many Requests`.
4. **Roteamento:** consulta a tabela de rotas e encaminha ao serviço de pedidos, levando a identidade do usuário num cabeçalho interno.
5. **Observabilidade:** registra latência e status e propaga um ID de rastreamento para o tracing distribuído (cap. 47).

A tabela de rotas costuma ser declarativa:

```
/pedidos/*     → servico-pedidos      exige token    100 req/min
/catalogo/*    → servico-catalogo     público       1000 req/min
/pagamentos    → servico-pagamentos   exige token     10 req/min
```

Se amanhã o catálogo for dividido em dois serviços, muda a tabela, não o app.

Quando clientes diferentes precisam de respostas muito diferentes, entra o padrão BFF (Backend for Frontend): um backend por tipo de cliente, que junta várias chamadas internas numa resposta sob medida para cada tela. É um bom lugar para a adaptação das respostas, que assim não incha o gateway compartilhado.

Analogia

Pense na central telefônica do seu banco. Você liga para um número só. A URA pede o CPF e a senha (autenticação), avisa que a ligação poderá ser gravada (logs), diz que todas as linhas estão ocupadas quando há gente demais (rate limit) e transfere para cartões, conta ou seguros (roteamento). Quem resolve o seu problema é o setor certo, não a URA. Quando ela tenta resolver tudo sozinha, com menus e mais menus, vira o gargalo que todo mundo detesta.

NO MUNDO REAL

- O **Amazon API Gateway** é o serviço gerenciado da AWS: autentica com IAM, Cognito ou autorizadores em Lambda, aplica throttling e encaminha para funções Lambda ou serviços HTTP.
- **Kong**, construído sobre o NGINX, e **Apigee**, do Google Cloud, são gateways populares, com plugins ou políticas de autenticação, rate limiting e logs.
- O **Zuul** é o gateway de borda que a Netflix liberou como código aberto em 2013, feito para receber o tráfego de todos os dispositivos e encaminhá-lo aos serviços internos.

TRADE-OFFS

	Clientes chamam os serviços direto	Com API gateway
O cliente conhece	o endereço de cada serviço	um único endpoint
Autenticação e rate limit	repetidos em cada serviço	num lugar só
Reorganizar os serviços internos	quebra os clientes	muda só a tabela de rotas

	Clientes chamam os serviços direto	Com API gateway
Latência	uma ida direta	um salto a mais, em geral de poucos milissegundos
Risco principal	regras inconsistentes entre serviços	gargalo e ponto único de falha, se não for redundante e enxuto

NA ENTREVISTA

- Desenhe o gateway na borda assim que houver vários serviços e clientes externos, e liste o que ele faz: TLS, autenticação, rate limit, roteamento e logs.
- Diga também o que ele *não* faz: regra de negócio e orquestração de fluxos ficam nos serviços. *"O gateway cuida do que vale para todas as rotas; o que é de um domínio fica no serviço."*
- Diferencie do load balancer: o LB reparte o tráfego entre cópias do mesmo serviço; o gateway escolhe *qual* serviço atende e aplica políticas. Na prática, há LBs na frente das instâncias do gateway e atrás dele.

CUIDADO COM

- **Autenticar não é autorizar.** O gateway confirma *quem* é o usuário. Se o pedido 123 pertence a ele, só o serviço de pedidos sabe. Esquecer essa checagem no serviço gera a falha que lidera o OWASP API Security Top 10, conhecida como BOLA: trocar o ID na URL e ver o pedido de outra pessoa.
- **Ponto único de falha.** Se o gateway cai, todas as APIs caem juntas. Rode várias instâncias, atrás de um LB, em mais de uma zona.
- **Gateway não é service mesh.** O gateway cuida do tráfego que entra no sistema (norte-sul). A comunicação entre serviços (leste-oeste) passa direto ou por um service mesh, como Istio ou Linkerd.

TESTE-SE

1 **Que problemas um API gateway resolve para os clientes?**

Os clientes falam com um único endpoint, sem conhecer os serviços internos, e autenticação, rate limit e logs ficam num lugar só, em vez de repetidos em cada serviço.

2 **Uma tela do app mobile junta perfil, pedidos e recomendações, e a rede do celular é lenta. Onde você monta essa resposta?**

Num BFF (Backend for Frontend) do mobile: ele faz as três chamadas internas, que são rápidas, e devolve uma resposta só, enxuta para a tela. O app faz uma ida e volta pela rede lenta em vez de três, e o gateway geral continua enxuto.

3 **O gateway já validou o token. O serviço de pedidos ainda precisa checar alguma coisa antes de devolver o pedido 123?**

Sim: se o pedido 123 pertence a esse usuário. O gateway garante quem é a pessoa (autenticação); se ela pode ver aquele recurso (autorização) depende de dados que só o serviço tem.

CONECTA COM

9 Microsserviços vs. monólito
13 Proxy reverso vs. proxy direto
32 Rate limiting
47 Tracing distribuído
49 OAuth 2.0 e OIDC

CDN

CDN (Content Delivery Network)

EM UMA FRASE

Uma CDN é uma rede de servidores de borda espalhados pelo mundo que guardam cópias do seu conteúdo perto dos usuários, cortando a latência e a carga nos servidores de origem.

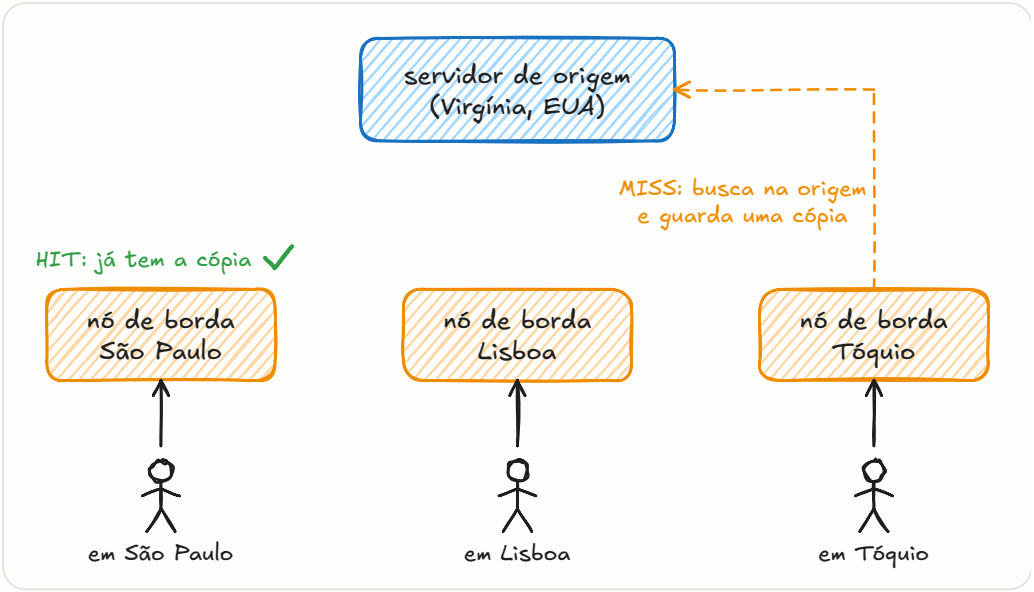
O ESSENCIAL · tradução do artigo original

Uma CDN (rede de distribuição de conteúdo) é uma rede de servidores distribuídos geograficamente que guardam em cache conteúdo estático, como imagens, vídeos e scripts, mais perto dos usuários. Quando um usuário pede um conteúdo, ele é direcionado ao nó da CDN mais próximo, o que reduz muito a latência. Isso também tira tráfego dos seus servidores de origem, melhorando a escalabilidade e a resiliência.

CDNs são essenciais para aplicações globais e para o desempenho do front-end. Pense nelas como “cópias locais” dos arquivos pesados do seu site espalhadas pelo mundo.

APROFUNDANDO

A distância custa caro: a luz na fibra percorre uns 200 km por milissegundo, e nenhuma otimização muda isso. Uma ida e volta entre São Paulo e a Virgínia, nos EUA, leva uns 120 ms. Abrir uma conexão HTTPS nova (TCP com TLS 1.3) custa três idas e voltas até a primeira resposta: uma do TCP, uma do TLS e uma da requisição. São ~360 ms só de espera. Com um nó da CDN na mesma cidade, a ~10 ms de distância, a conta cai para ~30 ms. Até num miss há ganho: a borda mantém conexões já abertas com a origem, então só a requisição faz a viagem longa, sem os handshakes.



Cada usuário fala com o nó de borda da sua região. Em São Paulo, o arquivo já está no cache (hit); em Tóquio, o nó busca na origem e guarda uma cópia (miss).

Como o usuário chega ao nó certo

Há duas técnicas principais. No **roteamento por DNS**, o servidor DNS da CDN devolve o IP de um nó próximo de quem perguntou (cap. 16). No **anycast**, muitos nós anunciam o mesmo IP, e o próprio roteamento da internet entrega cada pacote ao nó mais próximo em termos de rede.

Hit, miss e TTL

O nó de borda procura a URL no seu cache. Se acha (**hit**), responde na hora. Se não acha (**miss**), busca na origem, entrega ao usuário e guarda uma cópia pelo tempo que a origem mandou no cabeçalho `Cache-Control`:

```
HTTP/1.1 200 OK
Content-Type: text/javascript
Cache-Control: public, max-age=31536000, immutable
```

Esse arquivo pode ficar um ano em cache (31.536.000 segundos). Isso só é seguro porque o nome muda a cada versão (`app.3f9a2c.js`): publicar uma versão nova é trocar o nome no HTML, sem apagar nada da CDN. Já o HTML costuma ter TTL curto ou nenhum.

A métrica que importa é a **taxa de acerto** (hit ratio). Com 95% de hits, a origem atende 1 em cada 20 requisições: 20 vezes menos carga. E, para que centenas de nós em miss não corram todos à origem ao mesmo tempo, muitas CDNs põem uma camada intermediária (origin shield) que concentra os misses.

Analogia

Uma grande loja online não despacha tudo da fábrica. Ela mantém os produtos mais vendidos em centros de distribuição regionais, perto dos clientes. Se o seu pedido está no CD da sua região, chega amanhã (hit). Se não está, o CD pede à central, a entrega demora mais, e ele aproveita para estocar algumas unidades para os próximos pedidos (miss). Quando um produto sai de linha, alguém precisa recolhê-lo de todos os CDs (invalidação).

NO MUNDO REAL

- A **Netflix** tem a própria CDN, a **Open Connect**: instala servidores dentro de provedores de internet e em pontos de troca de tráfego e os abastece com o catálogo fora do horário de pico. O vídeo sai de perto do assinante.
- A **Cloudflare** usa anycast: o mesmo IP é anunciado por todos os seus data centers, e cada usuário cai, em geral, no mais próximo.
- **Amazon CloudFront**, **Akamai** e **Fastly** são outras CDNs comerciais. Em junho de 2021, um bug na Fastly, disparado pela mudança de configuração de um cliente, tirou do ar por quase uma hora sites como o do governo britânico e os de grandes jornais: a CDN vira uma dependência crítica.

TRADE-OFFS

Você ganha

- Latência menor: o usuário fala com um servidor a poucos milissegundos dele.
- Menos carga e menos tráfego na origem: com 95% de hits, ela atende 1 requisição em 20.
- Proteção: a CDN absorve picos e ataques DDoS antes que cheguem à origem.

Você paga

- Conteúdo que pode ficar velho até o TTL vencer ou até você invalidar.

- Custo por GB entregue e mais uma dependência externa.
- Configuração delicada: um erro na chave de cache ou no `Cache-Control` pode servir a página de um usuário para outro.

NA ENTREVISTA

- Em sistemas com usuários espalhados e conteúdo pesado, ponha a CDN logo na frente: "*imagens, vídeos e JS saem da CDN; a origem só atende a API.*"
- Fale de invalidação: nomes versionados com TTL longo para estáticos, TTL curto para o que muda, purge só em emergência.
- Mencione push vs. pull. Na CDN pull, a mais comum, a borda busca na origem no primeiro miss. Na push, você publica os arquivos na CDN antes, o que serve para conteúdo grande e previsível, como um catálogo de vídeos.

CUIDADO COM

- `ATUALIZAÇÃO:` "CDN é só para estático" ficou para trás. Hoje as CDNs também aceleram conteúdo dinâmico, mantendo conexões abertas e aquecidas com a origem, guardam respostas de API por alguns segundos, filtram ataques e rodam código na borda, como os Cloudflare Workers e o Lambda@Edge.
- **Não faça cache do que é privado.** Uma resposta com dados de um usuário marcada como `public` pode ser servida a outra pessoa. Use `Cache-Control: private` ou `no-store` nas respostas personalizadas.
- **"Nó mais próximo" é aproximado.** No roteamento por DNS, a CDN enxerga a localização do resolvedor, não a do usuário. Quem usa um resolvedor público distante pode cair num nó longe; a extensão EDNS Client Subnet ameniza isso.

TESTE-SE

1 O que acontece num cache miss da CDN?

O nó de borda não tem o arquivo: busca na origem, entrega ao usuário e guarda uma cópia pelo tempo do `Cache-Control`, para que os próximos usuários da região tenham um hit.

2 Você publicou uma nova versão do `app.js`, mas os usuários continuam recebendo a antiga pela CDN. Como evitar isso nas próximas versões?

Coloque a versão no nome do arquivo (`app.3f9a2c.js`) a cada deploy e sirva com TTL longo. O HTML, com TTL curto, aponta para o nome novo, e a versão antiga simplesmente deixa de ser pedida. O purge fica para emergências.

3 Pôr uma CDN na frente de uma API que devolve o extrato bancário de cada usuário alivia a origem?

Quase nada. Cada resposta é única e privada e não deve ficar no cache da borda, então toda requisição continua chegando à origem. A CDN ainda pode acelerar a abertura das conexões, que passa a acontecer perto do usuário, mas não tira carga da origem.

CONECTA COM

5 Throughput vs. latência · 13 Proxy reverso vs. proxy direto · 16 DNS · 41 Cache · 42 Estratégias de cache

DNS

DNS (Domain Name System)

EM UMA FRASE

O DNS traduz nomes como www.loja.com.br em endereços IP, consultando uma hierarquia de servidores (raiz, TLD e autoritativo) e guardando cada resposta em cache pelo tempo definido no TTL.

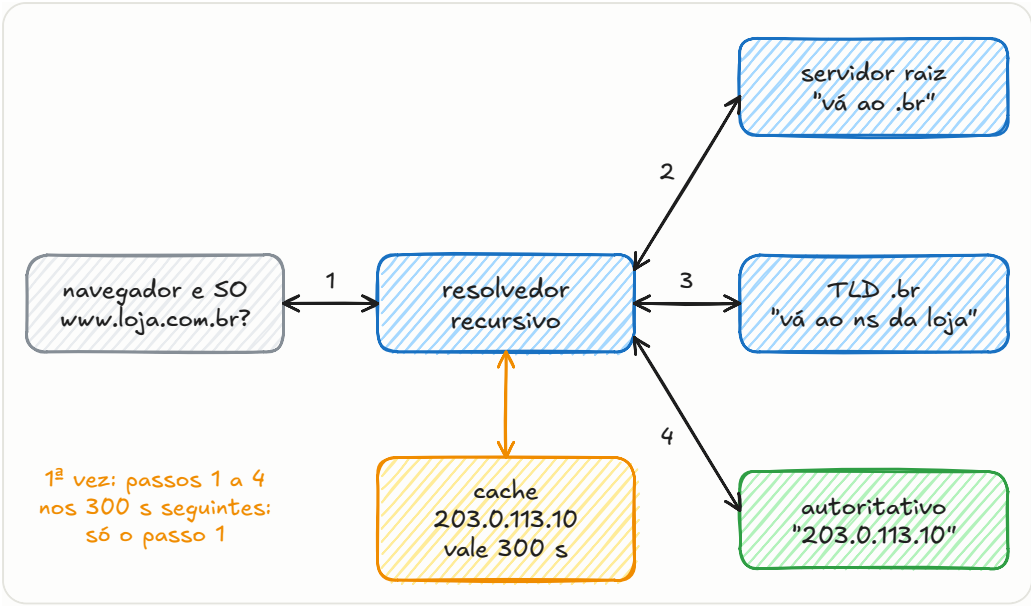
O ESSENCIAL · tradução do artigo original

O DNS (sistema de nomes de domínio) associa nomes de domínio legíveis por humanos a endereços IP. Quando você digita o nome de um site, o seu dispositivo consulta o DNS para descobrir o endereço numérico do servidor. O DNS tem várias camadas de cache, então as respostas são rápidas depois da primeira consulta.

Ele também pode ser usado para fazer um balanceamento de carga simples, devolvendo IPs diferentes para o mesmo nome. Entender o DNS ajuda você a raciocinar sobre por que mudanças de DNS levam tempo para se propagar e por que algumas quedas são causadas por DNS mal configurado.

APROFUNDANDO

O DNS é um banco de dados distribuído, hierárquico e eventualmente consistente (cap. 7). Ninguém tem a lista inteira: cada nível só sabe a quem delegar o próximo. Veja a primeira visita a `www.loja.com.br`, com todos os caches vazios:



A primeira consulta percorre a raiz, o TLD .br e o servidor autoritativo. Durante o TTL, as próximas param no cache do resolvedor.

1. O navegador e o sistema operacional olham seus caches locais. Nada. O SO pergunta ao **resolvedor recursivo**, em geral o do provedor de internet ou um público, como o 8.8.8.8.
2. O resolvedor pergunta a um **servidor raiz**. A raiz não sabe o IP, mas sabe quem cuida do `.br` e devolve essa indicação.
3. O resolvedor pergunta aos servidores do **TLD** `.br`, mantidos pelo Registro.br. Eles indicam os servidores de nomes (registros NS) que a loja cadastrou.
4. O resolvedor pergunta ao **servidor autoritativo** da loja, a fonte oficial, que responde:

```
www.loja.com.br.   300   IN   A   203.0.113.10
```

O `300` é o **TTL**: por 300 segundos, quem guardou essa resposta pode reutilizá-la sem perguntar de novo. O resolvedor guarda, devolve o IP ao SO, e o navegador enfim abre a conexão.

Cada pergunta costuma ir num único pacote UDP para a porta 53 (cap. 17), mas a resolução completa soma várias idas e voltas: de dezenas a centenas de milissegundos. Nos 300 segundos seguintes, qualquer cliente desse resolvedor recebe o IP direto do

cache, em poucos milissegundos. Os passos 2 e 3 são ainda mais raros, porque as indicações da raiz e do TLD ficam em cache por horas ou dias.

Balanceamento e "propagação"

Um nome pode ter vários registros A. O servidor alterna a ordem ou escolhe por região, e os clientes costumam tentar o primeiro da lista. É um balanceamento grosseiro: o DNS simples não sabe se um servidor caiu, e a resposta fica nos caches até o TTL vencer. Pelo mesmo motivo, ninguém "propaga" uma mudança. Com TTL de um dia, alguns resolvedores ainda vão servir o IP antigo por até um dia, até seus caches expirarem.

> *Analogia*
>
> Imagine procurar um paciente num hospital enorme. A recepção da entrada não sabe o leito, mas sabe que a cardiologia fica no bloco B. A recepção do bloco B manda você ao posto de enfermagem do quarto andar, e o posto diz: leito 412. Você anota no celular e, na visita de amanhã, vai direto. Só que, se o paciente trocar de leito, a anotação fica errada até você perguntar de novo. Por isso o DNS põe prazo de validade em cada resposta.

NO MUNDO REAL

- **Raiz e resolvedores públicos:** existem 13 identidades de servidores raiz, de a.root-servers.net a m.root-servers.net, cada uma replicada por anycast em vários pontos do mundo, num total de mais de mil instâncias. Resolvedores públicos como o 8.8.8.8 (Google) e o 1.1.1.1 (Cloudflare) substituem o do provedor para muita gente.
- **Amazon Route 53:** DNS gerenciado que escolhe a resposta por latência, geolocalização ou peso e faz failover com health checks.
- **Queda do Facebook, em outubro de 2021:** um comando de manutenção derrubou a rede que liga os datacenters da empresa. Sem enxergá-los, os servidores DNS do Facebook retiraram os próprios anúncios de rota (BGP), e o resto da internet passou cerca de seis horas sem conseguir resolver facebook.com.

TRADE-OFFS

	TTL curto (ex.: 60 s)	TTL longo (ex.: 1 dia)
Uma troca de IP chega a todos	em cerca de 1 minuto	em até 1 dia
Consultas ao seu servidor autoritativo	muitas	poucas
Para o usuário	mais resoluções completas, mais lentas	quase sempre resposta de cache
Se o seu DNS autoritativo cair	os caches expiram logo e o nome para de resolver	os caches seguram as respostas por mais tempo
Bom para	failover, migrações, balanceamento por DNS	registros estáveis, como NS e MX

NA ENTREVISTA

- Em "o que acontece quando você digita uma URL?", o DNS é o primeiro passo. Saiba a ordem: cache local, resolvedor recursivo, raiz, TLD, autoritativo.
- Em desenhos globais, use DNS com roteamento por latência ou geografia para mandar cada usuário à região mais próxima. Mas diga que o failover por DNS é lento, porque depende do TTL e de clientes que nem sempre o respeitam. Para failover rápido, use anycast ou um load balancer.
- Tenha o roteiro de migração pronto: "*baixo o TTL para 60 s com antecedência, troco o IP, mantenho o servidor antigo até o tráfego zerar e depois subo o TTL de novo.*"

CUIDADO COM

- **Baixar o TTL na hora da troca não adianta.** Os caches ainda guardam a resposta antiga com o TTL antigo. Baixe-o pelo menos um TTL antigo antes da troca: com TTL de um dia, um dia antes.
- **Nem todo cliente respeita o TTL.** Algumas aplicações e bibliotecas guardam o IP por mais tempo, às vezes até reiniciar. Mantenha o servidor antigo respondendo por um tempo depois da troca.

- ATUALIZAÇÃO: o “balanceamento simples” do artigo evoluiu. Serviços de DNS gerenciado, como o Route 53 e o da Cloudflare, combinam health checks, geolocalização e latência medida, e a resposta muda conforme a saúde dos servidores. Mesmo assim, ela continua sujeita aos caches e ao TTL.

TESTE-SE

1 Qual a diferença entre o resolvedor recursivo e o servidor autoritativo?

O recursivo faz o trabalho de perguntar à raiz, ao TLD e ao autoritativo em nome do cliente, e guarda as respostas em cache. O autoritativo é a fonte oficial de um domínio: responde com os registros que o dono configurou.

2 Você vai migrar o site para um servidor com outro IP, e o registro tem TTL de 86.400 s (um dia). Como planeja a troca?

Pelo menos um dia antes, baixe o TTL para 60 ou 300 s e espere o TTL antigo vencer. Aí troque o IP, mantenha o servidor antigo no ar até o tráfego zerar e, por fim, suba o TTL de novo.

3 Seu DNS devolve três IPs para o mesmo nome. Um dos servidores cai. O tráfego para de ir para ele?

Não necessariamente. O DNS simples não sabe que o servidor caiu e continua devolvendo os três IPs, e os clientes ainda guardam respostas antigas até o TTL vencer. Para tirá-lo da rotação rápido, você precisa de health check, no DNS gerenciado ou num load balancer.

CONECTA COM

7 Consistência forte vs. eventual
11 Balanceamento de carga
15 CDN
17 TCP vs. UDP
41 Cache

TCP vs. UDP

TCP vs UDP

EM UMA FRASE

O TCP abre uma conexão e garante entrega completa e em ordem, ao custo de handshake e retransmissões; o UDP dispara datagramas sem garantia nenhuma, ideal quando chegar rápido vale mais que chegar tudo.

O ESSENCIAL · tradução do artigo original

- O **TCP** é um protocolo confiável e orientado a conexão. Ele garante a entrega em ordem e com verificação de erros usando confirmações de recebimento (acknowledgments) e retransmissões.
- O **UDP** não usa conexão e não garante nem a entrega nem a ordem, o que o torna muito mais rápido e leve.

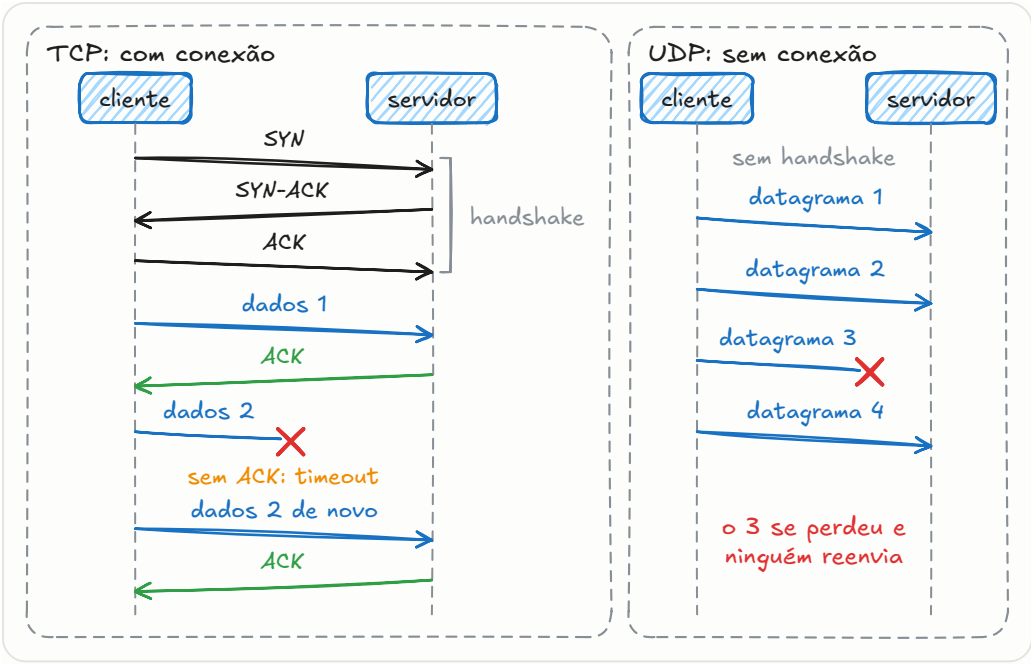
O TCP é indicado para APIs, páginas web e transferências de arquivos, em que a exatidão importa. O UDP funciona bem em aplicações de tempo real, como chamadas de vídeo ou jogos, em que perder um pacote de vez em quando é aceitável. Pense no TCP como uma carta registrada e no UDP como cartões-postais rápidos.

APROFUNDANDO

Os dois protocolos ficam na camada de transporte, logo acima do IP. O IP só sabe levar um pacote de uma máquina a outra, sem promessa nenhuma: pacotes se perdem, chegam duplicados ou fora de ordem. TCP e UDP são duas respostas opostas a essa bagunça.

TCP: conexão, confirmação e ordem

Antes de qualquer dado, cliente e servidor fazem o three-way handshake (aperto de mão em três etapas): SYN (“quero conversar; meu número inicial é x”), SYN-ACK (“combinado; o meu é y”) e ACK. Isso custa uma ida e volta (1 RTT) antes do primeiro byte útil. Com 100 ms de RTT, são 100 ms só para começar, e o TLS ainda soma mais uma ou duas idas e voltas (cap. 50).



O TCP combina a conexão antes de mandar dados, confirma cada segmento e reenvia o que se perde; o UDP dispara datagramas sem conexão, e o que se perde fica perdido.

Depois, cada byte enviado ganha um número de sequência. O receptor confirma o que chegou (ACK) e reordena o que veio fora de ordem. Se um segmento some, o emissor percebe, porque o ACK não chega antes de o timer estourar (ou chegam ACKs repetidos), e retransmite. Enquanto isso, o que chegou depois do segmento perdido espera no buffer, porque a aplicação só recebe bytes em ordem. Esse efeito se chama head-of-line blocking (bloqueio pelo primeiro da fila) e é um dos motivos da existência do HTTP/3 (cap. 18).

O TCP ainda faz controle de fluxo (não afoga um receptor lento) e controle de congestionamento: uma conexão nova começa enviando cerca de 10 segmentos (~14 KB) e dobra esse volume a cada ida e volta, até encontrar o limite da rede. Por isso reaproveitar conexões abertas vale tanto.

UDP: cada datagrama por si

O UDP quase não faz nada, e esse é o ponto. O cabeçalho tem 8 bytes (portas de origem e destino, tamanho e checksum), contra no mínimo 20 do TCP. Não há handshake, ACK, retransmissão nem ordem: se o datagrama 3 se perde, o 4 é entre-

gue mesmo assim. Quem precisa de alguma garantia a implementa na aplicação, só a que fizer sentido.

Em tempo real, isso é vantagem. Numa chamada de voz, cada pacote costuma levar 20 ms de áudio; se ele chega 300 ms atrasado, já não serve para nada. Melhor tocar um chiado e seguir em frente do que congelar a conversa esperando uma retransmissão.

Analogia

O TCP é ditar a linha digitável de um boleto por telefone. Primeiro vem o "alô?", "alô, pode falar"; depois, a cada bloco, a outra pessoa repete para conferir, e o número que ela não ouviu você dita de novo. Demora, mas o pagamento sai certo. O UDP é a narração do jogo no rádio: se o sinal chia na hora do lance, ninguém volta a fita. O jogo já seguiu, e o que interessa é o lance de agora.

NO MUNDO REAL

- **DNS:** a maioria das consultas vai por UDP, na porta 53: uma pergunta, uma resposta, sem handshake. Se a resposta não cabe no datagrama, o servidor avisa que a truncou e o cliente refaz a consulta por TCP.
- **Videochamadas no navegador (WebRTC):** áudio e vídeo trafegam sobre UDP. Só quando a rede bloqueia UDP a mídia cai para TCP, por um servidor TURN, com mais atraso.
- **Jogos online em tempo real** costumam enviar o estado da partida por UDP várias vezes por segundo: se uma atualização de posição se perde, a próxima a substitui.
- **Sobre TCP** rodam o HTTP/1.1, o HTTP/2, o SSH, o e-mail (SMTP) e os protocolos de bancos como PostgreSQL e MySQL.

TRADE-OFFS

	TCP	UDP
Conexão	handshake antes de enviar (1 RTT)	nenhuma: envia na hora
Garantias	entrega, ordem e sem duplicatas	nenhuma (só o checksum)

	TCP	UDP
Pacote perdido	retransmite; o que vem depois espera	segue em frente; a perda fica
Cabeçalho	20 a 60 bytes	8 bytes
Congestionamento	controlado pelo protocolo	problema da aplicação
Bom para	APIs, páginas, arquivos, bancos	voz, vídeo, jogos, DNS

NA ENTREVISTA

- Traga o UDP quando o problema for tempo real (voz, vídeo, jogos, telemetria) e diga o porquê: "*Um pacote de áudio que chega atrasado já não serve; retransmitir só aumenta o atraso.*"
- Para o resto (APIs, pagamentos, uploads), o padrão é TCP, e ninguém vai pedir justificativa longa.
- Mostre que confiabilidade não é tudo ou nada: o QUIC (cap. 18) e o WebRTC constroem sobre o UDP só as garantias de que precisam.
- Ao falar de latência, lembre o custo de abrir conexão: TCP mais TLS custam 2 ou 3 idas e voltas antes de a primeira requisição poder sair. Pool de conexões e keep-alive evitam pagar isso a cada requisição.

CUIDADO COM

- ATUALIZAÇÃO: "páginas web vão por TCP" ficou datado. O HTTP/3 roda sobre QUIC, que usa UDP e refaz por cima dele a entrega confiável e o controle de congestionamento. E o UDP não é mais rápido no fio: ele só não espera handshake nem retransmissão.
- **TCP é um fluxo de bytes, não de mensagens.** Dois envios podem chegar numa única leitura, ou um envio chegar picado. A aplicação precisa delimitar as mensagens.
- **ACK não é "pedido processado".** Ele só diz que o sistema operacional do outro lado recebeu os bytes. Daí retries (cap. 35) e idempotência (cap. 36).

- **O UDP também tem checksum.** Datagrama corrompido é descartado; o que o UDP não faz é pedir de novo.

TESTE-SE

1 **O que o three-way handshake do TCP estabelece, e quanto ele custa?**

Que os dois lados se alcançam e quais são os números de sequência iniciais de cada um. Custa uma ida e volta (1 RTT) antes de o cliente poder enviar dados, sem contar o TLS.

2 **Num app de videochamada com chat ao lado, o que vai por UDP e o que vai por TCP?**

Áudio e vídeo por UDP: um quadro atrasado é inútil e retransmitir só aumenta o atraso. O chat vai por TCP (ou outro canal confiável), porque nenhuma mensagem pode sumir.

3 **Se o TCP garante a entrega, por que a aplicação ainda precisa de retry e idempotência?**

Porque o ACK confirma só a chegada dos bytes ao sistema operacional do outro lado, não o processamento. Se a conexão cai no meio, o cliente não sabe se o pedido foi executado e precisa poder repeti-lo sem efeito duplicado.

CONECTA COM

5 Throughput vs. latência · 18 HTTP/2 e HTTP/3 (QUIC) · 35 Retries e backoff exponencial · 36 Idempotência · 50 Handshake TLS

HTTP/2 e HTTP/3 (QUIC)

HTTP/2 and HTTP/3 (QUIC)

EM UMA FRASE

O HTTP/2 multiplexa várias requisições numa única conexão TCP, mas uma perda de pacote ainda trava todas; o HTTP/3 roda sobre QUIC (UDP), com streams independentes e conexões mais rápidas de abrir.

O ESSENCIAL · tradução do artigo original

- O **HTTP/2** introduziu a multiplexação, que permite que várias requisições compartilhem uma única conexão TCP, reduzindo o overhead. Ele também trouxe recursos como compressão de cabeçalhos e server push.
- O **HTTP/3** roda sobre o QUIC, que é construído sobre UDP, reduz o tempo para estabelecer a conexão e melhora o desempenho em redes instáveis. Essas versões têm como objetivo principal reduzir a latência e aproveitar melhor as condições das redes modernas.

Para você, como engenheiro, a ideia principal é: abrir menos conexões e aproveitar melhor uma única conexão.

APROFUNDANDO

O problema do HTTP/1.1

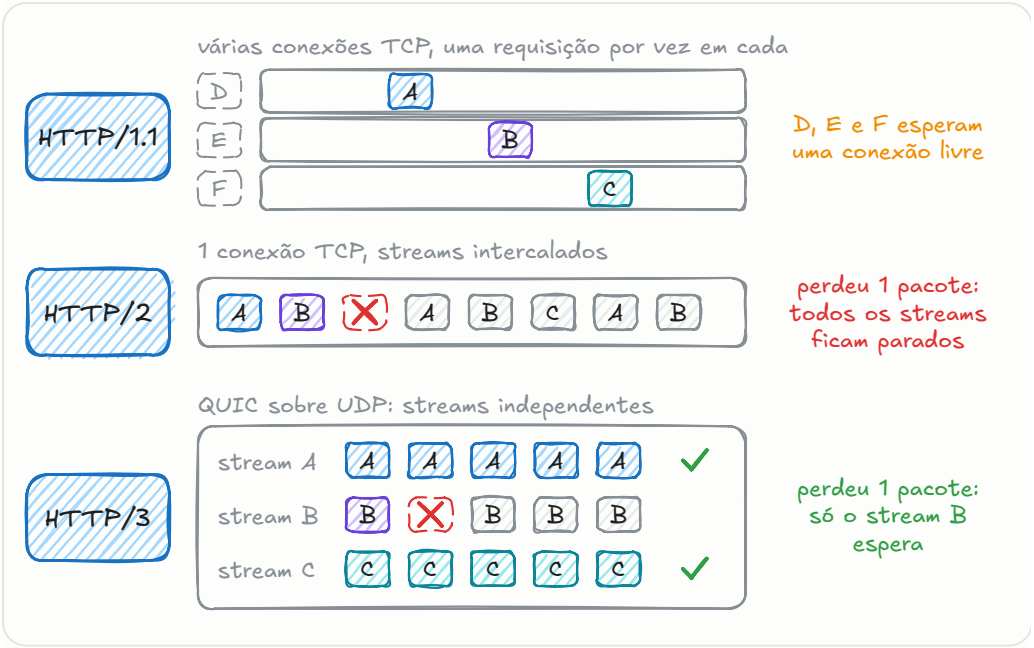
No HTTP/1.1, uma conexão TCP carrega uma requisição por vez: a próxima só sai quando a resposta anterior termina. (O pipelining existe na especificação, mas ficou desligado nos navegadores.) Para não enfileirar tudo, o navegador abre até 6 conexões por domínio, cada uma com seu handshake TCP, seu TLS e seu slow start (cap. 17). Uma página com 80 arquivos espera nessas 6 filas.

HTTP/2: uma conexão, vários streams

O HTTP/2 troca o texto por frames binários. Cada requisição vira um stream numerado, e frames de streams diferentes se intercalam na mesma conexão: o CSS, o JavaScript e as imagens vão e voltam ao mesmo tempo. Os cabeçalhos são comprimi-

dos com o HPACK, que guarda numa tabela o que já foi enviado; o cookie de 1 KB que se repetia em toda requisição passa a custar poucos bytes.

O defeito ficou embaixo. O TCP entrega bytes estritamente em ordem e não sabe que existem streams. Se um único pacote se perde, todos os streams esperam a retransmissão, mesmo os que não tinham nada a ver com ele: é o head-of-line blocking do TCP. E ele aparece justamente onde perda é comum, em Wi-Fi lotado e rede móvel.



No HTTP/1.1, cada requisição ocupa uma conexão inteira; no HTTP/2, uma perda trava todos os streams da conexão TCP; no HTTP/3, só o stream afetado espera.

HTTP/3: QUIC e streams independentes

O QUIC reimplementa sobre UDP o que o TCP fazia (confirmação, retransmissão, controle de congestionamento), mas garante a ordem dentro de cada stream, e não da conexão inteira. Um pacote perdido com dados da imagem trava só a imagem.

Ele também funde transporte e criptografia: o TLS 1.3 vem embutido, e uma conexão nova fica pronta em 1 RTT, contra 2 do TCP com TLS 1.3. Num celular com 150 ms de RTT, são 150 ms a menos em cada conexão nova. Ao voltar a um servidor conhecido, o cliente pode mandar a requisição já no primeiro pacote (0-RTT). E como a conexão é identificada por um ID, e não pelo par IP e porta, ela sobrevive quando o celular troca do Wi-Fi para o 4G.

Analogia

Pense em caminhões levando carga de uma fábrica até o porto. No HTTP/1.1, cada caminhão tem uma estradinha só dele, e o próximo só sai quando o anterior volta. No HTTP/2, todos dividem uma rodovia de faixa única: passa muito mais carga, mas se um caminhão quebra, a fila inteira para atrás dele, até quem ia para outro armazém. No HTTP/3, cada pedido ganha sua faixa na mesma rodovia, e o caminhão quebrado só trava a dele.

NO MUNDO REAL

- **Google:** criou o QUIC e o usa há anos no Chrome, na Busca e no YouTube. A IETF padronizou o QUIC em 2021 (RFC 9000) e o HTTP/3 em 2022 (RFC 9114).
- **Meta:** contou em 2020 que mais de 75% do seu tráfego de internet já usava QUIC e HTTP/3.
- **CDNs** como a Cloudflare oferecem HTTP/3 na borda: o site ganha o protocolo novo sem mexer no servidor de origem (cap. 15).
- **gRPC** (cap. 19) se apoia na multiplexação do HTTP/2 para fazer muitas chamadas simultâneas numa mesma conexão.

TRADE-OFFS

	HTTP/1.1	HTTP/2	HTTP/3
Transporte	TCP	TCP	QUIC, sobre UDP
Requisições por conexão	uma por vez	muitas, intercaladas	muitas, independentes
Cabeçalhos	texto, repetidos	comprimidos (HPACK)	comprimidos (QPACK)
Um pacote perdido	trava aquela conexão	trava todos os streams	trava só o stream afetado
Conexão nova com TLS 1.3	2 RTT	2 RTT	1 RTT (0-RTT ao retomar)
Troca de rede (Wi-Fi para 4G)	reconecta	reconecta	a conexão continua

NA ENTREVISTA

- Traga o assunto quando o requisito for latência para usuários móveis ou distantes, ou páginas com muitos recursos pequenos.
- Resuma assim: "O HTTP/2 *resolveu* o *head-of-line blocking* do HTTP; o HTTP/3 *resolveu* o do TCP."
- Diga onde isso se liga na prática: na borda (CDN, load balancer, proxy reverso). Atrás dela, os serviços podem continuar em HTTP/1.1 ou HTTP/2.
- Se citar 0-RTT, mencione o risco de replay: um atacante pode reenviar aquele primeiro pacote, então ele só serve para requisições idempotentes (cap. 36).

CUIDADO COM

- ATUALIZAÇÃO: o server push, citado no artigo, foi abandonado. Era pouco usado e fácil de errar (empurrava arquivos que o navegador já tinha em cache), e o Chrome o desligou em 2022. Hoje, para adiantar recursos, usa-se a resposta `103 Early Hints`, que só sugere ao navegador o que começar a baixar.
- **HTTP/2 não acabou com o head-of-line blocking**, só o tirou da camada HTTP. Com perda alta, uma conexão HTTP/2 pode render menos que seis conexões HTTP/1.1.
- **HTTP/3 depende de UDP liberado.** Redes que bloqueiam UDP fazem o navegador voltar ao HTTP/2. E a primeira visita costuma ir por HTTP/2, até o navegador receber o cabeçalho `Alt-Svc`, que anuncia o HTTP/3.

TESTE-SE

1 **O que é a multiplexação do HTTP/2?**

É enviar várias requisições e respostas ao mesmo tempo pela mesma conexão TCP, quebradas em frames intercalados. Cada uma vive num stream com seu próprio número.

2 **Seu app tem milhões de usuários em redes móveis instáveis. Vale ativar HTTP/3?**

Sim. Perdas de pacote só travam o stream afetado, a conexão abre em 1 RTT em vez de 2 e sobrevive à troca de Wi-Fi para 4G. Em geral, basta ligar no CDN ou no load balancer.

3 **O HTTP/2 eliminou o head-of-line blocking?**

Só na camada HTTP. Como todos os streams passam por uma única conexão TCP, que entrega bytes em ordem, um pacote perdido trava todos até a retransmissão. Quem elimina isso é o QUIC, no HTTP/3.

CONECTA COM

5 Throughput vs. latência
15 CDN
17 TCP vs. UDP
19 gRPC vs. REST
50 Handshake TLS

19 PARTE 2 · REDES E COMUNICAÇÃO

gRPC vs. REST

gRPC vs REST

EM UMA FRASE

REST expõe recursos via HTTP, quase sempre com JSON legível, e domina as APIs públicas; gRPC chama métodos remotos com protobuf binário sobre HTTP/2, contrato forte e streaming, e brilha entre serviços internos.

O ESSENCIAL · tradução do artigo original

- O **REST** normalmente usa HTTP com JSON e se organiza em torno de recursos, como `/users` ou `/orders`. É simples, legível por humanos e muito usado em APIs públicas.
- O **gRPC** usa HTTP/2 e mensagens codificadas em binário (protobuf), que são menores e mais rápidas de transmitir. Ele também oferece streaming bidirecional e tipagem forte.

Em microsserviços, o gRPC costuma ser o preferido nas chamadas entre serviços, enquanto o REST é comum para clientes externos. Use REST quando legibilidade e compatibilidade importarem, e gRPC quando desempenho e contratos importarem.

APROFUNDANDO

A mesma chamada nos dois estilos

Imagine um serviço de pedidos. Em REST, você expõe o recurso e usa os verbos do HTTP:

```
GET /pedidos/1042 HTTP/1.1
Host: api.loja.example
Accept: application/json
```

```
{
  "id": 1042,
  "status": "PAGO",
  "total_centavos": 15990
}
```

Em gRPC, você começa pelo contrato, num arquivo `.proto`:

```
syntax = "proto3";
package loja.v1;

service Pedidos {
  rpc BuscarPedido(BuscarPedidoRequest) returns (Pedido);
}

message BuscarPedidoRequest {
  int64 id = 1;
}

message Pedido {
  int64 id = 1;
  string status = 2;
  int64 total_centavos = 3;
}
```

A partir do contrato, o compilador `protoc` gera o cliente e o esqueleto do servidor em Go, Java, Python, C# e outras linguagens. O cliente chama `BuscarPedido` como se fosse uma função local; por baixo, isso vira um `POST` HTTP/2 para `/loja.v1.Pedidos/BuscarPedido`.

Por que o protobuf é menor

A resposta acima, em JSON compacto (sem espaços), tem 50 bytes. Em protobuf, a mesma mensagem tem 12. O protobuf não envia o nome dos campos: envia o número deles (o `= 1`, `= 2`, `= 3` do contrato) junto com o tipo, num único byte para os campos de 1 a 15. Os inteiros vão em varint, que usa só os bytes necessários: 1042 cabe em 2. O preço é que, sem o `.proto`, os bytes são ilegíveis.

Quatro jeitos de chamar

Tipo de chamada	Quem envia várias mensagens	Exemplo
Unária	ninguém: um pedido, uma resposta	buscar um pedido
Streaming do servidor	o servidor	acompanhar o status de uma entrega
Streaming do cliente	o cliente	enviar leituras de um sensor em lote
Bidirecional	os dois, ao mesmo tempo	chat, transcrição de áudio ao vivo

Duas coisas completam o pacote. Deadlines: o cliente diz quanto aceita esperar, o prazo viaja com a chamada e, se estourar, ela termina com `DEADLINE_EXCEEDED` em vez de ficar pendurada. E regras de evolução: campo novo ganha número novo, clientes antigos ignoram o que não conhecem, e um número já usado nunca é reaproveitado.

> *Analogia*
>
> REST é o cardápio do restaurante: qualquer cliente lê, entende e faz o pedido em linguagem comum. gRPC é o sistema de comandas entre o salão e a cozinha: "mesa 12, item 304, sem cebola", com campos fixos e códigos combinados. É rápido e não deixa margem para erro, mas só funciona entre quem usa o mesmo sistema. Por isso o cliente fala com o garçom em REST, e o garçom fala com a cozinha em gRPC.

NO MUNDO REAL

- **Stripe e GitHub** expõem APIs públicas REST com JSON: qualquer linguagem consome, e um `curl` basta para testar.
- **Kubernetes:** o API server fala REST (com JSON ou protobuf), mas o kubelet conversa com o runtime de containers (containerd, CRI-O) pela CRI, uma API gRPC.
- **etcd:** a API da versão 3 é gRPC, e um gateway embutido a traduz para JSON sobre HTTP para quem não fala gRPC.

- **Google Cloud:** muitas APIs aceitam REST com JSON e também gRPC, e várias bibliotecas oficiais usam gRPC por baixo.

TRADE-OFFS

	REST	gRPC
Modelo	recursos e verbos (`GET /pedidos/1042`)	métodos remotos (`BuscarPedido`)
Formato	texto, quase sempre JSON	binário (protobuf)
Contrato	opcional (OpenAPI)	obrigatório (`.proto`), com código gerado
Streaming	não nativo (usa SSE ou WebSocket)	quatro modos, inclusive bidirecional
Navegador	nativo (`fetch`)	só via gRPC-Web (proxy ou suporte no servidor)
Cache HTTP	`GET` cacheável por CDN e proxies	não aproveita (toda chamada é `POST`)
Depuração	`curl` e o navegador bastam	ferramentas próprias (`grpcurl`)

NA ENTREVISTA

- A frase que resume: "*REST na borda, gRPC no miolo: REST para parceiros e navegadores, gRPC entre os nossos serviços.*"
- Justifique com requisitos, não com moda: alcance, legibilidade e cache HTTP de um lado; contrato, código gerado, streaming e menos CPU do outro.
- Trate o `.proto` como fonte da verdade: mudança de API vira revisão de contrato e segue as regras de evolução.
- Fale de deadlines: numa cadeia de cinco serviços, propagar o prazo evita que o último trabalhe numa resposta que ninguém mais espera.

CUIDADO COM

- `ATUALIZAÇÃO:` “binário é menor e mais rápido” depende do caso. Com gzip ou Brotli, a diferença de tamanho encolhe em respostas grandes; o ganho do gRPC costuma vir mais da CPU de serialização, do contrato e da conexão reaproveitada. Meça antes de migrar.
- **Balanceamento:** um load balancer de camada 4 distribui conexões, não chamadas. Como o gRPC reaproveita uma conexão HTTP/2 longa, todo o tráfego de um cliente cai num servidor só. Use balanceamento L7 ou no cliente (cap. 11).
- **Navegador não fala gRPC puro:** precisa do gRPC-Web, traduzido por um proxy (como o Envoy) ou aceito pelo próprio servidor.
- **REST não é “JSON sobre HTTP”.** É modelar recursos e usar os verbos e códigos de status do HTTP. `POST /criarPedido` é RPC com JSON.

TESTE-SE

1 **Por que uma mensagem protobuf costuma ser bem menor que o JSON equivalente?**

Porque não carrega o nome dos campos, só um número e o tipo (geralmente 1 byte), codifica inteiros em varint e dispensa aspas, chaves e espaços.

2 **Você vai expor uma API para parceiros e ligar 30 microsserviços internos. O que usa em cada ponta?**

REST com JSON para os parceiros: qualquer linguagem consome, há OpenAPI, `curl` e cache HTTP. gRPC entre os serviços internos: contrato forte, código gerado, streaming e menos custo por chamada.

3 **Por que, atrás de um load balancer de camada 4, todo o tráfego gRPC de um cliente pode cair num único servidor?**

Porque o gRPC abre uma conexão HTTP/2 longa e multiplexa todas as chamadas nela, e o balanceador L4 só escolhe o destino quando a conexão é aberta. A saída é balancear por chamada (L7) ou no próprio cliente.

CONECTA COM

- 9 Microsserviços vs. monólito
- 11 Balanceamento de carga
- 14 API Gateway
- 18 HTTP/2 e HTTP/3 (QUIC)
- 20 WebSocket e Server-Sent Events

WebSocket e Server-Sent Events

WebSocket and Server-Sent Events (SSE)

EM UMA FRASE

O WebSocket abre um canal persistente em que cliente e servidor falam a qualquer momento; no SSE, só o servidor fala, enviando eventos ao cliente por uma resposta HTTP que não termina.

O ESSENCIAL · tradução do artigo original

O **WebSocket** cria uma conexão full-duplex, em que cliente e servidor podem enviar mensagens um ao outro a qualquer momento. O **SSE** (Server-Sent Events) permite que o servidor envie eventos ao cliente por um canal de mão única, usando HTTP. WebSockets são ótimos para chats, jogos multiplayer e colaboração ao vivo. O SSE é mais simples e se encaixa em casos como placares ao vivo ou notificações, em que só o servidor precisa enviar atualizações. Os dois resolvem problemas de comunicação em tempo real que o HTTP comum não resolve bem.

APROFUNDANDO

WebSocket: um upgrade e depois outro protocolo

O WebSocket começa como uma requisição HTTP comum, que pede para trocar de protocolo:

```
GET /chat HTTP/1.1
Host: chat.loja.example
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Se o servidor aceita, responde `101 Switching Protocols` e, daí em diante, aquela conexão TCP deixa de falar HTTP. Os dois lados trocam frames (texto, binário, ping/pong e fechamento) quando quiserem, com 2 a 14 bytes de cabeçalho por frame, contra as centenas de bytes de cabeçalhos de uma requisição HTTP típica.

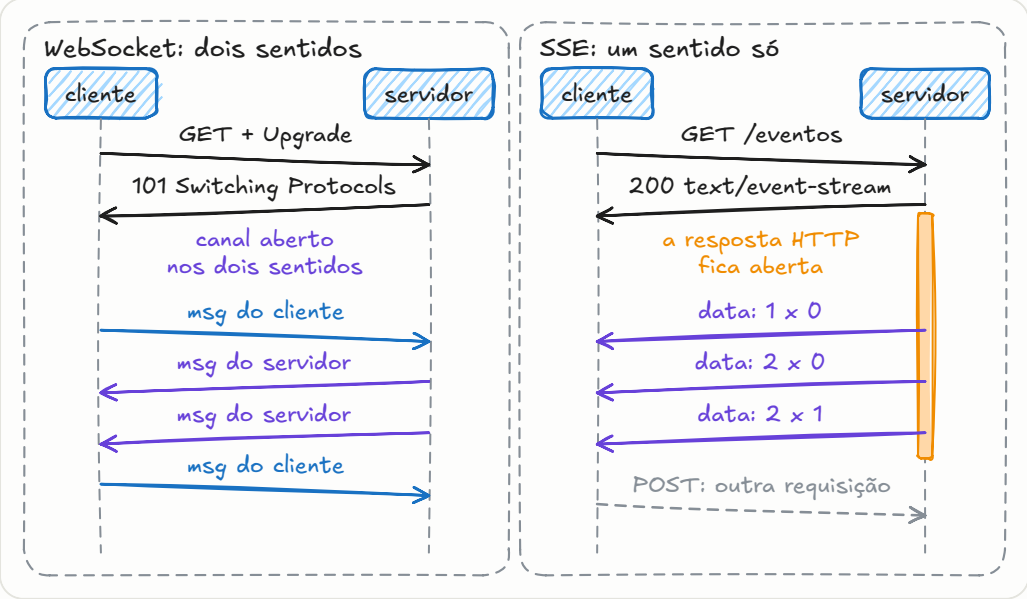
SSE: uma resposta HTTP que não termina

No SSE, o cliente faz um `GET` comum e o servidor responde com `Content-Type: text/event-stream`, mas não fecha a resposta. Vai escrevendo eventos em texto, separados por uma linha em branco:

```
id: 41
data: {"jogo": "BRA x ARG", "placar": "1 x 0"}

id: 42
data: {"jogo": "BRA x ARG", "placar": "2 x 0"}
```

No navegador, a API `EventSource` cuida do resto: se a conexão cai, ela reconecta sozinha e envia o cabeçalho `Last-Event-ID: 42`, para o servidor continuar de onde parou. Tudo é HTTP: a mesma autenticação por cookie, os mesmos proxies e, sobre HTTP/2 (cap. 18), só mais um stream na conexão que a página já tem.



O WebSocket troca de protocolo e abre um canal nos dois sentidos; no SSE, o servidor escreve eventos numa resposta HTTP que fica aberta, e o cliente, para falar com o servidor, faz outra requisição.

Conexões abertas mudam a arquitetura

Nos dois casos, cada cliente ocupa uma conexão aberta por minutos ou horas. O servidor passa a guardar estado (cap. 8): o load balancer (cap. 11) precisa aceitar conexões longas, um deploy derruba milhares de clientes de uma vez, e uma mensagem

gerada no servidor B para alguém conectado ao servidor A precisa de um pub/sub (cap. 45) entre eles. A conta de capacidade também muda: se cada servidor segura 50 mil conexões, 1 milhão de usuários online exigem 20 servidores só para manter os canais abertos.

Analogia

O WebSocket é a conversa no privado do WhatsApp: depois que ela começa, qualquer um dos dois escreve quando quiser, sem esperar a vez. O SSE é o grupo de avisos do condomínio em que só o síndico pode postar: os moradores recebem tudo na hora em que ele manda. Se um morador quiser reclamar do barulho, não escreve no grupo; manda mensagem no privado, que é outro canal.

NO MUNDO REAL

- **Discord:** os clientes recebem mensagens, presença e outros eventos em tempo real por uma conexão WebSocket com o gateway da plataforma.
- **Figma:** a edição multiplayer sincroniza as alterações de todos os participantes de um arquivo por WebSockets.
- **APIs de LLM (OpenAI, Anthropic):** com streaming ligado, a resposta chega aos pedaços, token a token, como Server-Sent Events.
- **Placares e painéis ao vivo** são o caso típico de SSE: muita gente lendo, só o servidor escrevendo.

TRADE-OFFS

	WebSocket	SSE
Direção	os dois sentidos	só do servidor para o cliente
Protocolo	próprio, depois de um upgrade do HTTP	HTTP comum (`text/event-stream`)
Dados	texto ou binário	só texto (UTF-8)
Reconexão	por sua conta	automática, com `Last-Event-ID`

	WebSocket	SSE
Infraestrutura no caminho	proxies e LBs precisam suportar o upgrade	passa por quase tudo que fala HTTP
Bom para	chat, jogos, edição colaborativa	notificações, placar, feed, tokens de LLM

NA ENTREVISTA

- Comece pela pergunta certa: "*Quem precisa falar com quem, e com que frequência?*" Só o servidor fala: SSE. Os dois falam o tempo todo: WebSocket.
- Mostre que pensou na escala: quantas conexões por servidor, pub/sub para entregar a mensagem no servidor certo e reconexão com backoff e jitter (cap. 35), para que um deploy não vire uma avalanche de reconexões.
- Lembre do heartbeat (cap. 37): ping/pong no WebSocket e um comentário vazio no SSE, em intervalos menores que o timeout de ociosidade dos proxies (60 s é um padrão comum), mantêm a conexão viva.

CUIDADO COM

- **SSE sobre HTTP/1.1 esbarra no limite de 6 conexões por domínio**, somando as abas do navegador: a sétima aba fica esperando. Com HTTP/2, os eventos viram streams de uma única conexão e o problema praticamente some.
- `EventSource` **não envia cabeçalhos próprios nem faz POST.** Para autenticar com token, use cookie ou leia o stream com `fetch`, como fazem os clientes das APIs de LLM.
- **Proxies com buffer** seguram os eventos e entregam tudo de uma vez. No Nginx, desligue o buffer dessa rota.
- **Depois do 101, não é mais HTTP.** Cache, códigos de status e ferramentas HTTP não enxergam as mensagens; a autenticação precisa acontecer no handshake ou na primeira mensagem.

TESTE-SE

1 Qual a diferença fundamental entre WebSocket e SSE?

O WebSocket é full-duplex: depois do upgrade, os dois lados enviam mensagens quando quiserem, num protocolo próprio. O SSE é de mão única, do servidor para o cliente, dentro de uma resposta HTTP comum que não termina.

2 Placar ao vivo de um campeonato, com milhões de leitores: WebSocket ou SSE?

SSE. Só o servidor fala, tudo continua sendo HTTP (aproveitando HTTP/2 e a infraestrutura existente) e o navegador reconecta sozinho, retomando do último evento. WebSocket traria complexidade sem ganho.

3 Com SSE, como o cliente envia dados ao servidor?

Não pelo canal SSE, que só desce. O cliente faz outra requisição HTTP comum, como um `POST`, e o servidor responde pelo stream se precisar.

CONECTA COM

8 Arquitetura stateful vs. stateless
11 Balanceamento de carga
18 HTTP/2 e HTTP/3 (QUIC)
21 Long polling
45 Pub/Sub

Long polling

Long Polling

EM UMA FRASE

No long polling, o cliente pergunta e o servidor segura a requisição até ter novidade ou estourar o timeout; ao receber a resposta, o cliente pergunta de novo, simulando tempo real com HTTP comum.

O ESSENCIAL · tradução do artigo original

Long polling é uma técnica em que o cliente envia uma requisição e o servidor a mantém aberta até haver dados novos ou estourar um timeout. Quando a resposta volta, o cliente abre outra requisição imediatamente. Isso simula atualizações em tempo real sobre HTTP comum, sem protocolos especiais. É menos eficiente que WebSockets, mas é mais fácil de implementar e passa pela maioria dos proxies e firewalls. Pense nele como perguntar "alguma novidade?" e esperar em silêncio até que haja uma resposta.

APROFUNDANDO

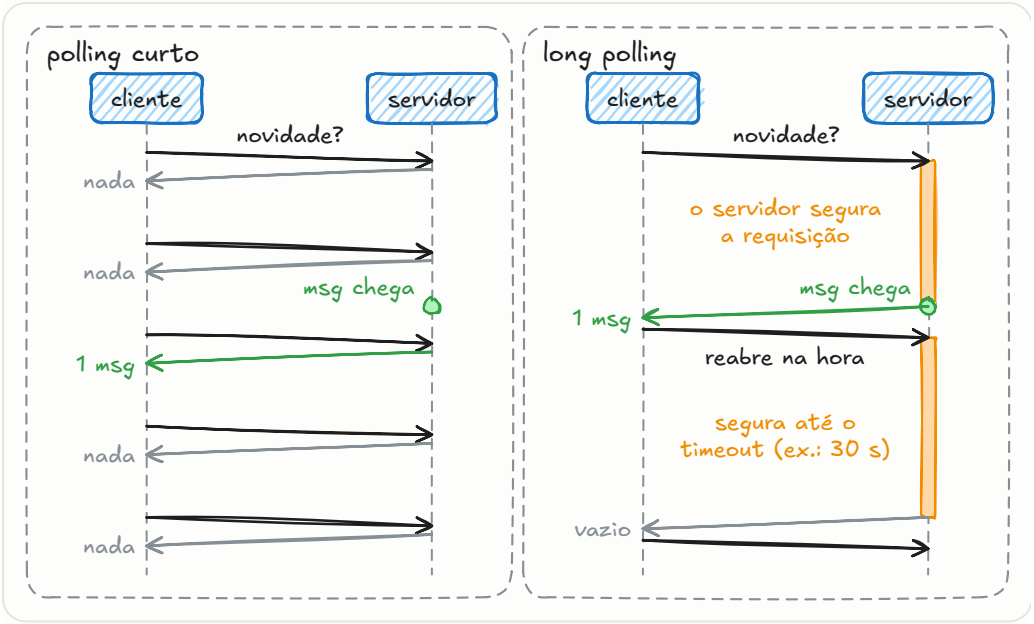
Polling curto vs. long polling

No polling curto (short polling), o cliente pergunta em intervalos fixos: "tem novidade?" a cada 2 segundos. Quase sempre a resposta é "não". Faça a conta: com 100 mil usuários, são 50 mil requisições por segundo, a maioria vazia, e ainda assim uma novidade espera em média 1 segundo para ser vista (metade do intervalo).

O long polling inverte a lógica: o servidor só responde quando tem o que dizer.

1. O cliente faz `GET /mensagens?depois=1234`.
2. Se já há mensagens depois da 1234, o servidor responde na hora.
3. Se não há, ele segura a requisição aberta, sem responder.
4. Quando chega uma mensagem, responde imediatamente com ela.
5. Se nada chega em, digamos, 30 segundos, responde vazio (por exemplo, `204 No Content`).

6. Em qualquer caso, o cliente processa a resposta e abre a próxima requisição na hora.



No polling curto, a maioria das perguntas volta vazia e a mensagem espera a próxima pergunta; no long polling, o servidor segura a requisição e responde assim que a mensagem chega.

Com os mesmos 100 mil usuários e nenhuma novidade, o tráfego cai para cerca de 3.300 requisições por segundo (uma a cada 30 s por usuário), e a novidade chega em uma viagem de rede, não em segundos.

O que o servidor precisa aguentar

- **Muitas requisições paradas ao mesmo tempo.** São 100 mil conexões abertas esperando. Num servidor que dedica uma thread a cada requisição, as threads acabam; o long polling pede um modelo assíncrono (Node.js, Go, Netty, asyncio), em que esperar custa só um pouco de memória.
- **Timeouts em toda a cadeia.** O timeout do long polling precisa ser menor que o de tudo no meio do caminho: por padrão, o Nginx desiste do servidor de trás depois de 60 s sem receber nada (`proxy_read_timeout`), e o load balancer de aplicação da AWS (ALB) fecha conexões ociosas em 60 s. Um timeout de uns 30 s fica longe desses limites.

- **O buraco entre duas requisições.** Entre uma resposta e a próxima requisição, o cliente não está ouvindo. Por isso a requisição leva um cursor (`depois=1234`): o servidor guarda as mensagens recentes e entrega tudo o que veio depois dele, e nada se perde.

Analogia

Polling curto é ir à padaria a cada dez minutos perguntar se a fornada das cinco já saiu: quase sempre a resposta é "ainda não", e você gasta sola de sapato à toa. Long polling é perguntar uma vez e ficar esperando no balcão: o padeiro só fala com você quando o pão sai do forno. Se demorar demais, ele avisa "hoje atrasou" (o timeout), e você pergunta de novo na hora.

NO MUNDO REAL

- **Telegram:** na Bot API, o método `getUpdates` é long polling: o bot passa um `timeout`, e o servidor segura a chamada até chegar uma mensagem ou o tempo acabar.
- **Amazon SQS:** com `WaitTimeSeconds` (até 20 s), o `ReceiveMessage` espera chegar mensagem em vez de responder vazio na hora, o que corta requisições vazias e custo (cap. 44).
- **Consul:** as blocking queries seguram uma consulta até o dado mudar ou até o `wait` acabar (o padrão é 5 minutos).
- **Socket.IO:** por padrão, abre a conexão com long polling e depois tenta passar para WebSocket; se a rede não deixar, segue no long polling.

TRADE-OFFS

	Polling curto	Long polling	SSE ou WebSocket (cap. 20)
Atraso para ver a novidade	até um intervalo inteiro	uma viagem de rede	uma viagem de rede
Requisições vazias	a maioria	só nos timeouts	nenhuma
Conexões presas no servidor	poucas e curtas	uma por cliente, quase sempre	uma por cliente, sempre

	Polling curto	Long polling	SSE ou WebSocket (cap. 20)
Cabeçalhos HTTP por mensagem	sim	sim	não, só na abertura
Implementação	trivial	simples, mas pede cursor e servidor assíncrono	SSE simples; WebSocket com infraestrutura própria

NA ENTREVISTA

- Traga o long polling quando o tempo real for desejável mas não crítico, ou quando algo no caminho (proxy corporativo, firewall, plataforma antiga) não deixar abrir WebSocket.
- Mostre que conhece o mecanismo inteiro: timeout abaixo do limite dos intermediários, cursor para não perder mensagens e servidor assíncrono para segurar milhares de requisições paradas.
- Frase pronta: "*Começo com SSE; se algo no caminho atrapalhar, long polling com timeout de 30 s e cursor resolve sem mudar o resto do desenho.*"

CUIDADO COM

- ATUALIZAÇÃO: o artigo compara o long polling só com WebSockets. Hoje, quando só o servidor precisa avisar o cliente, o SSE (cap. 20) costuma ser o meio-termo natural: também é HTTP comum, mas sem uma requisição nova a cada mensagem. O long polling ficou mais como plano B, como no Socket.IO.
- **Avalanche de reconexões:** um aviso para todos faz todos os clientes receberem a resposta e voltarem no mesmo instante. Espalhe com um atraso aleatório (jitter, cap. 35).
- **Rajadas:** se chegam 50 mensagens em um segundo, devolva todas numa resposta só, não 50 idas e voltas.

- **No HTTP/1.1, cada long poll ocupa uma das 6 conexões** que o navegador abre por domínio; com várias abas abertas, falta conexão para o resto da página.

TESTE-SE

1 O que acontece quando o long polling chega ao timeout sem nenhum dado novo?

O servidor responde vazio (por exemplo, `204 No Content`) e o cliente abre outra requisição imediatamente, recomeçando a espera.

2 Um painel interno recebe atualizações algumas vezes por hora, para 1.000 usuários, e a rede da empresa bloqueia WebSocket. O que você usa?

Long polling (ou SSE, se nada no caminho atrapalhar). Polling curto a cada 2 s geraria 500 requisições por segundo quase sempre vazias; long polling com timeout de 30 s fica em cerca de 33 por segundo e entrega as novidades na hora.

3 O long polling, sozinho, garante que o cliente não perca nenhuma mensagem?

Não. Entre uma resposta e a próxima requisição o cliente não está ouvindo. O servidor precisa guardar as mensagens recentes, e o cliente precisa dizer a partir de qual quer receber (um cursor ou o ID da última).

CONECTA COM

11 Balanceamento de carga
20 WebSocket e Server-Sent Events
35 Retries e backoff exponencial
44 Filas de mensagens

Protocolo gossip

Gossip Protocol

EM UMA FRASE

Num protocolo gossip, cada nó conta periodicamente o que sabe a alguns pares sorteados; a informação se espalha como uma epidemia e chega a todos em poucas rodadas, sem coordenador central.

O ESSENCIAL · tradução do artigo original

Um protocolo gossip (“fofoca”) permite que os nós de um sistema distribuído compartilhem informações conversando periodicamente com pares escolhidos ao acaso. Com o tempo, a informação se espalha como fofoca num grupo de pessoas, até que todos tenham mais ou menos a mesma visão. Ele é usado para compartilhar quem faz parte do cluster (membership), o estado de saúde dos nós ou configurações, de forma tolerante a falhas. O protocolo é eventualmente consistente e não depende de uma autoridade central. Isso o torna ideal para clusters grandes, em que nós entram e saem com frequência.

APROFUNDANDO

Uma rodada

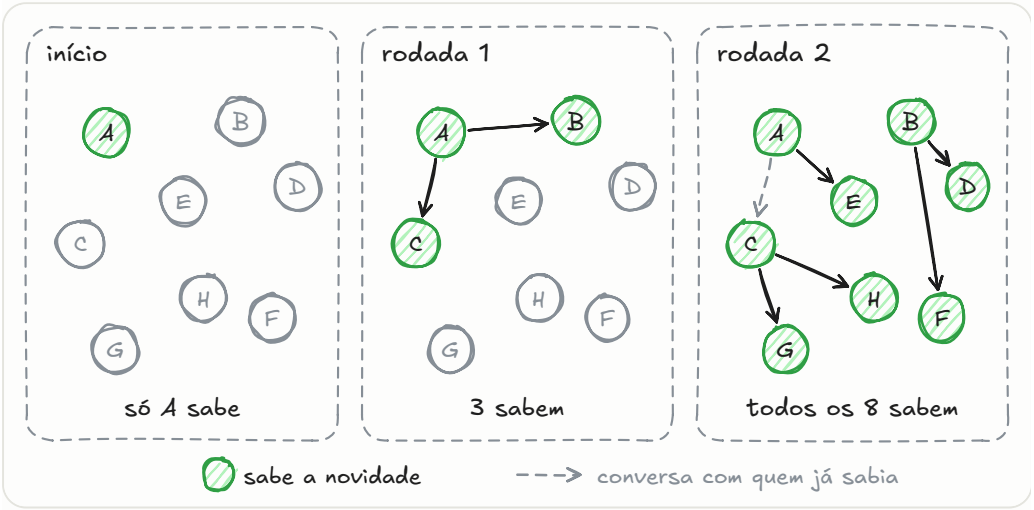
A cada intervalo fixo (no Cassandra, 1 segundo), cada nó:

1. sorteia alguns pares (o fanout, normalmente de 1 a 3);
2. envia um resumo do que sabe: para cada nó do cluster, uma versão, como um contador de heartbeat que o próprio nó incrementa;
3. compara com o resumo do outro lado, e cada um envia ao outro o que tiver de mais novo (o estilo push-pull).

Como toda informação tem versão, conflito se resolve sozinho: vale a mais nova. Ninguém coordena nada.

Por que espalha tão rápido

No começo, um nó sabe. Na rodada seguinte, ele e os pares que ouviram dele; depois, cada um desses conta para outros. O número de informados cresce de forma exponencial, como numa epidemia. Não por acaso, a literatura chama essa família de protocolos epidêmicos, desde um artigo de 1987 da Xerox PARC sobre replicação de bancos de dados. Por isso a novidade alcança N nós em cerca de log N rodadas. Numa simulação simples, com cada nó conversando com um par sorteado por rodada (push-pull), 1.000 nós ficam todos sabendo em cerca de 9 rodadas, e 10.000 nós em cerca de 12: dez vezes mais nós custam só umas 3 rodadas a mais.



Em cada rodada, quem já sabe conta para dois nós sorteados. Uma das conversas cai em quem já sabia; mesmo assim, ao fim da rodada 2, os oito nós sabem.

O mesmo mecanismo detecta falhas

Se o contador de heartbeat de um nó para de subir por tempo demais, os outros o marcam como suspeito e, depois, como morto. O SWIM (2002), usado pelo Consul, troca os contadores por sondagens: um nó pinga um par sorteado e, sem resposta, pede a alguns outros nós que tentem também antes de levantar suspeita. Assim, um nó com um único link ruim não é declarado morto por engano.

O custo

Cada nó envia poucas mensagens por rodada, não importa o tamanho do cluster: a carga é igual para todos, e não há gargalo nem ponto único de falha. Em troca, muitas conversas repetem o que o outro já sabia, e, enquanto a novidade se espalha, os

nós discordam entre si.

Analogia

Pense em como uma música chiclete se espalha no Carnaval. Ninguém marca reunião para ensinar a letra: quem ouviu canta perto de dois ou três amigos, cada um deles canta para outros, e na Quarta-Feira de Cinzas o país inteiro sabe o refrão. Se um folião vai embora mais cedo, a música chega pelos outros. Não existe um ponto central que, se cair, interrompe tudo.

NO MUNDO REAL

- **Cassandra:** o gossip roda a cada segundo e troca estado com até três outros nós. É assim que cada nó sabe quem está no anel (cap. 25), quem está vivo e como os outros estão.
- **Consul (HashiCorp):** usa o Serf, baseado no SWIM, para manter a lista de membros do cluster e detectar falhas.
- **Redis Cluster:** os nós trocam pings e pongs que carregam informações sobre outros nós; quando a maioria dos mestres considera um nó inacessível, ele é marcado como falho e o failover começa.
- **Amazon Dynamo:** o artigo de 2007 descreve gossip para propagar a lista de membros e as mudanças no anel.

TRADE-OFFS

	Gossip	Coordenador central
Ponto único de falha	não existe	o coordenador (ou o cluster dele)
Carga	igual em todos os nós	concentrada no coordenador
Tempo até todos saberem	~log N rodadas (segundos)	uma rodada de mensagens
Consistência	eventual: por um tempo, nós discordam	pode ser forte, com consenso
Mensagens	muitas repetidas	uma por nó

	Gossip	Coordenador central
Bom para	membership, saúde, metadados	líder, locks, configuração crítica

NA ENTREVISTA

- Traga o gossip quando o desenho tiver um cluster grande e dinâmico (banco distribuído, cache, service discovery) e surgir a pergunta: "*Como cada nó sabe quem está vivo?*"
- Descreva os parâmetros: intervalo da rodada, fanout, push-pull e informação versionada. Cite a convergência em O(log N) rodadas.
- Deixe claro o limite: gossip espalha estado, não decide nada. Para eleger líder ou travar um recurso, use consenso (cap. 38). Os dois convivem bem: o Consul usa gossip para membership e Raft para replicar o catálogo.

CUIDADO COM

- **Gossip não é consenso.** Durante a propagação, dois nós podem discordar sobre quem está vivo, e nada garante uma decisão única.
- **Lento não é morto.** Um nó numa pausa longa de garbage collection parece morto por alguns segundos. Use detectores pacientes, como o phi accrual do Cassandra, que se adapta ao histórico de atrasos, ou uma fase de suspeita antes da sentença, como no SWIM.
- **O estado cresce com o cluster.** Se cada mensagem levar tudo sobre todos, o tráfego explode; por isso se trocam primeiro só as versões, e depois o que mudou.
- **Partição não se cura com fofoca.** Cada lado converge para a própria visão e acha que o outro morreu.

TESTE-SE

1 **Em quantas rodadas uma novidade chega a todos os N nós num protocolo gossip?**

Em cerca de log N rodadas, porque o número de nós informados cresce de forma exponencial: cada nó que sabe conta para outros a cada rodada.

2 **Num cluster de 500 nós em que máquinas entram e saem o tempo todo, como cada nó descobre quem está vivo sem um servidor central?**

Com gossip: a cada segundo, cada nó troca com alguns pares sorteados a lista de membros e os contadores de heartbeat. Não há ponto único de falha, e cada nó envia o mesmo punhado de mensagens por rodada, tenha o cluster 50 ou 500 nós.

3 **Dá para usar gossip para eleger o líder do cluster?**

Não. Gossip é eventualmente consistente: durante a propagação, nós diferentes podem ter visões diferentes e "eleger" líderes diferentes. Eleição de líder exige consenso, como Raft ou Paxos.

CONECTA COM

2 Teorema CAP · 7 Consistência forte vs. eventual · 25 Hashing consistente · 37 Heartbeat · 38 Eleição de líder (Paxos e Raft)

Parte 3

Bancos de dados e armazenamento

Database and Storage Internals

Servidores de aplicação sem estado (cap. 8) se multiplicam com um clique; os dados, não. Mais cedo ou mais tarde, todo sistema que cresce esbarra no banco. Esta parte abre o capô do armazenamento. Primeiro, como espalhar os dados por várias máquinas: dividindo (sharding, cap. 23), copiando (replicação, cap. 24) e decidindo quem guarda cada chave sem embaralhar tudo quando o cluster muda de tamanho (hashing consistente, cap. 25).

Depois, o que acontece dentro de um único nó: as estruturas que tornam a busca rápida (B-Tree e LSM-Tree, cap. 26), o log que protege cada escrita contra quedas (WAL, cap. 27) e o filtro que poupa idas inúteis ao disco (Bloom filter, cap. 30). Por fim, decisões de modelagem que aparecem em toda entrevista: normalizar ou duplicar (cap. 28), usar um banco ou vários (cap. 29) e buscar por semelhança em vez de igualdade, com bancos vetoriais (cap. 31).

23 Sharding
24 Padrões de replicação
25 Hashing consistente
26 Índices: B-Tree e LSM-Tree
27 Write-Ahead Logging (WAL)
28 Normalização vs. desnormalização
29 Persistência poliglota
30 Bloom filter
31 Bancos de dados vetoriais

Sharding

Sharding (Data Partitioning)

EM UMA FRASE

Sharding divide os dados entre várias máquinas, cada uma com um pedaço; o difícil é escolher uma chave de shard que espalhe a carga por igual e conviver com o custo de mover dados depois.

O ESSENCIAL · tradução do artigo original

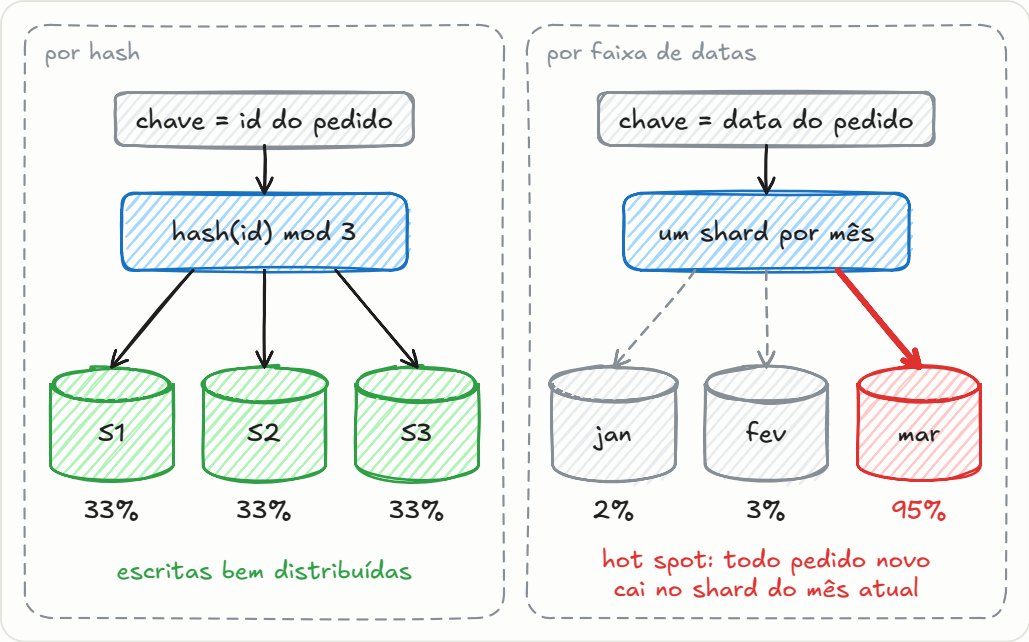
Sharding divide os dados entre várias máquinas, e cada uma guarda um subconjunto deles. As estratégias mais comuns são o sharding por faixa (range-based), o sharding por hash (hash-based) e o sharding por diretório (directory-based). O objetivo principal é escalar o armazenamento e o throughput, evitando um único nó de banco de dados gigante. A parte difícil é escolher uma chave de shard (shard key) que evite hot spots, em que um shard recebe a maior parte do tráfego. Depois que você faz sharding, mover dados entre shards (resharding) se torna um desafio operacional importante.

APROFUNDANDO

Cada linha tem uma **chave de shard**, uma coluna como `cliente_id`. Uma função transforma essa chave no número do shard, e um roteador (na aplicação, num proxy ou no próprio banco) manda a consulta para lá. A função define o comportamento:

- **Por faixa:** cada shard guarda um intervalo contíguo de chaves (clientes de A a F no shard 1, de G a M no shard 2...). Consultas por intervalo ficam baratas, porque vizinhos moram juntos. O risco são chaves que crescem em sequência, como datas e IDs autoincrementais: toda escrita nova cai no último shard.
- **Por hash:** o shard é `hash(chave) mod N`. A distribuição fica uniforme, mas uma consulta por intervalo precisa perguntar a todos os shards e juntar as respostas (scatter-gather).
- **Por diretório:** uma tabela de consulta diz em que shard está cada chave ou grupo de chaves. É a mais flexível (dá para mover um cliente gigante para um shard só dele), mas o diretório vira uma peça crítica, que precisa ser rápida e altamente

disponível.



Com hash, as escritas se dividem por igual; com faixa de datas, o shard do mês atual vira um hot spot.

Um exemplo hipotético: um sistema de pedidos com 2 TB de dados e 40 mil escritas por segundo, dividido em 8 shards por hash de `cliente_id`, deixa cada máquina com uns 250 GB e 5 mil escritas por segundo. E a consulta mais comum, "pedidos deste cliente", toca um shard só.

Só que hash espalha chaves, não acessos. Se uma única chave é muito acessada (a conta de uma celebridade, um cliente que é metade do faturamento), o shard dela sofre de qualquer jeito. As saídas são dividir essa chave (acrescentar um sufixo aleatório na escrita e juntar as partes na leitura) ou dar a ela um shard próprio.

O **resharding** dói porque `mod N` é frágil: passar de 4 para 5 shards muda o shard de cerca de 80% das chaves. Por isso, muitos sistemas criam de início muito mais **partições lógicas** do que máquinas (por exemplo, 1.024 partições em 8 servidores) e, para crescer, movem partições inteiras; outros usam hashing consistente (cap. 25). Mover dados com o sistema no ar exige copiar, acompanhar as escritas que chegam durante a cópia e só então virar o roteamento.

Analogia

Um cartório com milhões de certidões não cabe numa sala só, então divide o arquivo por sobrenome: de A a F na sala 1, de G a M na sala 2, e assim por diante. É sharding por faixa. Só que a sala do S vive lotada, porque Silva, Santos e Souza estão entre os sobrenomes mais comuns do Brasil: um hot spot. Distribuir as certidões por um cálculo sobre o CPF (um hash) equilibraria as filas, mas aí achar "todos os Silva" exigiria visitar todas as salas.

NO MUNDO REAL

- O **MongoDB** faz sharding nativo por faixa ou por hash e move blocos de dados (chunks) entre shards para equilibrá-los. Desde a versão 5.0, permite trocar a chave de shard de uma coleção em produção.
- O **Vitess**, criado no YouTube para fatiar o MySQL, roteia cada consulta ao shard certo e é usado pelo Slack.
- O **Instagram** documentou, em 2012, milhares de shards lógicos no PostgreSQL mapeados para poucos servidores físicos, com o número do shard embutido em cada ID gerado.
- O **DynamoDB** divide as tabelas por hash da chave de partição. Cada partição aguenta até 1.000 WCU (cerca de mil escritas de 1 KB por segundo), então um item muito acessado fica preso a esse teto, seja qual for a capacidade da tabela.

TRADE-OFFS

	Por faixa	Por hash	Por diretório
Distribuição	Depende dos dados	Uniforme por chave	Você decide, chave a chave
Consulta por intervalo	Barata: um ou poucos shards	Cara: todos os shards	Depende de como o diretório agrupa
Risco de hot spot	Alto com datas e IDs crescentes	Só com chaves muito populares	Controlável, movendo chaves
Resharding	Dividir uma faixa ao meio	Difícil com `mod N`; use partições lógicas	Atualizar o diretório e mover os dados

	Por faixa	Por hash	Por diretório
Peça extra	Mapa de faixas	Nenhuma	O diretório, rápido e sempre disponível

NA ENTREVISTA

- Não comece por sharding. Diga antes o que se esgotou: "Já usei réplicas de leitura e cache; o problema agora é o volume de escritas, que não cabe num primário."
- Justifique a chave pelo **padrão de acesso**: "Faço sharding por `conversa_id`, porque a consulta mais comum é 'últimas mensagens desta conversa', e ela fica num shard só."
- Antecipe as perguntas difíceis: o que fazer com a chave quente, como ficam as consultas que cruzam shards e como crescer de 8 para 16 shards sem parar o sistema.

CUIDADO COM

- **Sharding não é replicação.** Sharding divide os dados; replicação copia (cap. 24). Na prática, cada shard costuma ser um conjunto replicado.
- **Joins e transações entre shards** deixam de ser triviais. Uma transação que toca dois shards exige 2PC (cap. 40) ou SAGA (cap. 39), então escolha a chave para que as operações comuns fiquem num shard só.
- **"Partição" tem dois sentidos.** Aqui é um pedaço dos dados; no CAP (cap. 2), é uma falha de rede.
- `ATUALIZAÇÃO:` bancos distribuídos como Spanner, CockroachDB e TiDB dividem e movem faixas automaticamente conforme crescem. O resharding manual que o artigo descreve é a dor típica de quem faz sharding por conta própria sobre MySQL ou PostgreSQL.

TESTE-SE

1 **Qual a diferença entre sharding por faixa e por hash?**

Por faixa, cada shard guarda um intervalo contíguo de chaves: consultas por intervalo são baratas, mas chaves sequenciais criam hot spots. Por hash, a distribuição é uniforme, mas uma consulta por intervalo precisa ir a todos os shards.

2 **Num app de chat com bilhões de mensagens, qual chave de shard você usaria para a tabela de mensagens?**

O id da conversa, com hash: a consulta dominante ("últimas mensagens desta conversa") fica num shard só, e as conversas se espalham. A data criaria um hot spot, e o id do remetente espalharia cada conversa por vários shards.

3 **Com `hash(chave) mod N`, passar de 4 para 5 shards move só um quinto dos dados?**

Não. Cerca de 80% das chaves mudam de shard, porque o resto da divisão muda para quase todas. Hashing consistente (cap. 25) ou partições lógicas fixas evitam isso.

CONECTA COM

1 Escalabilidade vertical vs. horizontal

24 Padrões de replicação

25 Hashing consistente

40 Two-Phase Commit (2PC)

Padrões de replicação

Replication Patterns (Master Slave, Master Master)

EM UMA FRASE

Replicação mantém cópias dos dados em vários nós; com um primário, só ele aceita escritas e as réplicas atendem leituras; com vários primários, todos escrevem, mas os conflitos precisam ser resolvidos.

O ESSENCIAL · tradução do artigo original

Replicação significa manter várias cópias dos dados em nós diferentes.

- No modelo master-slave (ou primário-réplica), um nó cuida das escritas e replica as mudanças para os outros, que atendem as leituras.
- No modelo master-master (multi-primário), vários nós aceitam escritas e reconciliam os conflitos.

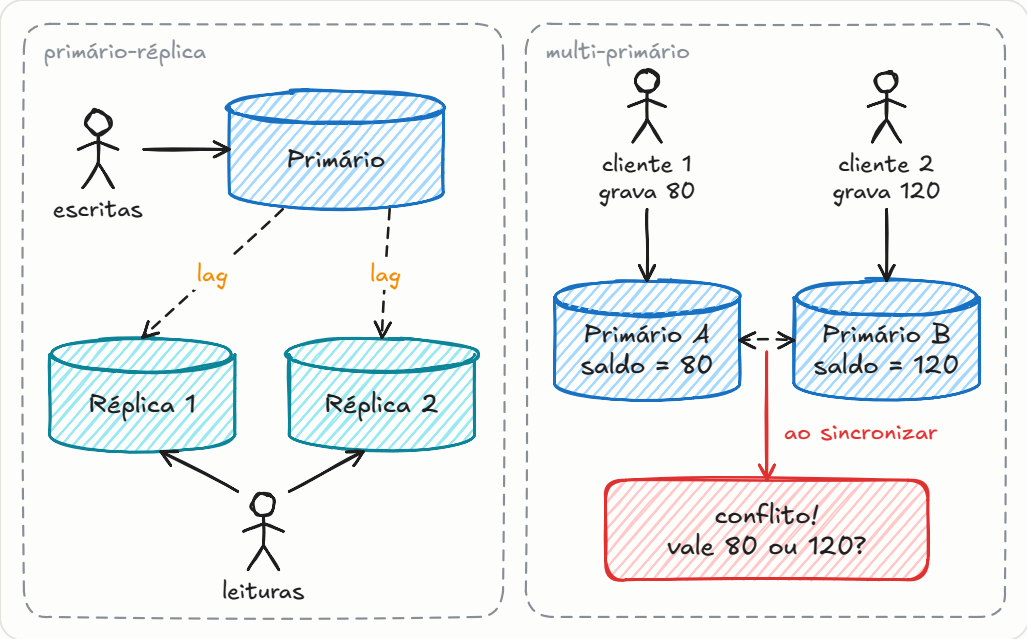
A replicação melhora o desempenho de leitura e a disponibilidade, mas torna a consistência mais difícil, principalmente quando as escritas vão para vários nós. Em entrevistas, espere falar sobre como o atraso de replicação (replication lag) afeta as leituras e como funciona o failover quando um master cai.

APROFUNDANDO

Primário-réplica

Toda escrita vai para o primário. Ele registra a mudança no próprio log (o WAL do cap. 27, ou o binlog no MySQL) e envia esse fluxo às réplicas, que aplicam as mudanças na mesma ordem. Como só existe um ponto de escrita, não há conflito. A questão é **quando** o primário confirma ao cliente:

- **Assíncrona:** confirma assim que grava localmente. É rápida, mas, se o primário morrer, as últimas escritas que ainda não chegaram às réplicas se perdem.
- **Síncrona:** só confirma depois que uma ou mais réplicas também confirmam. Não perde dados no failover, mas cada escrita paga um round trip a mais, e uma réplica lenta segura todo mundo.



À esquerda, um primário replica para duas réplicas com atraso; à direita, dois primários aceitam escritas concorrentes e entram em conflito.

O **replication lag** costuma ficar abaixo de um segundo, mas cresce para segundos ou minutos sob carga pesada. O sintoma clássico: o usuário troca a foto de perfil, a página recarrega lendo de uma réplica atrasada e mostra a foto antiga. A correção se chama read-your-writes: ler do primário os dados que o próprio usuário acabou de alterar, ou tudo dele por alguns segundos depois de cada escrita.

O **failover** tem três passos: detectar que o primário caiu (heartbeat, cap. 37), promover a réplica mais atualizada e redirecionar os clientes. O perigo é o split brain: o primário antigo volta achando que ainda manda, e passam a existir dois. Por isso, as ferramentas maduras usam consenso (cap. 38) e isolam (fencing) o nó antigo.

Multi-primário

Aqui vários nós aceitam escritas. Faz sentido quando usuários em várias regiões precisam escrever com baixa latência, ou quando o app precisa funcionar offline. O preço são os conflitos: se o saldo é R$ 100 e, ao mesmo tempo, uma região grava 80 e outra grava 120, qual vale? As opções são last-write-wins (o timestamp mais recente vence e a outra escrita some), mesclar na aplicação, usar CRDTs ou evitar o conflito mandando cada registro sempre para o mesmo primário "dono".

Analogia

Uma rede de lanchonetes tem o cardápio oficial na matriz (o primário). Só a matriz muda preços, e ela manda a tabela nova para as filiais (as réplicas). Até o e-mail chegar, quem pede um X-tudo na filial paga o preço antigo: é o replication lag. No modelo multi-primário, cada gerente pode mudar o preço e avisar os outros. É mais ágil, até o dia em que duas filiais mudam o preço do mesmo X-tudo na mesma hora.

NO MUNDO REAL

- O **PostgreSQL** replica enviando o WAL para os standbys (streaming replication). É assíncrona por padrão e vira síncrona com `synchronous_standby_names`. Ferramentas como o Patroni automatizam o failover apoiadas em etcd, Consul ou ZooKeeper.
- O **MySQL** replica pelo binlog, também de forma assíncrona por padrão, com opção semissíncrona. O Group Replication tem até um modo multi-primário.
- As **global tables do DynamoDB** aceitam escritas em várias regiões e, no modo padrão, resolvem conflitos por last-writer-wins.
- Em 2018, uma queda de rede de 43 segundos no **GitHub** disparou o failover do MySQL para a outra costa dos EUA. As duas costas ficaram com escritas que a outra não tinha, e o site passou mais de 24 horas degradado.

TRADE-OFFS

	Primário-réplica	Multi-primário
Escritas	Passam todas por um nó	Aceitas em vários nós, perto do usuário
Conflitos	Não existem: há uma ordem única	Inevitáveis; exigem regra de resolução
Leituras	Escalam com réplicas, sujeitas a lag	Escalam também, sujeitas a lag
Se um primário cai	Failover: promover uma réplica	Os outros primários seguem escrevendo

	Primário-réplica	Multi-primário
Bom para	A grande maioria dos sistemas	Multirregião, apps offline, edição colaborativa

NA ENTREVISTA

- Se as leituras dominam (digamos, 100 leituras por escrita), proponha réplicas de leitura antes de qualquer coisa mais exótica.
- Fale do lag sem esperar a pergunta: "Logo depois de uma escrita, as leituras daquele usuário vão para o primário, para ele não ver o dado antigo."
- Explique o failover em termos de negócio: com replicação assíncrona, você pode perder os últimos segundos de escritas (o RPO, quanto dado você aceita perder, não é zero); com síncrona, cada escrita fica mais lenta.
- Só proponha multi-primário com um motivo claro, como escrita em várias regiões, e uma estratégia de conflito.

CUIDADO COM

- **Replicação não é backup.** Um `DELETE` sem `WHERE` chega a todas as réplicas em milissegundos. Para isso existem backups e recuperação para um ponto no tempo.
- **Last-write-wins depende de relógios.** Com relógios dessincronizados, uma escrita mais antiga pode vencer, e a outra some sem erro nenhum.
- **Réplica que responde não é réplica em dia.** Um health check que só testa se ela está no ar pode mandar tráfego para uma cópia com minutos de atraso.
- `ATUALIZAÇÃO:` os termos master/slave caíram em desuso; hoje se diz primário/réplica ou líder/seguidor. O MySQL trocou `SHOW SLAVE STATUS` por `SHOW REPLICA STATUS`, e o Redis passou a usar `REPLICAOF`.

TESTE-SE

1 **O que é replication lag e qual o sintoma típico para o usuário?**

É o atraso entre uma escrita no primário e sua aplicação numa réplica. O sintoma típico é salvar um dado, recarregar a página e ver o valor antigo, porque a leitura foi para uma réplica atrasada.

2 **Um portal de notícias, numa única região, tem mil leituras para cada escrita. Que padrão de replicação você usa?**

Primário-réplica, com várias réplicas de leitura e cache na frente. As escritas cabem folgadas num primário, e as leituras escalam adicionando réplicas.

3 **"Tenho três réplicas, então não preciso de backup." Certo?**

Errado. Réplicas copiam também os erros: um `DROP TABLE` acidental chega a todas. Só um backup com recuperação para um ponto no tempo desfaz isso.

CONECTA COM

2 Teorema CAP · 7 Consistência forte vs. eventual · 27 Write-Ahead Logging (WAL) · 37 Heartbeat · 38 Eleição de líder (Paxos e Raft)

Hashing consistente

Consistent Hashing

EM UMA FRASE

Hashing consistente põe nós e chaves no mesmo anel e entrega cada chave ao próximo nó no sentido horário; quando um nó entra ou sai, só as chaves de um trecho do anel mudam de dono.

O ESSENCIAL · tradução do artigo original

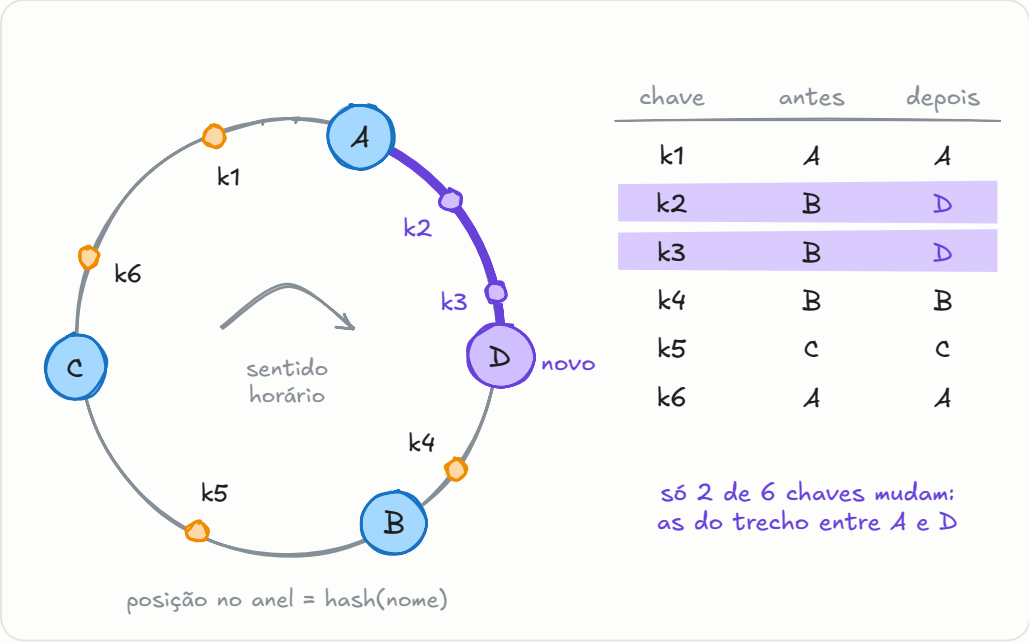
O hashing consistente é uma técnica para distribuir chaves entre nós de um jeito que minimiza a movimentação de dados quando nós são adicionados ou removidos. Chaves e nós são posicionados num anel lógico, e cada chave pertence ao próximo nó do anel. Quando um nó entra ou sai, só uma pequena parte das chaves precisa se mover. Essa propriedade é muito útil em caches e bancos de dados distribuídos. Pense nele como um mapeamento suave, que não se embaralha quando o tamanho do cluster muda.

APROFUNDANDO

O problema aparece com `hash(chave) mod N`. Com 4 servidores de cache e 1 milhão de chaves, passar para 5 servidores muda o dono de cerca de 800 mil chaves. Num cache, isso é uma onda de misses de uma vez só, e todas essas leituras caem no banco ao mesmo tempo.

O hashing consistente troca o `mod N` por um anel:

1. Pegue o espaço de saída da função de hash (por exemplo, de 0 a $2^{32} - 1$) e dobre-o num círculo.
2. Cada nó ganha uma posição no anel: `hash("servidor-A")`.
3. Cada chave também: `hash("usuario:42")`.
4. O dono da chave é o primeiro nó que você encontra andando no sentido horário a partir dela.



O nó D entra entre A e B; só as chaves do trecho entre A e D passam para ele, e o resto continua onde estava.

Quando o nó D entra, ele assume só as chaves entre o antecessor dele (A) e ele mesmo, que antes eram do sucessor (B). Nenhuma outra chave muda de dono. Quando um nó sai, as chaves dele vão para o próximo nó do anel. Em média, só K/N chaves se movem (K chaves, N nós): no exemplo do cache, uns 200 mil em vez de 800 mil.

Para achar o dono, basta guardar as posições dos nós numa lista ordenada e fazer uma busca binária, em O(log N).

Nós virtuais

Com poucos nós em posições aleatórias, os trechos do anel ficam desiguais, e um nó pode carregar o dobro de outro. Pior: quando um nó sai, toda a carga dele cai num único vizinho. A solução são os **nós virtuais (vnodes)**: cada servidor físico aparece em dezenas ou centenas de posições do anel (`hash("A#1")`, `hash("A#2")`...). A carga se espalha melhor, a saída de um nó divide as chaves dele entre muitos vizinhos, e uma máquina maior pode receber mais vnodes.

Em bancos como o Cassandra, o anel também define a replicação: cada chave fica no seu dono e nos próximos nós distintos no sentido horário.

Analogia

Imagine um condomínio com uma rua circular e algumas guaritas ao longo dela. Cada encomenda fica na primeira guarita que o entregador encontra andando no sentido horário a partir da casa do morador. Se o condomínio abre uma guarita nova, só as casas do trecho logo antes dela mudam de guarita; o resto dos moradores nem fica sabendo. Com a regra "casa número X vai para a guarita X mod N", abrir uma guarita mudaria o ponto de retirada de quase todo mundo.

NO MUNDO REAL

- A técnica nasceu num artigo de **Karger e colegas no MIT**, em 1997, pensado para caches distribuídos na web; dois dos autores fundaram a **Akamai** no ano seguinte.
- O **Dynamo** da Amazon (artigo de 2007) e o **Cassandra** usam um anel com nós virtuais para distribuir e replicar os dados.
- Clientes de **memcached** popularizaram o algoritmo **ketama**, que o **NGINX** usa na diretiva `hash ... consistent` para escolher servidores de upstream.
- O **Envoy** oferece as políticas de balanceamento ring hash e Maglev, esta inspirada no balanceador do Google: o mesmo cliente cai sempre no mesmo servidor, e quase nada muda quando servidores entram ou saem.

TRADE-OFFS

Você ganha:

- Adicionar ou remover um nó move só cerca de 1/N das chaves, em vez de quase todas.
- Crescer ou encolher o cluster sem invalidar o cache inteiro.
- Nenhum coordenador central: qualquer cliente que conheça a lista de nós calcula o dono.

Você paga:

- Distribuição desigual, se não houver nós virtuais.

- Mais complexidade que um `mod N`, e todos os clientes precisam ver a mesma lista de nós.
- Uma chave muito acessada continua sobrecarregando um nó só.

NA ENTREVISTA

- Traga o conceito sempre que houver um cache distribuído ou um banco particionado que precisa crescer: "Vou distribuir as chaves com hashing consistente, para que adicionar um nó mova só uma fração delas."
- Mostre a conta: com `mod N`, ir de 4 para 5 nós move 80% das chaves; com o anel, uns 20%.
- Fale de nós virtuais antes que o entrevistador pergunte sobre carga desigual.
- Se houver replicação, diga que as réplicas ficam nos próximos nós do anel.

CUIDADO COM

- **Hashing consistente não resolve chave quente.** Ele distribui chaves, não acessos: se uma chave recebe 30% do tráfego, um nó recebe esses 30%.
- **Nem todo sistema particionado usa anel.** O Redis Cluster, por exemplo, tem 16.384 hash slots fixos (`CRC16(chave) mod 16384`) e move slots inteiros entre os nós. Resolve o mesmo problema de outro jeito.
- `ATUALIZAÇÃO:` o anel de 1997 é a forma clássica, mas não a única. Rendezvous hashing, jump consistent hash (Google, 2014) e Maglev (Google, 2016) trazem distribuição mais uniforme ou consulta mais rápida e aparecem em sistemas modernos.

TESTE-SE

1 **Num anel de hashing consistente, a quem pertence uma chave?**

Ao primeiro nó encontrado andando no sentido horário a partir da posição da chave no anel.

2 **Seu cache distribuído tem 10 nós e vai ganhar mais 2 para a Black Friday. Por que usar hashing consistente em vez de `hash mod N`?**

Com `mod N`, passar de 10 para 12 nós troca o dono de cerca de 83% das chaves, gerando uma tempestade de misses no banco justamente no pico. Com o anel, só as chaves que os nós novos assumem, cerca de 1/6, mudam de lugar.

3 **Com hashing consistente e nós virtuais, uma chave muito popular deixa de ser problema?**

Não. Todas as requisições daquela chave continuam indo para o mesmo nó. Chave quente pede outra técnica, como replicá-la em vários nós ou guardar uma cópia em cache local.

CONECTA COM

12 Algoritmos de balanceamento · 22 Protocolo gossip · 23 Sharding · 41 Cache

Índices: B-Tree e LSM-Tree

Database Indexing (B Trees, LSM Trees)

EM UMA FRASE

Índices são estruturas extras que aceleram buscas; a B-Tree mantém páginas ordenadas e as atualiza no lugar, boa para leituras, e a LSM-Tree acumula escritas em memória e grava arquivos imutáveis, boa para escritas.

O ESSENCIAL · tradução do artigo original

Índices aceleram consultas ao organizar os dados de um jeito que permite encontrá-los rápido. B-Trees são árvores balanceadas que mantêm os dados ordenados e permitem encontrar faixas (intervalos) com eficiência; são comuns em bancos relacionais. LSM-Trees acumulam as escritas em memória e, de tempos em tempos, as descarregam no disco, o que torna as escritas muito rápidas, mas as leituras mais complexas. O trade-off é entre cargas com muita escrita e cargas com muita leitura. A ideia-chave é que índices são uma estrutura separada, que precisa ser atualizada a cada escrita, e é por isso que índices demais prejudicam o desempenho das inserções.

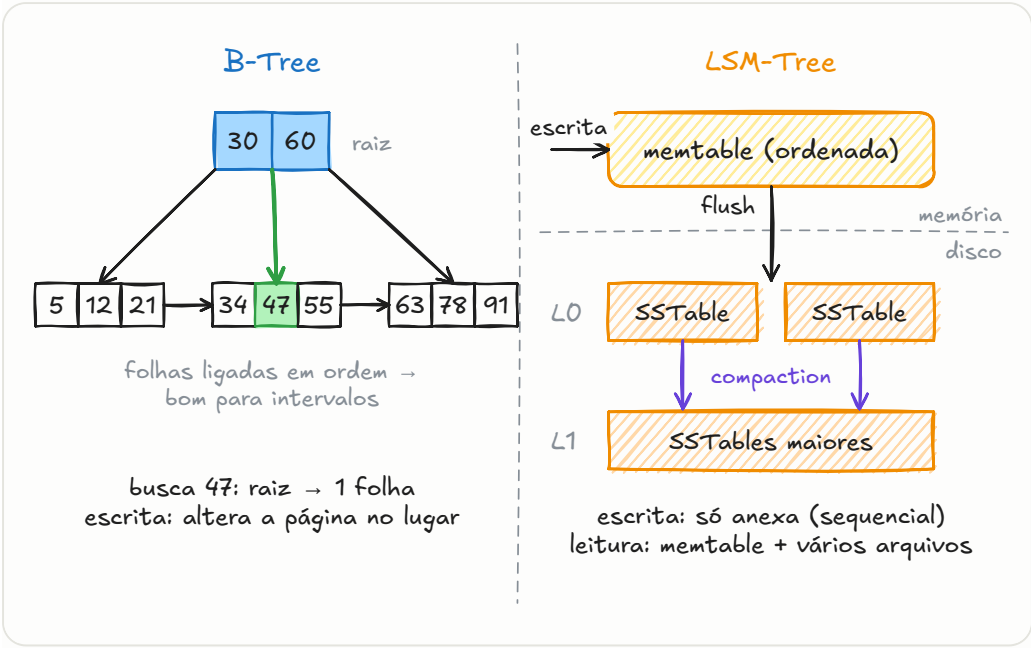
APROFUNDANDO

Sem índice, achar um cliente pelo e-mail numa tabela de 10 milhões de linhas exige ler todas elas (full table scan). Com índice, bastam algumas leituras de página.

B-Tree

O banco guarda o índice em **páginas** de tamanho fixo (8 KB no PostgreSQL, 16 KB no InnoDB do MySQL). Cada página interna tem centenas de chaves e ponteiros para as páginas filhas, então a árvore é larga e baixa: três ou quatro níveis bastam para centenas de milhões de linhas, e os níveis de cima quase sempre estão em memória. Buscar é descer da raiz até uma folha. As folhas são encadeadas em ordem, então uma consulta como `WHERE data BETWEEN ...` acha o começo da faixa e segue de folha em folha.

Escrever é atualizar a página certa **no lugar**; se ela enche, se divide em duas (page split). Cedo ou tarde, cada escrita vira a gravação de uma página inteira num ponto aleatório do disco, mesmo que a mudança seja de poucos bytes.



À esquerda, a B-Tree acha o 47 descendo da raiz a uma folha; à direita, a LSM-Tree acumula escritas na memtable, descarrega em SSTables e as junta por compaction.

LSM-Tree

A LSM-Tree (Log-Structured Merge-Tree) nunca altera um arquivo no lugar:

1. A escrita vai para o WAL (cap. 27) e para a **memtable**, uma estrutura ordenada em memória.
2. Quando a memtable atinge um limite (no RocksDB, 64 MB por padrão), ela é gravada inteira no disco como uma **SSTable**: um arquivo ordenado e imutável. É gravação sequencial, o que discos e SSDs fazem de mais rápido.
3. Em segundo plano, a **compaction** junta SSTables em arquivos maiores, organizados em níveis, e descarta valores sobrescritos e marcas de exclusão (tombstones).

O custo aparece na leitura: a chave pode estar na memtable ou em qualquer SSTable, da mais nova para a mais antiga. Para não abrir todos os arquivos, cada SSTable tem um índice esparso e um Bloom filter (cap. 30), que descarta na hora os arquivos que

com certeza não têm a chave.

Por que índices demais atrapalham

Cada índice secundário é mais uma árvore para atualizar. Uma tabela com seis índices secundários faz sete atualizações a cada `INSERT`: a tabela e os seis índices. O índice acelera a leitura que ele atende e encarece toda escrita.

> *Analogia*
>
> Pense em dois jeitos de organizar as figurinhas do álbum da Copa. No primeiro, você cola cada figurinha no lugar certo assim que abre o pacote: dá trabalho a cada pacote, mas achar a número 237 é imediato (B-Tree). No segundo, você junta as figurinhas do dia num envelope, ordena e guarda; de vez em quando, junta os envelopes antigos num só e joga fora as repetidas (LSM-Tree). Abrir pacote fica rapidíssimo, mas achar a 237 exige olhar vários envelopes.

NO MUNDO REAL

- A **B-Tree**, na variante B+Tree, é o índice padrão do PostgreSQL, do MySQL, do SQL Server e do Oracle. No InnoDB, a própria tabela é guardada numa B+Tree ordenada pela chave primária.
- A **LSM-Tree** está no Cassandra, no ScyllaDB, no HBase e no RocksDB. O RocksDB, por sua vez, roda por baixo de outros sistemas, como o TiKV (do TiDB) e o MyRocks, motor de armazenamento do MySQL que a Meta usa no banco do seu grafo social.
- O artigo do **Bigtable** (Google, 2006) popularizou os termos memtable e SSTable.

TRADE-OFFS

	B-Tree	LSM-Tree
Escrita	Atualiza páginas no lugar (I/O aleatório)	Só anexa e grava em lote (I/O sequencial)
Leitura de uma chave	Um caminho raiz → folha, previsível	Memtable e vários arquivos, com ajuda de Bloom filters

	B-Tree	LSM-Tree
Consulta por faixa	Excelente: folhas encadeadas	Boa, mas junta dados de vários arquivos
Trabalho em segundo plano	Pouco	Compaction constante, que consome disco e CPU
Bom para	Leituras e transações (OLTP clássico)	Escrita intensa: logs, métricas, eventos, IoT

NA ENTREVISTA

- Num sistema com muita escrita (telemetria, logs, mensagens), diga: "Prefiro um motor LSM, como Cassandra ou RocksDB, porque ele transforma escritas aleatórias em sequenciais."
- Para leitura com consultas por faixa, B-Tree é o padrão seguro.
- Mostre que conhece o custo de cada índice: "Vou indexar só as colunas das consultas críticas, porque cada índice encarece todas as escritas."
- Ponto extra: citar os Bloom filters que poupam leituras inúteis nas LSM-Trees.

CUIDADO COM

- **A ordem das colunas num índice composto importa.** Um índice em `(cliente_id, data)` serve para buscar por cliente, ou por cliente e data, mas ajuda pouco ou nada numa busca só por data.
- **Índice não é mágico.** Numa coluna com poucos valores distintos (um `status` com três valores bem distribuídos), o banco prefere ler a tabela inteira e ignora o índice.
- `ATUALIZAÇÃO:` "B-Tree para leitura, LSM para escrita" é uma regra de bolso, não uma lei. Com Bloom filters e cache, motores LSM atendem bem muitas cargas mistas (o MyRocks da Meta é um exemplo), e o custo que mais aparece na prática é a compaction, que disputa disco com o tráfego e causa picos de latência.

TESTE-SE

1 O que é uma SSTable?

Um arquivo imutável, ordenado por chave, que a LSM-Tree grava no disco quando a memtable enche. Ele nunca é alterado; a compaction o junta com outros em arquivos novos.

2 Você vai guardar as leituras de 1 milhão de sensores, cada um enviando um valor por segundo. B-Tree ou LSM-Tree?

LSM-Tree. São um milhão de escritas por segundo, quase todas inserções, e transformá-las em gravações sequenciais em lote é exatamente o forte da LSM.

3 Criar um índice para cada coluna da tabela deixa o sistema mais rápido?

Só as leituras que usam esses índices. Cada `INSERT`, `UPDATE` e `DELETE` passa a manter todos eles em dia, o que deixa as escritas mais lentas e ocupa mais disco e memória.

CONECTA COM

5 Throughput vs. latência
27 Write-Ahead Logging (WAL)
30 Bloom filter

Write-Ahead Logging (WAL)

Write Ahead Logging (WAL)

EM UMA FRASE

No WAL, o banco grava cada mudança primeiro num log sequencial e o força para o disco; só depois confirma e atualiza os arquivos de dados, e, após uma queda, refaz o log.

O ESSENCIAL · tradução do artigo original

O Write-Ahead Logging registra as mudanças num log antes de aplicá-las ao banco de dados principal. Se uma queda acontecer no meio de uma transação, o sistema pode refazer (replay) o log para restaurar um estado consistente. O WAL garante a durabilidade e a atomicidade das transações. Ele também permite técnicas como a replicação a partir do fluxo do log. Vou explicar por que isso é importante: sem WAL, uma queda poderia deixar seus dados num estado corrompido, atualizado pela metade.

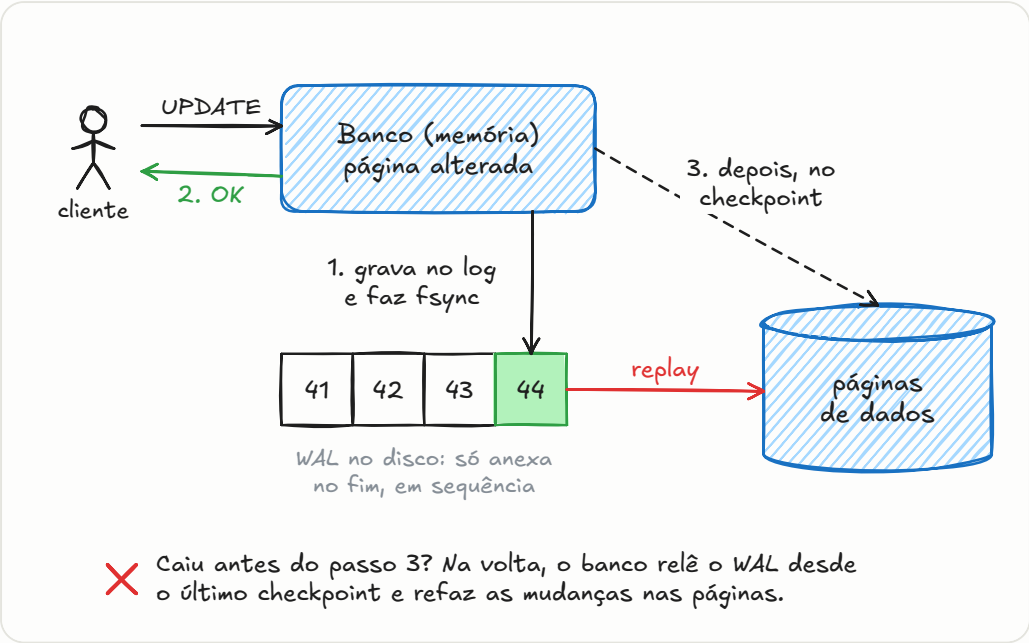
APROFUNDANDO

Um único `UPDATE` pode mexer em várias páginas: a da tabela e as de cada índice. Gravar essas páginas direto nos arquivos de dados a cada commit tem dois problemas. É lento, porque são escritas aleatórias em vários pontos do disco. E é perigoso: se a luz cai depois de gravar uma página e antes da outra, o índice aponta para uma linha que não existe.

O WAL resolve as duas coisas com uma regra: **nenhuma mudança chega aos arquivos de dados antes de estar descrita no log, no disco.** O caminho de uma escrita fica assim:

1. O banco altera a página **em memória** e anexa ao fim do log um registro descrevendo a mudança. No commit, força o log para o disco com `fsync`.
2. Só então confirma ao cliente. A transação já é durável, mesmo que nenhum arquivo de dados tenha mudado.

3. Mais tarde, num **checkpoint**, as páginas alteradas são gravadas nos arquivos de dados, em lote.



O banco grava o registro no WAL e faz fsync, confirma ao cliente e só depois grava as páginas; numa queda, refaz o log.

Anexar ao fim de um arquivo é escrita sequencial, o tipo mais barato para o disco. E vários commits simultâneos podem dividir um único `fsync` (group commit), o que multiplica o throughput.

Cada registro tem um número crescente, o **LSN** (log sequence number), e cada página guarda o LSN da última mudança aplicada nela. Na recuperação, o banco começa no último checkpoint, relê o log e reaplica o que ainda não está nas páginas. Transações que não chegaram a registrar o commit são descartadas ou desfeitas. O log anterior ao checkpoint deixa de ser necessário para isso e pode ser reciclado, a menos que réplicas ou o arquivamento ainda precisem dele. Checkpoints mais frequentes encurtam a recuperação, ao custo de mais gravações.

O log também é um fluxo ordenado de tudo o que mudou. Mande-o para outra máquina e você tem uma réplica (cap. 24); leia-o com uma ferramenta de change data capture (CDC) e você publica cada mudança para outros sistemas (cap. 29).

Analogia

O caixa de uma padaria anota cada venda num caderno, sempre na próxima linha, na hora em que ela acontece. A planilha de estoque, que dá mais trabalho, ele só atualiza no fim do dia. Se a luz acabar às três da tarde, nada se perde: quando ela volta, o caixa relê o caderno a partir do último fechamento e acerta a planilha. O caderno é o WAL; a planilha, os arquivos de dados.

NO MUNDO REAL

- O **PostgreSQL** grava o WAL na pasta `pg_wal`, em segmentos de 16 MB por padrão, e usa o mesmo log na streaming replication e na recuperação para um ponto no tempo.
- O **InnoDB**, do MySQL, tem o redo log. Com `innodb_flush_log_at_trx_commit = 1`, o padrão, ele força o log para o disco a cada commit.
- O **SQLite** tem um modo WAL que permite que leitores e o escritor trabalhem ao mesmo tempo.
- O **Debezium** lê o WAL do PostgreSQL e o binlog do MySQL para publicar cada mudança no Kafka.

TRADE-OFFS

Você ganha:

- Durabilidade: o que foi confirmado sobrevive a uma queda.
- Recuperação sem nada atualizado pela metade.
- Commits rápidos: um append sequencial agora, as páginas em lote depois.
- Replicação e CDC lendo o mesmo log.

Você paga:

- Cada mudança é gravada duas vezes: no log e, depois, nos arquivos de dados.
- A latência de um `fsync` por commit (ou o risco de perder dados, se você relaxar isso).
- Espaço em disco para o log e uma recuperação que cresce com o log desde o último checkpoint.

NA ENTREVISTA

- Quando o assunto for durabilidade, diga: “A escrita só é confirmada depois que o registro está no log, no disco. Se a máquina cair, o banco refaz o log na volta.”
- Ligue o WAL à replicação: “As réplicas recebem esse mesmo log e o aplicam na mesma ordem.”
- Se o assunto for desempenho, mostre o trade-off: relaxar o `fsync` por commit (como o `synchronous_commit = off` do PostgreSQL) aumenta o throughput, mas aceita perder as últimas transações confirmadas numa queda, ainda que sem corromper o banco.

CUIDADO COM

- **WAL não é backup.** Ele protege contra quedas, não contra um disco que morre nem contra um `DELETE` errado. Para isso, backups e arquivamento do log.
- **“Confirmado” depende do `fsync`.** Configurações que só forçam o disco de tempos em tempos trocam durabilidade por velocidade: o Redis com AOF em `appendfsync everysec`, por exemplo, pode perder cerca de um segundo de escritas.
- `ATUALIZAÇÃO:` o artigo diz que o WAL garante a atomicidade e a durabilidade. Ele é a base, mas não age sozinho: a atomicidade também depende de como o banco desfaz ou esconde transações incompletas (undo log no InnoDB, MVCC no PostgreSQL), e a durabilidade depende de o disco realmente honrar o `fsync`.

TESTE-SE

1 **Qual é a regra central do write-ahead logging?**

Nenhuma mudança é gravada nos arquivos de dados antes de estar registrada no log, no disco. E a transação só é confirmada depois que o registro do commit foi forçado para o disco.

2 **O banco cai logo depois de confirmar uma transação, antes de gravar as páginas de dados. A transação se perde?**

Não. O registro dela já estava no WAL, no disco. Na volta, o banco relê o log a partir do último checkpoint e reaplica a mudança nas páginas.

3 **Se o WAL faz cada mudança ser gravada duas vezes, por que ele deixa o banco mais rápido, e não mais lento?**

Porque o commit só exige um append sequencial no log, em vez de várias gravações aleatórias em páginas espalhadas. As páginas vão para o disco depois, em lote, e vários commits podem dividir o mesmo `fsync`.

CONECTA COM

4 ACID vs. BASE
24 Padrões de replicação
26 Índices: B-Tree e LSM-Tree
29 Persistência poliglota

Normalização vs. desnormalização

Normalization vs Denormalization

EM UMA FRASE

Normalizar é guardar cada fato num só lugar para evitar anomalias; desnormalizar é duplicar dados de propósito para ler sem joins, aceitando o trabalho de manter as cópias em dia.

O ESSENCIAL · tradução do artigo original

- A normalização organiza os dados em tabelas que reduzem a redundância e as dependências, seguindo regras como a primeira forma normal, a segunda forma normal e assim por diante. Isso evita anomalias em atualizações e inserções.
- A desnormalização duplica dados de propósito para acelerar as leituras e reduzir joins. Em sistemas de grande escala, desnormalizar é comum nos caminhos com muita leitura, como guardar o nome do usuário junto com os posts em vez de fazer um join toda vez.

A verdadeira habilidade é saber onde você pode desnormalizar com segurança sem quebrar a consistência.

APROFUNDANDO

As formas normais, sem decoreba

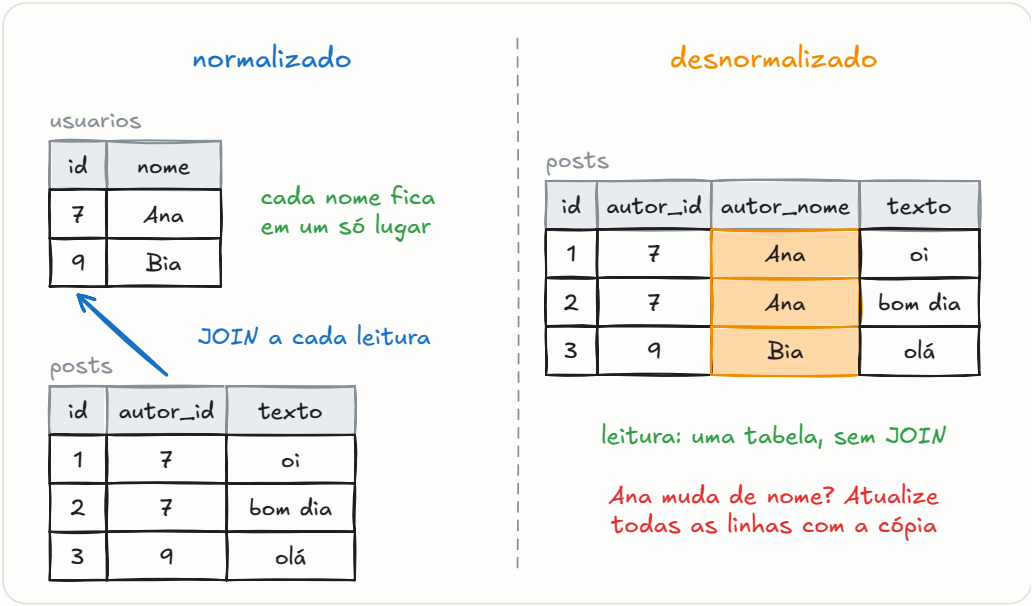
Cada forma normal elimina um tipo de repetição:

- **1FN:** cada coluna guarda um valor atômico. Nada de `telefones = "1111, 2222"` numa célula só.
- **2FN:** numa tabela de chave composta, nenhuma coluna depende só de parte da chave. Em `itens_pedido(pedido_id, produto_id, quantidade, nome_produto)`, o nome depende só do produto, então sai dali.
- **3FN:** nenhuma coluna depende de outra coluna que não é chave. Em `clientes(id, cep, cidade)`, a cidade depende do CEP, não do cliente.

O resumo clássico: cada coluna depende "da chave, da chave inteira e de nada além da chave".

As anomalias que ela evita

Se o endereço do cliente está copiado em 300 pedidos, mudar o endereço exige atualizar 300 linhas; se uma ficar para trás, o banco passa a ter dois endereços para a mesma pessoa (**anomalia de atualização**). Há também as de inserção (se produto e pedido moram na mesma tabela, não dá para cadastrar um produto que nunca foi vendido) e as de exclusão (apagar o último pedido apaga o único registro do cliente).



Normalizado, o nome do autor fica só na tabela de usuários e a leitura faz JOIN; desnormalizado, o nome vai copiado em cada post.

Quando desnormalizar

Num feed, mostrar 20 posts com o nome de cada autor exige um JOIN a cada leitura. Com milhares de leituras por segundo, ou com os usuários em outro shard (cap. 23), esse JOIN pesa. Copiar `autor_nome` para dentro do post resolve a leitura, mas cria uma obrigação: quando alguém muda de nome, todas as cópias precisam ser atualizadas, em geral de forma assíncrona.

Outras formas comuns: contadores pré-calculados (`total_curtidas`), views materializadas e documentos que embutem os dados lidos juntos.

A desnormalização segura tem três características: o dado muda pouco, um atraso na atualização das cópias é aceitável e existe uma **fonte da verdade** normalizada, a partir da qual as cópias podem ser reconstruídas.

Analogia

Numa escola, a secretaria guarda o cadastro de cada aluno, e o diário de classe usa só o número de matrícula: para saber quem é o 17, o professor consulta a secretaria (normalizado). Se cada diário trouxer nome, telefone e responsável de cada aluno (desnormalizado), a consulta fica mais rápida, mas quando uma família troca de telefone, alguém precisa corrigir todos os diários. O que ficar esquecido vira ligação para o número errado.

NO MUNDO REAL

- Bancos transacionais como **PostgreSQL** e **MySQL** costumam ser modelados até a 3FN: é o ponto de partida padrão para dados que mudam muito.
- **Data warehouses** seguem o caminho oposto: o modelo dimensional (esquema estrela) de Ralph Kimball desnormaliza as dimensões de propósito, porque ali quase tudo é leitura.
- **Cassandra** e **DynamoDB** são modelados a partir das consultas: é comum ter a mesma informação em várias tabelas, uma para cada padrão de acesso.
- No **MongoDB**, a recomendação é embutir no mesmo documento os dados que são lidos juntos.

TRADE-OFFS

	Normalizado	Desnormalizado
Leitura	Exige joins, que pesam em escala	Uma consulta, sem join
Escrita	Cada fato muda num lugar só	Várias cópias para atualizar
Consistência	Garantida pelo próprio modelo	Depende do código que sincroniza as cópias
Espaço	Menor	Maior

	Normalizado	Desnormalizado
Consultas novas	Flexível para perguntas que ninguém previu	Otimizado para as consultas que já existem

NA ENTREVISTA

- Comece normalizado e desnormalize só os caminhos quentes, com um motivo: “O feed tem 50 mil leituras por segundo; vou copiar o nome e a foto do autor no post.”
- Diga sempre **como** as cópias ficam em dia: na mesma transação, por um evento assíncrono ou por um job que reconstrói tudo.
- Deixe claro qual é a fonte da verdade e quanto atraso cada cópia tolera. É isso que o entrevistador quer ouvir.

CUIDADO COM

- **Nem toda cópia é desnormalização.** O preço guardado no item do pedido é o preço daquele momento, um fato diferente do preço atual do catálogo. Ele deve ser copiado e não deve mudar.
- **JOIN não é vilão.** Com índices, juntar poucas linhas é barato. O problema aparece em volumes muito altos ou quando as tabelas estão em shards diferentes.
- **Cópia sem responsável pela sincronização** é inconsistência com hora marcada. Se nenhum processo atualiza as cópias, elas vão divergir.

TESTE-SE

1 **O que é uma anomalia de atualização?**

É quando o mesmo fato está guardado em várias linhas e uma atualização muda só parte delas, deixando o banco com informações contraditórias, como dois endereços para o mesmo cliente.

2 **Um feed mostra o nome do autor em cada post e recebe 20 mil leituras por segundo. Nomes mudam raramente. O que você faz?**

Desnormalizo: guardo o nome do autor junto do post (ou no cache do feed) e, quando alguém troca de nome, atualizo as cópias de forma assíncrona. A troca é rara, e um atraso de alguns minutos é aceitável.

3 **Copiar o preço do produto para a linha do pedido é desnormalização?**

Não. O preço no pedido registra o valor cobrado naquele momento, um fato do próprio pedido. Ele precisa ser copiado justamente para não mudar quando o catálogo mudar.

CONECTA COM

4 ACID vs. BASE · 23 Sharding · 29 Persistência poliglota · 41 Cache

Persistência poliglota

Polyglot Persistence

EM UMA FRASE

Persistência poliglota é usar vários tipos de banco no mesmo sistema, cada um no papel em que é melhor, pagando com mais operação, mais conhecimento no time e o trabalho de manter os dados sincronizados.

O ESSENCIAL · tradução do artigo original

Persistência poliglota significa usar vários tipos de banco de dados dentro do mesmo sistema, cada um escolhido por aquilo que faz melhor. Você pode usar um banco relacional para transações, um banco de documentos para logs, um banco chave-valor para cache e um banco de grafos para relacionamentos. Em vez de forçar tudo para dentro de um único banco, você escolhe a ferramenta certa para cada trabalho. O trade-off é mais complexidade operacional e mais conhecimento exigido do time.

APROFUNDANDO

O termo, popularizado por Martin Fowler, empresta a ideia da programação poliglota: assim como um sistema pode ter várias linguagens, pode ter vários bancos. O ponto de partida é o **padrão de acesso** de cada tipo de dado. Num e-commerce hipotético:

Dado	Como é usado	Escolha típica
Pedidos e pagamentos	Transações ACID, consistência forte	Relacional (PostgreSQL, MySQL)
Sessões e carrinho	Chave → valor, em milissegundos	Chave-valor em memória (Redis)
Busca no catálogo	Texto livre, relevância, filtros	Motor de busca (Elasticsearch, OpenSearch)
“Quem comprou isto também comprou”	Percorrer relacionamentos	Grafo (Neo4j)

Dado	Como é usado	Escolha típica
Fotos dos produtos	Arquivos grandes e imutáveis	Object storage (S3)
Relatórios	Varrer bilhões de linhas por coluna	Data warehouse (BigQuery, Redshift)

O problema que ninguém mostra no slide

Os bancos precisam concordar entre si: o produto cadastrado no PostgreSQL tem que aparecer na busca. A tentação é a **escrita dupla** (dual write): o código grava no PostgreSQL e, em seguida, no Elasticsearch. Se a segunda escrita falhar, ou o processo cair entre as duas, os bancos divergem em silêncio.

As saídas confiáveis partem de uma **fonte da verdade**:

- **Change data capture (CDC):** uma ferramenta lê o log do banco principal (o WAL do cap. 27) e publica cada mudança num tópico (cap. 45); consumidores atualizam a busca, o cache e o warehouse.
- **Outbox transacional:** na mesma transação do pedido, o serviço grava um evento numa tabela `outbox`; um processo separado lê essa tabela e publica os eventos.

Assim, os bancos derivados (busca, cache, warehouse) ficam reconstruíveis: se algo der errado, você os recria a partir da fonte.

O custo de cada banco novo

Cada tecnologia a mais é mais um sistema para fazer backup, monitorar, atualizar, proteger e entender às três da manhã, quando ele cai. Por isso, a regra prática é começar com o mínimo (muitas vezes, um PostgreSQL resolve quase tudo) e acrescentar um banco especializado quando surgir um requisito que o atual não atende bem.

Analogia

Uma família guarda a vida em vários lugares: o dinheiro no banco, os documentos na pasta do armário, as fotos na nuvem, os contatos no celular. Cada um é ótimo no seu papel, e ninguém guardaria fotos no internet banking. O preço aparece na mudança de endereço: é preciso avisar o banco, o plano de saúde e a escola das crianças, um por um. Esquecer um deles é a versão doméstica da escrita dupla que falhou.

NO MUNDO REAL

- A combinação de **PostgreSQL ou MySQL com Redis e Elasticsearch/OpenSearch** é tão comum em aplicações web que virou quase um padrão.
- A **Netflix** usa o Cassandra para grandes volumes de dados e o EVCache, um cache distribuído baseado em memcached, entre outros armazenamentos.
- O **Debezium** é a ferramenta open source mais conhecida de CDC: lê o log do PostgreSQL, do MySQL e de outros bancos e publica as mudanças no Kafka.
- A **AWS** defende abertamente bancos "purpose-built" (feitos para um propósito) e oferece um para cada modelo, como DynamoDB (chave-valor), DocumentDB (documentos) e Neptune (grafos).

TRADE-OFFS

Você ganha:

- Cada tipo de dado num banco feito para o padrão de acesso dele.
- Desempenho e escala melhores onde importa: busca, cache, análise.
- Liberdade para evoluir uma parte sem mexer nas outras.

Você paga:

- Mais sistemas para operar, monitorar, proteger e pagar.
- Sincronização entre bancos, com consistência eventual entre eles.
- Um time que precisa dominar várias tecnologias, e mais pontos de falha.

NA ENTREVISTA

- Propor dois ou três bancos num desenho de sistema é natural. O que conta é justificar cada um pelo padrão de acesso: “Pedidos no PostgreSQL, pelas transações; busca no OpenSearch, pela relevância; sessões no Redis, pela latência.”
- Diga qual é a fonte da verdade, como os outros bancos são alimentados (CDC ou outbox) e o que o usuário vê enquanto eles estão atrasados.
- Mostre parcimônia: “Não adicionaria um banco de grafos só para recomendações simples; uma consulta recursiva no PostgreSQL resolve no começo.”

CUIDADO COM

- **Escrita dupla não é sincronização.** Gravar em dois bancos no mesmo código, sem uma transação que cubra os dois, leva a divergências que ninguém percebe até o cliente reclamar.
- **Banco derivado não é fonte da verdade.** Índices de busca e caches precisam poder ser apagados e reconstruídos.
- ATUALIZAÇÃO: a fronteira entre os tipos de banco ficou borrada. O PostgreSQL tem JSONB, busca textual e, com o pgvector, busca vetorial; MongoDB, Redis e Elasticsearch também ganharam busca vetorial. Hoje, muitos times consolidam no que já têm e só partem para um banco especializado quando o requisito exige.

TESTE-SE

1 **O que é persistência poliglota?**

É usar mais de um tipo de banco de dados no mesmo sistema, escolhendo cada um pelo padrão de acesso que ele atende melhor.

2 **Num marketplace, onde você guardaria os pedidos, a busca de produtos e as sessões dos usuários?**

Pedidos num banco relacional, pelas transações ACID; a busca num motor como Elasticsearch ou OpenSearch, pela relevância e pelos filtros; as sessões num chave-valor em memória, como o Redis, pela latência.

3 **"Meu serviço grava o produto no PostgreSQL e logo depois no Elasticsearch, então os dois sempre batem." Certo?**

Não. Se a segunda escrita falhar, ou o processo cair entre as duas, os bancos divergem. O seguro é ter uma fonte da verdade e propagar as mudanças por CDC ou por outbox.

CONECTA COM

9 Microsserviços vs. monólito

27 Write-Ahead Logging (WAL)

28 Normalização vs. desnormalização

31 Bancos de dados vetoriais

45 Pub/Sub

Bloom filter

Bloom Filters

EM UMA FRASE

Um Bloom filter é um array de bits com algumas funções de hash que diz, gastando pouquíssima memória, se um item talvez esteja num conjunto ou se com certeza não está, sem nunca dar falso negativo.

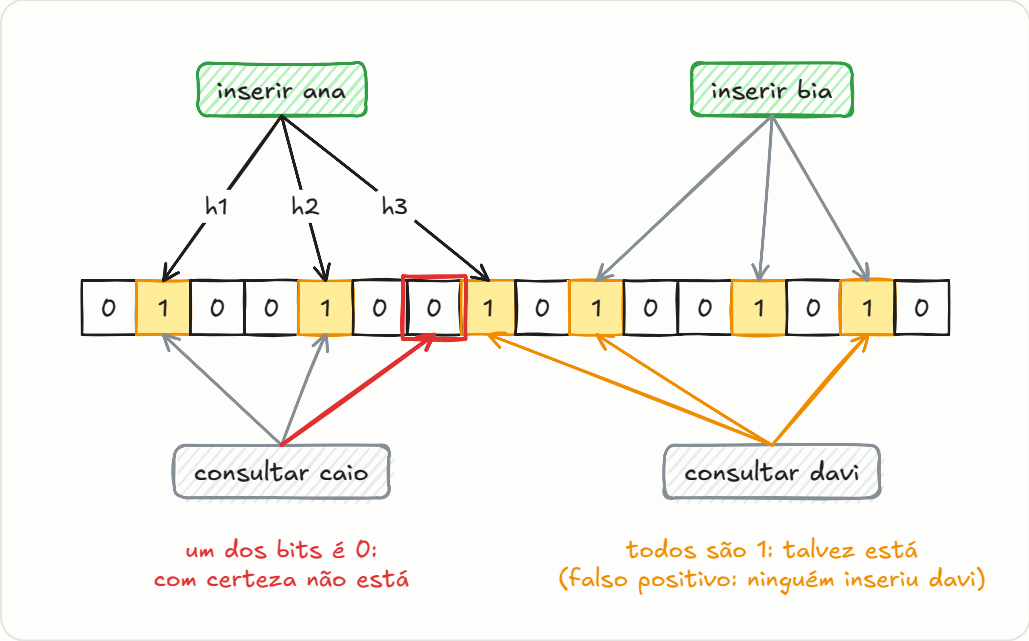
O ESSENCIAL · tradução do artigo original

Um Bloom filter é uma estrutura de dados eficiente em espaço que responde rapidamente à pergunta "este item pode estar no conjunto?", com possíveis falsos positivos, mas nenhum falso negativo. Ele usa várias funções de hash para ligar bits num array de bits quando os itens são inseridos. Para verificar se um item pertence ao conjunto, você testa os mesmos bits: se algum bit for zero, o item com certeza não está lá. Bancos de dados e caches usam Bloom filters para evitar buscas desnecessárias no disco ou cache misses. Pense neles como porteiros rápidos que dizem "com certeza não" ou "talvez".

APROFUNDANDO

Um Bloom filter tem só duas peças: um array de **m bits**, todos começando em zero, e **k funções de hash**, cada uma levando um item a uma posição do array.

- **Inserir:** calcule as k posições do item e ligue esses bits.
- **Consultar:** calcule as mesmas k posições. Se algum bit estiver desligado, o item **com certeza não** foi inserido, porque a inserção teria ligado todos. Se todos estiverem ligados, a resposta é **talvez**: outros itens podem ter ligado aqueles bits.



Inserir ana e bia liga três bits cada; a consulta de caio acha um bit zerado e responde "com certeza não", e a de davi só acha bits ligados e responde "talvez".

No diagrama, o array tem 16 bits e há três funções de hash. Ana liga os bits 1, 4 e 7 (contando do zero), e Bia liga 9, 12 e 14. Caio cai em 1, 4 e 6: o bit 6 está desligado, então ele com certeza não está no conjunto. Davi cai em 7, 9 e 14, todos ligados por outros itens: o filtro responde "talvez", e é um **falso positivo**.

Quanto erra e quanto ocupa

A taxa de falsos positivos (p) depende de quantos itens você inseriu (n), do tamanho do array (m) e do número de funções de hash (k):

```
p ≈ (1 - e^(-k·n/m))^k
```

A regra de bolso: com cerca de **10 bits por item** e 7 funções de hash, a taxa fica perto de **1%**. Para um bilhão de itens, isso dá pouco mais de 1 GB, contra 32 GB só para guardar IDs de 32 bytes. E o filtro nunca guarda os itens, só bits.

A taxa sobe à medida que o filtro enche. Inserir muito mais itens do que o planejado liga quase todos os bits, e o filtro passa a dizer "talvez" para tudo. Por isso ele é dimensionado para um número esperado de itens.

O uso clássico

O filtro fica na frente de algo caro. Numa LSM-Tree (cap. 26), cada SSTable tem o seu Bloom filter em memória. Para buscar uma chave, o banco consulta o filtro de cada arquivo e só lê do disco os que responderam "talvez". Uma busca por uma chave inexistente, que poderia abrir dezenas de arquivos, quase nunca toca o disco.

> *Analogia*
>
> Pense num teste de triagem que nunca deixa passar quem tem a doença, mas às vezes acusa quem não tem. Resultado negativo: você vai para casa tranquilo. Resultado positivo: você faz o exame completo, caro e demorado, para confirmar. O Bloom filter é essa triagem barata na frente do disco: só manda para a consulta cara quem pode mesmo estar lá.

NO MUNDO REAL

- **Cassandra**, **HBase** e **RocksDB** mantêm um Bloom filter por arquivo de dados; no Cassandra, a taxa de falsos positivos é configurável por tabela (`bloom_filter_fp_chance`).
- O artigo do **Bigtable** (Google, 2006) já descrevia Bloom filters para evitar idas ao disco atrás de linhas que não existem.
- A **Akamai** documentou o uso de Bloom filters para não ocupar o cache com "one-hit wonders", objetos pedidos uma única vez: um objeto só entra no cache quando é pedido pela segunda vez.
- O **Redis** oferece Bloom filters prontos (`BF.ADD`, `BF.EXISTS`), úteis, por exemplo, para checar se um nome de usuário já foi usado antes de ir ao banco.

TRADE-OFFS

Você ganha:

- Memória mínima: uns 10 bits por item para 1% de erro, seja o item um número ou uma URL inteira.
- Inserção e consulta em tempo constante (k cálculos de hash).
- Um "com certeza não" confiável, que poupa idas ao disco, à rede ou ao banco.

Você paga:

- Falsos positivos, que pedem uma verificação de verdade depois.
- Nada de remover itens: desligar um bit pode "apagar" outros.
- Nada de listar o conteúdo, e o tamanho precisa ser planejado antes.

NA ENTREVISTA

- Traga o Bloom filter quando houver muitas consultas por coisas que provavelmente **não** existem: "Antes de ir ao banco checar se o nome de usuário está livre, consulto um Bloom filter; se ele disser 'com certeza não', respondo na hora."
- Saiba dizer de que lado fica o erro: falso positivo pode, falso negativo nunca.
- Cite os usos clássicos: LSM-Trees e crawlers ("esta URL já foi visitada?").
- Se perguntarem sobre remoção: counting Bloom filter ou cuckoo filter.

CUIDADO COM

- **Não inverta a garantia.** O "talvez" pode estar errado; o "com certeza não", nunca. Uma estrutura que às vezes "esquece" itens inseridos não é um Bloom filter.
- **Filtro lotado perde a graça.** Passar muito do número de itens planejado faz a taxa de falsos positivos disparar. Dimensione com folga ou use um filtro que cresce em camadas.
- ATUALIZAÇÃO: o artigo diz que Bloom filters evitam cache misses. Mais preciso: eles evitam consultas inúteis, como pedidos de chaves que não existem e que, sem o filtro, passariam pelo cache e chegariam ao banco toda vez (o chamado cache penetration).

TESTE-SE

1 **Um Bloom filter pode dar falso negativo?**

Não. Se o item foi inserido, todos os bits dele foram ligados e nunca são desligados, então a consulta sempre responde "talvez". O único erro possível é o falso positivo.

2 **Seu crawler já visitou 5 bilhões de URLs e precisa evitar revisitá-las sem guardar todas em memória. O que você usa?**

Um Bloom filter das URLs visitadas. Com uns 10 bits por URL, são cerca de 6 GB para 1% de falsos positivos, e o custo do erro é só deixar de visitar algumas URLs novas.

3 **Dá para remover um item de um Bloom filter desligando os bits dele?**

Não. Esses bits podem ter sido ligados também por outros itens, e desligá-los criaria falsos negativos. Para remover, use um counting Bloom filter (contadores no lugar de bits) ou um cuckoo filter.

CONECTA COM

15 CDN · 26 Índices: B-Tree e LSM-Tree · 41 Cache

Bancos de dados vetoriais

Vector Databases

EM UMA FRASE

Bancos vetoriais guardam embeddings, vetores de números que representam o significado de textos, imagens ou áudios, e respondem consultas por semelhança encontrando os vetores mais próximos, quase sempre de forma aproximada.

O ESSENCIAL · tradução do artigo original

Bancos de dados vetoriais armazenam e consultam vetores, que são representações numéricas de dados como texto, imagens ou áudio. Esses vetores vêm de modelos, como os de embeddings, e permitem a busca por similaridade, como "encontre os documentos mais parecidos com este". Em vez de comparações exatas de igualdade, eles usam métricas de distância, como a similaridade de cosseno ou a distância euclidiana. Isso é essencial para os sistemas modernos de busca, recomendação e assistentes de IA. Em entrevistas, basta saber que bancos vetoriais suportam a busca por vizinhos mais próximos (nearest neighbor search) em dados de alta dimensão.

APROFUNDANDO

De dado para vetor

Um modelo de embeddings transforma um item num vetor de tamanho fixo, em geral de algumas centenas a alguns milhares de números (384, 768, 1.536...). O modelo é treinado para que itens de significado parecido fiquem perto nesse espaço: "cachorro" e "cão" ficam vizinhos, e "boleto" fica longe dos dois. Para buscar, você passa a consulta pelo **mesmo** modelo e procura os k vetores mais próximos dela (k-NN).



Palavras parecidas ficam perto no espaço de embeddings; a consulta "filhote" cai entre os animais, e o banco devolve os três vizinhos mais próximos.

Medindo "perto"

- **Similaridade de cosseno:** compara o ângulo entre os vetores e ignora o comprimento deles.
- **Distância euclidiana (L2):** a distância em linha reta.
- **Produto interno (dot product):** comum quando o modelo foi treinado para ele.

Com vetores normalizados (comprimento 1), as três produzem o mesmo ranking. Na dúvida, use a métrica para a qual o modelo foi treinado.

Por que "aproximado"

A busca exata compara a consulta com todos os vetores. Com 10 milhões de vetores de 768 dimensões, são uns 7,7 bilhões de multiplicações por consulta: viável em lote, caro demais para responder em milissegundos a milhares de consultas por segundo. E as estruturas clássicas de índice espacial, como as árvores, perdem a eficácia com centenas de dimensões.

Por isso, bancos vetoriais usam índices de **vizinhos mais próximos aproximados (ANN)**:

- **HNSW:** um grafo em camadas. A busca começa numa camada esparsa, dá saltos largos e desce para camadas mais densas, refinando. É rápido e preciso, mas gasta bastante memória.
- **IVF:** agrupa os vetores em clusters (com k-means) e, na consulta, examina só os clusters mais próximos.
- **Quantização (PQ):** comprime os vetores para caber mais em memória, perdendo um pouco de precisão.

O preço é o **recall**: em vez de garantir os 10 vizinhos exatos, o índice devolve, digamos, 9 deles. Você ajusta parâmetros para trocar recall por latência e memória.

Na prática, a busca vetorial quase nunca vem sozinha: ela se combina com filtros de metadados ("só documentos deste cliente") e, muitas vezes, com busca por palavra-chave (busca híbrida).

Analogia

Num supermercado, ninguém acha o requeijão pelo código de barras: vai ao corredor dos frios, onde ficam as coisas parecidas, como manteiga e cream cheese. Uma busca exata pergunta "existe o produto de código 789...?"; uma busca vetorial pergunta "o que está perto disto?". O mercado só funciona porque alguém organizou os corredores por semelhança. O modelo de embeddings faz esse papel com os seus dados.

NO MUNDO REAL

- O **pgvector** acrescenta ao PostgreSQL um tipo `vector` e índices HNSW e IVFFlat, com distância euclidiana, cosseno e produto interno.
- Bancos dedicados como **Pinecone**, **Milvus**, **Weaviate** e **Qdrant** nasceram para isso, e o **FAISS**, da Meta, é a biblioteca de referência para busca de similaridade.
- O **Spotify** abriu o código do **Annoy**, biblioteca de vizinhos aproximados criada lá para recomendar músicas.
- **Elasticsearch**, **OpenSearch**, **MongoDB Atlas** e **Redis** também oferecem busca vetorial, o que permite combiná-la com filtros e busca textual no mesmo lugar.

TRADE-OFFS

	Busca exata (k-NN)	Busca aproximada (ANN)
Resultado	Os k mais próximos, garantido	Quase sempre os mesmos (recall ajustável, como 95%)
Custo por consulta	Proporcional ao número de vetores	Cresce bem mais devagar
Memória	Só os vetores	Vetores mais o índice (o grafo HNSW pesa)
Atualizações	Triviais	Mais trabalhosas: o índice precisa ser mantido e, às vezes, reconstruído
Bom para	Coleções pequenas ou reordenar poucos candidatos	Milhões a bilhões de vetores com latência baixa

NA ENTREVISTA

- Em perguntas de busca semântica, recomendação ou RAG, desenhe o fluxo: dividir os documentos em trechos, gerar os embeddings, indexar, buscar os top-k e, se precisar, reordenar (rerank) antes de mandar ao modelo de linguagem.
- Diga que usaria um índice ANN (HNSW, por exemplo) e que existe um trade-off entre recall, latência e memória.
- Lembre dos filtros (“só documentos que este usuário pode ver”) e da busca híbrida com palavras-chave.
- Ponto extra: trocar o modelo de embeddings obriga a reindexar tudo.

CUIDADO COM

- **Vetores de modelos diferentes não se comparam.** Cada modelo cria o próprio espaço. Mudar de modelo, ou de versão, exige gerar todos os embeddings de novo.
- **Parecido não é exato.** Para códigos de produto, nomes próprios e mensagens de erro, a busca por palavra-chave (como o BM25) costuma ganhar; por isso a busca híbrida é tão comum.

- ATUALIZAÇÃO: “banco vetorial” deixou de ser só uma categoria de produto. Muitos bancos populares ganharam índice vetorial, então a decisão costuma ser entre ativá-lo no banco que você já usa ou adotar um dedicado para escala muito grande. E, em vagas que envolvem IA, espere perguntas além do “vizinho mais próximo”: ANN, recall e RAG.

TESTE-SE

1 O que é um embedding?

Um vetor de números, de tamanho fixo, gerado por um modelo para representar o significado de um item. Itens parecidos ficam com vetores próximos.

2 Você precisa de busca semântica sobre 50 milhões de trechos de documentos, com resposta em menos de 100 ms. Busca exata ou aproximada?

Aproximada, com um índice ANN como o HNSW. Comparar cada consulta com 50 milhões de vetores é caro demais; aceita-se perder um pouco de recall em troca de latência baixa.

3 Dá para comparar vetores gerados por dois modelos de embedding diferentes?

Não. Cada modelo cria o próprio espaço, e a distância entre vetores de modelos diferentes não significa nada. Trocar de modelo exige gerar os embeddings de novo.

CONECTA COM

5 Throughput vs. latência
26 Índices: B-Tree e LSM-Tree
29 Persistência poliglota

Parte 4

Confiabilidade e tolerância a falhas

Reliability and Fault Tolerance

Num sistema distribuído, falhar é rotina: um disco morre, uma dependência fica lenta, a rede perde pacotes, um cliente entra em loop e dispara milhares de requisições. A pergunta de design não é "e se falhar?", e sim "quando falhar, quanto do sistema vai junto?". Os capítulos 32 a 37 são as defesas do dia a dia: limitar o que entra, parar de chamar quem está quebrado, isolar recursos, repetir com calma, garantir que repetir não faça estrago e perceber quando um nó morreu.

Os capítulos 38 a 40 sobem um degrau: como vários nós concordam sobre quem manda e como manter correta uma operação que atravessa vários serviços ou bancos. Repare como os padrões se apoiam: retry só é seguro com idempotência, circuit breaker e backoff impedem que os retries virem um ataque, e é a falta de heartbeats que dispara a eleição de um novo líder.

32 Rate limiting

33 Circuit breaker

34 Bulkhead

35 Retries e backoff exponencial

36 Idempotência

37 Heartbeat

38 Eleição de líder (Paxos e Raft)

39 Transações distribuídas: o padrão SAGA

40 Two-Phase Commit (2PC)

Rate limiting

Rate Limiting

EM UMA FRASE

Rate limiting controla quantas requisições um usuário, IP ou chave de API pode fazer numa janela de tempo, protegendo o sistema contra abuso, picos acidentais e loops descontrolados.

O ESSENCIAL · tradução do artigo original

O rate limiting (limitação de taxa) controla quantas requisições um usuário, um IP ou uma chave de API pode fazer numa determinada janela de tempo. Ele protege o seu sistema contra abuso, picos acidentais de tráfego e loops descontrolados. Estratégias comuns incluem janela fixa, janela deslizante e token bucket. Os limites costumam ser aplicados no API gateway ou no load balancer. Pense neles como freios de segurança que impedem que recursos compartilhados fiquem sobrecarregados.

APROFUNDANDO

Todo rate limiter responde à mesma pergunta a cada requisição: “esta chave ainda tem cota?”. A chave pode ser o usuário, o IP, a chave de API ou uma combinação, como usuário mais endpoint. O que muda é o jeito de contar.

Janela fixa

Um contador por chave e por janela: “no máximo 100 requisições por minuto”, zerado às 12:00:00, às 12:01:00 e assim por diante. É o mais simples e barato, mas falha nas bordas: um cliente pode mandar 100 requisições às 12:00:59 e mais 100 às 12:01:00. São 200 em dois segundos, o dobro do limite.

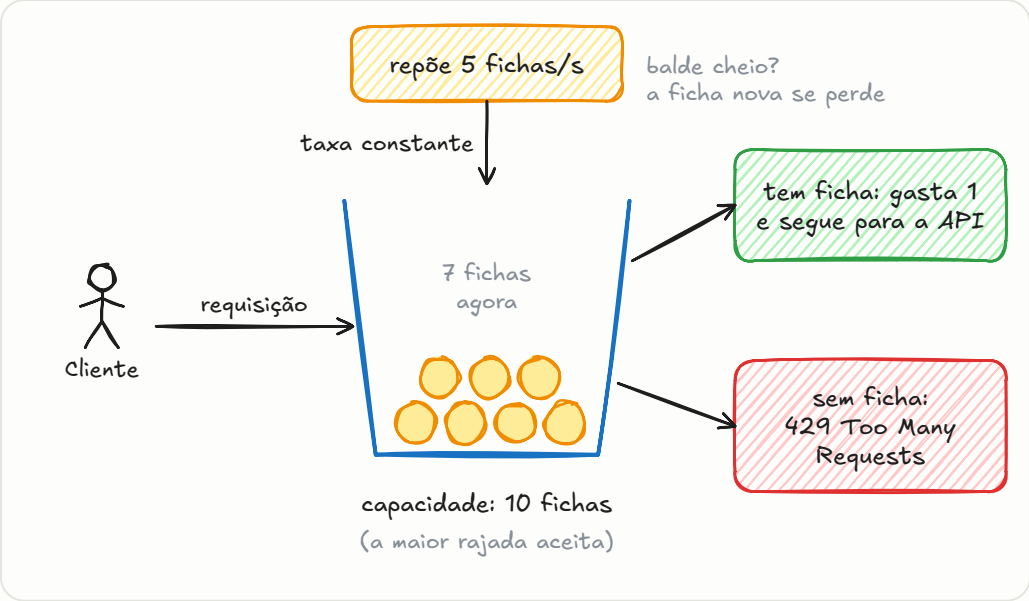
Janela deslizante

Olha sempre para “os últimos 60 segundos”. A versão exata guarda o horário de cada requisição (um log), o que custa memória. A versão aproximada guarda só dois contadores, o da janela anterior e o da atual, e faz uma média ponderada. Exemplo: li-

mite de 100 por minuto, 80 requisições no minuto anterior, 30 no atual e 15 segundos já corridos do minuto atual. Os últimos 60 segundos incluem 45 segundos (75%) do minuto anterior, então a estimativa é 30 + 80 × 0,75 = 90. Ainda cabe.

Token bucket

Cada chave tem um balde (bucket) com capacidade fixa de fichas (tokens), reabastecido a uma taxa constante. Cada requisição gasta uma ficha; sem ficha, a resposta é **429 Too Many Requests**.



O balde recebe fichas a uma taxa constante até encher; cada requisição gasta uma, e sem ficha a resposta é 429.

Com capacidade 10 e reposição de 5 por segundo, um cliente parado acumula 10 fichas e pode disparar uma rajada de 10 requisições de uma vez; depois disso, fica limitado à média de 5 por segundo. São dois botões independentes: a taxa média (reposição) e a maior rajada aceita (capacidade). Um parente próximo é o **leaky bucket** (balde furado): as requisições entram numa fila que escoa num ritmo constante, o que achata as rajadas em vez de permiti-las.

Onde guardar a contagem

Com um servidor só, o contador fica em memória. Com dez réplicas atrás de um load balancer, cada uma vê só um pedaço do tráfego, então o estado costuma ir para um armazenamento compartilhado, como o Redis, com operações atômicas (`INCR` com expiração, ou um script Lua que lê e atualiza o balde de uma vez). Ao recusar, devolva `429` com o cabeçalho `Retry-After`, para o cliente saber quando tentar de novo (cap. 35).

> *Analogia*
>
> O limite do Pix noturno é um rate limiter. Pela regra criada pelo Banco Central em 2021, das 20h às 6h o banco limita quanto você pode transferir para outra pessoa física (R$ 1.000, por padrão), mesmo que haja saldo sobrando. Não é desconfiança de você: é um teto que limita o estrago se o celular for roubado. E, como num plano de API mais caro, dá para pedir ao banco um limite maior.

NO MUNDO REAL

- **AWS API Gateway:** limita com token bucket. Na maioria das regiões, cada conta tem por padrão 10.000 requisições por segundo, com um balde de 5.000 para rajadas; quem passa disso recebe 429.
- **NGINX:** o módulo `limit_req` usa o método leaky bucket, com o parâmetro `burst` para enfileirar o excesso. Por padrão, ele recusa com 503, e não 429, a menos que você configure `limit_req_status`.
- **GitHub:** a API REST aceita 5.000 requisições por hora de usuários autenticados e 60 por hora sem autenticação, e informa a cota nos cabeçalhos `x-ratelimit-remaining` e `x-ratelimit-reset`.
- **Stripe:** o blog de engenharia da empresa descreve limitadores com token bucket sobre Redis, além de um limitador de requisições simultâneas.

TRADE-OFFS

Algoritmo	Estado por chave	Rajadas	Ponto fraco
Janela fixa	1 contador	Até 2× o limite na virada da janela	Imprecisão nas bordas

Algoritmo	Estado por chave	Rajadas	Ponto fraco
Janela deslizante (log)	1 horário por requisição	Respeita o limite à risca	Memória cresce com o limite
Janela deslizante (contador)	2 contadores	Suaviza a borda	É uma estimativa
Token bucket	Fichas e hora da última reposição	Aceita rajada até a capacidade	Dois parâmetros para calibrar
Leaky bucket	Fila e ritmo de saída	Achata as rajadas	Requisições esperam na fila

NA ENTREVISTA

- "Projete um rate limiter" é uma pergunta clássica por si só. Comece pelos requisitos: limitar por quê (abuso, custo, justiça entre clientes), por qual chave e com que precisão.
- Diga onde ele mora (na borda, no API gateway, com estado no Redis) e o que acontece se o Redis cair: falhar aberto (deixa passar) ou fechado (recusa tudo). Em geral, falhar aberto é o menos ruim, exceto em endpoints sensíveis, como login.
- Frase pronta: "*Token bucket por chave de API no gateway, com estado no Redis: capacidade de 20 para absorver rajadas e reposição de 10 por segundo. Quem estoura recebe 429 com Retry-After.*"

CUIDADO COM

- **Nem toda requisição custa igual.** Num limite simples, exportar um relatório e ler um item contam 1 cada. A API GraphQL do GitHub cobra pontos conforme o tamanho da consulta.
- **Limitar só por IP pune inocentes.** Redes móveis e provedores com CGNAT colocam centenas de clientes atrás do mesmo IP. Prefira usuário ou chave de API quando houver.
- **Rate limiting não é load shedding.** Um limita a cota de cada cliente; o outro descarta trabalho quando o próprio servidor está sobrecarregado, venha de quem vier.

- ATUALIZAÇÃO: além das três estratégias do artigo, o leaky bucket é muito usado (é o do NGINX), e hoje se limita em camadas: na borda contra DDoS, no gateway por chave de API e no serviço para operações caras.

TESTE-SE

1 **Numa janela fixa de 100 requisições por minuto, o que acontece se um cliente manda 100 requisições às 12:00:59 e mais 100 às 12:01:00?**

As duas levas passam, porque caem em janelas diferentes: são 200 requisições em dois segundos, o dobro do limite. A janela deslizante e o token bucket evitam esse pico.

2 **Uma API pública precisa aceitar rajadas curtas (um app que sincroniza 20 itens ao abrir), mas manter a média em 5 requisições por segundo. Qual algoritmo você usa e como configura?**

Token bucket com capacidade 20, que é a rajada, e reposição de 5 fichas por segundo, que é a média. Cada parâmetro atende a um dos requisitos.

3 **A API tem 10 réplicas, cada uma com um limitador em memória de 100 requisições por minuto por usuário. Qual é o limite real?**

Até 1.000 por minuto, se o load balancer espalhar as requisições do usuário entre as réplicas. Para um limite global, o estado precisa ser compartilhado (no Redis, por exemplo) ou a cota precisa ser dividida entre as réplicas.

CONECTA COM

11 Balanceamento de carga
14 API Gateway
33 Circuit breaker
35 Retries e backoff exponencial

Circuit breaker

Circuit Breaker Pattern

EM UMA FRASE

O circuit breaker monitora as chamadas a um serviço remoto e, quando as falhas passam de um limite, para de chamá-lo por um tempo e falha na hora, impedindo que um serviço doente derrube os outros.

O ESSENCIAL · tradução do artigo original

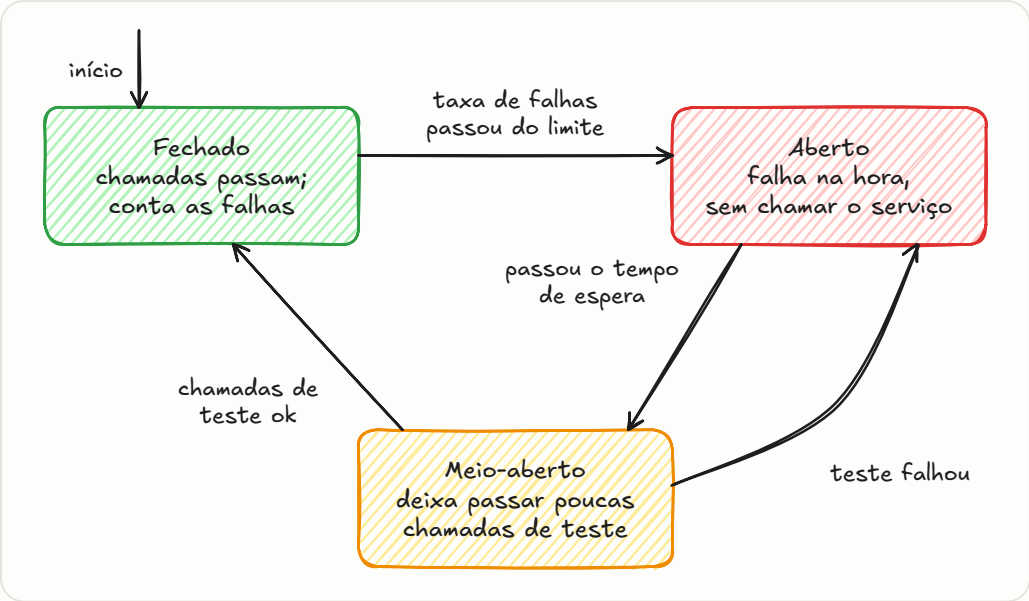
Um circuit breaker (disjuntor) monitora as chamadas a um serviço remoto e "abre" se houver falhas demais. Quando está aberto, ele faz as novas requisições falharem imediatamente, em vez de tentar de novo o serviço quebrado. Depois de um período de espera, ele deixa passar algumas chamadas de teste para ver se o serviço se recuperou, e fecha se elas tiverem sucesso. Esse padrão evita falhas em cascata, em que um serviço lento arrasta o sistema inteiro para baixo. Agora, a parte delicada: circuit breakers precisam ser calibrados com cuidado, para não abrirem agressivamente demais nem tarde demais.

APROFUNDANDO

Por que um serviço lento é pior que um morto

Um serviço morto recusa a conexão em milissegundos. Um serviço lento prende quem o chama. Imagine que o serviço de pedidos tem 200 threads e chama o serviço de frete, que passou a responder só no timeout de 30 segundos. Chegando 10 requisições por segundo que precisam de frete, cada uma fica presa 30 s: em 20 segundos, as 200 threads estão esperando, e o serviço de pedidos para de atender até quem nem precisava de frete. A falha subiu um andar. Se outros serviços dependem de pedidos, sobe mais um. Isso é a falha em cascata.

Os três estados



O breaker abre quando as falhas passam do limite, testa a recuperação no meio-aberto e só então volta a fechar.

- **Fechado:** a situação normal. As chamadas passam e o breaker registra o resultado numa janela deslizante, por exemplo as últimas 100 chamadas. Se a taxa de falhas passa do limite (digamos, 50%) depois de um número mínimo de chamadas, ele abre.
- **Aberto:** nenhuma chamada sai. Quem chamou recebe erro na hora ou um fallback (plano B): um valor em cache, uma resposta padrão, a página sem aquela seção. As threads ficam livres, e o serviço doente ganha fôlego para se recuperar.
- **Meio-aberto:** passado o tempo de espera (por exemplo, 30 s), o breaker deixa passar algumas chamadas de teste. Se derem certo, ele fecha. Se falharem, volta a abrir e recomeça a contar o tempo.

O que conta como falha

Timeouts, erros de conexão e respostas 5xx contam. Respostas 4xx, em geral, não: um 404 diz que o pedido estava errado, não que o serviço está doente. As bibliotecas atuais também podem contar **chamadas lentas** como falha (acima de 2 s, por exemplo), porque lentidão é justamente o sintoma que derruba quem chama.

O padrão foi popularizado por Michael Nygard no livro *Release It!* (2007). Ele não substitui o timeout: o breaker decide *se* a chamada sai; o timeout limita *quanto tempo* cada chamada pode prender um recurso.

Analogia

Você pede almoço sempre no mesmo restaurante pelo aplicativo de delivery. Três pedidos seguidos chegam com mais de uma hora de atraso. Você para de pedir lá por uma semana e almoça em outro lugar (o fallback), sem nem abrir o cardápio deles. Passada a semana, arrisca um pedido só. Chegou no horário? Volta à rotina. Atrasou de novo? Mais uma semana de castigo.

NO MUNDO REAL

- **Resilience4j:** a biblioteca padrão no ecossistema Java, inclusive por trás do Spring Cloud Circuit Breaker. Por padrão, abre com 50% de falhas numa janela das últimas 100 chamadas e espera 60 s antes de passar para meio-aberto.
- **Netflix Hystrix:** popularizou o padrão em microsserviços, mas está em modo de manutenção desde 2018; a própria Netflix passou a indicar o Resilience4j para projetos novos.
- **Polly:** a biblioteca de resiliência do .NET implementa a mesma máquina de três estados.
- **Envoy e Istio:** chamam de "circuit breaking" outra coisa: limites de conexões, requisições pendentes e retries por cluster. O que tira um host doente de rotação, mais parecido com o padrão clássico, é a outlier detection (detecção de pontos fora da curva).

TRADE-OFFS

	Breaker sensível (abre cedo)	Breaker tolerante (abre tarde)
A dependência caiu de vez	Corta rápido e protege as threads	Acumula timeouts antes de cortar
Soluço passageiro de rede	Abre à toa e recusa chamadas que dariam certo	Aguenta sem cortar

	Breaker sensível (abre cedo)	Breaker tolerante (abre tarde)
Risco típico	Fica oscilando entre aberto e fechado	Deixa a cascata começar
Ajuste que ajuda	Número mínimo de chamadas e janela maior	Timeout curto e chamadas lentas contando como falha

NA ENTREVISTA

- Traga o circuit breaker em toda chamada síncrona entre serviços, junto com timeout e retry (cap. 35). O entrevistador quer ver que você pensou no que acontece quando a dependência fica **lenta**, e não só quando cai.
- Descreva os três estados e, principalmente, o fallback de cada funcionalidade. É aí que o design aparece.
- Frase pronta: "*Se o serviço de recomendações degradar, o breaker abre e a página de produto sai sem a vitrine personalizada, com os mais vendidos do cache. O checkout não depende dele e nunca espera por ele.*"

CUIDADO COM

- **Breaker não substitui timeout.** Sem timeout, cada chamada no estado fechado ainda pode prender uma thread por minutos antes de contar como falha.
- **Retry contra breaker aberto é desperdício.** Cada tentativa falha na hora; o certo é desistir rápido e usar o fallback.
- **Cada instância tem o seu breaker.** Com 50 réplicas, são 50 visões independentes da mesma dependência, cada uma abrindo no seu tempo. Em geral isso é aceitável, mas não espere um interruptor global.
- ATUALIZAÇÃO: muito material ainda ensina com o Hystrix, que não recebe versões novas. Em código novo, use Resilience4j, Polly ou os recursos da malha de serviços (service mesh).

TESTE-SE

1 **Quais são os três estados de um circuit breaker?**

Fechado (as chamadas passam e as falhas são contadas), aberto (as chamadas falham na hora, sem tocar no serviço) e meio-aberto (algumas chamadas de teste decidem se ele fecha ou volta a abrir).

2 **O serviço de recomendações começou a responder em 30 segundos. O que o breaker do serviço de produto deve fazer, e o que o usuário vê?**

Contar as chamadas lentas como falha, abrir e servir um fallback, como os mais vendidos em cache. O usuário vê a página normal, só sem a vitrine personalizada.

3 **Configurar o breaker para abrir na primeira falha deixa o sistema mais seguro?**

Não. Qualquer soluço de rede abriria o circuito e recusaria chamadas que dariam certo, e ele ficaria oscilando. É preciso um número mínimo de chamadas e uma taxa de falhas medida numa janela.

CONECTA COM

9 Microsserviços vs. monólito
32 Rate limiting
34 Bulkhead
35 Retries e backoff exponencial

Bulkhead

Bulkhead Pattern

EM UMA FRASE

O bulkhead divide os recursos do sistema em compartimentos isolados, como pools de conexões ou de threads por funcionalidade, para que a falha ou a sobrecarga de uma parte não afunde as outras.

O ESSENCIAL · tradução do artigo original

O padrão bulkhead (antepara) isola partes de um sistema para que uma falha numa área não afunde tudo. Isso pode significar pools de conexões separados, pools de threads separados ou até clusters de serviço inteiros separados para funcionalidades diferentes. Se um compartimento é inundado de tráfego, os outros continuam funcionando. O nome vem das anteparas dos navios, que contêm a inundação num único compartimento. Em discussões de design, usar bulkheads mostra que você está pensando em isolamento de falhas e em raio de impacto (blast radius).

APROFUNDANDO

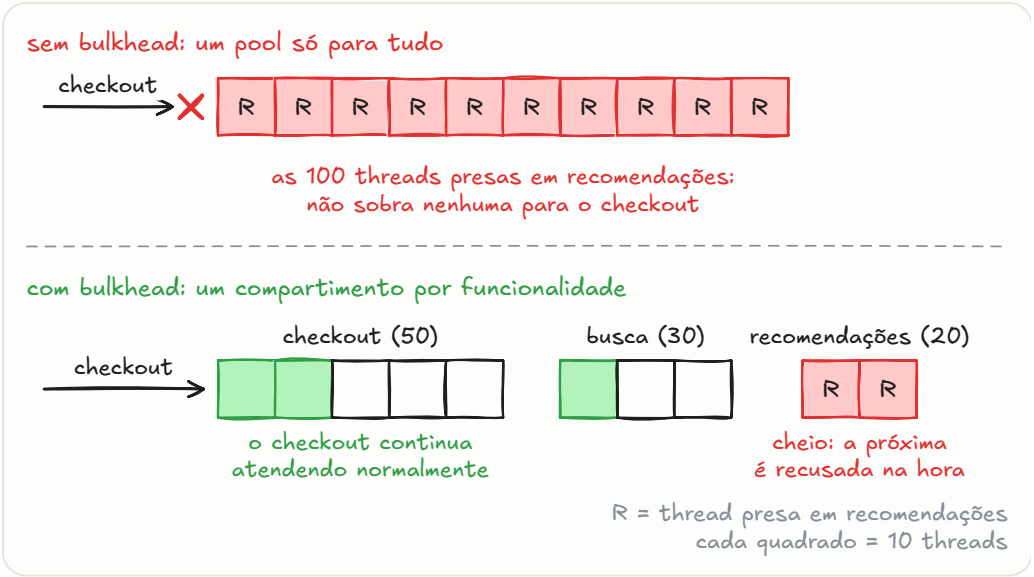
O problema do pool compartilhado

Imagine o serviço de uma loja virtual com um único pool de 100 threads atendendo três funcionalidades: checkout, busca e recomendações. O serviço de recomendações, uma dependência externa, começa a levar 10 segundos para responder. Cada chamada a ele segura uma thread por 10 s, e em pouco tempo as 100 estão presas esperando. Chega uma requisição de checkout e não há thread livre. O checkout caiu por causa de uma vitrine que nem é essencial.

Compartimentos

O bulkhead dá a cada funcionalidade, ou a cada dependência, o seu próprio recurso, com limite: 50 threads para o checkout, 30 para a busca, 20 para as recomendações. Quando recomendações trava, só os 20 lugares dela enchem; a 21ª chamada simultâ-

nea é recusada na hora (ou espera numa fila curta e limitada), e o checkout segue com os 50 lugares dele intactos.



Com um pool só, a dependência lenta ocupa todas as threads; com compartimentos, ela enche só o dela, e o checkout continua atendendo.

O mesmo raciocínio vale em vários níveis:

- **Dentro do processo:** pools de threads ou semáforos por dependência ("no máximo 20 chamadas simultâneas ao serviço de recomendações").
- **Conexões:** um pool de conexões com o banco para o tráfego transacional e outro, menor, para relatórios pesados, ou uma réplica de leitura só para eles.
- **Instâncias e clusters:** máquinas separadas para a API pública e para o processamento interno, ou para clientes grandes e pequenos.
- **Células:** cópias completas da pilha, cada uma atendendo uma fatia dos clientes. Uma célula com problema afeta só os clientes dela.

Quanto dar para cada um

A lei de Little ajuda a dimensionar: concorrência média = vazão × latência. Se o checkout faz 40 chamadas por segundo ao serviço de pagamento e cada uma leva 250 ms, há em média 10 chamadas em andamento. Um compartimento de 20 dá folga para picos sem deixar que uma lentidão do pagamento consuma o resto do sistema.

Analogia

O caixa rápido do supermercado, para até 10 volumes, é um bulkhead. Em dia de promoção, a fila dos carrinhos cheios pode dar a volta no corredor, mas quem só foi comprar pão e leite não fica preso atrás dela: existe um recurso reservado para o seu tipo de compra. Sem essa separação, as compras lentas travariam todas as rápidas.

NO MUNDO REAL

- **Hystrix e Resilience4j:** o Hystrix isolava cada dependência num pool de threads próprio por padrão. O Resilience4j oferece um bulkhead por semáforo, mais leve, e outro com pool de threads fixo e fila limitada.
- **AWS Route 53:** usa shuffle sharding, uma forma refinada de compartimento. Cada domínio de cliente recebe uma combinação de 4 servidores de nomes virtuais, entre 2.048. Um ataque contra um domínio atinge os 4 servidores dele, mas nenhum outro domínio divide mais de 2 servidores com ele.
- **Kubernetes:** requests e limits de CPU e memória por contêiner, e cotas por namespace (ResourceQuota), impedem que um serviço ou um time consuma os recursos do cluster inteiro.

TRADE-OFFS

	Pool compartilhado	Compartimentos (bulkhead)
Uma dependência trava	Pode esgotar o pool e derrubar tudo	Esgota só o compartimento dela
Uso dos recursos	Alto: quem precisa pega	Parte fica ociosa, reservada
Configuração	Um número para ajustar	Um tamanho por compartimento, mais filas e alertas
Prioridades	Todo mundo disputa igual	O essencial tem lugar garantido

NA ENTREVISTA

- Traga o bulkhead quando um serviço tem dependências de importâncias diferentes (“o checkout não pode cair por causa da vitrine”) ou quando vários clientes dividem a mesma infraestrutura, o problema do vizinho barulhento (noisy neighbor).
- Use o vocabulário: isolamento de falhas, raio de impacto, células.
- Combine com o circuit breaker (cap. 33): o bulkhead limita o estrago; o breaker para de alimentá-lo.
- Frase pronta: *“Cada dependência tem o seu limite de chamadas simultâneas. Se recomendações travar, perco só a vitrine; o checkout tem 50 lugares só dele.”*

CUIDADO COM

- **Compartimento sem timeout só adia o problema.** Ele enche e fica cheio. Precisa de timeout nas chamadas e de recusa rápida quando lota.
- **Fila ilimitada anula o isolamento.** Se o excesso espera numa fila sem fim, a memória vira o recurso compartilhado que afunda tudo.
- **Pools dentro do mesmo processo não isolam CPU nem memória.** Um vazamento de memória numa funcionalidade derruba o processo inteiro; para isso, separe em processos, contêineres ou máquinas.
- ATUALIZAÇÃO: com I/O assíncrono e threads virtuais (Java 21 em diante), threads deixaram de ser o recurso escasso. O bulkhead moderno costuma ser um limite de chamadas simultâneas por dependência, e não um pool de threads.

TESTE-SE

1 **O que é o raio de impacto (blast radius) de uma falha, e como o bulkhead o reduz?**

É a parte do sistema afetada quando algo quebra. O bulkhead separa recursos por funcionalidade, dependência ou cliente, então a falha esgota só o compartimento dela.

2 **Numa API usada por várias empresas, um cliente dispara relatórios gigantes e deixa todo mundo lento. Como aplicar bulkhead?**

Separando o trabalho pesado do interativo: pool, fila ou instâncias próprios para relatórios, com limite. Se o problema é um cliente específico, ele ganha um compartimento ou uma célula própria, junto com rate limiting (cap. 32).

3 **Em vez de criar compartimentos, não basta aumentar o pool compartilhado de 100 para 1.000 threads?**

Não. Pela lei de Little, a dependência lenta ocupa vazão × latência threads: se a latência dela sobe de 100 ms para 10 s, ela passa a precisar de 100 vezes mais, e o pool maior também esgota, só que mais tarde e gastando mais memória. O problema é a falta de isolamento, não o tamanho.

CONECTA COM

9 Microsserviços vs. monólito
23 Sharding
32 Rate limiting
33 Circuit breaker

Retries e backoff exponencial

Retry Patterns and Exponential Backoff

EM UMA FRASE

Retries recuperam falhas passageiras; o backoff exponencial faz cada nova tentativa esperar mais que a anterior, e o jitter sorteia essas esperas para que os clientes não voltem todos juntos contra um serviço que já está sofrendo.

O ESSENCIAL · tradução do artigo original

Retries (novas tentativas) ajudam a se recuperar de erros passageiros, como timeouts de rede ou sobrecarga temporária. Backoff exponencial significa que cada nova tentativa espera mais do que a anterior, por exemplo 1 segundo, 2 segundos, 4 segundos, e assim por diante. Isso evita que o seu cliente fique martelando um serviço que já está com dificuldade. Boas políticas de retry também usam jitter (um pouco de aleatoriedade) para evitar o efeito manada (thundering herd). Explicando melhor: retries sem backoff podem piorar uma queda em vez de ajudar.

APROFUNDANDO

O que vale repetir

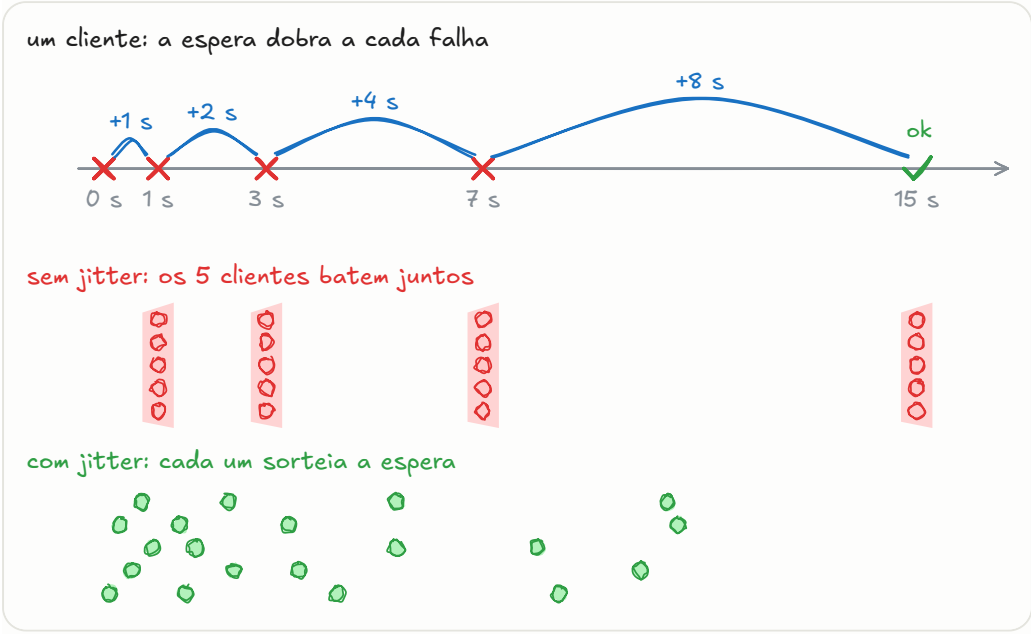
Só erros **passageiros**: timeout, conexão recusada ou interrompida, `503 Service Unavailable` e `429 Too Many Requests` (respeitando o cabeçalho `Retry-After`). Um `400`, `401`, `403` ou `404` vai dar o mesmo erro na décima tentativa. E só repita operações **idempotentes** (cap. 36); do contrário, o retry de um pagamento pode cobrar duas vezes.

A conta do backoff

```
espera(n) = min(teto, base × 2^n)        n = 0, 1, 2, ...
base = 1 s, teto = 30 s  →  1 s, 2 s, 4 s, 8 s, 16 s, 30 s, 30 s...
```

Com um limite de, digamos, 4 retries, o cliente desiste cerca de 15 s depois da primeira falha e devolve o erro, ou manda a mensagem para uma dead letter queue (cap. 46).

Por que o jitter



As esperas dobram a cada falha (1, 2, 4 e 8 s); sem jitter, os clientes voltam juntos, em picos; com jitter, cada um sorteia a espera e a carga se espalha.

O backoff resolve o problema de *um* cliente insistente, mas não o de mil. Se o servidor reinicia e mil clientes falham no mesmo segundo, todos voltam juntos em t = 1 s, depois em t = 3 s, em t = 7 s. Cada onda é um pico que pode derrubar o servidor de novo. O jitter quebra essa sincronia. Na variante full jitter (jitter total), a espera inteira é sorteada entre zero e o teto exponencial:

```
espera(n) = aleatório(0, min(teto, base × 2^n))
```

As mil tentativas de cada rodada se espalham pelo intervalo, em vez de chegarem no mesmo milissegundo.

Retries se multiplicam

Se o app repete 3 vezes, o gateway repete 3 vezes e o serviço repete 3 vezes a chamada ao banco, uma falha no banco vira até 4 × 4 × 4 = 64 chamadas, porque cada camada faz a chamada original mais 3 retries. Por isso, repita em uma camada só e use um **orçamento de retries** (retry budget). O livro de SRE do Google descreve um limite de 3 tentativas por requisição e um orçamento por cliente: retries só acontecem enquanto forem menos de 10% das requisições dele.

> *Analogia*
>
> Sexta à noite, você liga para a pizzaria e dá ocupado. Você não fica rediscando sem parar: espera um minuto, depois dois, depois quatro. Agora imagine o bairro inteiro, que ligou no mesmo instante, rediscando exatamente um minuto depois: a linha continua ocupada para sempre. O que faz as ligações entrarem é cada um esperar um tempo um pouco diferente. Isso é o jitter.

NO MUNDO REAL

- **Kubernetes:** reinicia contêineres que caem com backoff exponencial, 10 s, 20 s, 40 s, até o teto de 5 minutos. O `CrashLoopBackOff` que aparece no `kubectl` é exatamente esse estado de espera.
- **gRPC:** a política de retry é declarada na configuração do serviço: máximo de tentativas, backoff inicial, multiplicador, teto e quais códigos repetir (como `UNAVAILABLE`). A espera é sorteada entre zero e o valor exponencial, ou seja, já vem com jitter.
- **SDKs da AWS:** repetem as chamadas com backoff exponencial e jitter e mantêm uma cota de retries (um token bucket) que para de repetir quando as falhas se generalizam. O full jitter foi popularizado por um artigo do blog de arquitetura da AWS, de 2015.

TRADE-OFFS

	Retry imediato	Backoff exponencial	Backoff com jitter
Carga sobre o serviço doente	Rajadas seguidas	Menor, mas em ondas sincronizadas	Menor e espalhada

	Retry imediato	Backoff exponencial	Backoff com jitter
Espera de cada cliente	Curta	Cresce a cada tentativa	Cresce, com valor sorteado
Previsibilidade	Alta	Alta	Menor
Quando usar	Quase nunca	Poucos clientes	Padrão quando há muitos clientes

NA ENTREVISTA

- Toda vez que desenhar uma chamada de rede, diga a política: quais erros repetir, quantas tentativas, backoff com jitter, teto e timeout por tentativa.
- Amarre com idempotência (cap. 36): "posso repetir porque a operação tem chave de idempotência".
- Mostre que conhece o lado ruim: a amplificação entre camadas e o efeito manada depois de uma queda.
- Frase pronta: "*Retry só para erros transitórios, no máximo 3 tentativas, backoff exponencial com full jitter e teto de 10 s. Esgotou, a mensagem vai para a dead letter queue.*"

CUIDADO COM

- ATUALIZAÇÃO: o artigo fala em "um pouco de aleatoriedade", mas um ruído pequeno, de milissegundos sobre esperas de segundos, mal desfaz a sincronia. Na análise da AWS de 2015, as melhores variantes sorteiam a espera inteira, como o full jitter.
- **Retry sem timeout não ajuda.** Se a primeira tentativa fica pendurada, a segunda nunca acontece. E todas juntas precisam caber no tempo que o usuário espera.
- **Timeout abaixo da latência normal gera retries à toa.** Com p99 de 800 ms e timeout de 500 ms, pelo menos 1 chamada em 100 é repetida sem necessidade, e sob carga isso piora.
- **Backoff longo não cabe em requisição de usuário.** Esperar 15 s com a tela travada já é uma falha; o backoff longo é para jobs e filas.

TESTE-SE

1 **Com backoff exponencial de base 1 s, quanto o cliente espera antes da quarta nova tentativa, e em que instante ela acontece?**

Espera 8 s, porque as esperas são 1, 2, 4 e 8 s. Ela acontece 1 + 2 + 4 + 8 = 15 s depois da primeira falha, sem contar a duração das próprias chamadas.

2 **Um app com um milhão de celulares perde a conexão quando o servidor reinicia. O que configurar nos clientes para que a volta não derrube o servidor de novo?**

Reconexão com backoff exponencial, teto e jitter (full jitter). Assim as reconexões se espalham por vários segundos, em vez de chegarem todas no mesmo instante.

3 **Vale a pena dar retry num `404 Not Found`?**

Não. O recurso não existe, e repetir dá o mesmo resultado. Retries são para falhas passageiras: timeouts, `503`, `429` e erros de conexão.

CONECTA COM

32 Rate limiting · 33 Circuit breaker · 36 Idempotência · 46 Dead letter queue

Idempotência

Idempotency

EM UMA FRASE

Uma operação é idempotente quando executá-la várias vezes tem o mesmo efeito que executá-la uma vez, o que torna seguros os retries e a entrega "pelo menos uma vez", sem cobranças em dobro.

O ESSENCIAL · tradução do artigo original

Uma operação é idempotente se executá-la várias vezes tem o mesmo efeito que executá-la uma vez. Por exemplo, “definir o status do usuário como ativo” é idempotente, enquanto “incrementar o saldo da conta em 10” não é. A idempotência é crítica quando os sistemas usam retries, porque a mesma requisição pode ser enviada mais de uma vez. APIs costumam exigir chaves de idempotência em operações como pagamentos, para evitar cobrar duas vezes. Em entrevistas, sempre mencione idempotência quando falar de entrega “pelo menos uma vez” (at-least-once) ou de retries.

APROFUNDANDO

O cliente não sabe o que aconteceu

O app manda “pagar R$ 100” e a conexão cai antes da resposta. Há duas histórias possíveis: a requisição se perdeu no caminho, ou o pagamento foi feito e a *resposta* se perdeu. Do lado do cliente, as duas parecem iguais: um timeout. Se ele repete e a verdadeira era a segunda, cobra duas vezes. A idempotência faz a dúvida deixar de importar, porque repetir passa a ser sempre seguro.

No HTTP

Pela especificação (RFC 9110), `GET`, `HEAD`, `PUT` e `DELETE` são idempotentes: “deixe o recurso 42 com este conteúdo” pode ser repetido à vontade. `POST` não é: cada chamada pode criar um recurso novo. É no `POST` que entram as chaves.

Chave de idempotência

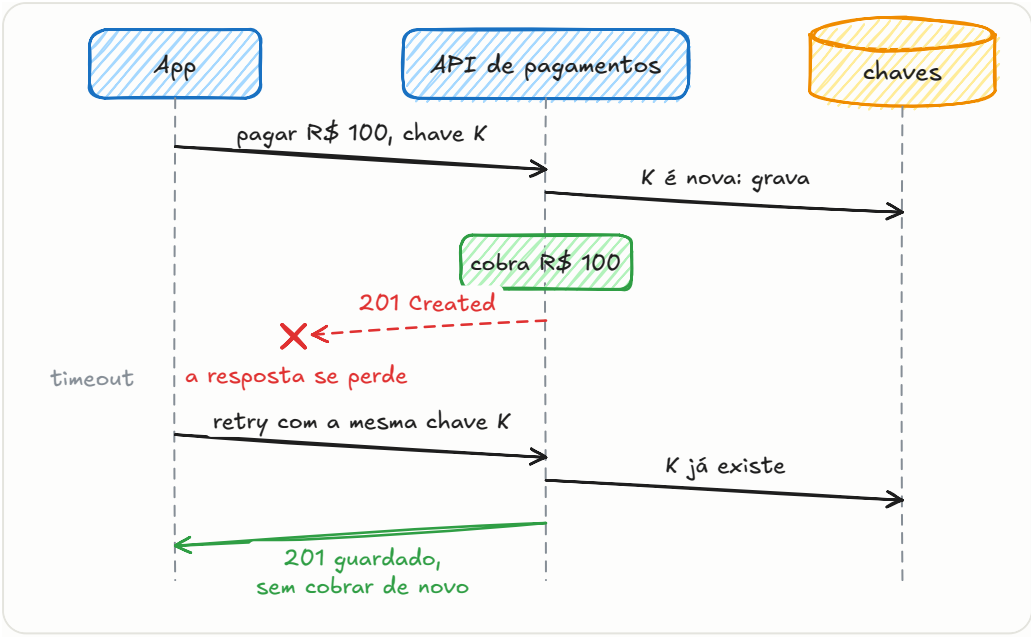
O cliente gera um identificador único, como um UUID, para a **operação lógica** e o reenvia em todos os retries dela:

```
POST /v1/pagamentos
Idempotency-Key: 8f14e45f-ceea-4f6a-9a3b-2c1d5e7b9a10
Content-Type: application/json

{"pedido": "A-1027", "valor": 10000}
```

No servidor:

1. Tenta gravar a chave numa tabela com restrição de unicidade, com status “em processamento”.
2. Se conseguiu, executa a cobrança e salva a resposta junto da chave, de preferência na mesma transação.
3. Se a chave já existia com resposta pronta, devolve a resposta guardada, sem cobrar de novo. Se o corpo veio diferente do original, recusa: é uso errado da chave.
4. Se a chave existe mas ainda está em processamento (duas requisições simultâneas), responde com erro de conflito (`409`), e o cliente tenta depois.
5. As chaves expiram depois de um prazo, de horas a dias, para a tabela não crescer para sempre.



A resposta da primeira tentativa se perde; o retry chega com a mesma chave, e o servidor devolve a resposta guardada sem cobrar de novo.

Outros caminhos

Nem sempre é preciso uma chave: dá para tornar a própria operação idempotente. "Some 10 ao saldo" vira "registre o crédito 7781, de R$ 10", e um índice único na coluna do identificador do crédito impede a duplicata. Em filas com entrega at-least-once (cap. 44), o consumidor guarda os IDs das mensagens já processadas e ignora as repetidas.

> **Analogia**
>
> No bar, pedir "mais uma" ao garçom não é idempotente: se ele não ouviu direito e você repete, podem chegar duas cervejas. Já dizer "a comanda 27 fica com três cervejas no total" é idempotente: repetir a frase dez vezes não muda a conta. A chave de idempotência é o garçom anotar "pedido 3 da mesa 27": se você repete, ele vê que esse pedido já está na comanda e não anota de novo.

NO MUNDO REAL

- **Stripe:** todas as requisições `POST` aceitam o cabeçalho `Idempotency-Key`. A API guarda o código de status e o corpo da primeira resposta, devolve o mesmo resultado nos retries, recusa a mesma chave com parâmetros diferentes e pode descartar chaves com mais de 24 horas.
- **Kafka:** o produtor idempotente numera as mensagens por partição, e o broker descarta as duplicatas criadas por retries. Nas versões recentes, ele vem ligado por padrão.
- **Amazon SQS FIFO:** cada mensagem tem um ID de deduplicação; reenvios com o mesmo ID dentro de 5 minutos não entram duplicados na fila.

TRADE-OFFS

Abordagem	Como funciona	Custo
Operação idempotente por natureza	`PUT` com o estado final, "definir status = ativo"	Nem toda regra de negócio cabe nesse formato
Chave de idempotência	O cliente manda um ID por operação; o servidor guarda ID e resposta	Tabela extra, consulta a cada requisição e expiração das chaves
Restrição de unicidade	Índice único num ID natural (pedido, crédito)	Precisa de um identificador estável vindo de fora
Deduplicação no consumidor	Guarda os IDs das mensagens já processadas	Estado extra e janela de deduplicação limitada

NA ENTREVISTA

- Sempre que disser "retry" ou "pelo menos uma vez", complete: "...e por isso o consumidor é idempotente". O entrevistador costuma cobrar isso.
- Em pagamentos, descreva a chave: quem gera (o cliente), por quanto tempo vale e onde fica (numa tabela com unicidade, na mesma transação do efeito).
- Frase pronta: "*O cliente gera um UUID por compra e o reenvia em todos os retries. O servidor grava a chave com restrição de unicidade na mesma transação da cobrança; uma repetição recebe a resposta guardada.*"

CUIDADO COM

- **Idempotente não quer dizer mesma resposta.** O primeiro `DELETE /pedidos/42` devolve `204`; o segundo pode devolver `404`. O que se repete é o efeito no servidor: o pedido não existe mais.
- **Efeitos colaterais quebram a idempotência.** "Definir status = ativo" é idempotente no banco, mas não se cada chamada dispara um e-mail de boas-vindas.
- **"Consultar e depois inserir" tem corrida.** Duas requisições simultâneas consultam, não acham a chave e inserem as duas. Quem garante é a restrição de unicidade do banco.
- **"Exatamente uma vez" tem asterisco.** Sistemas que prometem entrega exactly-once, como o Kafka, combinam entrega pelo menos uma vez com idempotência e deduplicação. Na rede, exatamente uma vez não existe de graça.

TESTE-SE

1 **Por que "incrementar o saldo em 10" não é idempotente e "definir o status como ativo" é?**

Cada repetição do incremento soma mais 10, enquanto repetir a definição deixa o status no mesmo valor. O efeito de várias execuções é diferente do de uma no primeiro caso e igual no segundo.

2 **O app envia um pagamento, a conexão cai e ele não sabe se a cobrança aconteceu. Como repetir sem risco de cobrar em dobro?**

Reenviando a mesma chave de idempotência gerada para esse pagamento. O servidor reconhece a chave e devolve o resultado da primeira execução, em vez de cobrar de novo.

3 **O cliente gera uma chave de idempotência nova a cada retry. Está protegido?**

Não. Para o servidor, cada chave nova é uma operação nova, e a cobrança pode acontecer duas vezes. A chave identifica a operação lógica e precisa ser a mesma em todas as tentativas.

CONECTA COM

35 Retries e backoff exponencial
39 Transações distribuídas: o padrão SAGA
44 Filas de mensagens
46 Dead letter queue

Heartbeat

Heartbeat

EM UMA FRASE

Heartbeat é um sinal periódico que um nó envia para avisar que está vivo; quando os sinais param de chegar por tempo demais, o monitor o considera fora do ar e dispara failover ou substituição.

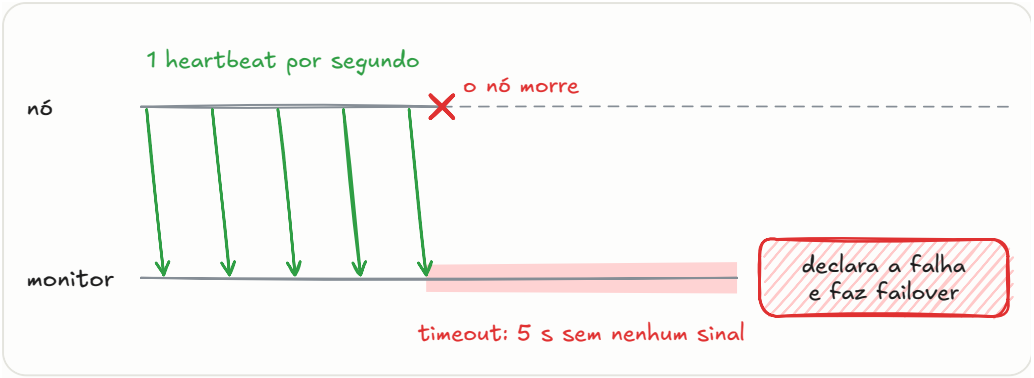
O ESSENCIAL · tradução do artigo original

Um heartbeat (batimento cardíaco) é um sinal periódico enviado por um serviço ou nó para indicar que está vivo e saudável. Sistemas de monitoramento ou coordenadores ficam escutando esses heartbeats. Se param de recebê-los, marcam o nó como fora do ar e disparam ações de failover ou de escalonamento. Heartbeats são ferramentas simples, mas poderosas, para detectar se um nó está vivo (liveness). Pense neles como a "checagem de pulso" do sistema.

APROFUNDANDO

Dois números: intervalo e timeout

Um heartbeat é definido por **com que frequência** o sinal sai e **quanto silêncio** o monitor tolera. Com intervalo de 1 s e timeout de 5 s, um nó que morre é declarado fora do ar cerca de 5 segundos depois. Timeout curto detecta rápido, mas confunde lentidão com morte; timeout longo evita alarme falso, mas deixa um nó morto recebendo tráfego por mais tempo. Por isso é comum exigir vários intervalos perdidos seguidos antes de agir.



O nó manda um sinal por segundo; quando os sinais param, o monitor espera o timeout inteiro antes de declarar a falha e disparar o failover.

Empurrar ou perguntar

No heartbeat clássico, o nó **empurra** o sinal ("estou vivo") para o monitor. No health check, o monitor **pergunta** ("você está bem?"), como fazem os load balancers (cap. 11). O Kubernetes usa os dois: o kubelet de cada nó renova periodicamente um objeto Lease no control plane, e o próprio kubelet consulta cada contêiner, pelas probes, para saber se ele está vivo (liveness) e se está pronto para receber tráfego (readiness).

Silêncio não é morte

O problema de fundo: o monitor não sabe *por que* o sinal parou. O nó pode ter morrido, mas pode estar numa pausa longa de garbage collection, ou a rede entre os dois pode ter caído enquanto ele continua atendendo clientes. Se o coordenador promove outro nó e o antigo "ressuscita" achando que ainda manda, o sistema fica com dois líderes (split brain). Sistemas sérios se defendem de três jeitos:

- **Confirmar com outros:** no Redis Sentinel, um sentinela que deixa de ouvir o master só registra uma suspeita (SDOWN); o master só é dado como fora do ar (ODOWN), e o failover pode começar, quando um quórum de sentinelas concorda.
- **Detectores adaptativos:** o Cassandra usa o phi accrual failure detector, que aprende o ritmo normal de chegada dos sinais e calcula um nível contínuo de suspeita, em vez de um prazo fixo.
- **Fencing:** cada novo líder recebe um número de época maior, e o armazenamento recusa ordens com número antigo. Um nó que ressuscita não consegue estragar nada.

Num cluster grande, heartbeat de todos para todos custa N^2 mensagens a cada rodada; protocolos gossip (cap. 22) espalham essa informação de forma muito mais barata.

Analogia

Todo dia às 7h, a tia manda "bom dia" com figurinha no grupo da família. No dia em que o bom-dia não chega até as 10h, alguém liga para saber se está tudo bem. Mas ninguém chama a ambulância no primeiro atraso: pode ser só o celular sem bateria. Esse é o dilema do heartbeat: o silêncio é um sinal de alerta, não uma prova.

NO MUNDO REAL

- **etcd (Raft):** o líder manda heartbeats a cada 100 ms por padrão, e um seguidor que passa 1 s sem ouvi-lo, o election timeout padrão, inicia uma eleição (cap. 38).
- **Kubernetes:** se o Lease de um nó para de ser renovado, o control plane marca a condição `Ready` do nó como `Unknown` e, depois de um prazo (5 minutos, por padrão), começa a despejar os pods dele para que sejam recriados em outros nós.
- **HAProxy:** por padrão, verifica cada servidor a cada 2 s, tira de rotação o que falhar 3 verificações seguidas e só o devolve depois de 2 verificações bem-sucedidas.

TRADE-OFFS

	Timeout curto (ex.: 2 s)	Timeout longo (ex.: 30 s)
Detecção de nó morto	Rápida: failover em segundos	Lenta: o nó morto segue recebendo tráfego
Alarme falso	Frequente: uma pausa de GC ou um soluço de rede derruba o nó	Raro
Efeito colateral	Failovers à toa e risco de dois líderes	Indisponibilidade mais longa
Tráfego de heartbeats	Maior, se o intervalo também for curto	Menor

NA ENTREVISTA

- Sempre que houver líder, réplica, pool de workers ou service discovery, diga como uma falha é detectada: "heartbeat a cada X, falha declarada depois de Y sem sinal".
- Mostre o trade-off do timeout e a defesa contra o falso positivo: confirmação por quórum e fencing.
- Separe vivo de pronto: a liveness reinicia o contêiner; a readiness só o tira do balanceamento.
- Frase pronta: "Os *workers mandam heartbeat a cada 2 s; com 10 s de silêncio, o coordenador redistribui as tarefas deles. Como um worker 'morto' pode estar só lento, cada tarefa carrega um token de fencing, e o resultado atrasado é descartado.*"

CUIDADO COM

- ATUALIZAÇÃO: o artigo diz que, sem heartbeats, o nó é marcado como fora do ar. Mas silêncio não prova morte: o nó pode estar lento ou isolado e voltar. Sistemas atuais tratam a falta de sinal como suspeita (detectores adaptativos, quórum) e se protegem do nó que volta (fencing).
- **Vivo não é saudável.** Um processo pode responder ao heartbeat com o disco cheio ou sem conexão com o banco.
- **Meça o silêncio com o relógio de quem recebe.** Comparar o horário escrito no heartbeat com o relógio do monitor engana, porque relógios de máquinas diferentes divergem. O monitor conta o tempo desde o último sinal com o próprio relógio.
- **Thread de heartbeat isolada pode mentir.** Ela continua batendo enquanto o trabalho principal está travado.

TESTE-SE

1 Qual a diferença entre heartbeat e health check?

No heartbeat, o nó envia sinais periódicos por conta própria; no health check, o monitor pergunta ativamente e o nó responde. Nos dois casos, a falta de resposta dentro de um prazo é tratada como falha.

2 Um coordenador declara workers mortos depois de 1 s sem heartbeat, e tarefas estão sendo executadas em dobro. O que está acontecendo e o que você muda?

O timeout curto gera falsos positivos (pausas de GC, picos de rede): o worker "morto" continua trabalhando enquanto outro recebe a mesma tarefa. Aumente o timeout para vários intervalos e torne as tarefas idempotentes ou protegidas por fencing.

3 A liveness probe do seu serviço consulta o banco de dados. O que acontece quando o banco cai?

Todas as réplicas falham a checagem ao mesmo tempo, e o Kubernetes reinicia todas em loop, o que derruba até o que ainda funcionava. Dependências externas vão, se forem, na readiness, que só tira a réplica do balanceamento.

CONECTA COM

11 Balanceamento de carga
22 Protocolo gossip
24 Padrões de replicação
38 Eleição de líder (Paxos e Raft)

Eleição de líder (Paxos e Raft)

Leader Election (Paxos, Raft)

EM UMA FRASE

A eleição de líder escolhe um único nó para coordenar os demais; algoritmos de consenso como Paxos e Raft usam votos da maioria para garantir no máximo um líder por mandato e trocá-lo automaticamente quando ele cai.

O ESSENCIAL · tradução do artigo original

A eleição de líder é o processo de escolher um único nó, entre muitos, para atuar como coordenador. Algoritmos como Paxos e Raft garantem que só um líder seja escolhido e que todos os nós acabem concordando sobre quem é ele. O líder cuida de tarefas como distribuir trabalho, gerenciar metadados ou ordenar as escritas. Se o líder falha, um novo é eleito automaticamente. Você não precisa decorar a matemática para as entrevistas, mas deve saber que algoritmos de consenso sustentam muitos sistemas críticos, como repositórios de metadados e logs distribuídos.

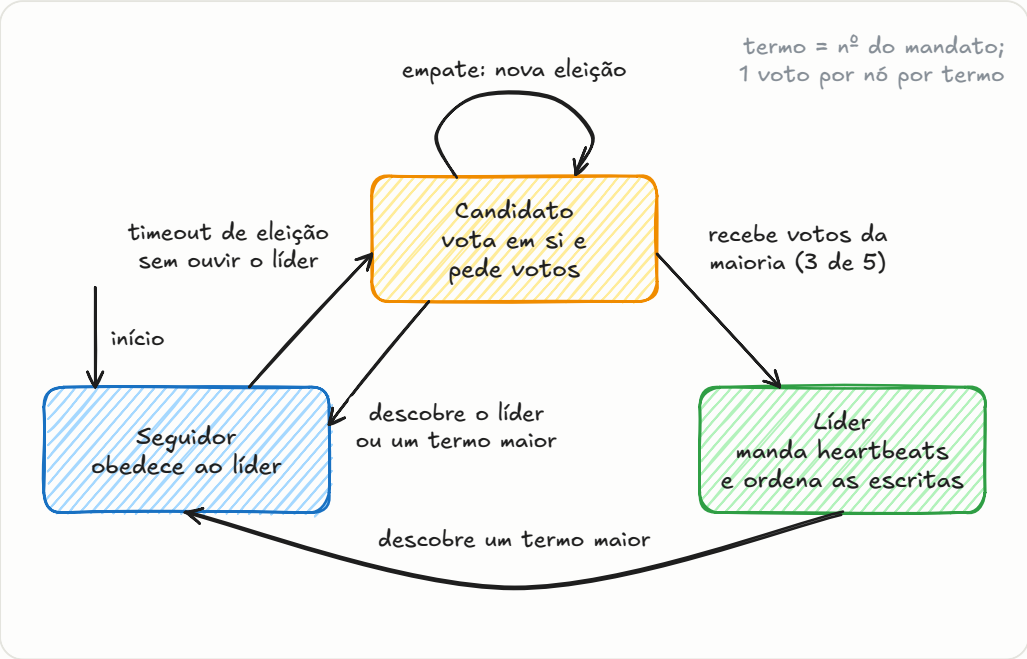
APROFUNDANDO

Por que maioria

Ter um líder simplifica tudo: um só nó decide a ordem das escritas, e os outros copiam. O perigo é haver **dois** líderes ao mesmo tempo (split brain), cada um aceitando escritas diferentes. A defesa é exigir maioria: num cluster de 5 nós, um líder precisa de 3 votos. Como duas maiorias de 5 sempre têm pelo menos um nó em comum, e cada nó vota uma vez só por mandato, dois candidatos não conseguem vencer a mesma eleição. A conta geral: **2f + 1 nós toleram f falhas.** Três nós aguentam 1 falha; cinco aguentam 2.

Raft, passo a passo

O Raft (Diego Ongaro e John Ousterhout, Stanford, 2014) foi criado para ser mais fácil de entender que o Paxos. O tempo é dividido em **termos** (mandatos) numerados, e cada nó está sempre em um de três estados:



O seguidor vira candidato quando o líder some; o candidato com maioria vira líder; quem descobre um termo maior volta a ser seguidor.

1. Todos começam como **seguidores**. O líder manda heartbeats periódicos (cap. 37).
2. Se um seguidor passa o seu election timeout (tempo de eleição) sem ouvir o líder, vira **candidato**: incrementa o termo, vota em si e pede votos aos demais.
3. Cada nó vota em no máximo um candidato por termo, no primeiro que pedir, e só se o log do candidato estiver pelo menos tão atualizado quanto o seu.
4. Com a maioria dos votos, o candidato vira **líder** e começa a mandar heartbeats.
5. Se os votos se dividem e ninguém chega à maioria, os candidatos esperam um novo timeout e tentam de novo. Os timeouts são sorteados (150 a 300 ms, no artigo original) justamente para que, quase sempre, um só nó acorde primeiro.
6. Quem descobre um termo maior que o seu volta a ser seguidor, inclusive um líder antigo que estava isolado.

Exemplo: o líder A, do termo 7, cai. O seguidor C é o primeiro a estourar o timeout, abre o termo 8 e recebe os votos de B e D. São 3 de 5, contando o dele, e C vira líder. Quando A volta, vê o termo 8 e vira seguidor.

E o Paxos?

O Paxos, de Leslie Lamport, oferece a mesma garantia de segurança, com papéis de proponente, aceitador e aprendiz e duas fases de mensagens. Na forma básica, ele decide um único valor; o Multi-Paxos, usado na prática, elege um líder estável para não repetir a primeira fase a cada decisão.

> *Analogia*
>
> Pense na eleição de síndico. O síndico tem mandato, que é o termo, e precisa da maioria na assembleia. Se ele some e para de responder, qualquer morador pode convocar uma assembleia e se candidatar; se os votos empatam, marca-se outra. E se o síndico antigo volta de viagem e tenta dar ordens, ninguém obedece: há um mandato novo, e o dele acabou.

NO MUNDO REAL

- **Raft:** o etcd (o banco de configuração do Kubernetes), o Consul, o CockroachDB e o TiKV. O Kafka também: o modo KRaft, baseado em Raft, gerencia os metadados do cluster e, desde a versão 4.0 (2025), é o único modo.
- **Paxos:** o Chubby, serviço de locks do Google, e o Spanner, que replica cada fatia dos dados com Paxos.
- **ZAB:** o ZooKeeper usa um protocolo próprio, parecido com o Paxos, o ZooKeeper Atomic Broadcast.

TRADE-OFFS

	Com líder (Raft, Multi-Paxos)	Sem líder (quóruns no estilo Dynamo)
Ordem das escritas	Uma só, decidida pelo líder	Não há ordem global; conflitos se resolvem depois

	Com líder (Raft, Multi-Paxos)	Sem líder (quóruns no estilo Dynamo)
Consistência	Forte: todos aplicam as mesmas escritas na mesma ordem	Eventual ou ajustável por quórum
Quando um nó cai	Se for o líder, as escritas param até a nova eleição	Sem pausa: as outras réplicas aceitam
Gargalo	Todas as escritas passam pelo líder	Escritas espalhadas entre as réplicas

NA ENTREVISTA

- Traga a eleição de líder quando o design precisa de um coordenador único: scheduler, dono de uma partição, lock distribuído, metadados.
- Não implemente consenso no quadro. Diga: "*Uso um serviço de coordenação pronto, como etcd ou ZooKeeper. O líder é quem segura um lease, um lock com prazo; se ele morrer, o lease expira e outro assume.*"
- Mostre que sabe as contas: maioria, número ímpar de nós (3 ou 5) e tempo de failover, que é o timeout de eleição mais a própria eleição.
- O entrevistador quer ouvir como você evita dois líderes: maioria, termos e fencing.

CUIDADO COM

- ATUALIZAÇÃO: muito material ainda descreve o Kafka elegendo o controller pelo ZooKeeper. O Kafka 4.0, de 2025, removeu o suporte ao ZooKeeper; o KRaft, baseado em Raft, virou o único modo.
- **"Um líder só" vale por termo.** Um líder antigo, isolado por uma partição, pode continuar achando que manda; ele só não consegue confirmar escritas sem maioria. Por isso clientes e armazenamento precisam checar o termo.
- **Consenso cobra em disponibilidade.** O Raft é CP (cap. 2): durante a eleição, e do lado minoritário de uma partição, ninguém confirma escritas. E mais nós deixam cada escrita mais lenta: com 7, ela só é confirmada quando 4 nós a gravam.

- **O ZooKeeper não usa Paxos**, e sim o ZAB, um protocolo parecido.

TESTE-SE

1 **Por que um cluster Raft de 5 nós tolera 2 falhas, e não 3?**

Porque o líder precisa da maioria, 3 de 5, para ser eleito e para confirmar escritas. Com 3 nós fora, sobram 2, que não formam maioria, e o cluster para de aceitar escritas.

2 **Um job agendado precisa rodar em só uma de 10 instâncias por vez. Como você garante isso?**

Com eleição de líder sobre um serviço de coordenação (etcd, ZooKeeper, Consul): a instância que obtém o lease roda o job; se ela cair, o lease expira e outra assume. Para o caso de o líder antigo voltar no meio, proteja o efeito do job com um token de fencing.

3 **Um cluster de 4 nós é mais tolerante a falhas que um de 3?**

Não. Com 4 nós, a maioria é 3, então ele tolera só 1 falha, igual ao de 3, e ainda exige mais votos e mensagens por escrita.

CONECTA COM

2 Teorema CAP · 24 Padrões de replicação · 37 Heartbeat · 40 Two-Phase Commit (2PC)

Transações distribuídas: o padrão SAGA

Distributed Transactions (SAGA Pattern)

EM UMA FRASE

O padrão SAGA divide uma transação que atravessa vários serviços numa sequência de transações locais; se um passo falha, ações de compensação desfazem os anteriores, em ordem inversa, sem travar nada globalmente.

O ESSENCIAL · tradução do artigo original

Uma transação distribuída abrange vários serviços ou bancos de dados. O padrão SAGA modela essa transação como uma sequência de passos locais, com ações de compensação para os rollbacks. Em vez de travar tudo, como numa única transação ACID, cada serviço executa a sua parte e publica um evento. Se algo falha, passos de compensação tentam desfazer as mudanças anteriores. Isso combina naturalmente com microsserviços e consistência eventual. A contrapartida é uma lógica mais complexa e a possibilidade de falhas parciais, que precisam ser tratadas de forma controlada.

APROFUNDANDO

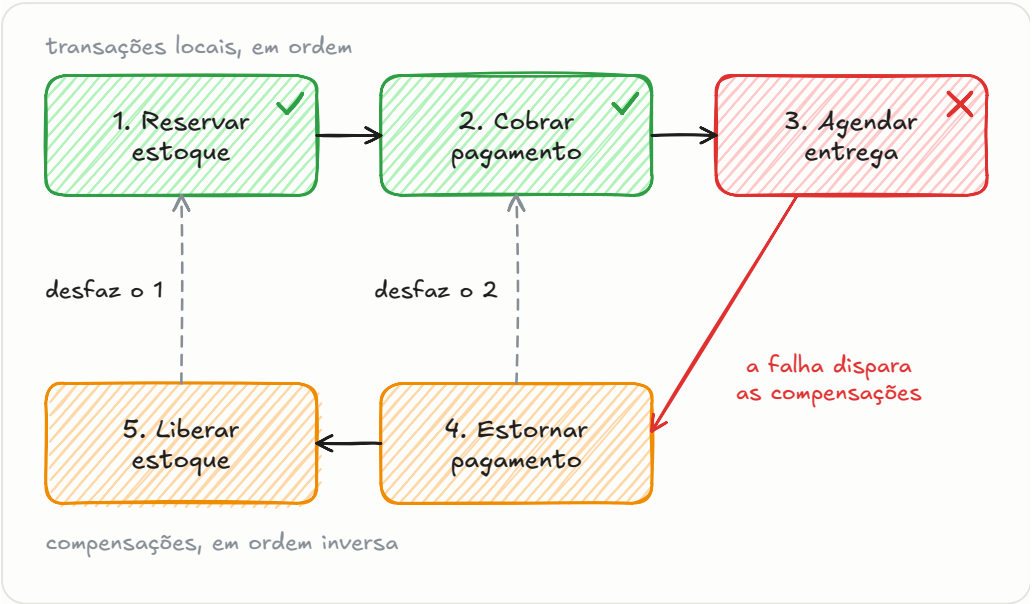
De onde vem

O termo vem de um artigo de 1987 de Hector Garcia-Molina e Kenneth Salem sobre transações longas dentro de um único banco. Voltou com força nos microsserviços, em que cada serviço tem o seu banco (cap. 9) e não existe uma transação ACID (cap. 4) que abranja todos.

Um pedido em três passos

Cada passo é uma transação local, com commit no banco do seu serviço, e tem uma **compensação** correspondente:

Passo	Serviço	Compensação
1. Reservar o estoque	Estoque	Liberar o estoque
2. Cobrar o pagamento	Pagamentos	Estornar o pagamento
3. Agendar a entrega	Logística	Nenhuma: é o último passo



Os passos 1 e 2 dão certo e o 3 falha; as compensações rodam em ordem inversa: primeiro o estorno, depois a liberação do estoque.

Se a logística não consegue agendar a entrega, as compensações rodam de trás para a frente: estornar, depois liberar o estoque. Se a falha fosse na cobrança, só o estoque seria liberado, porque um passo que não chegou a fazer commit não tem o que desfazer.

Coreografia ou orquestração

- **Coreografia:** cada serviço reage aos eventos dos outros. O estoque publica `EstoqueReservado`, pagamentos escuta, cobra e publica `PagamentoAprovado`, e assim por diante. Não há ponto central, mas o fluxo fica espalhado e difícil de enxergar.

- **Orquestração:** um orquestrador, uma máquina de estados com o progresso salvo, chama cada passo e decide as compensações. O fluxo fica num lugar só, mais fácil de testar e monitorar.

Regras para não se machucar

- Compensações precisam ser **idempotentes** (cap. 36) e repetidas até dar certo: uma compensação não pode "falhar de vez".
- Ponha por último os passos difíceis de desfazer. Despachar o pacote vem depois da cobrança, nunca antes.
- Grave no banco e publique o evento de forma atômica, com o padrão **outbox**: o evento entra numa tabela na mesma transação local, e um processo separado o publica.

> *Analogia*
>
> Você monta as férias em três compras separadas: passagem aérea, hotel e aluguel de carro. Não existe um botão que desfaça as três de uma vez. Se a locadora fica sem carro, você cancela o hotel e pede o reembolso da passagem, na ordem inversa, talvez pagando multa. Compensar não é voltar no tempo: é fazer uma nova operação que anula o efeito da anterior.

NO MUNDO REAL

- **AWS Step Functions:** o guia de padrões de nuvem da AWS mostra uma saga orquestrada por uma máquina de estados, com passos de pedido, estoque e pagamento e as compensações correspondentes, como reverter o pagamento e o estoque.
- **Temporal:** plataforma de workflows, criada pelos autores do Cadence, da Uber, em que a saga vira código comum, com o estado de cada passo persistido e retomado depois de falhas.
- **microservices.io:** o catálogo de padrões de Chris Richardson, a referência mais citada sobre o tema, descreve as duas formas de coordenar uma saga, coreografia e orquestração.

TRADE-OFFS

	Coreografia (eventos)	Orquestração (orquestrador)
Quem decide o próximo passo	Cada serviço, reagindo a eventos	Um orquestrador central
Visão do fluxo	Espalhada pelos serviços	Num lugar só, fácil de monitorar
Acoplamento	Baixo, mas com risco de dependências cíclicas	Os serviços dependem do orquestrador
Bom para	Sagas curtas, de 2 a 4 passos	Fluxos longos, com muitas regras e compensações

NA ENTREVISTA

- Traga a saga sempre que uma operação de negócio atravessa serviços com bancos próprios: checkout, reserva de viagem, transferência entre carteiras.
- Liste os passos e as compensações em voz alta e justifique a ordem: o que é difícil de desfazer vai por último.
- Mostre que sabe o preço: não há isolamento, o usuário pode ver um pedido "pendente" e cada passo precisa ser idempotente.
- Frase pronta: "*Cada serviço tem o seu banco, então não existe transação ACID única. Uso uma saga orquestrada: reservar estoque, cobrar, agendar entrega; se a entrega falhar, estorno e libero o estoque. Os passos são idempotentes, e os eventos saem por um outbox.*"

CUIDADO COM

- **Compensação não é rollback.** O estado intermediário existiu e foi visto, e efeitos externos não voltam: não dá para "desenviar" um e-mail, só mandar outro corrigindo.
- **Falta o I do ACID.** Enquanto a saga roda, outra requisição pode ver o estoque reservado de um pedido que vai ser cancelado. Estados explícitos, como `PENDENTE` e `CONFIRMADO`, avisam aos outros o que ainda está em andamento.

- `ATUALIZAÇÃO:` o artigo descreve só a coreografia (“cada serviço publica um evento”). Na prática, a orquestração com motores de workflow, como Step Functions e Temporal, é igualmente comum, principalmente em fluxos longos.

TESTE-SE

1 O que é uma ação de compensação?

Uma nova operação que anula, do ponto de vista do negócio, o efeito de um passo já confirmado, como estornar uma cobrança ou liberar uma reserva. Ela não apaga o passo original; registra outro por cima.

2 Uma saga de viagem tem três passos: emitir a passagem (sem reembolso), reservar o hotel (cancelável) e alugar o carro (cancelável). Em que ordem você os coloca?

Hotel e carro primeiro, passagem por último. Se o hotel ou o carro falharem, as compensações são simples cancelamentos; a passagem, que não dá para desfazer, só é emitida quando o resto já deu certo.

3 Uma saga garante o mesmo isolamento de uma transação ACID?

Não. Cada passo faz commit sozinho, então outros processos veem os estados intermediários. O design precisa lidar com isso, por exemplo com estados como `PENDENTE`.

CONECTA COM

4 ACID vs. BASE
7 Consistência forte vs. eventual
9 Microsserviços vs. monólito
36 Idempotência
40 Two-Phase Commit (2PC)

Two-Phase Commit (2PC)

Two Phase Commit (2PC)

EM UMA FRASE

O two-phase commit torna atômica uma transação em vários nós; o coordenador pergunta se todos podem confirmar e só manda confirmar se todos votarem sim, mas, se ele cair no meio, os participantes ficam travados.

O ESSENCIAL · tradução do artigo original

O Two-Phase Commit (confirmação em duas fases) é um protocolo que tenta oferecer transações atômicas em vários nós.

- Na primeira fase, o coordenador pergunta a todos os participantes se eles podem confirmar (commit).
- Na segunda fase, se todos concordarem, ele manda todos confirmarem; caso contrário, manda todos desfazerem (rollback).

O 2PC oferece garantias fortes, mas pode travar se o coordenador falhar, e é caro em escala por causa dos bloqueios (locks). Em sistemas modernos na nuvem, o 2PC costuma ser evitado nos caminhos de alto throughput e substituído por padrões como SAGA.

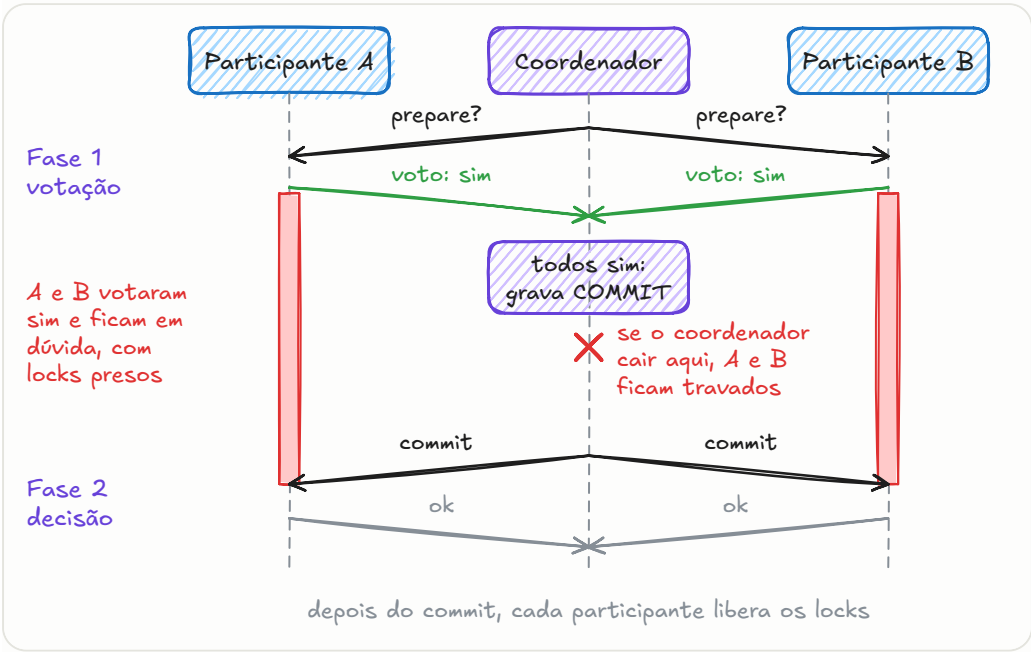
APROFUNDANDO

As duas fases, numa transferência

Imagine uma transferência de R$ 100 da conta X, no banco A, para a conta Y, no banco B. Um coordenador conduz a transação.

Fase 1, votação. O coordenador manda `prepare` para A e B. Cada um executa a sua parte (A debita, B credita), grava tudo de forma durável no log em disco (cap. 27), mantém os locks das linhas e responde com o voto: **sim**, se pode confirmar, ou **não**, se algo impede, como saldo insuficiente.

Fase 2, decisão. Se todos votaram sim, o coordenador grava "commit" no próprio log. Esse é o ponto de commit: a partir dali, a transação está decidida. Então manda `commit` para todos. Se alguém votou não, ou não respondeu a tempo, manda `abort`. Os participantes aplicam a decisão, liberam os locks e confirmam o recebimento.



Depois de votar sim, os participantes seguram os locks à espera da decisão; se o coordenador cair nesse intervalo, eles ficam travados.

O voto é uma promessa

Votar sim significa "eu confirmo, aconteça o que acontecer, mesmo se reiniciar". Por isso o participante grava tudo antes de votar, e por isso ele perde o direito de desistir sozinho. Suponha que o coordenador caia logo depois de receber os votos. O participante A fica **em dúvida**: não pode confirmar, porque talvez B tenha votado não, nem desfazer, porque talvez o coordenador tenha decidido commit e B já tenha aplicado. Ele espera, com as linhas da conta X travadas, até o coordenador voltar e ler a decisão no log. Qualquer outra transação que tocar a conta X espera junto.

Por que é caro

São pelo menos duas idas e voltas de rede e gravações forçadas em disco em cada participante, com os locks presos durante tudo isso. O participante mais lento dita o ritmo de todos, e uma transação que envolve 5 nós depende de os 5 estarem saudáveis

ao mesmo tempo.

Quando o coordenador volta

Ele lê o próprio log. Se encontra "commit", reenvia `commit` a todos; se não encontra decisão nenhuma, aborta, porque nenhum participante pode ter confirmado sem ela. É por isso que o ponto de commit é a gravação no log, e não o envio das mensagens.

> *Analogia*
>
> Você organiza um churrasco com quatro amigos e pergunta no grupo: "dá para sábado?". Essa é a fase 1. Quem responde "dá" guarda o sábado e recusa outros convites. Só quando todos dizem sim você confirma; se um diz não, cancela para todos. Essa é a fase 2. Agora imagine que você some depois de colher as respostas: os amigos ficam com o sábado travado, sem saber se vai ter churrasco.

NO MUNDO REAL

- **PostgreSQL:** suporta as duas fases com `PREPARE TRANSACTION` e `COMMIT PREPARED`, mas vem com o recurso desligado (`max_prepared_transactions = 0`). A documentação avisa que uma transação preparada e esquecida continua segurando locks e atrapalha o `VACUUM`.
- **XA e JTA:** gerenciadores de transação no Java usam 2PC para gravar num banco e numa fila de mensagens de forma atômica.
- **Google Spanner:** transações que tocam vários grupos de réplicas usam 2PC, mas o coordenador e os participantes são grupos replicados por Paxos. Se uma máquina cai, outra réplica do grupo assume, e o protocolo não trava.

TRADE-OFFS

	2PC	SAGA (cap. 39)
Atomicidade	Tudo ou nada, de verdade	Eventual, via compensações
Isolamento	Locks seguram os dados até o fim	Nenhum: estados intermediários ficam visíveis

	2PC	SAGA (cap. 39)
Se o coordenador cai	Os participantes ficam travados, em dúvida	Sem bloqueio: o fluxo é retomado do estado salvo
Latência e escala	Duas idas e voltas com locks; o mais lento dita o ritmo	Passos locais e rápidos; serviços independentes
Exige	Que todos os participantes falem o protocolo (XA, por exemplo)	Uma compensação para cada passo

NA ENTREVISTA

- Quando a pergunta for "como garantir atomicidade entre dois bancos?", cite o 2PC, explique as fases e logo depois o problema: bloqueio se o coordenador cair e locks presos durante duas idas e voltas.
- Entre microsserviços, proponha saga (cap. 39) com outbox. Dentro de um banco distribuído que já faz 2PC por baixo, como Spanner ou CockroachDB, deixe o banco cuidar disso.
- Frase pronta: "*2PC dá atomicidade de verdade, mas segura locks durante duas idas e voltas e trava se o coordenador cair depois dos votos. No caminho de alto volume, prefiro saga; onde preciso de atomicidade forte entre shards, uso um banco que faz 2PC sobre réplicas com consenso.*"

CUIDADO COM

- **2PC não é consenso.** O 2PC exige que *todos* votem sim e trava se o coordenador sumir; Paxos e Raft (cap. 38) precisam só da maioria e seguem funcionando com nós fora. Um confirma uma transação em todos; os outros fazem um grupo concordar sobre um valor.
- **O three-phase commit (3PC) não resolve.** Ele reduz o bloqueio, mas supõe atrasos de rede limitados; com partições, pode levar a decisões inconsistentes, e quase ninguém o usa.

- ATUALIZAÇÃO: “evitar 2PC” vale entre microsserviços. Bancos distribuídos como Spanner e CockroachDB usam 2PC ou variantes por dentro, com o estado da transação replicado por Paxos ou Raft, o que elimina o bloqueio pela queda de uma única máquina.

TESTE-SE

1 O que acontece em cada fase do 2PC?

Na fase 1, o coordenador manda `prepare`, e cada participante grava tudo de forma durável e vota sim ou não. Na fase 2, se todos votaram sim, o coordenador grava a decisão e manda `commit`; se alguém votou não ou não respondeu, manda `abort`.

2 Um sistema interno, de baixo volume, precisa debitar num PostgreSQL e creditar em outro, de forma atômica. 2PC é uma opção razoável?

Sim. Com poucos participantes e pouco volume, `PREPARE TRANSACTION` e `COMMIT PREPARED`, ou um gerenciador XA, dão atomicidade real. Só é preciso monitorar transações preparadas órfãs, que seguram locks.

3 Depois de votar sim, o participante pode desistir sozinho se o coordenador demorar?

Não. O voto sim é uma promessa de confirmar; ele precisa esperar a decisão do coordenador, ou de outro participante que a conheça, mesmo segurando locks. É exatamente aí que o 2PC trava.

CONECTA COM

4 ACID vs. BASE
27 Write-Ahead Logging (WAL)
38 Eleição de líder (Paxos e Raft)
39 Transações distribuídas: o padrão SAGA

Parte 5

Cache e mensageria

Caching and Messaging

Depois de escalar e de sobreviver a falhas, sobram duas ferramentas que aparecem em quase todo design: o cache, que evita refazer trabalho, e a mensageria, que evita esperar por ele. O cache guarda perto de quem lê os dados mais pedidos e corta a latência e a carga do banco. A mensageria põe uma fila ou um tópico entre quem produz e quem processa, para que cada lado escale e falhe no próprio ritmo.

Os capítulos 41 a 43 tratam do cache: por que ele funciona, como mantê-lo em sintonia com o banco nas leituras e nas escritas e quem sai quando ele enche. Os capítulos 44 a 46 tratam da mensageria: a fila ponto a ponto, que distribui trabalho; o pub/sub, que distribui eventos; e a dead letter queue, que guarda as mensagens que falham. As duas famílias cobram um preço parecido: dados que podem chegar velhos, duplicados ou fora de ordem.

41 Cache

42 Estratégias de cache

43 Políticas de remoção (LRU, LFU)

44 Filas de mensagens

45 Pub/Sub

46 Dead letter queue

41 PARTE 5 · CACHE E MENSAGERIA

Cache

Caching

EM UMA FRASE

Cache é uma cópia dos dados mais usados num armazenamento rápido, quase sempre a memória; corta a latência e a carga do banco, mas cria um problema novo, o de saber quando essa cópia ficou velha.

O ESSENCIAL · tradução do artigo original

Cache é guardar os dados acessados com frequência numa camada de armazenamento rápida, geralmente a memória, para reduzir a latência e a carga sobre o backend. Camadas de cache comuns incluem caches dentro do próprio processo, armazenamentos chave-valor externos e CDNs. O cache é especialmente eficaz em cargas com muita leitura e em computações caras. Agora vem a parte complicada: dados desatualizados e a invalidação tornam o cache mais difícil do que parece à primeira vista. Como diz o ditado, invalidar cache é um dos problemas difíceis da ciência da computação.

APROFUNDANDO

O cache funciona porque o acesso aos dados quase nunca é uniforme. Numa loja online, algumas centenas de produtos em destaque recebem a maior parte das visitas; numa rede social, o post do momento é lido milhões de vezes. Guardar essa fatia quente em memória resolve a maioria das leituras.

Taxa de acerto

A métrica central é a **taxa de acerto** (hit ratio): a fração das leituras que o cache responde. Quando o dado está lá, é um **hit**; quando não está, um **miss**, e a leitura segue para a origem. Faça a conta com um Redis que responde em 1 ms e um banco que leva 20 ms. Toda leitura paga o cache; só os misses pagam também o banco:

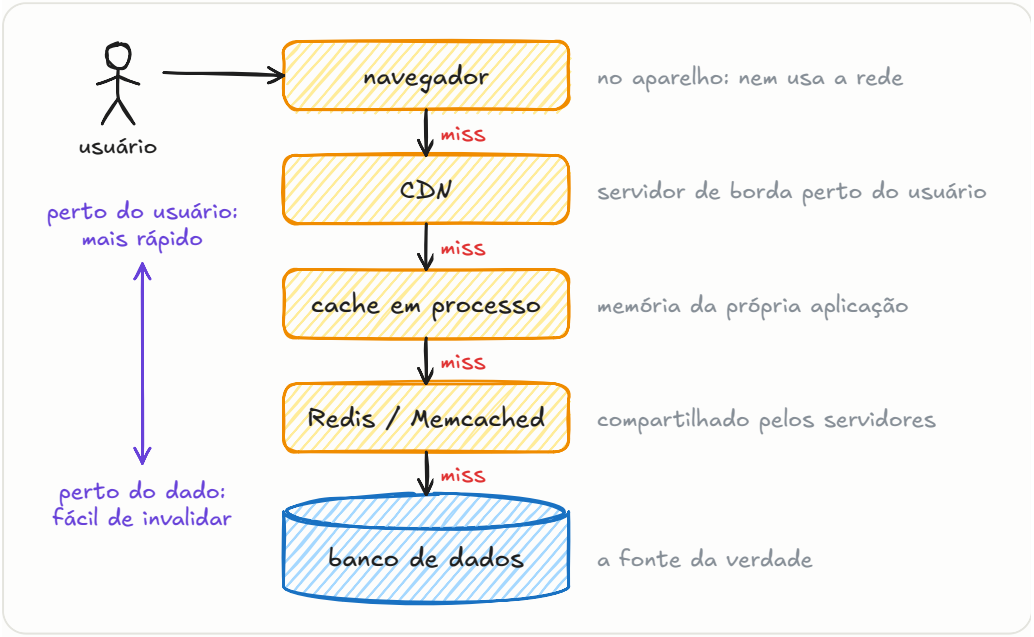
Taxa de acerto	Latência média	Leituras que chegam ao banco
90%	1 + 0,10 × 20 = 3 ms	10%

Taxa de acerto	Latência média	Leituras que chegam ao banco
99%	1 + 0,01 × 20 = 1,2 ms	1%

Subir de 90% para 99% parece pouco, mas divide por dez a carga do banco. O banco só enxerga os misses.

Camadas

Entre o usuário e o banco pode haver várias camadas, e a leitura só desce para a seguinte num miss:



Cada miss desce uma camada. As de cima são mais rápidas para o usuário; as de baixo, mais fáceis de manter atualizadas.

- **Navegador:** com `Cache-Control: max-age=3600`, o aparelho reaproveita a resposta por uma hora sem tocar a rede.
- **CDN** (cap. 15): servidores de borda guardam imagens, scripts e até páginas inteiras perto do usuário.
- **Cache em processo:** um mapa na memória da própria aplicação. Responde em menos de um microssegundo, mas cada instância tem a sua cópia.

- **Cache distribuído:** Redis ou Memcached, compartilhado por todas as instâncias. Custa uma ida pela rede: cerca de meio milissegundo no mesmo datacenter.
- **Banco:** a fonte da verdade, que também mantém em memória as páginas mais lidas.

Quanto mais perto do usuário, mais difícil invalidar: você apaga uma chave no Redis com um comando, mas não alcança o navegador de quem já baixou a resposta.

Validade

Toda entrada precisa de uma regra para sair. O **TTL** (time to live, tempo de vida) limita a idade do dado: com TTL de 60 s, ninguém vê um preço com mais de um minuto de atraso. A **invalidação explícita** apaga a chave quando o dado muda na origem. Use as duas juntas: a invalidação mantém o dado fresco no caso comum, e o TTL corrige o que ela deixar escapar. Não por acaso, a frase citada pelo autor, atribuída a Phil Karlton, diz que só há duas coisas difíceis em computação: invalidar cache e dar nome às coisas.

Analogia

O caixa da padaria tem um papel colado no balcão com o preço dos vinte itens que mais saem. Pão francês, café, pão de queijo: ele olha o papel e cobra na hora. Para um item raro, vai até o escritório consultar a tabela completa. O problema aparece quando o dono reajusta o pão e ninguém troca o papel: o caixa continua cobrando o preço antigo, rápido e errado. Decidir quando trocar o papel é a invalidação.

NO MUNDO REAL

- **Facebook:** o artigo *Scaling Memcache at Facebook* (NSDI 2013) descreve a frota de Memcached que fica na frente do MySQL e já atendia bilhões de requisições por segundo.
- **Redis e Memcached** são os caches distribuídos mais comuns, inclusive como serviço gerenciado (o Amazon ElastiCache, por exemplo). Depois que o Redis mudou de licença, em 2024, a Linux Foundation lançou o **Valkey**, um fork aberto e compatível.

- **HTTP:** navegadores e CDNs seguem as regras de cache da RFC 9111, com cabeçalhos como `Cache-Control` e `ETag`. É o cache mais barato que existe, porque a requisição nem chega ao seu servidor.
- **Bancos:** o PostgreSQL (`shared_buffers`) e o InnoDB do MySQL (buffer pool) guardam em memória as páginas mais lidas.

TRADE-OFFS

Você ganha

- Latência menor: um dado em memória volta em micro ou nanossegundos; uma consulta ao banco custa milissegundos.
- Menos carga na origem, o que adia a hora de escalar o banco.
- Fôlego em picos de leitura concentrados em poucos itens.

Você paga

- Dados que podem estar velhos por até um TTL.
- Mais uma peça para operar, monitorar e dimensionar.
- Risco de enxurrada no banco quando o cache esvazia ou uma chave quente expira.

NA ENTREVISTA

- Traga o cache assim que o requisito disser "muita leitura" ou "latência baixa", e diga **o que** vai no cache, **com qual chave** e **com qual TTL**: "Cacheio o perfil em `user:{id}` no Redis, com TTL de 5 minutos e invalidação quando o perfil muda."
- Estime o efeito no banco: com 95% de acerto, 20 mil leituras por segundo viram mil no banco.
- Diga quanto atraso cada dado tolera: "Na vitrine, um preço com até um minuto de atraso é aceitável; no checkout, leio direto do banco." É a consistência eventual (cap. 7) aplicada.

CUIDADO COM

- **Avalanche de misses (cache stampede):** quando uma chave muito acessada expira, centenas de requisições dão miss juntas e vão todas ao banco. Defesas: deixar só uma recarregar a chave enquanto as outras esperam, renovar antes de expirar e espalhar os TTLs com um pouco de aleatoriedade.
- **Chave incompleta vaza dados:** se a resposta depende do usuário, do idioma ou da permissão, isso precisa estar na chave. No Natal de 2015, um erro de configuração de cache na Steam mostrou páginas de conta de cerca de 34 mil usuários a outras pessoas.
- **Cache frio:** depois de um reinício, o cache começa vazio e o banco leva a carga inteira.
- **Cache não conserta consulta lenta:** só a esconde enquanto a taxa de acerto é alta.

TESTE-SE

1 **O que é a taxa de acerto (hit ratio) de um cache e por que ela importa tanto?**

É a fração das leituras que o cache responde. Ela decide quanto da carga chega ao banco: com 99% de acerto, o banco recebe 1% das leituras; com 90%, recebe dez vezes mais.

2 **Num e-commerce, você trataria do mesmo jeito o estoque exibido na página do produto e o estoque conferido ao fechar o pedido?**

Não. Na vitrine, um número com alguns segundos de atraso é aceitável, e um cache com TTL curto resolve. No fechamento, a conferência vai à fonte da verdade, com transação ou reserva; senão você vende o que não tem.

3 **Colocar um cache na frente do banco sempre reduz a latência?**

Não. Num miss você paga o cache e o banco, então, com taxa de acerto baixa (chaves muito variadas, dados lidos uma vez só), o cache só acrescenta latência e custo.

CONECTA COM

42 Estratégias de cache

43 Políticas de remoção (LRU, LFU)

15 CDN

7 Consistência forte vs. eventual

5 Throughput vs. latência

Estratégias de cache

Caching Strategies (Cache Aside, Write Through, etc.)

EM UMA FRASE

As estratégias de cache definem quem preenche o cache nas leituras e o que acontece com ele nas escritas; cache-aside, write-through e write-back trocam frescor, simplicidade e velocidade de escrita de jeitos diferentes.

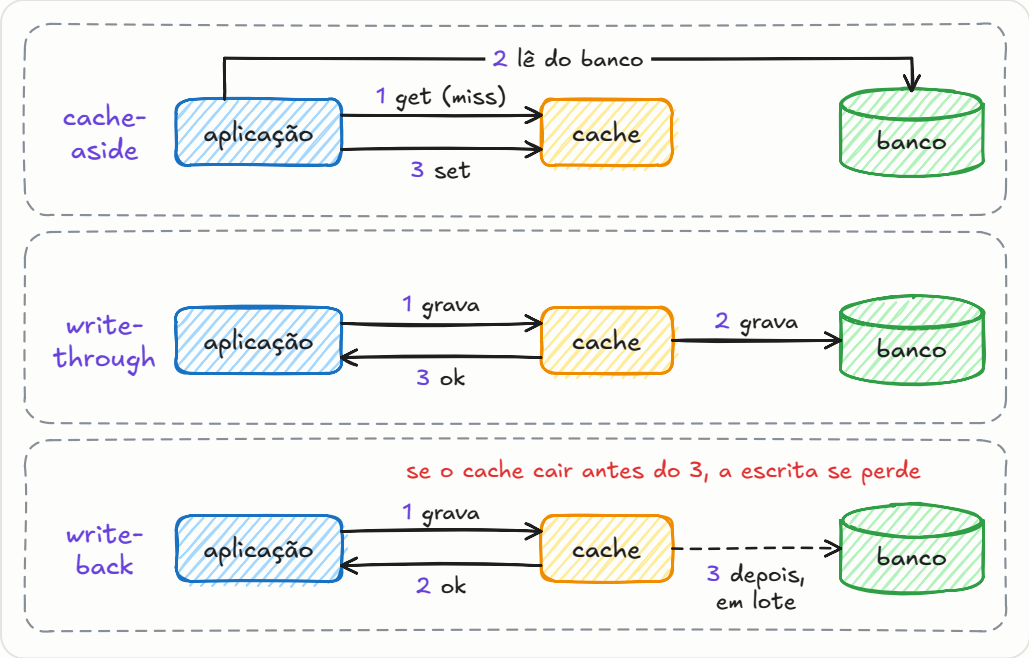
O ESSENCIAL · tradução do artigo original

- **Cache-aside** significa que a aplicação lê do cache e, quando não encontra o dado (um miss), carrega do banco de dados e grava no cache.
- **Write-through** grava no cache e no banco ao mesmo tempo, garantindo que o cache e a fonte fiquem sempre sincronizados.
- **Write-back** grava primeiro no cache e só depois descarrega no banco, o que é rápido, mas arriscado se o cache falhar.

Cada estratégia equilibra de um jeito diferente o frescor dos dados, a complexidade e o desempenho. Os entrevistadores adoram quando você diz qual estratégia escolheria para um determinado cenário.

APROFUNDANDO

Toda estratégia responde a duas perguntas: **quem preenche o cache nas leituras** e **o que acontece com ele nas escritas**. O diagrama mostra as três do artigo, passo a passo.



Mesmas peças, ordens diferentes. No write-through, o ok só volta depois de o banco gravar; no write-back, volta antes.

Cache-aside (lazy loading)

A aplicação conversa com os dois, e só entra no cache o que alguém leu:

```
def buscar_produto(id):
    produto = cache.get(f"produto:{id}")
    if produto is None:                        # miss
        produto = banco.buscar_produto(id)
        cache.set(f"produto:{id}", produto, ttl=300)
    return produto

def mudar_preco(id, preco):
    banco.atualizar_preco(id, preco)
    cache.delete(f"produto:{id}")              # apaga em vez de atualizar
```

A escrita **apaga** a chave em vez de gravar o valor novo. Se gravasse, duas atualizações concorrentes poderiam chegar ao cache na ordem inversa da que chegaram ao banco, e o preço antigo ficaria no cache até o TTL vencer. Apagando, a próxima lei-

tura busca o valor atual. Ainda sobra uma corrida rara (uma leitura lenta que repõe o valor velho logo depois do delete), e por isso o TTL continua lá como rede de segurança.

Read-through

Igual ao cache-aside, mas quem vai ao banco no miss é o próprio cache; a aplicação só fala com ele. A lógica de carga sai do código e vira configuração da biblioteca ou do serviço.

Write-through

A escrita passa pelo cache, que grava no banco e só então confirma. Uma leitura logo depois da escrita já encontra o dado no cache, mas cada escrita paga as duas gravações, e o cache se enche de dados que talvez ninguém leia.

Write-back (write-behind)

A escrita é confirmada assim que chega ao cache, e o banco é atualizado depois, em lote. Um contador que recebe mil incrementos por segundo pode virar uma única gravação por segundo no banco. O preço: se o nó de cache cair antes de descarregar, essas escritas somem. Por isso o write-back aparece onde há proteção contra a perda, como a bateria das controladoras de disco, ou onde perder alguns segundos é tolerável.

Write-around

A escrita vai só para o banco, apagando a chave se ela existir, e o cache se enche apenas nas leituras. É o que o próprio cache-aside faz nas escritas, e serve bem para dados muito escritos e pouco lidos, como logs de auditoria.

Analogia

Na recepção de um consultório, a secretária tem a agenda do dia impressa na mesa (o cache) e o sistema oficial no computador (o banco). **Cache-aside:** o paciente pergunta o horário; se não estiver na folha, ela consulta o sistema e anota. **Write-through:** ao marcar consulta, lança no sistema e na folha antes de liberar o paciente. **Write-back:** na correria, anota só na folha e passa tudo para o sistema no fim do dia. Se o café cair na folha antes disso, as marcações somem.

NO MUNDO REAL

- **Amazon ElastiCache:** a documentação da AWS apresenta o lazy loading (o cache-aside) e o write-through como as estratégias básicas e recomenda somar um TTL às duas.
- **DynamoDB Accelerator (DAX):** fica no caminho das requisições e faz read-through e write-through: grava primeiro no DynamoDB e depois atualiza o próprio cache.
- **Hazelcast:** o `MapStore` faz write-through quando o atraso de escrita é zero e write-behind quando você configura alguns segundos de atraso.
- **Linux:** o page cache é write-back. Um `write()` volta antes de o dado chegar ao disco; quem precisa de durabilidade chama `fsync`, como os bancos fazem com o WAL (cap. 27).

TRADE-OFFS

	Cache-aside	Write-through	Write-back
Quem grava no banco	A aplicação	O cache, antes de confirmar	O cache, depois, em lote
Latência da escrita	A do banco	Cache + banco	Só a do cache
Cache após a escrita	Vazio até a próxima leitura	Atualizado	Atualizado; o banco fica para trás
Principal risco	Corridas e misses em massa	Escritas mais lentas	Perder escritas se o cache cair

	Cache-aside	Write-through	Write-back
Bom para	Leitura pesada em geral	Ler logo depois de escrever	Contadores e escrita intensa

NA ENTREVISTA

- Escolha por cenário e justifique em uma frase: “Para o catálogo, cache-aside com TTL de 10 minutos e delete na atualização. Para o contador de visualizações, write-back: perder alguns segundos de contagem num crash é aceitável, e o banco deixa de receber milhares de escritas por segundo.”
- Mencione a corrida do cache-aside e explique por que apaga em vez de atualizar; isso mostra que você já pensou em concorrência.
- Se escolher write-back, diga como protege o dado: réplica do cache, persistência em disco (o AOF do Redis) ou perda tolerada por contrato.

CUIDADO COM

- ATUALIZAÇÃO: o “ao mesmo tempo” do write-through é uma simplificação. Cache e banco são dois sistemas, sem transação entre eles: se a segunda gravação falhar, os dois divergem, e escritas concorrentes podem chegar em ordens diferentes a cada um. “Sempre sincronizados” é o objetivo, não uma garantia; mantenha um TTL mesmo com write-through.
- **Ordem no cache-aside:** grave no banco e só depois apague a chave. Apagar antes abre uma janela em que uma leitura recarrega o valor antigo.
- **Write-back e quem lê o banco direto:** relatórios e integrações que consultam o banco veem dados atrasados.

TESTE-SE

1 **Qual a diferença entre cache-aside e read-through?**

No cache-aside, a aplicação trata o miss: vai ao banco e grava no cache. No read-through, o próprio cache busca no banco, e a aplicação só conversa com ele.

2 **Um contador de visualizações de vídeo recebe milhares de incrementos por segundo, e perder alguns segundos de contagem num crash é aceitável. Qual estratégia de escrita você usa?**

Write-back. O cache acumula os incrementos e grava o total no banco em lote, poupando milhares de escritas por segundo; a perda possível está dentro do tolerado.

3 **Com write-through, cache e banco nunca divergem?**

Podem divergir. Sem uma transação entre os dois, uma falha entre as gravações ou escritas concorrentes podem deixar valores diferentes; por isso o TTL continua útil.

CONECTA COM

41 Cache

43 Políticas de remoção (LRU, LFU)

7 Consistência forte vs. eventual

27 Write-Ahead Logging (WAL)

Políticas de remoção (LRU, LFU)

Cache Eviction Policies (LRU, LFU)

EM UMA FRASE

Quando o cache enche, a política de remoção escolhe quem sai; o LRU tira o item usado há mais tempo, o LFU tira o menos usado no total, e cada um erra em padrões de acesso diferentes.

O ESSENCIAL · tradução do artigo original

As políticas de remoção (eviction) de cache decidem quais itens tirar quando o cache está cheio.

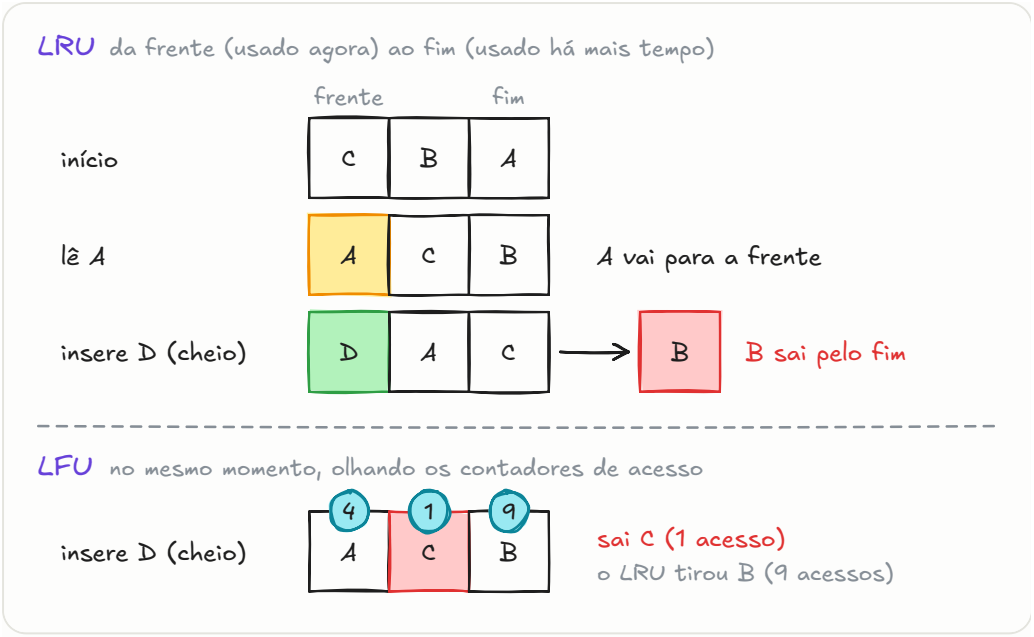
- **LRU** (Least Recently Used, o usado menos recentemente) remove os itens que não foram acessados recentemente, supondo que os itens recentes têm mais chance de ser usados de novo.
- **LFU** (Least Frequently Used, o usado com menos frequência) remove os itens raramente acessados, com foco na popularidade de longo prazo.

Alguns sistemas usam remoção aleatória, FIFO ou algoritmos mais avançados. A ideia central é que o espaço do cache é limitado, então você quer manter na memória os itens mais valiosos.

APROFUNDANDO

LRU: uma lista ordenada pelo último acesso

Imagine o cache como uma lista que vai do item usado agora (a frente) ao usado há mais tempo (o fim). Todo acesso move o item para a frente; quando o cache está cheio e chega um item novo, sai o do fim.



Em cima, o LRU com três vagas: ler A o leva para a frente, e inserir D expulsa B, o último. Embaixo, o LFU no mesmo momento: tiraria C, com um acesso só, e não B, o mais popular.

Para que cada operação custe O(1), a implementação clássica junta um **mapa de hash** (da chave para o nó, para achar o item sem percorrer a lista) com uma **lista duplamente encadeada** (para tirar um nó do meio e pô-lo na frente sem varrer nada).

LFU: um contador por item

O LFU guarda quantas vezes cada item foi acessado e remove o de menor contagem; no empate, o menos recente. Ele protege o que é popular de verdade: um item lido 500 vezes não sai só porque ficou alguns minutos sem acesso.

Onde cada um erra

- **LRU e a varredura:** um relatório noturno que lê 50 mil produtos uma única vez empurra para fora os mil produtos quentes do dia, e a taxa de acerto despenca até o cache se refazer.
- **LFU e a popularidade velha:** a notícia que bombou na semana passada acumulou milhões de acessos e fica no cache indefinidamente, enquanto itens novos entram com contagem 1 e são os primeiros a sair. A correção é envelhecer os contadores, por exemplo dividindo todos por dois de tempos em tempos.

Na prática, tudo é aproximado

Uma lista exata custa dois ponteiros por chave e uma atualização a cada leitura, um gargalo quando muitas threads leem ao mesmo tempo. Por isso os sistemas aproximam. O Redis sorteia algumas chaves (5, por padrão) e remove a de acesso mais antigo entre elas; o LFU dele usa um contador logarítmico de 8 bits que decai com o tempo. O Caffeine, do Java, usa o W-TinyLFU: uma pequena janela LRU recebe os itens novos, e um item só entra no cache principal se a frequência estimada dele, guardada num sketch probabilístico parente do Bloom filter (cap. 30), for maior que a da vítima.

> *Analogia*
>
> Seu guarda-roupa tem 30 cabides; para pendurar uma camisa nova, alguma precisa sair. Pela regra **LRU**, sai a que você não veste há mais tempo, e em janeiro o casaco vai embora, para fazer falta em junho. Pela **LFU**, sai a que você vestiu menos vezes no total, e a camisa da seleção, usada em todos os jogos da última Copa, segura o cabide por anos sem ser vestida de novo.

NO MUNDO REAL

- **Redis:** com `maxmemory` definido, as políticas `allkeys-lru` e `allkeys-lfu` escolhem a vítima entre todas as chaves, e as variantes `volatile-*`, só entre as que têm TTL. O padrão, `noeviction`, não remove nada: quando o limite é atingido, as escritas passam a falhar.
- **Caffeine:** essa biblioteca de cache em memória para Java, sucessora do cache do Guava e suportada pelo Spring, usa W-TinyLFU em vez de LRU puro.
- **PostgreSQL:** o buffer pool usa o clock sweep, uma aproximação de LRU com um pequeno contador de uso por página.
- **ZFS:** o cache em memória é o ARC (Adaptive Replacement Cache), que divide o espaço entre itens recentes e frequentes e ajusta a proporção sozinho.

TRADE-OFFS

	LRU	LFU
Critério	Último acesso	Número de acessos

	LRU	LFU
Custo por acesso	Mover um nó na lista: O(1)	Atualizar o contador: O(1) com listas por frequência, mais memória
Vai bem quando	O interesse muda rápido (sessões, notícias do dia)	A popularidade é estável (catálogo, configurações)
Erra quando	Uma varredura única passa por cima dos itens quentes	Itens antigos acumulam contagem e itens novos não têm chance
Ajuste típico	LRU segmentado: o item só vira "protegido" no segundo acesso	Envelhecer os contadores

NA ENTREVISTA

- Ao propor um cache, diga o tamanho e o que acontece quando ele enche: "Com 10 GB de Redis e `allkeys-lru`, os perfis menos acessados saem primeiro."
- Se pedirem para implementar, é o clássico LRU em O(1): mapa de hash mais lista duplamente encadeada. Em Python, `OrderedDict` com `move_to_end` resolve em poucas linhas, mas explique a estrutura por baixo.
- Mostre que conhece a fraqueza de cada um: o LRU sofre com varreduras; o LFU precisa envelhecer os contadores.

CUIDADO COM

- `ATUALIZAÇÃO:` os "algoritmos avançados" que o original cita de passagem são o padrão hoje. Quase ninguém usa LRU ou LFU exatos: o Redis amostra chaves, o Caffeine usa W-TinyLFU, o ZFS usa ARC. E pesquisas recentes, como o S3-FIFO (2023) e o SIEVE (2024), mostraram que variações de FIFO, mais simples e baratas que o LRU, acertam tanto quanto ele, ou mais, em muitas cargas web reais.
- **Remoção não é expiração:** o TTL tira o dado por idade, mesmo com espaço sobrando; a remoção tira por falta de espaço.

- **Redis sem `maxmemory`:** em 64 bits não há limite, e o processo cresce até a máquina ficar sem memória.

TESTE-SE

1 Qual a diferença entre LRU e LFU?

O LRU remove o item acessado há mais tempo; o LFU remove o item acessado menos vezes. O primeiro olha a recência; o segundo, a frequência.

2 Um cache de catálogo é varrido toda noite por um job que lê todos os produtos uma vez, e de manhã a taxa de acerto despenca. Qual é o problema e o que você muda?

O LRU deixa a varredura expulsar os itens quentes. Troque por uma política resistente a varreduras (LFU com envelhecimento, W-TinyLFU, LRU segmentado) ou faça o job ler sem passar pelo cache.

3 Configurar TTL em todas as chaves dispensa uma política de remoção?

Não. O TTL remove por idade, mas, se as escritas chegarem mais rápido do que as chaves expiram, o cache enche do mesmo jeito. Sem política (o `noeviction` do Redis), as escritas passam a falhar.

CONECTA COM

41 Cache · 42 Estratégias de cache · 30 Bloom filter

Filas de mensagens

Message Queues (Point to Point)

EM UMA FRASE

Uma fila de mensagens guarda tarefas entre quem produz e quem processa; cada mensagem vai para um único consumidor e só sai da fila depois de confirmada, o que desacopla os serviços e absorve picos.

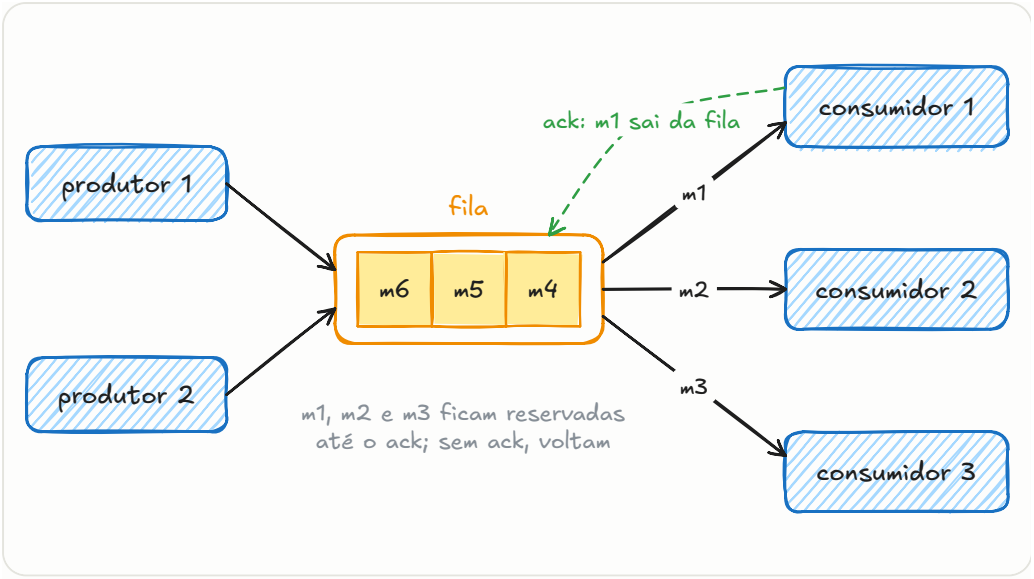
O ESSENCIAL · tradução do artigo original

Uma fila de mensagens permite que um componente envie mensagens a outro sem que os dois precisem estar online ao mesmo tempo. No modelo ponto a ponto, cada mensagem da fila é consumida por um único receptor e depois removida. Isso desacopla o remetente do destinatário, que podem escalar e falhar de forma independente. Filas são ótimas para tarefas em segundo plano, envio de e-mails e processamento assíncrono de tarefas pesadas. Pense nelas como uma lista de tarefas compartilhada entre serviços.

APROFUNDANDO

São três papéis: o **produtor** envia, o **broker** (RabbitMQ, Amazon SQS) guarda e o **consumidor** processa. O ciclo de vida de uma mensagem tem um detalhe que muda tudo:

1. O produtor envia a mensagem; o broker a grava (em disco e, em geral, em réplicas) e confirma o recebimento.
2. Um consumidor recebe a mensagem, que **não é apagada**: fica reservada, invisível para os outros consumidores.
3. O consumidor termina o trabalho e envia um **ack** (confirmação). Só então o broker apaga a mensagem.
4. Se o ack não chegar (o consumidor travou, caiu ou estourou o prazo), a mensagem volta para a fila e outro consumidor a recebe.



Cada mensagem vai para um único consumidor e continua guardada até o ack; sem ack no prazo, volta para a fila.

O passo 4 garante que a queda de um consumidor não perde mensagens, mas tem um efeito colateral: se o consumidor caiu depois de mandar o e-mail e antes do ack, o e-mail sai duas vezes. Essa garantia se chama **pelo menos uma vez** (at-least-once), e é por isso que consumidores precisam ser idempotentes (cap. 36).

Consumidores concorrentes

Vários consumidores na mesma fila dividem o trabalho. A conta de quantos você precisa é direta: se chegam 50 e-mails por segundo e cada envio leva 200 ms, pela lei de Little há em média 50 × 0,2 = 10 envios em andamento, então são pelo menos 10 consumidores, mais uma folga. Para processar mais rápido, basta subir mais consumidores; o produtor nem fica sabendo.

Amortecedor de picos

Numa Black Friday hipotética, o checkout gera 3 mil pedidos por minuto durante uma hora, mas o emissor de notas fiscais processa 2 mil por minuto. Sem fila, o checkout teria de recusar pedidos ou esperar. Com fila, ela acumula 60 mil mensagens no pico e esvazia quando o movimento cai, enquanto o cliente recebe "pedido recebido" na hora. Acompanhe o **tamanho da fila** e a **idade da mensagem mais antiga**: se crescem sem parar, faltam consumidores. E dê um teto à fila: cheia, ela passa a recusar

ou atrasar mensagens novas, e o produtor é obrigado a desacelerar. Esse sinal de "vá mais devagar" se chama **backpressure** (contrapressão); sem ele, a fila cresce até esgotar memória ou disco.

Ordem

Com consumidores concorrentes, a ordem global se perde: a mensagem 2 pode terminar antes da 1. Se a ordem importa por entidade (os eventos de um mesmo pedido), use uma fila FIFO com chave de grupo, como o `MessageGroupId` do SQS FIFO.

> *Analogia*
>
> Na cozinha de um restaurante, o garçom pendura a comanda no trilho e volta para o salão, sem esperar o prato ficar pronto. Cada cozinheiro livre pega a próxima comanda, e dois nunca fazem o mesmo pedido. Se um cozinheiro passa mal no meio do prato, a comanda volta para o trilho e outro assume. Na hora do rush, o trilho enche, mas o salão continua atendendo.

NO MUNDO REAL

- **Amazon SQS:** a mensagem recebida fica invisível por um prazo (o visibility timeout, 30 s por padrão) e reaparece se não for apagada até lá. Filas padrão entregam pelo menos uma vez e sem ordem garantida; filas FIFO preservam a ordem dentro de cada grupo de mensagens.
- **RabbitMQ:** o consumidor confirma com ack, e o prefetch limita quantas mensagens sem ack cada um pode segurar. Se a conexão cai, as pendentes voltam para a fila.
- **Jobs em segundo plano:** o Sidekiq (Ruby, sobre Redis) e o Celery (Python, sobre RabbitMQ ou Redis) usam filas para e-mails, relatórios e processamento de imagens.

TRADE-OFFS

Você ganha

- Produtor e consumidor sobem, caem e escalam de forma independente.
- Picos viram fila em vez de erro.
- Resposta rápida ao usuário, com o trabalho pesado feito depois.

Você paga

- Mais um componente crítico para operar (o broker também precisa de réplicas).
- Duplicatas e falta de ordem, que o consumidor precisa tolerar.
- Fluxo assíncrono: o resultado não sai na hora, e depurar fica mais difícil.

NA ENTREVISTA

- Traga a fila sempre que uma etapa for lenta, sujeita a falhas ou não precisar de resposta imediata: e-mails, transcodificação de vídeo, geração de relatórios.
- Fale do contrato de entrega: "A entrega é pelo menos uma vez, então o consumidor é idempotente; uso o ID do pedido como chave de deduplicação."
- Diga o que acontece com falhas: retries com backoff (cap. 35) e, depois de algumas tentativas, uma dead letter queue (cap. 46).
- Mostre que monitora a fila: o tamanho e a idade da mensagem mais antiga disparam o autoscaling dos consumidores.

CUIDADO COM

- ATUALIZAÇÃO: "consumida e depois removida" pula o passo mais importante. A mensagem só sai depois do ack; até lá, pode ser entregue de novo, inclusive a outro consumidor. O efeito de "exatamente uma vez", de ponta a ponta, depende de um consumidor idempotente.
- **Fila não é banco de dados:** as mensagens têm retenção limitada (no SQS, no máximo 14 dias) e não servem para consultas.
- **Kafka não é uma fila ponto a ponto clássica:** ler não apaga a mensagem; cada grupo de consumidores guarda a própria posição (offset) num log, como mostra o capítulo 45.

TESTE-SE

1 O que significa entrega "pelo menos uma vez" (at-least-once)?

Que o broker garante que toda mensagem chega a um consumidor, mas pode entregá-la mais de uma vez, por exemplo quando o consumidor cai depois de processar e antes de confirmar.

2 Seu serviço de upload precisa gerar miniaturas das fotos, o que leva 2 s por imagem. Como você desenha isso?

O upload grava a foto, publica uma mensagem numa fila e responde na hora. Consumidores geram as miniaturas, escalam pelo tamanho da fila e são idempotentes, para que uma entrega repetida só refaça a mesma miniatura.

3 Se cada mensagem vai para um único consumidor, como ela pode ser processada duas vezes?

Se o ack não chega a tempo (consumidor lento, travado ou que caiu depois de processar), o broker devolve a mensagem para a fila, e outro consumidor a processa de novo.

CONECTA COM

45 Pub/Sub · 46 Dead letter queue · 36 Idempotência · 35 Retries e backoff exponencial · 5 Throughput vs. latência

Pub/Sub

Pub Sub (Publish Subscribe)

EM UMA FRASE

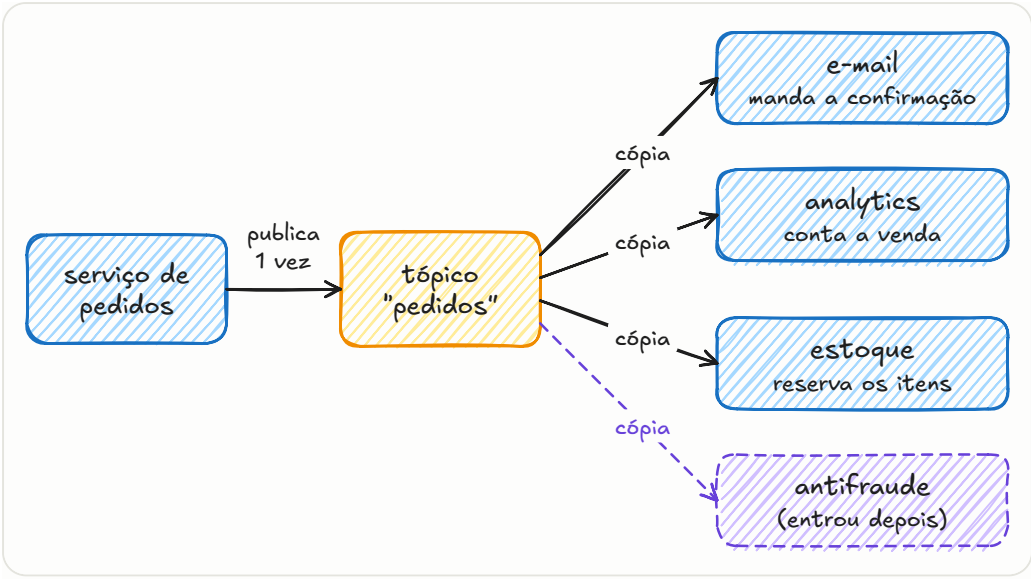
No pub/sub, o publicador envia cada evento a um tópico, e cada assinante recebe a sua cópia; vários serviços reagem ao mesmo fato sem que o publicador saiba quem eles são.

O ESSENCIAL · tradução do artigo original

No pub/sub, os publicadores enviam mensagens para tópicos, e não diretamente para os consumidores. Os assinantes escutam os tópicos que lhes interessam e recebem cópias das mensagens relevantes. Isso permite uma comunicação no estilo transmissão (broadcast) e um acoplamento fraco entre produtores e consumidores. Vários serviços podem reagir ao mesmo evento de formas diferentes, como registro de logs, analytics e notificações. Em entrevistas, o pub/sub costuma aparecer em designs orientados a eventos, como feeds de atividade ou event sourcing.

APROFUNDANDO

Pense no evento “pedido 123 criado”. O serviço de pedidos o publica uma vez no tópico `pedidos`, e o broker entrega uma cópia a cada assinatura: o e-mail manda a confirmação, o analytics conta a venda, o estoque reserva os itens.



Uma publicação, uma cópia por assinante. O antifraude chegou depois e só precisou criar a própria assinatura.

O ganho está no que o publicador **não** sabe. Se amanhã surgir um serviço antifraude, ele cria uma assinatura e passa a receber os eventos, sem nenhuma mudança no serviço de pedidos. Com chamadas diretas, o serviço de pedidos chamaria três APIs, ficaria esperando as três e precisaria de um deploy a cada consumidor novo.

Fila ou pub/sub? Os dois

Na fila (cap. 44), cada mensagem vai para **um** consumidor: é distribuição de trabalho. No pub/sub, cada mensagem vai para **todas** as assinaturas: é distribuição de fatos. Na prática, os dois se combinam: cada assinatura funciona como uma fila própria, e as instâncias de um mesmo serviço competem por ela. Se o serviço de e-mail roda em 4 instâncias, cada evento gera um e-mail, e não quatro: uma cópia por **serviço**, não por instância. É assim nos grupos de consumidores do Kafka, nas assinaturas do Google Cloud Pub/Sub e no fan-out em que um tópico do Amazon SNS entrega cada mensagem a várias filas SQS.

Assinatura durável ou efêmera

Se o estoque ficar fora do ar por dez minutos, o que acontece com os eventos desse intervalo? Depende do broker. No Redis Pub/Sub, a assinatura é efêmera: quem não estava conectado perde a mensagem. No Kafka, os eventos ficam num log durante o pe-

ríodo de retenção (7 dias, por padrão), e cada grupo de consumidores guarda onde parou (o offset); ao voltar, o estoque continua de onde estava. Para eventos de negócio, você quase sempre quer a durável.

Ordem

Com várias partições e várias instâncias, não existe ordem global. O Kafka garante a ordem dentro de cada partição: use o ID do pedido como chave para que "criado", "pago" e "enviado" do mesmo pedido caiam na mesma partição e cheguem em sequência.

> *Analogia*
>
> O grupo de WhatsApp do condomínio: a síndica escreve uma vez "a água será cortada amanhã às 9h", e cada morador recebe a sua cópia. Um enche baldes, outro avisa a diarista, o porteiro cola um cartaz no elevador. Quem estava sem internet recebe quando reconectar. Mas quem entra no grupo depois não vê o aviso: a assinatura só vale dali para a frente.

NO MUNDO REAL

- **Apache Kafka:** criado no LinkedIn para distribuir eventos de atividade, como visualizações de página, a vários sistemas ao mesmo tempo, e aberto como software livre em 2011.
- **Amazon SNS + SQS:** a AWS documenta o fan-out em que um tópico do SNS entrega cada mensagem a várias filas SQS, uma por serviço.
- **Google Cloud Pub/Sub:** cada assinatura recebe as mensagens publicadas depois da sua criação, e os clientes de uma mesma assinatura dividem o trabalho.
- **RabbitMQ:** exchanges do tipo `fanout` copiam a mensagem para todas as filas ligadas a elas; as do tipo `topic`, para as filas cujo padrão casa com a chave de roteamento.

TRADE-OFFS

	Fila (ponto a ponto)	Pub/sub
Cada mensagem vai para	Um consumidor	Cada assinatura

	Fila (ponto a ponto)	Pub/sub
Uso típico	Distribuir trabalho	Avisar que algo aconteceu
Novo interessado	Disputa as mesmas mensagens	Ganha a própria cópia
O que o produtor sabe	Que existe um trabalho a fazer	Nada sobre quem escuta
Risco	Fila crescendo sem consumidor	Evento mal desenhado vira contrato com muitos serviços

NA ENTREVISTA

- Use pub/sub quando **um fato** interessa a **vários serviços**: "pedido criado", "usuário cadastrado", "pagamento aprovado". Para uma tarefa que só um serviço executa, use fila.
- Diga o tópico, a chave de partição e quem assina: "Publico `PedidoCriado` no tópico `pedidos`, particionado por ID do pedido; e-mail, estoque e analytics têm cada um o seu grupo de consumidores."
- Em fluxos de vários passos, eventos podem coordenar uma saga coreografada (cap. 39).
- Em feeds, mostre o custo do fan-out: um post de uma conta com 10 milhões de seguidores vira 10 milhões de entregas, e muitos designs montam o feed dessas contas na hora da leitura.

CUIDADO COM

- **Assinatura efêmera perde mensagem:** no Redis Pub/Sub, quem não está conectado no momento da publicação nunca recebe aquela mensagem. Para eventos de negócio, use um broker com retenção (Kafka, Redis Streams, um serviço gerenciado).
- **Pub/sub não é event sourcing:** event sourcing é guardar o estado como uma sequência de eventos, que vira a fonte da verdade; pub/sub é só um jeito de distribuí-los. Dá para usar um sem o outro.
- **Acoplamento fraco não é acoplamento zero:** o formato do evento vira um contrato com todos os assinantes. Mudar um campo sem versionar quebra serviços que você nem sabe que existem.

TESTE-SE

1 **Qual a diferença entre uma fila ponto a ponto e um tópico de pub/sub?**

Na fila, cada mensagem é entregue a um único consumidor. No tópico, cada assinatura recebe a sua cópia de todas as mensagens.

2 **Quando um pagamento é aprovado, o sistema precisa enviar o recibo, liberar o pedido e atualizar o painel financeiro. Como você conecta isso?**

O serviço de pagamentos publica `PagamentoAprovado` num tópico, e os três serviços assinam, cada um com a sua assinatura. O pagamento não conhece nenhum deles, e um novo interessado só cria mais uma assinatura.

3 **O serviço de e-mail roda em 5 instâncias, todas ligadas ao tópico de pedidos. O cliente recebe 5 e-mails?**

Não, se as instâncias compartilharem a mesma assinatura (o mesmo grupo de consumidores): elas dividem as mensagens, e cada evento é processado uma vez pelo serviço. Se cada instância criar a própria assinatura, aí sim saem 5 e-mails.

CONECTA COM

44 Filas de mensagens
46 Dead letter queue
39 Transações distribuídas: o padrão SAGA
9 Microsserviços vs. monólito

Dead letter queue

Dead Letter Queues

EM UMA FRASE

A dead letter queue é a fila para onde vão as mensagens que falharam várias vezes; ela tira as mensagens envenenadas do caminho e as guarda para alguém investigar, corrigir e reprocessar.

O ESSENCIAL · tradução do artigo original

Uma dead letter queue (DLQ, ou fila de mensagens mortas) guarda as mensagens que não puderam ser processadas com sucesso depois de várias tentativas. Em vez de tentar para sempre e travar a fila principal, essas mensagens são colocadas de lado. Os engenheiros podem inspecionar a dead letter queue para depurar problemas, corrigir dados ou reprocessar as mensagens mais tarde. Esse padrão aumenta a resiliência e evita que o sistema fique travado em "mensagens envenenadas" (poison messages). Pense nela como uma área de espera para tarefas problemáticas.

APROFUNDANDO

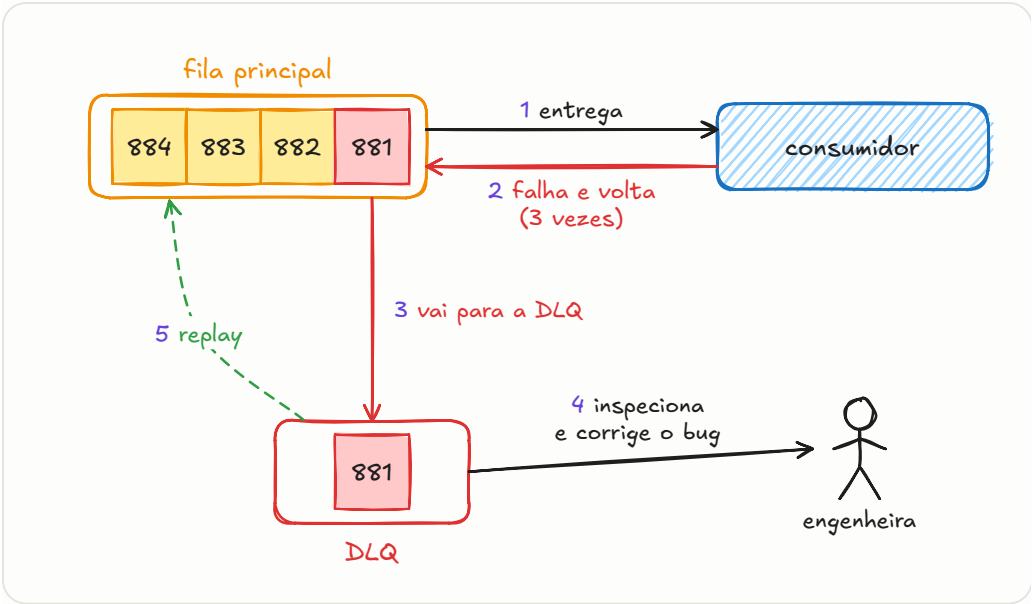
O nome vem dos correios: "dead letter" é a carta que não pode ser entregue nem devolvida. Numa fila, é a mensagem que o consumidor não consegue processar.

O caminho de uma mensagem envenenada

Imagine uma fila de pedidos com limite de 3 tentativas:

1. O consumidor recebe o pedido 881, cujo JSON traz uma data num formato que ele não entende. Ele lança uma exceção e não manda o ack.
2. A mensagem volta para a fila, e o broker conta mais uma entrega (no SQS, o `ApproximateReceiveCount`).
3. Na terceira falha, o broker para de devolvê-la à fila principal e a move para a DLQ.
4. Um alarme dispara porque a DLQ deixou de estar vazia. Alguém inspeciona a mensagem e corrige o parser.

5. Vem o **replay**: a mensagem volta para a fila principal e, agora, passa.



A 881 falha três vezes e sai do caminho, e as outras mensagens seguem. Depois da correção, o replay a devolve à fila principal.

Sem a DLQ, o pedido 881 voltaria para sempre. Numa fila comum, ocuparia consumidores sem parar; numa fila com ordem (um grupo FIFO, uma partição do Kafka), bloquearia todas as mensagens atrás dele.

Falha transitória ou permanente

Nem todo erro merece as mesmas tentativas. Um timeout do banco é **transitório**: vale tentar de novo com backoff exponencial (cap. 35). Um JSON inválido é **permanente**: tentar de novo só atrasa o inevitável, e o melhor é mandar direto para a DLQ. Um limite entre 3 e 5 tentativas é comum. Com 1, qualquer soluço momentâneo lota a DLQ; com 50, uma mensagem envenenada ocupa consumidores por muito tempo.

O que guardar e como reprocessar

Junto com a mensagem, guarde o contexto do erro: a exceção, o número de tentativas, a hora da primeira falha e a versão do consumidor. Sem isso, a investigação vira arqueologia. No replay, devolva as mensagens aos poucos e garanta que o consumidor é idempotente (cap. 36), porque algumas podem ter sido processadas pela metade. E lembre que elas chegam fora de ordem em relação às mais novas.

Analogia

O entregador tenta deixar uma encomenda três vezes e não consegue: o número da casa não existe. Em vez de rodar com o pacote para sempre, ele o deixa na agência, numa prateleira separada, e segue com as outras entregas. Lá, alguém confere a etiqueta, corrige o endereço com o remetente e põe o pacote de volta na rota.

NO MUNDO REAL

- **Amazon SQS:** a redrive policy define o `maxReceiveCount`; passada essa contagem de entregas, a mensagem vai para a DLQ configurada. O próprio SQS oferece o redrive de volta para a fila de origem depois da correção.
- **RabbitMQ:** mensagens rejeitadas sem reenfileiramento, expiradas por TTL ou descartadas por fila cheia vão para um dead letter exchange.
- **Kafka:** o broker não tem DLQ. O consumidor publica a mensagem problemática num tópico separado; o Kafka Connect faz isso nos conectores de destino (sink) com `errors.deadletterqueue.topic.name`.
- **Sidekiq:** depois de 25 novas tentativas com backoff exponencial, que somam cerca de 20 dias, o job vai para o conjunto Dead, de onde pode ser reexecutado pela interface web.

TRADE-OFFS

Você ganha

- A fila principal continua andando quando uma mensagem não tem conserto.
- Nenhuma mensagem some em silêncio: a falha fica guardada, com contexto.
- Uma forma controlada de reprocessar depois da correção.

Você paga

- Mais uma fila para monitorar; sem alarme, ela vira um cemitério que ninguém visita.
- Retenção e replay para planejar, porque a DLQ também expira mensagens.
- Ordem quebrada: quem passou pela DLQ volta depois das mensagens mais novas.

NA ENTREVISTA

- Toda vez que desenhar uma fila, complete a frase: “com retries e backoff para erros transitórios e uma DLQ depois de 5 tentativas, com alarme quando ela tiver qualquer mensagem”.
- Diferencie erro transitório de permanente; isso mostra maturidade operacional.
- Explique o replay: “Corrigido o bug, devolvo as mensagens da DLQ aos poucos; o consumidor é idempotente, então reprocessar é seguro.”

CUIDADO COM

- ATUALIZAÇÃO: a DLQ não é um recurso de todo broker. SQS, RabbitMQ e Google Cloud Pub/Sub têm suporte nativo; no Kafka, você implementa no consumidor, e, enquanto ele insistir numa mensagem envenenada, o offset não avança e a partição inteira fica parada.
- **A DLQ também expira mensagens:** a AWS recomenda que a retenção da DLQ seja maior que a da fila de origem, para não perder justamente o que você queria investigar.
- **DLQ sem dono:** se ninguém olha, ela só adia a perda de dados. Defina alarme, responsável e prazo para tratar.

TESTE-SE

1 **O que é uma mensagem envenenada (poison message)?**

Uma mensagem que falha toda vez que é processada, por dado inválido ou bug, e que, sem DLQ, voltaria para a fila indefinidamente.

2 **Durante 30 segundos de instabilidade no banco, um consumidor de pagamentos falha com timeouts, e dezenas de mensagens vão parar na DLQ. O que está errado?**

O limite de tentativas e o backoff estão curtos demais para erros transitórios. Com backoff exponencial, as novas tentativas se espalham por mais tempo, e a instabilidade passa antes de esgotar o limite; a DLQ deve receber, sobretudo, falhas permanentes.

3 **Corrigido o bug, basta devolver todas as mensagens da DLQ de uma vez para a fila principal?**

Não. O consumidor precisa ser idempotente, porque algumas mensagens podem ter sido processadas pela metade, e o replay deve ser gradual para não criar um pico; além disso, elas chegam fora de ordem em relação às mais novas.

CONECTA COM

44 Filas de mensagens · 45 Pub/Sub · 35 Retries e backoff exponencial · 36 Idempotência

Parte 6

Observabilidade e segurança

Observability and Security

Um sistema que funciona no quadro-branco ainda precisa responder a duas perguntas quando chega à produção: o *que está acontecendo lá dentro*? e *quem pode fazer o quê*? Esta última parte trata das duas. Os dois primeiros capítulos são de observabilidade: o tracing distribuído (cap. 47) reconstrói o caminho de uma requisição por dezenas de serviços, e SLA, SLO e SLI (cap. 48) transformam "o sistema está bom?" em números que dá para medir e cobrar.

Os outros três são de segurança e funcionam em camadas. OAuth 2.0 e OIDC (cap. 49) dizem quem é o usuário e o que cada aplicação pode fazer em nome dele. O handshake TLS (cap. 50) protege os bytes no caminho. O Zero Trust (cap. 51) junta tudo numa postura: nenhuma requisição é confiável só por vir de dentro da rede. Na entrevista, são esses temas que mostram que você pensou em operar e proteger o sistema, e não só em desenhá-lo.

47 Tracing distribuído

48 SLA vs. SLO vs. SLI

49 OAuth 2.0 e OIDC

50 Handshake TLS

51 Zero Trust

Tracing distribuído

Distributed Tracing

EM UMA FRASE

O tracing distribuído segue uma requisição por todos os serviços que ela atravessa e registra cada etapa como um span, com início e duração, para mostrar onde o tempo foi gasto e onde a falha nasceu.

O ESSENCIAL · tradução do artigo original

O tracing distribuído acompanha uma única requisição enquanto ela passa por vários serviços. Cada serviço adiciona um trace ID e informações de span, para que você consiga reconstruir o caminho completo da requisição. Isso ajuda muito a depurar respostas lentas ou falhas em arquiteturas de microsserviços. Sem tracing, você só vê erros isolados. Com ele, você vê a história inteira, passando por serviços, filas e bancos de dados.

APROFUNDANDO

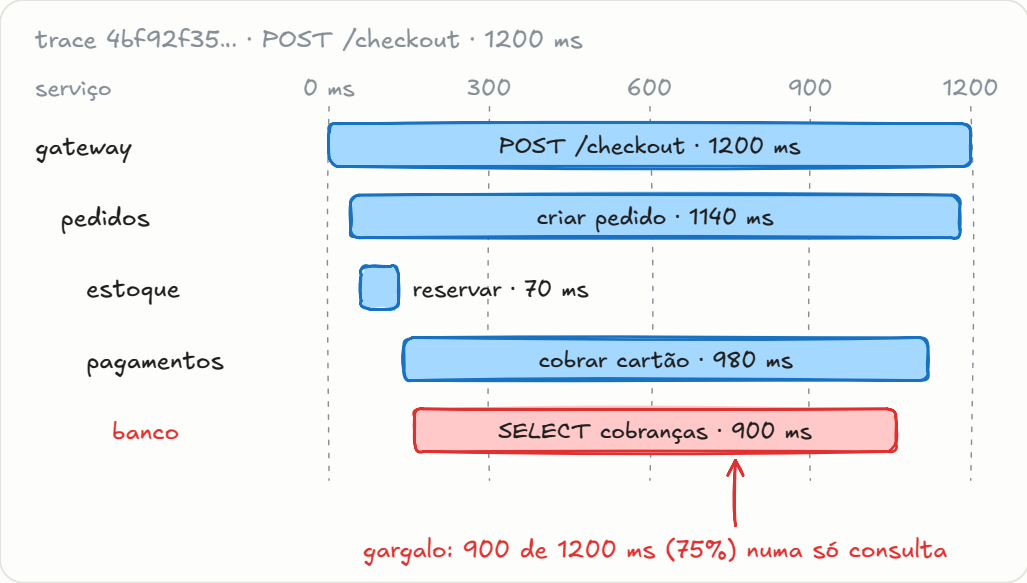
Dois conceitos sustentam tudo:

- **Trace:** a viagem completa de uma requisição. É identificado por um **trace ID** de 128 bits, gerado uma única vez, no primeiro ponto instrumentado do caminho (o próprio app do cliente ou o gateway).
- **Span:** uma etapa dessa viagem, como uma chamada HTTP, uma consulta ao banco ou a publicação numa fila. Cada span guarda o próprio ID, o ID do span pai, o nome da operação, o instante de início, a duração e atributos como o status HTTP ou a consulta SQL.

O que costura os serviços é a **propagação de contexto**. Ao chamar o próximo serviço, cada um envia o trace ID e o ID do span atual num cabeçalho. O padrão hoje é o `traceparent`, do W3C Trace Context:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

Os campos são a versão, o trace ID, o ID do span pai e as flags (o `01` diz que este trace foi amostrado). Quem recebe cria um span filho com o mesmo trace ID e, ao chamar o próximo, envia um `traceparent` novo, com o próprio span como pai. Os spans são enviados em lotes, de forma assíncrona, a um coletor; o backend junta tudo pelo trace ID e monta a árvore pelos IDs de pai.



Cascata de um trace de checkout: as barras estão em escala, e uma única consulta ao banco ocupa 900 dos 1200 ms.

O resultado é a cascata do diagrama. Imagine um checkout que levou 1,2 s. Os logs de cada serviço, sozinhos, diriam apenas "terminei com sucesso". A cascata mostra que pedidos e pagamentos passaram quase todo o tempo esperando, e que uma consulta ao banco consumiu 900 ms; talvez falte um índice (cap. 26). Chamadas em fila indiana que poderiam rodar em paralelo também saltam aos olhos.

Amostragem

Guardar todos os traces de um sistema com milhares de requisições por segundo custa caro, então se amostra:

- **Na entrada (head sampling):** a decisão sai no primeiro span, por exemplo 1% das requisições, e viaja nas flags do `traceparent`. É barato, mas descarta às cegas.

- **No fim (tail sampling):** o coletor espera o trace terminar e guarda os lentos e os que deram erro. Pega o que interessa, mas precisa de memória para segurar os traces até decidir.

Analogia

Pense no código de rastreio dos Correios. O pacote ganha um código na postagem, e cada unidade por onde ele passa (agência, centro de tratamento, centro de distribuição) registra entrada e saída com esse mesmo código. Quando a encomenda atrasa, você não liga para cada unidade: abre o rastreio e vê que ela ficou cinco dias parada num centro de tratamento. O código é o trace ID; cada parada, com hora de chegada e de saída, é um span.

NO MUNDO REAL

- O **Dapper**, do Google, descrito num artigo de 2010, popularizou trace, span e amostragem em larga escala. O **Zipkin**, criado no Twitter e aberto em 2012, nasceu inspirado nele.
- O **Jaeger**, criado na Uber e hoje projeto graduado da CNCF, e o **Grafana Tempo** guardam traces e desenham a cascata.
- O **OpenTelemetry**, também da CNCF, virou o padrão neutro de instrumentação: SDKs para as principais linguagens, um coletor e o W3C Trace Context como propagação padrão. Instrumentando com ele, você troca de backend (Jaeger, Tempo, Datadog, Honeycomb) sem reinstrumentar o código.

TRADE-OFFS

Você ganha

- Ver onde o tempo de cada requisição foi gasto, serviço por serviço, em vez de dar palpites cruzando logs soltos.
- O mapa real de dependências (quem chama quem), extraído dos próprios traces.
- Uma ponte entre sinais: do pico na métrica para um trace lento, e dele para as linhas de log com o mesmo trace ID.

Você paga

- Instrumentação em todos os serviços: um único elo que não propaga o cabeçalho parte o trace em pedaços.
- Volume e custo de armazenamento, que obrigam a amostrar, e a amostragem pode jogar fora justamente o trace que você queria.
- Um pouco de CPU, memória e rede por span. É pouco, mas não é zero.

NA ENTREVISTA

- Traga o tracing quando o desenho tiver vários serviços e o entrevistador perguntar como você vai investigar lentidão ou erros em produção.
- Mostre que sabe dividir o trabalho entre os sinais: as métricas avisam que a latência do p99 subiu (cap. 5), os traces mostram onde, os logs explicam por quê.
- Cite os detalhes de quem já operou: propagação pelo cabeçalho `traceparent`, instrumentação com OpenTelemetry, amostragem na entrada para controlar o custo e no fim para não perder os traces com erro.
- Frase pronta: *"As métricas me dizem que o p99 do checkout dobrou; o trace me mostra que 75% desse tempo está numa consulta do serviço de pagamentos."*

CUIDADO COM

- `ATUALIZAÇÃO:` o artigo diz que cada serviço "adiciona um trace ID". Na verdade, o trace ID nasce uma vez e só é repassado. O que cada serviço cria é o seu **span**, com um ID novo que aponta para o span pai.
- **Filas quebram o trace sem avisar.** Numa chamada assíncrona por fila (cap. 44), o contexto precisa ir nos cabeçalhos da mensagem; senão, o trace termina no produtor.
- **Relógios diferentes.** Cada span é medido pelo relógio da própria máquina. Com relógios dessincronizados, um filho pode aparecer começando antes do pai; confie mais nas durações do que em alinhamentos de poucos milissegundos.
- **Tracing não substitui métricas.** Traces amostrados servem para diagnosticar casos, não para contar. Alertas e taxas de erro saem das métricas.

TESTE-SE

1 Qual a diferença entre um trace e um span?

O trace é a viagem inteira de uma requisição, identificada por um trace ID. O span é uma etapa dessa viagem (uma chamada, uma consulta), com início, duração e o ID do span pai.

2 O p99 do checkout saltou de 300 ms para 2 s depois de um deploy. Como o tracing ajuda a achar a causa?

Você filtra os traces lentos do checkout e abre a cascata: o span mais longo e mais profundo aponta o culpado, por exemplo uma consulta nova sem índice no serviço de pagamentos. Depois, pelo trace ID, chega às linhas de log daquela requisição.

3 Se cada serviço gerar o próprio trace ID ao receber uma requisição, o tracing funciona?

Não. Você terá um trace solto por serviço, sem como ligar um ao outro. O trace ID nasce uma vez e é propagado (no cabeçalho `traceparent`); cada serviço só cria IDs novos de span.

CONECTA COM

5 Throughput vs. latência
9 Microsserviços vs. monólito
14 API Gateway
44 Filas de mensagens
48 SLA vs. SLO vs. SLI

SLA vs. SLO vs. SLI

SLA vs SLO vs SLI

EM UMA FRASE

O SLI é a medida real, o SLO é a meta interna para ela e o SLA é a promessa ao cliente, com multa; a folga entre o SLO e 100% é o orçamento de erro.

O ESSENCIAL · tradução do artigo original

Um SLA (Service Level Agreement, ou acordo de nível de serviço) é uma promessa externa feita aos clientes, como "99,9% de uptime por mês". Um SLO (Service Level Objective, ou objetivo de nível de serviço) é uma meta interna que os engenheiros buscam cumprir, normalmente mais rígida que o SLA. Um SLI (Service Level Indicator, ou indicador de nível de serviço) é a métrica efetivamente medida, como o uptime real ou a taxa de sucesso das requisições. Pense no SLA como o contrato, no SLO como a meta e no SLI como o placar. Em entrevistas, usar esses termos corretamente mostra maturidade ao pensar em confiabilidade.

APROFUNDANDO

Os três se montam de baixo para cima: primeiro a medida, depois a meta, por último a promessa.

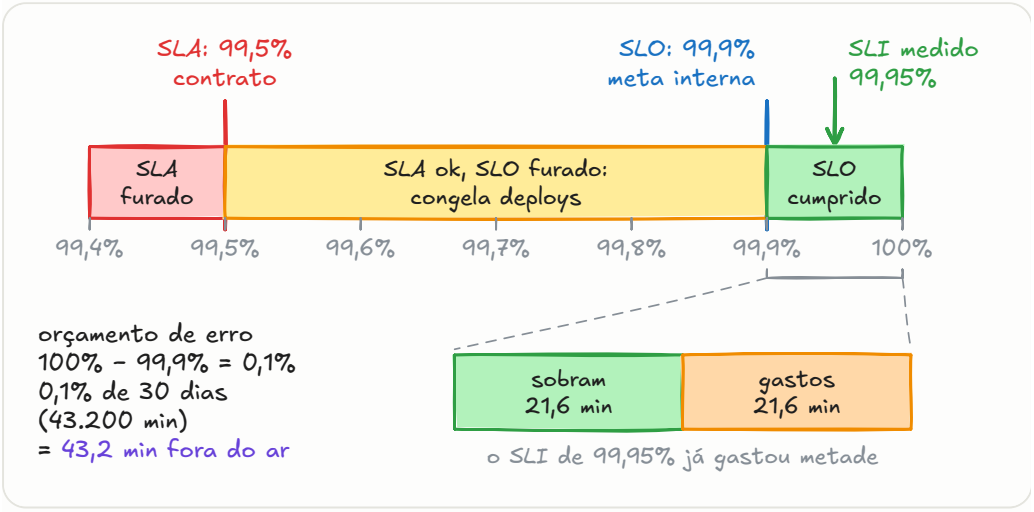
1. SLI: escolha o que medir. Um bom SLI é uma razão entre eventos bons e eventos válidos, medida do ponto de vista de quem usa:

- disponibilidade: requisições respondidas sem erro 5xx ÷ requisições válidas;
- latência: requisições respondidas em menos de 300 ms ÷ requisições válidas;
- frescor, num pipeline de dados: registros atualizados há menos de 10 minutos ÷ registros.

Erros 4xx causados pelo próprio cliente costumam ficar fora da conta, porque não são culpa do serviço.

2. SLO: dê uma meta e uma janela. Por exemplo: "99,9% das requisições do checkout sem erro, numa janela móvel de 30 dias". A meta nunca é 100%: cada nove a mais custa muito mais caro, e acima de certo ponto o usuário nem percebe a diferença, porque o Wi-Fi e o celular dele falham mais do que isso.

3. SLA: prometa menos do que mira. O contrato fica abaixo do SLO, digamos 99,5%, e diz o que acontece se for violado; em geral, créditos na fatura. A folga entre os dois é o seu aviso antecipado: você fura a meta interna bem antes de dever qualquer coisa ao cliente.



SLA, SLO e SLI na mesma régua. A faixa verde, entre o SLO e 100%, é o orçamento de erro: 43,2 minutos por mês com 99,9%.

O orçamento de erro

O que o SLO deixa de fora é o **orçamento de erro** (error budget). Com 99,9% em 30 dias, você pode falhar 0,1% do tempo: 0,1% de 43.200 minutos dá 43,2 minutos. Medindo por requisições, a conta é a mesma: com 10 milhões de requisições no mês, até 10 mil podem falhar.

Meta	Fora do ar permitido em 30 dias
99%	7 h 12 min
99,5%	3 h 36 min
99,9%	43 min 12 s

Meta	Fora do ar permitido em 30 dias
99,99%	4 min 19 s
99,999%	26 s

O orçamento vira ferramenta de decisão. Enquanto sobra, a equipe pode lançar e arriscar. Quando acaba, congela as mudanças arriscadas e põe o esforço em confiabilidade até a janela se recuperar.

Analogia

Uma pizzaria anuncia: “entrega em até 50 minutos ou a pizza sai de graça”. Essa é a promessa ao cliente, com multa embutida: o SLA. Na cozinha, o gerente cobra da equipe 35 minutos, para sobrar folga contra trânsito e chuva: o SLO. O sistema de pedidos anota a hora de saída e de chegada de cada pizza: o SLI. Quando os atrasos da semana se acumulam, o gerente adia a promoção nova antes de começar a dar pizza de graça.

NO MUNDO REAL

- O livro *Site Reliability Engineering* (Google, 2016) popularizou SLIs, SLOs e orçamento de erro. O *Site Reliability Workbook* (2018) traz exemplos de política de orçamento, como congelar lançamentos quando ele acaba.
- Os provedores de nuvem publicam SLAs com créditos na fatura. A AWS diz que o S3 Standard é projetado para 99,99% de disponibilidade, mas o SLA dele garante 99,9% ao mês: o alvo de engenharia acima da promessa, como manda o figurino.
- Na prática, os SLIs saem de contadores de requisições em ferramentas como Prometheus e Datadog. O próprio Workbook recomenda alertar pela velocidade com que o orçamento está sendo queimado (burn rate), e não por erro isolado.

TRADE-OFFS

	SLI	SLO	SLA
O que é	A medida real	A meta interna	A promessa ao cliente

	SLI	SLO	SLA
Exemplo	99,95% das requisições OK no mês	99,9% em 30 dias	99,5%, ou crédito na fatura
Quem define	Engenharia	Engenharia com produto	Comercial e jurídico, ouvindo a engenharia
Se não cumprir	Não se aplica: é o placar	Congela lançamentos e prioriza confiabilidade	Multa, créditos e cliente insatisfeito
Quem vê	A equipe, nos painéis	A empresa	O cliente

NA ENTREVISTA

- Transforme "precisa ser confiável" em números logo nos requisitos não funcionais: "99,9% de disponibilidade e p99 de leitura abaixo de 300 ms, em 30 dias".
- Traduza noves em tempo para mostrar que sabe o custo: 99,9% são uns 43 minutos por mês, o que cabe um incidente resolvido pelo plantão; 99,99% são 4 minutos, então o failover precisa ser automático.
- Lembre que dependências em série se multiplicam: três serviços de 99,9% em cadeia, com falhas independentes, entregam uns 99,7% ($0{,}999^3$), a menos que haja redundância ou fallback.
- Frase pronta: "*Proponho um SLO de 99,9% em 30 dias. Isso dá um orçamento de uns 43 minutos, e é ele que decide quando paramos de lançar para cuidar da estabilidade.*"

CUIDADO COM

- ATUALIZAÇÃO: o artigo fala em "uptime". Hoje o SLI mais comum é por requisição: a fração de requisições boas entre as válidas. Um servidor "no ar" que devolve erro 500 para metade dos usuários tem 100% de uptime e um SLI péssimo.
- **A média esconde a cauda.** Uma latência média de 150 ms pode esconder 1% das requisições levando 5 s. Use percentis (p95, p99) ou a fração de requisições abaixo de um limite (cap. 5).

- **SLO igual ao SLA não avisa nada.** Se a meta interna é a própria promessa, você só descobre o problema quando já deve a multa.
- **O "disponível" do CAP é outro.** No CAP (cap. 2), é uma propriedade formal durante partições; no SLA, é uma porcentagem medida num período.

TESTE-SE

1 Qual a diferença entre SLI, SLO e SLA?

O SLI é a medida real, como a fração de requisições sem erro. O SLO é a meta interna para essa medida, como 99,9% em 30 dias. O SLA é a promessa contratual ao cliente, com consequência (créditos, multa) se for violada.

2 O SLO do seu serviço é 99,9% em 30 dias, e a janela atual já soma 40 minutos de indisponibilidade. O que fazer?

Restam uns 3 dos 43 minutos do orçamento. Você congela deploys e lançamentos arriscados e põe a equipe em confiabilidade até a janela se recuperar. O SLA de 99,5% (216 minutos) ainda está longe, e é para isso que serve a folga.

3 Se o SLA prometido ao cliente é 99,9%, o SLO interno também deve ser 99,9%?

Não. Com a meta igual à promessa, o primeiro sinal de problema já é a violação do contrato. O SLO deve ser mais rígido, como 99,95%, para o alarme interno tocar antes da multa.

CONECTA COM

2 Teorema CAP · 5 Throughput vs. latência · 24 Padrões de replicação · 47 Tracing distribuído

OAuth 2.0 e OIDC

OAuth 2.0 and OIDC

EM UMA FRASE

O OAuth 2.0 deixa o usuário dar a uma aplicação acesso limitado a uma API, via access token, sem entregar a senha; o OIDC acrescenta o ID token, que diz à aplicação quem é o usuário.

O ESSENCIAL · tradução do artigo original

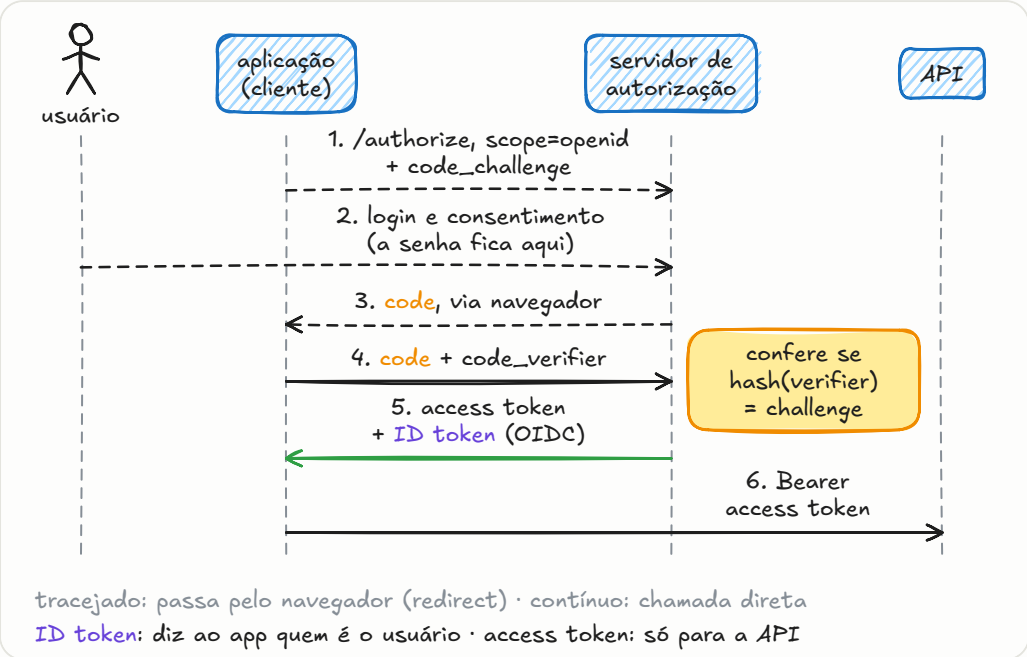
O OAuth 2.0 é um framework de autorização delegada. Ele permite que usuários concedam a uma aplicação acesso limitado aos seus recursos sem compartilhar senhas. O OIDC (OpenID Connect) é construído sobre o OAuth 2.0 para acrescentar autenticação: ele permite que os clientes verifiquem quem é o usuário e obtenham informações sobre a identidade dele. Essa é a base de muitos fluxos do tipo "Entrar com X". A ideia central é que um servidor de autorização emite tokens em que clientes e APIs podem confiar.

APROFUNDANDO

O OAuth 2.0 (RFC 6749) tem quatro papéis: o **usuário**, dono dos dados; a **aplicação** (o "cliente", no jargão do OAuth), que quer acessá-los; o **servidor de autorização**, que autentica o usuário e emite os tokens; e a **API** (o "servidor de recursos"), que guarda os dados. Para quase tudo que envolve um usuário, de site a app móvel, o fluxo recomendado hoje é o **Authorization Code com PKCE**:

1. A aplicação sorteia um segredo aleatório, o `code_verifier`, e calcula o hash SHA-256 dele, o `code_challenge`.
2. Ela manda o navegador para o `/authorize` do servidor com o `client_id`, os escopos pedidos e o `code_challenge`. Se os escopos incluem `openid`, o pedido vira OIDC.
3. O usuário faz login e consente **no servidor de autorização**. A aplicação nunca vê a senha.
4. O servidor devolve o navegador à aplicação com um **code**: de vida curta, de uso único e que, sozinho, não abre nada.

5. A aplicação troca o code pelos tokens numa chamada direta ao `/token`, enviando junto o `code_verifier`. O servidor só aceita se o hash do verifier bater com o challenge do passo 2. Quem interceptou o code no caminho não tem o verifier.
6. A resposta traz o **access token** e, no OIDC, o **ID token**. A aplicação chama a API com o cabeçalho `Authorization: Bearer <access token>`.



Authorization Code com PKCE: o code passa pelo navegador, mas só vira token junto com o code_verifier; o ID token do OIDC chega com o access token.

Dois tokens, dois destinatários

O **access token** é para a API. Diz o que o portador pode fazer (os escopos, como `pedidos:ler`) e costuma valer de minutos a uma hora. Pode ser opaco, e então a API pergunta ao servidor de autorização se ele vale, ou um JWT assinado, que a API valida sozinha com a chave pública do servidor.

O **ID token** é para a aplicação. É sempre um JWT assinado, com quem o emitiu (`iss`), quem é o usuário (`sub`), para quem foi emitido (`aud`, o `client_id` da aplicação) e até quando vale (`exp`). É ele que transforma o OAuth, que só autoriza, num login de verdade.

Para não pedir login de novo quando o access token expira, a aplicação pode receber também um **refresh token**, de vida mais longa, que só serve para buscar access tokens novos no servidor de autorização.

Analogia

Você está viajando e a diarista precisa entrar no seu apartamento. Em vez de emprestar a chave (a senha), você liga para a portaria, que confere que é você mesmo e libera um crachá de visitante para ela: só para o seu andar, só hoje, e que pode ser cancelado a qualquer hora. O crachá é o access token. Se a portaria ainda entregasse um bilhete carimbado dizendo "quem autorizou foi o morador do 72", esse bilhete faria o papel do ID token.

NO MUNDO REAL

- Os botões **"Entrar com Google"** e **"Entrar com Microsoft"** usam OpenID Connect: a aplicação recebe um ID token assinado pelo provedor.
- O **Login Único do gov.br** é integrado por OpenID Connect: o órgão público que usa o login nunca vê a senha do cidadão.
- No **Open Finance Brasil**, quem compartilha seus dados com outra instituição é redirecionado ao próprio banco, faz login e consente lá. A instituição que pediu recebe só tokens com o escopo autorizado, num perfil reforçado de OAuth e OIDC chamado FAPI.
- Para não escrever um servidor de autorização na mão, há opções prontas: **Keycloak** (código aberto), **Auth0**, **Okta**, **Microsoft Entra ID** e **Amazon Cognito**.

TRADE-OFFS

	OAuth 2.0	OIDC
Pergunta que responde	O que esta aplicação pode fazer?	Quem é o usuário?
Token principal	Access token	ID token
Destinatário	A API	A aplicação
Formato	Opaco ou JWT, à escolha do servidor	Sempre JWT assinado

	OAuth 2.0	OIDC
Como pedir	Escopos da API, como `pedidos:ler`	Escopo `openid`, mais `profile` e `email` se quiser
Peças extras	Refresh token, introspecção	Endpoint `/userinfo`, descoberta em `/.well-known/openid-configuration`

NA ENTREVISTA

- Quando o desenho tiver login, delegue a identidade a um provedor via OIDC, com Authorization Code e PKCE, em vez de guardar senhas. Entre serviços, sem usuário, o fluxo é o client credentials.
- Mostre onde o token é validado: o API Gateway (cap. 14) confere assinatura, validade e audiência do JWT com as chaves públicas do provedor, sem consultar o servidor de autorização a cada requisição.
- Antecipe o trade-off: JWT validado localmente é rápido, mas difícil de revogar antes de expirar; daí access token curto, refresh token com rotação e, onde for crítico, consulta ao servidor.
- Frase pronta: "*OAuth diz o que o app pode fazer; OIDC diz quem é o usuário. A API só aceita access token; o ID token fica com a aplicação.*"

CUIDADO COM

- `ATUALIZAÇÃO:` o artigo fala em tokens "em que clientes e APIs podem confiar" como se fossem um só. São dois, com destinatários diferentes: o ID token é da aplicação e não deve ir para a API como credencial; o access token é da API.
- **OAuth puro não é login.** Usar o access token como prova de identidade abre brecha: um token emitido para outra aplicação pode ser reaproveitado na sua. O `aud` do ID token evita isso.
- **Fluxos aposentados.** As boas práticas atuais (RFC 9700, de 2025) desaconselham o fluxo implícito (token devolvido na URL), proíbem o de senha (o app recebe a senha do usuário) e recomendam PKCE em todo cliente.

- **JWT assinado não é cifrado.** Qualquer um lê o conteúdo; não guarde segredos nele.

TESTE-SE

1 Qual a diferença entre OAuth 2.0 e OIDC?

O OAuth 2.0 é autorização delegada: entrega à aplicação um access token para chamar uma API em nome do usuário. O OIDC é uma camada de identidade por cima, que acrescenta o ID token, dizendo à aplicação quem é o usuário.

2 Você está fazendo um app móvel que consome a API da sua empresa, com login centralizado num provedor de identidade. Qual fluxo usar?

Authorization Code com PKCE, abrindo o login no navegador do sistema. O app é um cliente público (não consegue guardar segredo), e o PKCE impede que um code interceptado seja trocado por tokens. O ID token identifica o usuário; o access token vai para a API.

3 Posso mandar o ID token para a minha API como prova de acesso?

Não deveria. O ID token é endereçado à aplicação (o `aud` é o `client_id` dela) e diz quem é o usuário, não o que ele pode fazer. A API deve exigir um access token emitido para ela, com os escopos certos.

CONECTA COM

8 Arquitetura stateful vs. stateless · 14 API Gateway · 50 Handshake TLS · 51 Zero Trust

Handshake TLS

TLS/SSL Handshake

EM UMA FRASE

No handshake TLS, cliente e servidor combinam os algoritmos, chegam a uma chave secreta sem transmiti-la e verificam o certificado do servidor; no TLS 1.3 isso custa 1 RTT, e depois tudo trafega cifrado.

O ESSENCIAL · tradução do artigo original

O TLS/SSL protege a comunicação entre cliente e servidor cifrando os dados em trânsito. Durante o handshake, cliente e servidor combinam os algoritmos de criptografia, trocam chaves de forma segura e verificam certificados. Quando o handshake termina, todos os dados seguintes trafegam cifrados, a salvo de quem estiver escutando. É isso que faz aparecer o cadeadinho no seu navegador. Sem TLS, qualquer pessoa na rede poderia ler ou alterar tráfego sensível.

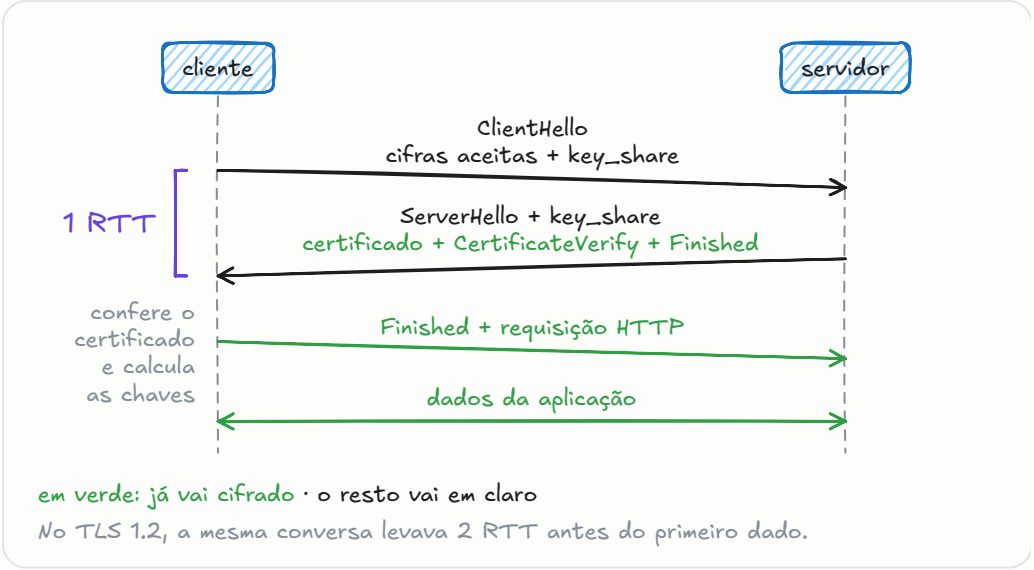
APROFUNDANDO

O handshake resolve três problemas antes do primeiro byte útil: **combinar** os algoritmos, **criar** uma chave secreta que só os dois lados conhecem e **provar** que o servidor é quem diz ser. No TLS 1.3 (RFC 8446, de 2018), tudo isso cabe numa ida e volta:

1. **ClientHello.** O cliente diz quais versões e cifras aceita, informa o nome do site (o SNI) e já manda a sua **key share**: a parte pública de uma troca de chaves efêmera, criada só para esta conexão. Em geral ela usa a curva X25519, que os clientes mais novos já combinam com o ML-KEM, um algoritmo pós-quântico. O cliente aposta no algoritmo que o servidor vai aceitar; se errar, o servidor pede outro e se perde 1 RTT.
2. **Resposta do servidor.** O ServerHello traz a cifra escolhida e a key share do servidor. Cada lado combina a própria chave privada efêmera com a key share do outro e chega ao mesmo segredo, que nunca passa pela rede. Daí em diante, tudo vai cifrado: o **certificado**; o **CertificateVerify**, uma assinatura feita com a chave pri-

vada do certificado sobre tudo o que foi trocado, provando que o servidor tem essa chave; e o **Finished**, um código de verificação do handshake inteiro, que denuncia qualquer adulteração.

3. **Finished do cliente.** O cliente confere a cadeia do certificado até uma autoridade certificadora em que confia, checa se o nome bate com o site e se está na validade, e responde com o seu Finished. Na mesma leva, já vai a primeira requisição HTTP.



TLS 1.3: a key share já vai no ClientHello, então o handshake fecha em 1 RTT; do certificado em diante, tudo trafega cifrado.

Como a troca de chaves do handshake completo é sempre efêmera, o TLS 1.3 tem **sigilo futuro** (forward secrecy): quem gravar o tráfego hoje e roubar a chave privada do servidor amanhã não consegue decifrar o que gravou.

Quanto custa

No TLS 1.2, o handshake completo leva **2 RTT**. Somado ao handshake do TCP (cap. 17), a requisição HTTP só sai depois de 3 RTT; no TLS 1.3, depois de 2. Com 150 ms de RTT entre continentes, são 450 ms contra 300 ms só de espera. Por isso se reaproveitam conexões, se termina o TLS numa CDN perto do usuário (cap. 15), e o QUIC do HTTP/3 (cap. 18) junta transporte e TLS 1.3 num handshake só. Numa reconexão, o TLS 1.3 permite até mandar dados no primeiro voo (**0-RTT**), com um porém: um atacante pode repetir esses dados.

Analogia

É como abrir conta numa agência que você nunca visitou. Primeiro você confere se o lugar é mesmo do banco: o alvará na parede tem o carimbo de um órgão em que você confia (o certificado assinado por uma autoridade certificadora). Depois, você e o gerente combinam, sem dizer em voz alta, um código que só os dois conhecem, e dali em diante tudo o que passa entre vocês vai em envelope lacrado. Quem está na fila vê que vocês conversam, mas não o que dizem.

NO MUNDO REAL

- O **Let's Encrypt**, autoridade certificadora gratuita e automatizada pelo protocolo ACME, tornou o HTTPS padrão até em sites pequenos, com certificados de vida curta renovados por robôs.
- A validade máxima dos certificados públicos está encolhendo: o CA/Browser Forum aprovou em 2025 a redução de 398 para 200 dias a partir de março de 2026, depois para 100 dias em 2027 e 47 dias em 2029. Renovar à mão deixa de ser opção.
- Em produção, o TLS costuma terminar no load balancer, no proxy reverso (cap. 13) ou na CDN; dali até os serviços, a conexão é reaberta, de preferência cifrada também.

TRADE-OFFS

	TLS 1.2 (2008)	TLS 1.3 (2018)
Handshake completo	2 RTT	1 RTT
Reconexão	1 RTT, retomando a sessão	1 RTT, ou 0-RTT com risco de replay
Troca de chaves	RSA estático ou (EC)DHE	Sempre (EC)DHE efêmero: sigilo futuro garantido
Certificado	Vai em claro	Vai cifrado
Cifras	Dezenas, inclusive combinações hoje fracas	Cinco, todas AEAD, como AES-GCM e ChaCha20-Poly1305

	TLS 1.2 (2008)	TLS 1.3 (2018)
Compatibilidade	Universal, inclusive clientes antigos	Clientes modernos; equipamentos de rede antigos atrasaram a adoção

NA ENTREVISTA

- Traga o TLS ao falar de latência de conexão nova, de onde terminar a criptografia e de tráfego interno.
- Diga onde ele termina (no load balancer ou na CDN, perto do usuário) e o que vem depois: TLS de novo até os serviços, ou mTLS (cap. 51), em que o cliente também apresenta certificado.
- Mostre que sabe reduzir o custo: conexões persistentes e multiplexadas (HTTP/2), retomada de sessão, TLS 1.3 e HTTP/3.
- Frase pronta: *"Termino o TLS na borda e uso mTLS entre os serviços. Com TLS 1.3, o handshake custa 1 RTT, e com conexões reaproveitadas eu pago isso uma vez, não a cada requisição."*

CUIDADO COM

- ATUALIZAÇÃO: "SSL" é nome histórico. O SSL 2.0 e o 3.0 estão quebrados e proibidos, e o TLS 1.0 e o 1.1 foram aposentados em 2021 (RFC 8996). O mínimo hoje é TLS 1.2; o recomendado, 1.3.
- ATUALIZAÇÃO: o cadeado não quer dizer "site confiável", só que a conexão está cifrada com quem tem o certificado daquele domínio, e sites de phishing também têm. Por causa desse mal-entendido, o Chrome trocou o cadeado por um ícone neutro em 2023.
- **0-RTT pode ser repetido.** Use só em requisições idempotentes (cap. 36).
- **TLS protege o caminho, não as pontas.** O servidor vê tudo em claro, e quem observa a rede ainda vê IPs, volumes e, em geral, o nome do site.

TESTE-SE

1 O que o handshake TLS garante antes do primeiro byte de dados?

Que os dois lados combinaram os algoritmos e chegaram à mesma chave sem transmiti-la, e que o servidor provou ter a chave privada de um certificado válido para aquele domínio. O Finished garante ainda que ninguém adulterou a conversa.

2 Um app abre uma conexão HTTPS nova para cada requisição, com um servidor a 150 ms de RTT. Quanto tempo passa até a requisição sair, com TLS 1.2 e com TLS 1.3? O que você mudaria?

Contando o TCP, são 3 RTT (450 ms) no TLS 1.2 e 2 RTT (300 ms) no TLS 1.3. Eu reaproveitaria a conexão (keep-alive, HTTP/2), adotaria TLS 1.3 ou HTTP/3 e terminaria o TLS numa borda mais perto do usuário.

3 O cadeado no navegador garante que o site é confiável?

Não. Garante só que a conexão está cifrada e que o servidor tem um certificado válido para aquele domínio. Um site de phishing com domínio parecido também consegue certificado.

CONECTA COM

13 Proxy reverso vs. proxy direto · 15 CDN · 17 TCP vs. UDP · 18 HTTP/2 e HTTP/3 (QUIC) · 51 Zero Trust

Zero Trust

Zero Trust Security

EM UMA FRASE

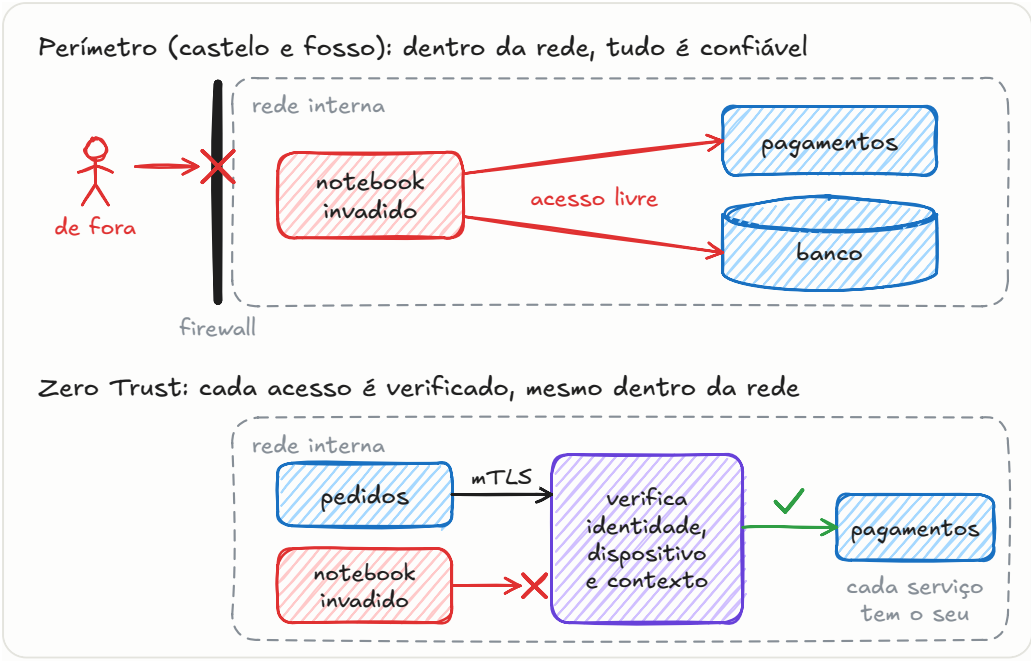
Zero Trust abandona a ideia de que a rede interna é segura; cada acesso, de pessoa ou de serviço, é autenticado, autorizado e cifrado com base em identidade, dispositivo e contexto, com o mínimo de privilégio.

O ESSENCIAL · tradução do artigo original

Zero Trust é um modelo de segurança que diz: “nunca confie, sempre verifique”. Ele parte do princípio de que ameaças podem existir tanto fora quanto dentro da rede. Toda requisição precisa ser autenticada, autorizada e cifrada, mesmo que venha de dentro do seu data center ou da sua VPC. O acesso é concedido com base em identidade, postura do dispositivo e contexto, e não apenas por estar “dentro do firewall”. Nas arquiteturas modernas, o Zero Trust está se tornando a abordagem padrão para projetar sistemas seguros.

APROFUNDANDO

O modelo antigo é o **perímetro**, apelidado de “castelo e fosso”: um firewall forte na borda, uma VPN para quem está fora e, lá dentro, uma rede em que todo mundo confia em todo mundo. O problema aparece depois da primeira invasão. Basta um notebook comprometido por phishing para o atacante alcançar bancos de dados e serviços internos, porque o único teste era “estar dentro”. Esse avanço de máquina em máquina se chama **movimento lateral**.



No perímetro, quem está dentro da rede alcança tudo; no Zero Trust, cada serviço verifica cada acesso, mesmo os que vêm de dentro.

O Zero Trust troca o endereço de rede por **identidade** como base da confiança. A arquitetura de referência do NIST (SP 800-207, de 2020) separa duas funções: um **ponto de aplicação de política** (PEP, de policy enforcement point) na frente de cada recurso, que barra ou deixa passar, e um **motor de política**, que decide. A cada acesso, a decisão considera:

- **Quem pede:** a identidade da pessoa, com login forte (SSO via OIDC, cap. 49, mais MFA), ou a do serviço, comprovada por certificado.
- **De onde:** a postura do dispositivo. É um notebook da empresa, com disco cifrado e sistema atualizado?
- **Em que contexto:** horário, localização, comportamento recente e sensibilidade do recurso.

A resposta segue o **mínimo privilégio**: acesso só àquele recurso, por tempo limitado.

Entre serviços

Para máquinas, o padrão é dar a cada serviço uma identidade criptográfica e usar **mTLS** (TLS mútuo, cap. 50): os dois lados apresentam certificado, e cada serviço sabe com certeza quem o chama. Um service mesh como Istio ou Linkerd automatiza isso com proxies ao lado de cada serviço (sidecars) e certificados que vencem em horas, renovados sozinhos. O padrão SPIFFE dá nome a essas identidades, como `spiffe://loja.com/ns/prod/sa/pedidos`. Sobre a identidade entram as políticas explícitas, por exemplo: só o serviço de pedidos pode chamar `POST /cobrancas` em pagamentos. Cada decisão fica registrada, e esses registros ajudam a detectar comportamento estranho.

> *Analogia*
>
> Num hospital, passar pela recepção não leva você à UTI. Cada ala confere de novo: quem é você (documento), quem veio visitar (o paciente certo), em que horário (fora da visita, não entra) e se está paramentado com máscara e avental, o equivalente à postura do dispositivo. Estar dentro do prédio não conta como credencial; cada porta pergunta outra vez.

NO MUNDO REAL

- O **BeyondCorp**, do Google, nasceu depois dos ataques da Operação Aurora, em 2009. O Google deixou de confiar na rede corporativa: funcionários acessam aplicações internas de qualquer rede, sem VPN, por um proxy que confere identidade e dispositivo. A empresa descreveu o modelo em artigos a partir de 2014.
- O **NIST SP 800-207** virou a referência do assunto, e o governo dos EUA mandou suas agências migrarem para Zero Trust (memorando M-22-09, de 2022).
- **Istio** e **Linkerd** entregam mTLS automático entre serviços no Kubernetes; o **SPIFFE/SPIRE**, da CNCF, padroniza identidades de serviço.
- Proxies cientes de identidade, como o **Identity-Aware Proxy** do Google Cloud e o **Cloudflare Access**, publicam ferramentas internas sem VPN.

TRADE-OFFS

	Perímetro (castelo e fosso)	Zero Trust
Base da confiança	Estar dentro da rede ou da VPN	Identidade, dispositivo e contexto, a cada acesso
Depois de uma invasão	Movimento lateral fácil numa rede plana	Estrago limitado ao que aquela identidade pode acessar
Tráfego interno	Muitas vezes em texto claro	Autenticado e cifrado (mTLS)
Acesso remoto	VPN que abre a rede inteira	Acesso por aplicação, sem VPN de rede inteira
Custo de adoção	Simples de montar	Exige identidade forte, inventário de dispositivos, certificados e políticas por recurso
Operação	Pouca verificação no caminho	Mais verificações, mais peças que podem falhar ou somar latência

NA ENTREVISTA

- Traga o assunto quando perguntarem como os microsserviços se autenticam entre si (cap. 9), como proteger ferramentas internas ou o que acontece se um servidor for invadido.
- Seja concreto: identidade para cada serviço, mTLS entre eles, tokens OIDC dos usuários validados no gateway (cap. 14), políticas de mínimo privilégio por endpoint e credenciais de vida curta.
- Mostre o raciocínio de dano limitado: "se o serviço de recomendações for invadido, ele não tem permissão para chamar pagamentos".
- Frase pronta: "*Não confio em nada só por estar na VPC. Cada serviço tem identidade própria, as chamadas internas usam mTLS, e cada endpoint confere se aquele chamador pode fazer aquela operação.*"

CUIDADO COM

- **Zero Trust não é um produto.** É uma estratégia. Ferramentas ajudam, mas o trabalho é inventariar identidades, dispositivos e recursos e escrever as políticas.
- **Não é o fim do firewall.** Segmentar a rede continua útil como defesa em profundidade; só deixa de ser o critério principal de confiança.
- **"Sempre verifique" não é pedir senha a cada clique.** Cada requisição é verificada pelo token ou pelo certificado que carrega; o login forte se repete quando o risco muda, como num dispositivo novo ou num lugar estranho.
- **A peça central vira ponto único de falha.** Se o provedor de identidade ou o motor de política cai, ninguém acessa nada. Eles precisam de redundância e de um plano de emergência.

TESTE-SE

1 O que significa "nunca confie, sempre verifique"?

Que nenhum acesso é liberado só pela localização na rede. Cada requisição, de pessoa ou de serviço, é autenticada e autorizada com base em identidade, dispositivo e contexto, e recebe só o privilégio mínimo necessário.

2 Você tem 40 microsserviços numa mesma VPC. Como aplicar Zero Trust entre eles?

Dê a cada serviço uma identidade (certificados emitidos e renovados automaticamente, como no SPIFFE), exija mTLS em toda chamada (um service mesh faz isso) e escreva políticas por serviço e endpoint, liberando só o necessário. Registre as decisões para auditoria.

3 Toda a equipe acessa os sistemas internos por VPN. Isso já é Zero Trust?

Não. A VPN verifica uma vez, na entrada, e depois libera a rede inteira: é o perímetro com uma porta a mais. Zero Trust verifica cada acesso a cada recurso, com identidade, dispositivo e contexto.

CONECTA COM

- 9 Microsserviços vs. monólito
- 14 API Gateway
- 47 Tracing distribuído
- 49 OAuth 2.0 e OIDC
- 50 Handshake TLS

FECHAMENTO

Para fechar

As lições principais

O autor do artigo original encerra com cinco ideias. Elas resumem o livro inteiro:

- **System design é, acima de tudo, entender trade-offs:** consistência vs. disponibilidade, latência vs. throughput, simplicidade vs. flexibilidade.
- **Escalar não é só "colocar mais servidores".** Você precisa pensar em balanceamento de carga, sharding, replicação e gargalos.
- **Padrões de confiabilidade** como rate limiting, circuit breakers, retries e bulkheads existem porque falhas são normais em sistemas distribuídos.
- **Cache, filas e pub/sub são seus melhores amigos** para desempenho e desacoplamento, mas trazem novos desafios de consistência e ordenação.
- **Observabilidade e segurança**, com conceitos como tracing, SLIs, OAuth, TLS e Zero Trust, são essenciais para sistemas que não sejam só rápidos, mas também seguros e fáceis de depurar.

A viagem de um clique

Os 51 conceitos parecem uma lista, mas eles se encaixam. Siga um único clique em "Finalizar compra" numa loja online:

1. O navegador descobre o IP da loja pelo **DNS** (cap. 16) e baixa imagens e scripts de uma **CDN** (cap. 15), já numa conexão **HTTP/2 ou HTTP/3** (cap. 18) protegida por **TLS** (cap. 50).
2. A requisição chega a um **load balancer** (cap. 11), que escolhe um servidor com algum **algoritmo** (cap. 12), e passa por um **API gateway** (cap. 14) que confere o login (**OAuth/OIDC**, cap. 49) e aplica **rate limiting** (cap. 32).
3. Os serviços de pedido, estoque e pagamento são **stateless** (cap. 8), então escalam **horizontalmente** (cap. 1). Cada um tem seu banco (**persistência poliglota**, cap. 29).

4. O pedido percorre uma **SAGA** (cap. 39): reservar estoque, cobrar, agendar entrega. A cobrança usa uma **chave de idempotência** (cap. 36) porque o cliente pode tentar de novo com **backoff** (cap. 35), e um **circuit breaker** (cap. 33) protege o serviço de pagamento quando o provedor externo fica lento.
5. Os dados do pedido ficam num banco com **WAL** (cap. 27), **replicado** (cap. 24) e **shardeado** (cap. 23). O catálogo vem do **cache** (cap. 41) com **cache-aside** (cap. 42).
6. O evento "pedido criado" sai por **pub/sub** (cap. 45) para e-mail, nota fiscal e analytics. A mensagem que falhar três vezes vai para uma **dead letter queue** (cap. 46).
7. Um **trace distribuído** (cap. 47) registra o caminho inteiro, e o tempo total entra no **SLI** que o time compara com o **SLO** (cap. 48).

Um clique, mais de vinte capítulos. Quando você conseguir contar essa história sem olhar, desenhando cada etapa, a entrevista vira uma conversa sobre coisas que você já sabe.

E agora?

- **Refaça o estudo de caso** (apêndice C) sem consultar, e depois compare.
- **Pratique com os problemas clássicos:** encurtador de links, feed de rede social, chat em tempo real, rate limiter distribuído, sistema de notificações, upload e streaming de vídeo. Para cada um, liste os capítulos deste livro que entram no desenho.
- **Para ir além:** *Designing Data-Intensive Applications*, de Martin Kleppmann, é a referência para a Parte 3 e boa parte da Parte 4; *System Design Interview*, de Alex Xu, traz dezenas de problemas resolvidos; e o livro *Site Reliability Engineering*, do Google, que é gratuito online, aprofunda SLOs, monitoramento e confiabilidade.

ESTUDO

Cola: os conceitos em uma linha

O "Em uma frase" de cada capítulo, em ordem. Leia de cima a baixo na véspera e marque os que você não consegue explicar.

Parte 1 · Princípios de arquitetura

1	**Escalabilidade vertical vs. horizontal**	Escalar na vertical é turbinar uma única máquina; escalar na horizontal é somar máquinas e dividir o trabalho entre elas, o que exige load balancer, serviços stateless e dados compartilhados.
2	**Teorema CAP**	Quando a rede entre os nós falha, um sistema distribuído precisa escolher entre responder com dados talvez velhos (disponibilidade) ou recusar para não responder errado (consistência).
3	**Teorema PACELC**	Se há partição de rede (P), o sistema escolhe entre disponibilidade (A) e consistência (C); senão (E), escolhe entre latência baixa (L) e consistência (C), porque manter réplicas em sincronia custa tempo.
4	**ACID vs. BASE**	ACID garante transações tudo-ou-nada, isoladas e duráveis, ideais para dinheiro e estoque; BASE aceita inconsistências temporárias em troca de disponibilidade e escala, com as réplicas convergindo depois.
5	**Throughput vs. latência**	Throughput é quantas requisições o sistema atende por segundo; latência é quanto tempo cada uma leva. Paralelismo e lotes aumentam a vazão, mas filas perto do limite fazem a latência disparar.
6	**Lei de Amdahl**	O ganho de velocidade com paralelismo é limitado pela parte do trabalho que continua sequencial; se 20% não se paraleliza, nem infinitas máquinas deixam o sistema mais de 5 vezes mais rápido.

7	**Consistência forte vs. eventual**	Com consistência forte, toda leitura vê a última escrita confirmada; com consistência eventual, as réplicas podem divergir por um tempo, mas convergem para o mesmo valor quando as escritas param.
8	**Arquitetura stateful vs. stateless**	Um serviço stateful guarda o contexto do usuário na própria memória entre requisições; um stateless trata cada requisição como nova e guarda o estado fora, então qualquer instância pode atender qualquer requisição.
9	**Microsserviços vs. monólito**	Um monólito reúne todas as funcionalidades numa única unidade de deploy; microsserviços as separam em serviços independentes que conversam pela rede, trocando simplicidade por autonomia dos times e escala separada.
10	**Arquitetura serverless**	No serverless, você entrega funções que a nuvem executa sob demanda, cuidando dos servidores e da escala; paga por execução e, em troca, aceita cold starts, limite de duração e custo alto com tráfego constante.

Parte 2 · Redes e comunicação

11	**Balanceamento de carga**	Um load balancer recebe o tráfego num endereço único e o reparte entre vários servidores, tirando da rotação os que falham no health check; é a primeira peça de qualquer escala horizontal.
12	**Algoritmos de balanceamento**	Round robin reveza os servidores em ordem, least connections escolhe o que tem menos conexões abertas e IP hash prende cada cliente ao mesmo servidor; a escolha define quão justa fica a distribuição.
13	**Proxy reverso vs. proxy direto**	O proxy direto fica do lado dos clientes e fala por eles com a internet; o proxy reverso fica do lado dos servidores e fala por eles, cuidando de TLS, cache e roteamento.
14	**API Gateway**	Um API gateway é o proxy reverso que serve de porta única para as APIs de um sistema de microsserviços, concentrando roteamento, autenticação, rate limit e logs; precisa ficar enxuto para não virar gargalo.

15	**CDN**	Uma CDN é uma rede de servidores de borda espalhados pelo mundo que guardam cópias do seu conteúdo perto dos usuários, cortando a latência e a carga nos servidores de origem.
16	**DNS**	O DNS traduz nomes como www.loja.com.br em endereços IP, consultando uma hierarquia de servidores (raiz, TLD e autoritativo) e guardando cada resposta em cache pelo tempo definido no TTL.
17	**TCP vs. UDP**	O TCP abre uma conexão e garante entrega completa e em ordem, ao custo de handshake e retransmissões; o UDP dispara datagramas sem garantia nenhuma, ideal quando chegar rápido vale mais que chegar tudo.
18	**HTTP/2 e HTTP/3 (QUIC)**	O HTTP/2 multiplexa várias requisições numa única conexão TCP, mas uma perda de pacote ainda trava todas; o HTTP/3 roda sobre QUIC (UDP), com streams independentes e conexões mais rápidas de abrir.
19	**gRPC vs. REST**	REST expõe recursos via HTTP, quase sempre com JSON legível, e domina as APIs públicas; gRPC chama métodos remotos com protobuf binário sobre HTTP/2, contrato forte e streaming, e brilha entre serviços internos.
20	**WebSocket e Server-Sent Events**	O WebSocket abre um canal persistente em que cliente e servidor falam a qualquer momento; no SSE, só o servidor fala, enviando eventos ao cliente por uma resposta HTTP que não termina.
21	**Long polling**	No long polling, o cliente pergunta e o servidor segura a requisição até ter novidade ou estourar o timeout; ao receber a resposta, o cliente pergunta de novo, simulando tempo real com HTTP comum.
22	**Protocolo gossip**	Num protocolo gossip, cada nó conta periodicamente o que sabe a alguns pares sorteados; a informação se espalha como uma epidemia e chega a todos em poucas rodadas, sem coordenador central.

Parte 3 · Bancos de dados e armazenamento

23	**Sharding**	Sharding divide os dados entre várias máquinas, cada uma com um pedaço; o difícil é escolher uma chave de shard que espalhe a carga por igual e conviver com o custo de mover dados depois.
24	**Padrões de replicação**	Replicação mantém cópias dos dados em vários nós; com um primário, só ele aceita escritas e as réplicas atendem leituras; com vários primários, todos escrevem, mas os conflitos precisam ser resolvidos.
25	**Hashing consistente**	Hashing consistente põe nós e chaves no mesmo anel e entrega cada chave ao próximo nó no sentido horário; quando um nó entra ou sai, só as chaves de um trecho do anel mudam de dono.
26	**Índices: B-Tree e LSM-Tree**	Índices são estruturas extras que aceleram buscas; a B-Tree mantém páginas ordenadas e as atualiza no lugar, boa para leituras, e a LSM-Tree acumula escritas em memória e grava arquivos imutáveis, boa para escritas.
27	**Write-Ahead Logging (WAL)**	No WAL, o banco grava cada mudança primeiro num log sequencial e o força para o disco; só depois confirma e atualiza os arquivos de dados, e, após uma queda, refaz o log.
28	**Normalização vs. desnormalização**	Normalizar é guardar cada fato num só lugar para evitar anomalias; desnormalizar é duplicar dados de propósito para ler sem joins, aceitando o trabalho de manter as cópias em dia.
29	**Persistência poliglota**	Persistência poliglota é usar vários tipos de banco no mesmo sistema, cada um no papel em que é melhor, pagando com mais operação, mais conhecimento no time e o trabalho de manter os dados sincronizados.
30	**Bloom filter**	Um Bloom filter é um array de bits com algumas funções de hash que diz, gastando pouquíssima memória, se um item talvez esteja num conjunto ou se com certeza não está, sem nunca dar falso negativo.

31	**Bancos de dados vetoriais**	Bancos vetoriais guardam embeddings, vetores de números que representam o significado de textos, imagens ou áudios, e respondem consultas por semelhança encontrando os vetores mais próximos, quase sempre de forma aproximada.

Parte 4 · Confiabilidade e tolerância a falhas

32	**Rate limiting**	Rate limiting controla quantas requisições um usuário, IP ou chave de API pode fazer numa janela de tempo, protegendo o sistema contra abuso, picos acidentais e loops descontrolados.
33	**Circuit breaker**	O circuit breaker monitora as chamadas a um serviço remoto e, quando as falhas passam de um limite, para de chamá-lo por um tempo e falha na hora, impedindo que um serviço doente derrube os outros.
34	**Bulkhead**	O bulkhead divide os recursos do sistema em compartimentos isolados, como pools de conexões ou de threads por funcionalidade, para que a falha ou a sobrecarga de uma parte não afunde as outras.
35	**Retries e backoff exponencial**	Retries recuperam falhas passageiras; o backoff exponencial faz cada nova tentativa esperar mais que a anterior, e o jitter sorteia essas esperas para que os clientes não voltem todos juntos contra um serviço que já está sofrendo.
36	**Idempotência**	Uma operação é idempotente quando executá-la várias vezes tem o mesmo efeito que executá-la uma vez, o que torna seguros os retries e a entrega "pelo menos uma vez", sem cobranças em dobro.
37	**Heartbeat**	Heartbeat é um sinal periódico que um nó envia para avisar que está vivo; quando os sinais param de chegar por tempo demais, o monitor o considera fora do ar e dispara failover ou substituição.
38	**Eleição de líder (Paxos e Raft)**	A eleição de líder escolhe um único nó para coordenar os demais; algoritmos de consenso como Paxos e Raft usam votos da maioria para garantir no máximo um líder por mandato e trocá-lo automaticamente quando ele cai.

39	**Transações distribuídas: o padrão SAGA**	O padrão SAGA divide uma transação que atravessa vários serviços numa sequência de transações locais; se um passo falha, ações de compensação desfazem os anteriores, em ordem inversa, sem travar nada globalmente.
40	**Two-Phase Commit (2PC)**	O two-phase commit torna atômica uma transação em vários nós; o coordenador pergunta se todos podem confirmar e só manda confirmar se todos votarem sim, mas, se ele cair no meio, os participantes ficam travados.

Parte 5 · Cache e mensageria

41	**Cache**	Cache é uma cópia dos dados mais usados num armazenamento rápido, quase sempre a memória; corta a latência e a carga do banco, mas cria um problema novo, o de saber quando essa cópia ficou velha.
42	**Estratégias de cache**	As estratégias de cache definem quem preenche o cache nas leituras e o que acontece com ele nas escritas; cache-aside, write-through e write-back trocam frescor, simplicidade e velocidade de escrita de jeitos diferentes.
43	**Políticas de remoção (LRU, LFU)**	Quando o cache enche, a política de remoção escolhe quem sai; o LRU tira o item usado há mais tempo, o LFU tira o menos usado no total, e cada um erra em padrões de acesso diferentes.
44	**Filas de mensagens**	Uma fila de mensagens guarda tarefas entre quem produz e quem processa; cada mensagem vai para um único consumidor e só sai da fila depois de confirmada, o que desacopla os serviços e absorve picos.
45	**Pub/Sub**	No pub/sub, o publicador envia cada evento a um tópico, e cada assinante recebe a sua cópia; vários serviços reagem ao mesmo fato sem que o publicador saiba quem eles são.
46	**Dead letter queue**	A dead letter queue é a fila para onde vão as mensagens que falharam várias vezes; ela tira as mensagens envenenadas do caminho e as guarda para alguém investigar, corrigir e reprocessar.

Parte 6 · Observabilidade e segurança

47	**Tracing distribuído**	O tracing distribuído segue uma requisição por todos os serviços que ela atravessa e registra cada etapa como um span, com início e duração, para mostrar onde o tempo foi gasto e onde a falha nasceu.
48	**SLA vs. SLO vs. SLI**	O SLI é a medida real, o SLO é a meta interna para ela e o SLA é a promessa ao cliente, com multa; a folga entre o SLO e 100% é o orçamento de erro.
49	**OAuth 2.0 e OIDC**	O OAuth 2.0 deixa o usuário dar a uma aplicação acesso limitado a uma API, via access token, sem entregar a senha; o OIDC acrescenta o ID token, que diz à aplicação quem é o usuário.
50	**Handshake TLS**	No handshake TLS, cliente e servidor combinam os algoritmos, chegam a uma chave secreta sem transmiti-la e verificam o certificado do servidor; no TLS 1.3 isso custa 1 RTT, e depois tudo trafega cifrado.
51	**Zero Trust**	Zero Trust abandona a ideia de que a rede interna é segura; cada acesso, de pessoa ou de serviço, é autenticado, autorizado e cifrado com base em identidade, dispositivo e contexto, com o mínimo de privilégio.

APÊNDICE A

Roteiro para a entrevista de system design

Uma entrevista de system design não tem gabarito. O entrevistador quer ver como você pensa: se pergunta antes de desenhar, se usa números para decidir e se sabe dizer quanto custa cada escolha. Este roteiro divide os 45 minutos em oito etapas. Se o entrevistador puxar a conversa para outro lado, vá junto; ter a ordem na cabeça serve para você não se afogar num detalhe nem chegar ao fim sem o sistema desenhado. O apêndice C aplica o roteiro a um encurtador de links.

As oito etapas

Etapa	Tempo	O que sai dela	Capítulos
1. Entender e delimitar o problema	0–6 min	requisitos funcionais e não funcionais	2, 3, 5, 7, 48
2. Estimar a ordem de grandeza	6–10 min	QPS médio e de pico, armazenamento, banda	5, 23, 41
3. Definir a API	10–13 min	endpoints, com entrada e saída	19, 20, 21, 36, 49
4. Esboçar o modelo de dados	13–16 min	entidades, chaves e tipo de banco	4, 26, 28, 29
5. Desenhar a visão de alto nível	16–25 min	diagrama e um fluxo de ponta a ponta	8, 9, 11, 14, 15, 16, 41, 44, 45
6. Aprofundar em 1 ou 2 componentes	25–37 min	o componente por dentro, com alternativas	23–27, 30, 38–40, 42, 43, 46
7. Caçar gargalos e falhas	37–42 min	pontos fracos e como mitigá-los	1, 6, 24, 32–35, 37, 50, 51
8. Encerrar	42–45 min	resumo, operação e próximos passos	47, 48

Se a sessão gastar alguns minutos com apresentações, encurte as etapas 2 a 4, nunca o aprofundamento.

1. Entender e delimitar o problema

O enunciado vem vago de propósito: "projete o Instagram". Antes de desenhar qualquer caixa, transforme isso numa lista curta.

- **Requisitos funcionais:** o que o usuário consegue fazer. Fique com três a cinco e diga em voz alta o que fica de fora.
- **Requisitos não funcionais:** escala, latência, disponibilidade, consistência e durabilidade.

Perguntas que quase sempre valem a pena:

- Quantos usuários ativos por dia? Quantas leituras para cada escrita?
- Que latência o usuário aceita no fluxo principal (cap. 5)?
- Se um dado aparecer com alguns segundos de atraso, alguém sai prejudicado? A resposta decide entre consistência forte e eventual (cap. 7) e o seu lado no CAP (cap. 2) e no PACELC (cap. 3).
- Que disponibilidade se espera? Ela vira o SLO do encerramento (cap. 48).
- Os usuários estão num país só ou no mundo todo?

Frase útil: *"Antes de desenhar, quero fechar o escopo: postar e ver o feed. Busca e notificações ficam de fora. Faz sentido?"*

2. Estimar a ordem de grandeza

As contas servem para decidir, não para impressionar. Calcule o QPS (requisições por segundo, o throughput do capítulo 5) médio e de pico, o armazenamento e a banda, arredondando sem dó: um dia tem ~10^5 segundos. O apêndice B traz as tabelas de referência e um exemplo resolvido.

Pare quando o número decidir alguma coisa. Cem escritas por segundo cabem num banco só; cem mil pedem sharding (cap. 23). Milhares de leituras por segundo do mesmo conteúdo pedem cache (cap. 41). Se o entrevistador preferir pular as contas,

faça-as mais tarde, quando forem decidir algo.

Frase útil: *"Vou arredondar para facilitar: 10 milhões de usuários com 10 leituras por dia dão 10^8 leituras, ou cerca de mil por segundo."*

3. Definir a API

Escreva os endpoints que cobrem os requisitos funcionais: verbo, caminho, o que entra e o que sai. Poucas linhas bastam.

```
POST /v1/posts           cria um post (com Idempotency-Key)
GET  /v1/posts/{id}      lê um post
GET  /v1/feed?cursor=X   lê o feed, 20 posts por página
```

O que mostra maturidade nesta etapa:

- REST para clientes externos e gRPC entre serviços internos (cap. 19). Atualização em tempo real pede WebSocket ou SSE (cap. 20), ou long polling quando a infraestrutura não aceita essas conexões (cap. 21).
- Idempotência em tudo o que cria recurso ou movimenta dinheiro (cap. 36): o retry do cliente não pode duplicar nada.
- Autenticação definida: quem chama e com que token (cap. 49).

4. Esboçar o modelo de dados

Liste as entidades, os campos que importam e, principalmente, os padrões de acesso: como cada tela lê os dados. É o padrão de acesso que escolhe o banco.

- Transações e consultas variadas pedem um banco relacional, com garantias ACID (cap. 4).
- Acesso por chave em volume gigante pede chave-valor ou wide-column (DynamoDB, Cassandra), em geral com garantias BASE.
- Muito mais leitura que escrita? Considere desnormalizar (cap. 28).
- Cada tipo de dado pode ficar no banco que o atende melhor (cap. 29). Diga também que índices você criaria (cap. 26).

Frase útil: *"Quase toda leitura é pelo id do usuário, então ele vai ser a chave de partição."*

5. Desenhar a visão de alto nível

Desenhe o caminho de uma requisição de ponta a ponta, com poucas caixas: cliente, DNS (cap. 16), CDN para o conteúdo estático (cap. 15), load balancer (cap. 11) ou API gateway (cap. 14), serviços stateless (cap. 8), cache (cap. 41), banco e, para o que pode esperar, uma fila (cap. 44) ou um tópico de pub/sub (cap. 45).

Comece pelo desenho mais simples que atende aos requisitos e só acrescente uma caixa quando um número da etapa 2 pedir; um monólito bem dividido é resposta legítima (cap. 9). Depois, percorra em voz alta um fluxo de escrita e um de leitura. É assim que aparecem os buracos.

Frase útil: *"Vou seguir uma leitura do celular até o banco e de volta; depois, uma escrita. Quer que eu aprofunde alguma parte?"*

6. Aprofundar em um ou dois componentes

É aqui que está a maior parte da avaliação, principalmente em vagas sênior. Pergunte se o entrevistador tem preferência; se não tiver, escolha o gargalo que as estimativas apontaram. Os aprofundamentos mais comuns:

- Particionar os dados: sharding (cap. 23) e hashing consistente (cap. 25).
- Replicar (cap. 24) e eleger um líder (cap. 38).
- Cache: escrita e invalidação (cap. 42) e o que remover quando enche (cap. 43).
- Por dentro do banco: B-Tree e LSM-Tree (cap. 26), WAL (cap. 27) e Bloom filter (cap. 30).
- Operações que atravessam serviços: saga (cap. 39) ou 2PC (cap. 40).
- Mensageria: garantias de entrega e dead letter queue (cap. 46).

Em cada ponto, apresente duas opções, compare e escolha com base num requisito.

Frase útil: *"Posso particionar por data ou por hash do id. Por data, toda escrita nova cai no mesmo shard; por hash, a carga se espalha. Como a leitura aqui é sempre por id, fico com hash."*

7. Caçar gargalos e falhas

Olhe para o seu desenho como quem quer derrubá-lo:

- **E se o tráfego multiplicar por 10?** Descubra o que satura primeiro. Escalar horizontalmente (cap. 1) só acelera a parte do trabalho que dá para paralelizar (cap. 6).
- **E se cada caixa cair?** Procure pontos únicos de falha. Réplicas (cap. 24), heartbeat (cap. 37) e failover cuidam deles.
- **E se uma dependência ficar lenta?** Timeouts, retries com backoff exponencial (cap. 35), circuit breaker (cap. 33) e bulkhead (cap. 34) impedem que a lentidão se espalhe.
- **E se alguém abusar?** Rate limiting (cap. 32), TLS (cap. 50) e zero trust entre serviços (cap. 51).

Diga o trade-off com todas as letras: o que você ganhou, o que pagou e em que cenário mudaria de ideia.

Frase útil: "*O ponto fraco é o perfil de uma celebridade, lido milhões de vezes. Se virar problema, replico essas chaves em mais nós do cache, ao custo de memória.*"

8. Encerrar

Guarde três minutos. Recapitule os requisitos atendidos, as duas ou três decisões principais e o que cada uma custou. Diga como operaria o sistema: que SLIs e SLOs acompanharia (cap. 48) e como acharia uma requisição lenta com tracing distribuído (cap. 47). Feche com o que faria se tivesse mais tempo.

Frase útil: "*Recapitulando: o feed prioriza disponibilidade, o cache segura as leituras e a fila absorve o que pode esperar. Com mais tempo, eu detalharia o feed de quem segue celebridades.*"

Erros comuns

- **Desenhar antes de perguntar.** Você resolve com capricho um problema que ninguém pediu.
- **Afogar-se nas contas.** Quinze minutos de aritmética roubam o aprofundamento.

- **Citar tecnologia sem motivo.** “Coloca um Kafka aqui” não é argumento; “preciso de uma fila porque o processamento leva 30 segundos” é.
- **Projetar para a escala do Google.** Sem um número que a justifique, complexidade conta contra você.
- **Desenhar só o caminho feliz.** Se você não falar de falhas, o entrevistador vai perguntar.
- **Pensar em silêncio.** O que se avalia é o raciocínio. Narre.
- **Brigar com a dica.** “E se esse banco cair?” quase sempre aponta um buraco. Reconheça e ajuste.
- **Não fechar.** Sem resumo, o desenho parece inacabado.

Para treinar, resolva um problema em voz alta com um cronômetro de 45 minutos, de preferência para outra pessoa, e depois releia os capítulos em que travou.

APÊNDICE B

Números e estimativas

Na entrevista, ninguém espera que você saiba de cor quanto tempo leva uma leitura em SSD. O que se espera é que você saiba a ordem de grandeza: se algo leva nanossegundos, microssegundos ou milissegundos, e se o sistema precisa de um servidor ou de mil. Este apêndice reúne os números que mais aparecem em entrevista e mostra como fazer as contas de cabeça.

Latências de referência

Os valores abaixo são **ordens de grandeza**, não medidas exatas. Eles variam com o hardware, a geração do processador, o tipo de SSD e a rota da rede: um SSD NVMe moderno, por exemplo, faz a leitura aleatória em dezenas de microssegundos. O que vale guardar é a distância entre as linhas. A última coluna estica o tempo para a escala humana: se um acesso à cache L1 levasse 1 segundo, quanto levaria cada operação?

Operação	Tempo típico	Se 1 ns fosse 1 s
Acesso à cache L1 da CPU	~1 ns	1 segundo
Acesso à memória RAM	~100 ns	~2 minutos
Enviar 1 KB numa rede de 1 Gbit/s	~10 µs	~3 horas
Leitura aleatória de 4 KB em SSD	~100 µs	~1 dia
Ida e volta na rede do mesmo datacenter	~0,5 ms	~6 dias
Leitura sequencial de 1 MB em SSD	~1 ms	~12 dias
Ida e volta entre zonas de disponibilidade da mesma região	~1 ms	~12 dias
Busca (seek) em disco rígido	~10 ms	~4 meses

Operação	Tempo típico	Se 1 ns fosse 1 s
Ida e volta entre continentes (São Paulo ↔ EUA ou Europa)	~150 ms (100 a 200 ms)	~5 anos

O que tirar da tabela:

- **A RAM é ~1.000 vezes mais rápida que o SSD** no acesso aleatório (100 ns contra 100 µs). Por isso o cache em memória (cap. 41) rende tanto.
- **Uma ida e volta no datacenter vale ~5.000 acessos à RAM** (0,5 ms ÷ 100 ns). Dez chamadas encadeadas a outros serviços somam ~5 ms antes de qualquer processamento.
- **Ler em sequência é muito mais barato que ler aos saltos**, sobretudo em disco rígido. É a ideia por trás do WAL (cap. 27) e das LSM-Trees (cap. 26).
- **A distância domina.** Uma ida e volta entre continentes vale ~300 dentro do datacenter (150 ÷ 0,5). Uma conexão HTTPS nova gasta uma ida e volta no TCP e outra no TLS 1.3 antes de a requisição sair (cap. 17, cap. 50): com o servidor do outro lado do oceano, são ~300 ms só de preparação. CDN (cap. 15), conexões reaproveitadas e QUIC (cap. 18) atacam exatamente isso.
- **Um GET no Redis do mesmo datacenter leva menos de 1 ms**, e quase todo esse tempo é rede.

Potências de 2 e de 10

Potência de 2	Valor exato	Aproximação	Unidade
2^{10}	1.024	mil (10^3)	KB (kilobyte)
2^{20}	1.048.576	um milhão (10^6)	MB (megabyte)
2^{30}	1.073.741.824	um bilhão (10^9)	GB (gigabyte)
2^{40}	~1,1 trilhão	um trilhão (10^{12})	TB (terabyte)
2^{50}	~1,13 quatrilhão	um quatrilhão (10^{15})	PB (petabyte)

A aproximação erra 2,4% no KB e ~10% no TB. Numa estimativa, ignore a diferença (a rigor, 2^{10} bytes é um KiB, e 10^3 bytes é um KB).

Dois limites que aparecem em entrevista:

- **$2^{31} - 1 \approx 2{,}1$ bilhões** é o maior inteiro de 32 bits com sinal. Uma tabela com id `INT` estoura aí; por isso tabelas grandes usam ids de 64 bits.
- **$2^{32} \approx 4{,}3$ bilhões** é o total de endereços IPv4.

Tamanhos úteis: um inteiro de 64 bits ou um timestamp ocupa 8 bytes; um UUID, 16 bytes (36 caracteres em texto); um caractere ASCII, 1 byte, e uma letra acentuada em UTF-8, 2 bytes; uma foto comprimida, de centenas de KB a alguns MB.

Atalhos para conta de cabeça

Atalho	De onde vem
1 dia $\approx 10^5$ s	86.400 s, arredondado para cima
1 mês $\approx 2{,}6 \times 10^6$ s	$30 \times 86.400 = 2.592.000$ s
1 ano $\approx 3 \times 10^7$ s	$365 \times 86.400 = 31.536.000$ s
1 milhão por dia ≈ 12 por segundo	$10^6 \div 86.400 \approx 11{,}6$
1 bilhão por dia ≈ 12 mil por segundo	$10^9 \div 86.400 \approx 11.574$
100 milhões por mês ≈ 40 por segundo	$10^8 \div 2.592.000 \approx 38{,}6$
1 Gbit/s ≈ 125 MB/s	10^9 bits ÷ 8 bits por byte
Pico ≈ 2 a 3 vezes a média	suposição comum; Black Friday e jogo da Copa passam disso

Dividir por 10^5 em vez de 86.400 subestima o resultado em ~14%. Para estimar, tudo bem: o fator de pico cobre a diferença com folga. As três fórmulas que resolvem quase tudo:

- **QPS** (requisições por segundo) = ações por dia ÷ 10^5.
- **Armazenamento** = itens por dia × tamanho × dias × número de cópias.
- **Banda** = QPS × tamanho de cada requisição ou resposta.

Disponibilidade: a tabela dos noves

Disponibilidade	Fora do ar por ano	Fora do ar por mês (30 dias)
99% (dois noves)	~3,65 dias	~7,2 horas
99,5%	~1,8 dia	~3,6 horas
99,9% (três noves)	~8,8 horas	~43 minutos
99,95%	~4,4 horas	~22 minutos
99,99% (quatro noves)	~53 minutos	~4,3 minutos
99,999% (cinco noves)	~5,3 minutos	~26 segundos

A conta é (1 − disponibilidade) × período. Com 99,9%: 0,001 × 525.600 minutos do ano ≈ 526 minutos, ou 8,76 horas. Esse tempo é o orçamento de erro de um SLO (cap. 48).

Quando há mais de um componente:

- **Em série, multiplique as disponibilidades.** Se a requisição depende de dois componentes com 99,9% cada, o conjunto fica com 0,999 × 0,999 ≈ 99,8%. Cada dependência a mais derruba o total.
- **Em paralelo, multiplique as chances de falha.** Com duas réplicas independentes de 99,9% (cap. 24), o serviço só cai se as duas caírem: 1 − 0,001 × 0,001 = 99,9999%. Na prática as falhas andam juntas (mesmo deploy, mesma região, mesmo bug), então trate esse número como teto otimista.

Exemplo resolvido: um app de fotos

Um app hipotético em que as pessoas postam fotos e rolam um feed. Premissas, combinadas com o entrevistador:

- 10 milhões de usuários ativos por dia.
- Cada usuário posta uma foto a cada cinco dias (0,2 por dia) e vê 50 fotos por dia.
- Cada foto comprimida tem 500 KB; os metadados (autor, legenda, data) têm ~1 KB.
- O pico é 3 vezes a média.

Passo 1: QPS.

```
Escritas: 10^7 × 0,2 = 2 × 10^6 fotos por dia
          2 × 10^6 ÷ 10^5 s = 20/s          pico: 60/s
Leituras: 10^7 × 50 = 5 × 10^8 fotos vistas por dia
          5 × 10^8 ÷ 10^5 s = 5.000/s       pico: 15.000/s
```

São 250 leituras para cada escrita: um sistema dominado por leitura.

Passo 2: armazenamento em 5 anos.

```
Fotos:     2 × 10^6 × 500 KB = 1 TB por dia
           1 TB × 365 × 5 ≈ 1,8 PB
Metadados: 2 × 10^6 × 1 KB = 2 GB por dia
           2 GB × 365 × 5 ≈ 3,7 TB
```

Guardando três cópias de cada foto, são ~5,5 PB.

Passo 3: banda.

```
Entrada: 1 TB ÷ 10^5 s = 10 MB/s = 80 Mbit/s
         pico: 240 Mbit/s
Saída:   5 × 10^8 × 500 KB = 250 TB por dia
         250 TB ÷ 10^5 s = 2,5 GB/s = 20 Gbit/s
         pico: 60 Gbit/s
```

O que os números dizem:

- 60 escritas por segundo no pico é pouco. O desafio está na leitura.
- 15 mil leituras por segundo pedem cache (cap. 41) e réplicas de leitura (cap. 24).
- Foto não vai para o banco: vai para um armazenamento de objetos, e o banco guarda só o endereço (cap. 29). Os 3,7 TB de metadados crescem sem parar, então vale planejar o sharding (cap. 23).
- De 20 a 60 Gbit/s de saída tornam a CDN obrigatória (cap. 15). Servir no feed versões menores das fotos derrubaria esse número.

Uma boa estimativa termina numa decisão de arquitetura. Se a conta não mudou nenhuma escolha, ela podia ter ficado de fora.

APÊNDICE C

Estudo de caso: um encurtador de links

Projetar um encurtador de links, no estilo do bit.ly, é um dos pedidos mais comuns em entrevista: cabe em 45 minutos e passa por boa parte deste livro. Aqui ele segue as oito etapas do apêndice A. Os números são hipotéticos, escolhidos para deixar as contas redondas.

1. Requisitos

Funcionais:

1. Criar um link curto a partir de uma URL longa, com apelido personalizado opcional.
2. Redirecionar quem abre o link curto para a URL original.
3. Mostrar ao dono quantos cliques o link teve, por dia e por país.

Ficam de fora: editar o destino, painel completo e QR code.

Não funcionais:

- **Leitura domina:** cada link é criado uma vez e clicado, em média, 100 vezes.
- **Redirecionamento rápido:** 99% das respostas em menos de 50 ms no servidor (p99).
- **Alta disponibilidade:** link que não abre quebra campanha, post e cartaz impresso. Meta de 99,99% (cap. 48).
- **Durabilidade:** link criado não some.
- **Consistência:** o link novo funciona em segundos; as estatísticas podem atrasar minutos (consistência eventual, cap. 7).
- **Códigos difíceis de adivinhar:** ninguém deve listar os links dos outros incrementando o código.

2. Estimativas

Premissas: 100 milhões de links novos por mês, 100 cliques por link e pico de 3 vezes a média.

```
Escritas: 10^8 por mês ÷ 2,6 × 10^6 s ≈ 40/s   pico: ~120/s
Leituras: 100 × 40/s = 4.000/s               pico: ~12.000/s
Registro: código + URL + metadados + índice ≈ 500 bytes
Por mês:  10^8 × 500 B = 50 GB
5 anos:   50 GB × 60 = 3 TB (6 bilhões de links)
Cache:    20% mais clicados do mês: 2 × 10^7 × 500 B = 10 GB
Banda:    4.000 respostas/s × ~500 B = 2 MB/s
Cliques:  10^10 por mês × 200 B = 2 TB de eventos por mês
```

O que os números dizem: 120 escritas por segundo é pouco para qualquer banco; 12 mil leituras por segundo pedem cache; 3 TB que nunca param de crescer pedem um banco que escale na horizontal; a banda é irrelevante; e o maior volume está nos eventos de clique, não nos links.

3. API

A criação exige um token OAuth (cap. 49):

```
POST /links HTTP/1.1
Authorization: Bearer <token>
Idempotency-Key: 7c0e2a9b-1f4d-4a8e-9b1a-3d2f6e8c5a10
Content-Type: application/json

{"url": "https://loja.example.com/ofertas?origem=insta"}
```

A resposta é `201 Created`, com o código (`aZ3kP9x`) e o link curto (`https://lnk.example/aZ3kP9x`). O redirecionamento é público:

```
GET /aZ3kP9x HTTP/1.1
Host: lnk.example

HTTP/1.1 302 Found
Location: https://loja.example.com/ofertas?origem=insta
```

Um terceiro endpoint, `GET /links/{codigo}/cliques`, devolve as contagens agregadas.

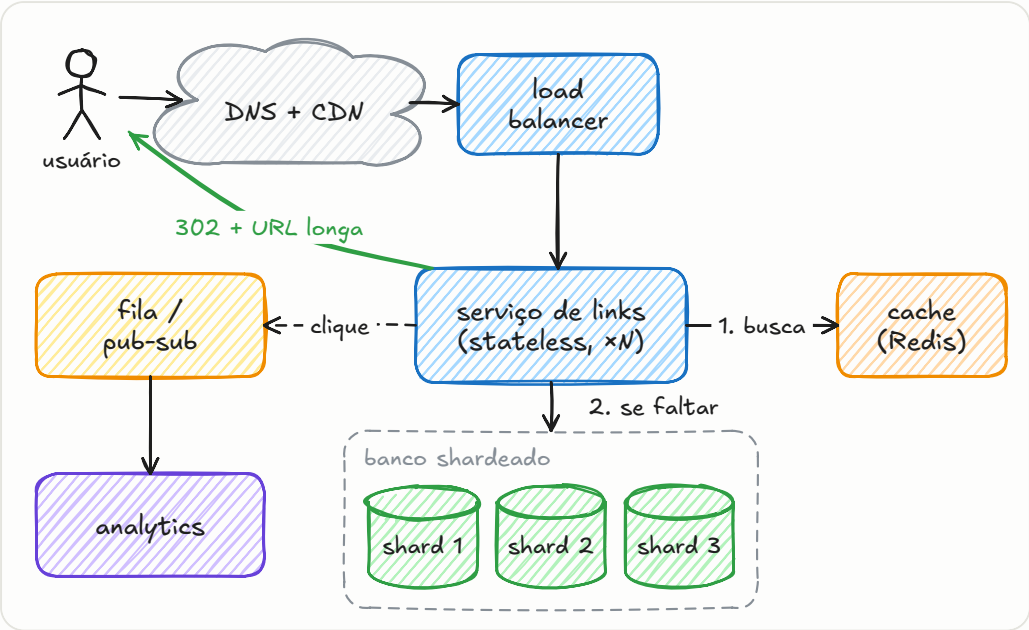
Por que 302 e não 301? O 301 (Moved Permanently) diz que o endereço mudou para sempre, e o navegador guarda essa resposta: nos cliques seguintes, nem passa pelo seu servidor. A carga cai, mas você deixa de contar esses cliques e perde o poder de desativar um link que virou golpe. O 302 (Found) é temporário, então todo clique passa pelo serviço. Como as estatísticas são requisito, fica o 302. (O 307 também serviria; preservar o método HTTP, que é a diferença dele, não pesa num GET.)

4. Modelo de dados

Campo	Tipo	Observação
`codigo`	texto curto	chave primária e chave de partição
`url_longa`	texto	até ~2 KB
`dono_id`	inteiro de 64 bits	quem criou
`criado_em`	timestamp	
`status`	texto	`ativo` ou `bloqueado`

O acesso é praticamente um só: dado o código, devolver a URL. Sem join nem consulta por intervalo, o caminho é um banco chave-valor distribuído, do tipo DynamoDB ou Cassandra. As estatísticas ficam num banco colunar, feito para agregações: cada dado no banco que o atende melhor (cap. 29).

5. Visão de alto nível



O caminho de um clique: o DNS leva à borda da CDN, o load balancer escolhe uma instância do serviço, e o serviço procura o código no cache e, se faltar, no banco shardeado. O 302 volta ao usuário, e o evento do clique segue para a fila (tracejado) sem atrasar a resposta.

O clique. O DNS (cap. 16) resolve `lnk.example` para a borda mais próxima da CDN (cap. 15), que termina o TLS perto do usuário, encurtando o handshake (cap. 50), e segura picos e ataques. Ela não guarda o 302, para que todo clique seja contado. O load balancer (cap. 11) distribui as requisições entre instâncias stateless do serviço (cap. 8), que procura o código no cache, vai ao banco se não achar, responde 302 e publica o evento do clique numa fila, sem esperar por ela.

A criação. O pedido passa pelo rate limiting e pela checagem de idempotência, ganha um código e é gravado no banco e no cache.

O mesmo serviço faz as duas coisas. Como ele não guarda estado, separá-lo em dois mais tarde é fácil, se as cargas divergirem (cap. 9).

6. Aprofundamento

Geração do código

Em base62 (0–9, a–z, A–Z), cada caractere tem 62 valores: seis caracteres dão $62^6 \approx$ 56,8 bilhões de códigos, e sete, $62^7 \approx$ 3,5 trilhões. Seis durariam ~47 anos no ritmo de 1,2 bilhão de links por ano, mas os 6 bilhões de links de 5 anos ocupariam 10% do espaço: um chute ao acaso acertaria um link a cada dez tentativas. Com sete, a ocupação fica em 0,17%. Ficamos com sete.

Há três jeitos de gerar o código.

Hash da URL. Calcula-se um hash (MD5, SHA-256) da URL e ficam os 7 primeiros caracteres em base62. Dispensa coordenação, mas o hash truncado colide, e muito. Pelo paradoxo do aniversário, com n = 6 bilhões de links e N = 3,5 trilhões de códigos possíveis:

```
colisões ≈ n² ÷ (2 × N)
         = (6 × 10^9)² ÷ (2 × 3,5 × 10^12)
         ≈ 5 milhões em 5 anos
```

Cada colisão exige checar se o código existe e tentar de novo com um sal. E quem encurtasse a mesma URL ganharia o mesmo link, com as estatísticas misturadas.

Sorteio. Sorteiam-se 7 caracteres e grava-se com escrita condicional ("só se o código não existir"). É simples e imprevisível, mas tem as mesmas colisões, e cada uma custa uma nova tentativa.

Contador. Um número que só cresce, convertido para base62. Nunca colide, mas o contador central vira gargalo e ponto único de falha, e códigos sequenciais são fáceis de enumerar.

A escolha: contador em faixas. Cada instância pede ao coordenador uma faixa de mil números e vai gastando localmente; se o coordenador cair, as instâncias seguem com o que têm. Ele precisa ser CP (cap. 2), porque entregar a mesma faixa duas vezes repetiria códigos. Por isso mora num armazenamento consistente, como o etcd, que usa Raft (cap. 38). Antes da conversão, cada número passa por uma permutação fixa e reversível, e códigos vizinhos deixam de parecer vizinhos. Uma instância que morre perde o resto da faixa, o que não faz falta num espaço de 3,5 trilhões.

```
ALFABETO = ("0123456789"
            "abcdefghijklmnopqrstuvwxyz"
            "ABCDEFGHIJKLMNOPQRSTUVWXYZ")

def base62(n: int) -> str:
    codigo = ""
    while n > 0:
        n, resto = divmod(n, 62)
        codigo = ALFABETO[resto] + codigo
    return codigo or "0"

base62(1_000_000_000)  # '15FTGg'
```

Apelidos usam o mesmo espaço, com escrita condicional; se o apelido já existir, a API responde `409 Conflict`. A gravação dos códigos do contador também é condicional: se um apelido já ocupou aquele texto, o serviço pula para o próximo número.

Leituras rápidas: cache e sharding

Com até 12 mil leituras por segundo e um conjunto quente de ~10 GB, o cache resolve quase tudo (cap. 41). A estratégia é cache-aside (cap. 42): o serviço procura no Redis; se não acha, lê do banco e guarda com TTL. Na criação, o link já entra no cache, porque quem cria costuma compartilhar na hora. Um link nunca muda, então não há invalidação, exceto no bloqueio, quando o serviço apaga a chave. Para remover, LRU (cap. 43) basta: os links quentes são lidos o tempo todo. Com 90% de acerto, o banco recebe ~1.200 leituras por segundo no pico.

Dois cuidados:

- **Link viral.** Dezenas de milhares de cliques por segundo no mesmo código caem num só nó do Redis. Um cache local em cada instância, com TTL de poucos segundos, absorve o pico.
- **Código inexistente.** Quem varre códigos ao acaso nunca acerta o cache. Guarde também o "não existe", por pouco tempo.

O banco guarda 3 TB em 5 anos e não para de crescer. Ele é particionado pelo hash do código (cap. 23), com hashing consistente (cap. 25): quando entra um nó novo, só a fração das chaves que passa a ser dele muda de lugar, e a carga, inclusive a das escritas novas, se espalha por igual.

7. Gargalos, falhas e trade-offs

Abuso: rate limiting

Encurtador atrai spam e golpe. Na criação, um token bucket por chave de API (ou por IP, para quem não se autentica) limita, digamos, 100 links por hora na conta gratuita; acima disso, a resposta é `429 Too Many Requests` (cap. 32). No redirecionamento, o alvo são as varreduras: um IP que acumula códigos inexistentes também recebe 429. Um verificador assíncrono confere as URLs novas contra listas de sites maliciosos e bloqueia o que for golpe.

Idempotência na criação

O app envia o POST, a resposta se perde num timeout e o app tenta de novo (cap. 35): sem cuidado, nascem dois links. Por isso o cliente manda um `Idempotency-Key` por operação (cap. 36). O serviço grava a chave com a resposta, por escrita condicional e com validade de 24 horas; se a chave voltar, devolve a resposta guardada. Deduplicar pela URL não serviria: duas pessoas podem querer links diferentes para o mesmo destino.

Cliques sem atrasar o redirecionamento

Incrementar um contador no banco a cada clique poria uma escrita no caminho crítico e faria do link viral uma hot key de escrita. Em vez disso, o serviço publica o evento (código, horário, país, origem) num tópico de pub/sub (cap. 45), sem esperar. Cada grupo de consumidores lê no seu ritmo: o agregador soma cliques por link, dia e país no banco colunar; o detector de abuso procura padrões estranhos; o arquivador guarda o evento bruto em armazenamento barato. Se o agregador atrasar, os eventos esperam na fila (cap. 44). A entrega é "pelo menos uma vez", então o agregador descarta repetidos pelo id do evento, e eventos malformados vão para uma dead letter queue (cap. 46). Com o broker fora do ar por muito tempo, perder alguns cliques é aceitável; atrasar o redirecionamento, não.

Replicação e disponibilidade

Cada shard tem três réplicas, uma por zona de disponibilidade (cap. 24), e a escrita espera duas confirmações (W = 2). Como um link nunca muda, qualquer réplica que tenha o registro tem o valor certo. Então a leitura vai a uma réplica (R = 1) e, se ela

não achar, confirma com duas (R = 2) antes do 404. Como R + W > N (2 + 2 > 3 réplicas), pelo menos uma das duas tem o link recém-criado.

Numa partição de rede (cap. 2), o redirecionamento prefere disponibilidade e responde com o que tem; o coordenador de faixas prefere consistência e recusa, enquanto as instâncias criam links com as faixas que já têm.

E se o cache cair? As 12 mil leituras por segundo iriam a um banco dimensionado para ~1.200. O cache local amortece o impacto, e um circuit breaker (cap. 33) diante do banco recusa o excesso na hora, em vez de deixar tudo lento.

8. Encerramento: como operar

SLIs (cap. 48): latência p99 do redirecionamento, taxa de 5xx, taxa de acerto do cache e eventos parados na fila de analytics. SLO: 99,99% dos redirecionamentos com sucesso em 30 dias. Com 10 bilhões de cliques por mês, o orçamento de erro é de 1 milhão de falhas, o equivalente a ~4 minutos fora do ar (apêndice B). Para achar onde o tempo se perde, tracing distribuído (cap. 47) com amostragem: 1% dos cliques já dá dezenas de traces por segundo, e a criação, rara, pode ser rastreada inteira.

Decisões e alternativas

Decisão	Alternativa descartada	Por quê
Redirecionar com 302	301	O navegador guarda o 301 e não volta ao serviço: somem a contagem e o poder de bloquear golpe.
Contador em faixas, permutação e base62	Hash da URL truncado	O hash truncado colide (~5 milhões de vezes em 5 anos) e junta usuários diferentes no mesmo link.
Código de 7 caracteres	6 caracteres	Com 6, um chute acertaria um link em cada dez; com 7, um em ~600.

Decisão	Alternativa descartada	Por quê
Banco chave-valor particionado com hashing consistente	Relacional num nó só	O acesso é só por chave e o volume cresce para sempre; um nó único vira teto e ponto único de falha.
Cache-aside com cache local	Ler sempre do banco	Tira ~90% da carga do banco; o cache local segura o link viral.
Cliques via pub/sub, agregados depois	Contador no banco a cada clique	A escrita síncrona aumenta a latência e cria uma hot key de escrita.
`Idempotency-Key` no POST	Deduplicar pela URL	A chave identifica a tentativa; a mesma URL pode virar vários links.
Ler de 1 réplica e confirmar com 2 antes do 404	Quórum em toda leitura	O link é imutável: só o raro “não achei” paga o quórum.

APÊNDICE D

Glossário

A

ACID: atomicidade, consistência, isolamento e durabilidade, as quatro garantias de uma transação confiável num banco de dados. Capítulo 4.

API Gateway: proxy reverso especializado que é a porta de entrada única das chamadas de API e cuida de roteamento, autenticação, rate limiting e logs. Capítulo 14.

At-least-once (*pelo menos uma vez*): garantia de entrega em que nenhuma mensagem se perde, mas algumas podem chegar repetidas, o que exige consumidores idempotentes. Capítulos 36 e 44.

B

Backoff exponencial (*exponential backoff*): esperar cada vez mais entre uma tentativa e a seguinte (1 s, 2 s, 4 s...) para não sobrecarregar um serviço que já está com problemas. Capítulo 35.

Backpressure (*contrapressão*): sinal que um componente sobrecarregado manda para quem produz trabalho diminuir o ritmo, em vez de deixar as filas crescerem sem limite. Capítulo 44.

Banco de dados vetorial (*vector database*): banco que guarda embeddings e encontra os mais próximos de uma consulta, a chamada busca por vizinhos mais próximos. Capítulo 31.

BASE: sigla de Basically Available, Soft state, Eventual consistency, modelo que prioriza continuar respondendo e aceita inconsistências temporárias, que se resolvem com o tempo. Capítulo 4.

Bloom filter (*filtro de Bloom*): estrutura compacta de bits e funções de hash que responde "com certeza não está" ou "talvez esteja": pode dar falso positivo, nunca falso negativo. Capítulo 30.

B-Tree (*árvore B*): árvore balanceada e ordenada, base dos índices dos bancos relacionais, que encontra uma chave ou um intervalo com poucas leituras de disco. Capítulo 26.

Bulkhead: isolar recursos (pools de conexões, threads, clusters) por funcionalidade, como

as anteparas de um navio, para que a falha de uma parte não afunde o resto. Capítulo 34.

C

Cache: camada de armazenamento rápido, quase sempre em memória, que guarda dados muito acessados para reduzir a latência e a carga sobre o banco. Capítulo 41.

Cache-aside: estratégia em que a aplicação procura no cache e, num miss, lê do banco e grava o resultado no cache; também chamada de lazy loading. Capítulo 42.

CAP: teorema segundo o qual, durante uma partição de rede, um sistema distribuído precisa escolher entre consistência (C) e disponibilidade (A). Capítulo 2.

CDN (*rede de distribuição de conteúdo*): sigla de Content Delivery Network, rede de servidores espalhados pelo mundo que guarda cópias do conteúdo estático perto dos usuários. Capítulo 15.

Circuit breaker (*disjuntor*): componente que, depois de falhas demais, recusa na hora as chamadas a um serviço com problema e só volta a testá-lo após um tempo, evitando falhas em cascata. Capítulo 33.

Cold start (*partida a frio*): atraso de uma chamada serverless que precisa de um ambiente de execução novo, criado e inicializado na hora. Capítulo 10.

Compaction (*compactação*): processo de fundo das LSM-Trees que mescla os arquivos do disco, descarta versões antigas e dados apagados e reduz o trabalho das leituras. Capítulo 26.

Consenso (*consensus*): acordo entre os nós de um cluster sobre um mesmo valor ou uma mesma ordem de operações, mesmo com falhas, obtido com algoritmos como Paxos e Raft. Capítulo 38.

Consistência eventual (*eventual consistency*): garantia de que, se as escritas pararem, todas as réplicas convergem para o mesmo valor; até lá, uma leitura pode ver dado velho. Capítulo 7.

Consistência forte (*strong consistency*): garantia de que toda leitura vê a escrita confirmada mais recente, como se existisse uma única cópia dos dados. Capítulo 7.

D

Desnormalização (*denormalization*): duplicar dados de propósito para acelerar leituras e evitar joins, pagando com escritas mais trabalhosas e o risco de cópias divergentes. Capítulo 28.

DLQ (*fila de mensagens mortas*): sigla de dead letter queue, a fila para onde vão as mensagens que falharam depois de várias tentativas, para análise e reprocessamento. Capítulo 46.

DNS (*sistema de nomes de domínio*): sigla de Domain Name System, o serviço que traduz nomes de domínio em endereços IP, com cache em várias camadas. Capítulo 16.

E

Embedding: vetor de números gerado por um modelo para representar o significado de um texto, imagem ou áudio, de modo que itens parecidos fiquem próximos. Capítulo 31.

Error budget (*orçamento de erro*): a margem de falha que o SLO tolera num período; um SLO de 99,9% de disponibilidade mensal deixa cerca de 43 minutos de indisponibilidade. Capítulo 48.

Escalabilidade horizontal (*horizontal scaling*): crescer adicionando máquinas e dividindo o trabalho entre elas (scale out). Capítulo 1.

Escalabilidade vertical (*vertical scaling*): crescer dando mais recursos a uma única máquina, como CPU, memória ou disco mais rápido (scale up). Capítulo 1.

Exactly-once (*exatamente uma vez*): garantia de que o efeito de cada mensagem acontece uma única vez; na prática, sai de entrega at-least-once com deduplicação ou processamento idempotente. Capítulos 36 e 44.

F

Failover: troca automática de um componente que falhou por um reserva, como promover uma réplica a primário quando o primário cai. Capítulos 24 e 37.

Fencing token: número sempre crescente entregue a cada novo dono de um lock ou da liderança, que permite ao recurso recusar ordens vindas de um dono antigo. Capítulo 38.

Fila de mensagens (*message queue*): estrutura em que um produtor deixa mensagens para processamento posterior, cada uma consumida por um único receptor, desacoplando os dois lados. Capítulo 44.

G

Gossip (*fofoca*): protocolo em que cada nó troca informações periodicamente com alguns pares aleatórios, até que todo o cluster tenha aproximadamente a mesma visão, sem coordenador central. Capítulo 22.

gRPC: framework de chamadas remotas que usa HTTP/2 e mensagens binárias em Protocol Buffers, com contratos tipados e streaming nos dois sentidos. Capítulo 19.

H

Hashing consistente (*consistent hashing*): técnica que posiciona chaves e nós num anel lógico, de modo que adicionar ou remover um nó mova só uma pequena parte das chaves. Capítulo 25.

Head-of-line blocking (*bloqueio de cabeça de fila*): quando um item atrasado no começo da fila segura todos os que vêm atrás; no HTTP/2, um pacote TCP perdido trava todos os streams da conexão. Capítulo 18.

Heartbeat: sinal periódico que um nó envia para avisar que está vivo; se ele para de chegar, o nó é dado como fora e o sistema pode disparar o failover. Capítulo 37.

Hot spot: shard, nó ou chave que recebe uma fatia desproporcional do tráfego, geralmente por causa de uma shard key mal escolhida. Capítulo 23.

HTTP/2: versão do HTTP que multiplexa várias requisições numa mesma conexão TCP e comprime os cabeçalhos. Capítulo 18.

HTTP/3: versão do HTTP que roda sobre QUIC, em cima de UDP, e acaba com o head-of-line blocking do TCP entre streams. Capítulo 18.

I

Idempotência (*idempotency*): propriedade de uma operação que, executada várias vezes, tem o mesmo efeito de uma execução só, como “definir status como ativo”. Capítulo 36.

Idempotency key (*chave de idempotência*): identificador único que o cliente envia com a requisição para o servidor reconhecer repetições e não executá-las de novo, como numa cobrança. Capítulo 36.

IP Hash: algoritmo de balanceamento que escolhe o servidor pelo hash do IP do cliente, mandando o mesmo cliente quase sempre para o mesmo servidor. Capítulo 12.

J

Jitter: aleatoriedade somada ao tempo de espera entre novas tentativas, para que os clientes não tentem todos no mesmo instante. Capítulo 35.

JWT: sigla de JSON Web Token, token assinado com informações (claims) que quem o recebe valida pela assinatura, sem consultar o emissor a cada requisição; o ID token do OIDC é um JWT. Capítulo 49.

L

Latência (*latency*): quanto tempo uma única requisição leva, do início ao fim. Capítulo 5.

Least Connections (*menos conexões*): algoritmo de balanceamento que manda cada requisição para o servidor com menos conexões ativas, bom quando as requisições têm durações muito diferentes. Capítulo 12.

Lei de Amdahl (*Amdahl's law*): o ganho com paralelismo é limitado pela parte do trabalho que precisa rodar em sequência; com 20% sequencial, o ganho máximo é de 5 vezes. Capítulo 6.

LFU: sigla de Least Frequently Used, política que remove do cache os itens acessados menos vezes. Capítulo 43.

Linearizabilidade (*linearizability*): a forma mais forte de consistência, em que o sistema se comporta como se houvesse uma única cópia dos dados; é o C do CAP. Capítulos 2 e 7.

Load balancer (*balanceador de carga*): componente que distribui o tráfego entre vários servidores e, com health checks, para de mandar requisições para os que estão com problema. Capítulo 11.

Long polling: técnica em que o servidor segura a requisição aberta até ter novidade ou estourar um timeout, e o cliente abre outra logo em seguida. Capítulo 21.

LRU: sigla de Least Recently Used, política que remove do cache o item que está há mais tempo sem ser acessado. Capítulo 43.

LSM-Tree: sigla de Log-Structured Merge-Tree, estrutura que acumula as escritas em memória e as grava em lote em arquivos ordenados no disco, com escrita muito rápida e leitura mais cara. Capítulo 26.

M

Memtable: estrutura ordenada em memória que recebe as escritas de uma LSM-Tree até encher e ser gravada no disco como um arquivo imutável (SSTable). Capítulo 26.

Microsserviços (*microservices*): arquitetura que divide as funcionalidades em serviços separados, com deploy independente, que conversam pela rede. Capítulo 9.

Monólito (*monolith*): aplicação única que reúne todas as funcionalidades numa só unidade de deploy. Capítulo 9.

mTLS: TLS mútuo, em que cliente e servidor apresentam certificados e um autentica o outro; é a base da comunicação entre serviços em Zero Trust. Capítulos 50 e 51.

Multi-primário (*multi-primary*): modelo de replicação, antes chamado de master-master, em que vários nós aceitam escritas e precisam resolver os conflitos entre elas. Capítulo 24.

N

Normalização (*normalization*): organizar os dados em tabelas sem redundância, seguindo as formas normais, para evitar anomalias de inserção e de atualização. Capítulo 28.

O

OAuth 2.0: framework de autorização delegada em que o usuário permite que um app acesse recursos dele por meio de um access token, sem entregar a senha. Capítulo 49.

OIDC: sigla de OpenID Connect, camada sobre o OAuth 2.0 que adiciona autenticação, entregando um ID token que diz quem é o usuário. Capítulo 49.

P

PACELC: extensão do CAP: se há partição (P), escolha entre disponibilidade (A) e consistência (C); senão (E), entre latência (L) e consistência (C). Capítulo 3.

Partição de rede (*network partition*): falha em que grupos de nós continuam funcionando, mas param de conseguir se comunicar entre si. Capítulo 2.

Paxos: família de algoritmos de consenso criada por Leslie Lamport, correta e influente, mas famosa por ser difícil de entender e de implementar. Capítulo 38.

Percentil (*percentile*): valor abaixo do qual fica uma fração das medições; o p99 de latência é o tempo que 99% das requisições não ultrapassam. Capítulos 5 e 48.

Persistência poliglota (*polyglot persistence*): usar vários tipos de banco no mesmo sistema, cada um no que faz melhor, como um relacional para pedidos e um chave-valor para sessões. Capítulo 29.

Primário-réplica (*primary-replica*): modelo de replicação, antes chamado de master-slave, em que só o primário aceita escritas e as réplicas recebem as mudanças e atendem leituras. Capítulo 24.

Proxy direto (*forward proxy*): intermediário que fica na frente dos clientes e os representa diante da internet, para filtrar conteúdo, fazer cache ou controlar acesso. Capítulo 13.

Proxy reverso (*reverse proxy*): intermediário que fica na frente dos servidores e os representa para os clientes, escondendo a topologia interna e fazendo terminação TLS, cache e roteamento. Capítulo 13.

Pub/Sub (*publicação/assinatura*): modelo em que publicadores enviam mensagens para tópicos e cada assinante do tópico recebe a sua própria cópia. Capítulo 45.

Q

QUIC: protocolo de transporte sobre UDP, com TLS 1.3 embutido e streams independentes, que serve de base para o HTTP/3. Capítulo 18.

Quórum (*quorum*): número mínimo de nós que precisa responder para uma leitura ou escrita valer; quóruns de maioria garantem que dois grupos quaisquer tenham pelo menos um nó em comum. Capítulos 3 e 38.

R

Raft: algoritmo de consenso feito para ser fácil de entender, que elege um líder por mandato e replica um log com confirmação da maioria; é usado no etcd e no Consul. Capítulo 38.

Rate limiting (*limitação de taxa*): limitar quantas requisições um usuário, IP ou chave de API pode fazer numa janela de tempo, com algoritmos como janela fixa, janela deslizante e token bucket. Capítulo 32.

Replication lag (*atraso de replicação*): intervalo entre uma escrita ser confirmada no primário e aparecer nas réplicas, que faz leituras em réplicas devolverem dados velhos. Capítulo 24.

Resharding: redistribuir os dados entre shards quando se adicionam ou removem shards ou quando se troca a shard key. Capítulo 23.

REST: estilo de API sobre HTTP organizado em recursos (/users, /orders) manipulados pelos verbos HTTP, normalmente com JSON. Capítulo 19.

Retry (*nova tentativa*): repetir automaticamente uma operação que falhou por um erro transitório, como um timeout. Capítulo 35.

Round Robin (*rodízio*): algoritmo de balanceamento que passa pelos servidores em ordem, um de cada vez, e recomeça do primeiro no fim da lista. Capítulo 12.

RTT (*tempo de ida e volta*): sigla de round-trip time, o tempo que uma mensagem leva para ir até o outro lado e a resposta voltar. Capítulos 18 e 50.

S

SAGA: padrão que divide uma transação distribuída em passos locais, cada um com uma ação compensatória que o desfaz se um passo seguinte falhar. Capítulo 39.

Serverless: modelo em que você roda funções na nuvem sem gerenciar servidores, paga só pela execução e deixa a escala por conta da plataforma. Capítulo 10.

Sharding (*particionamento*): dividir os dados entre várias máquinas, cada uma com um subconjunto, por intervalo, por hash ou por diretório. Capítulo 23.

Shard key (*chave de shard*): campo que decide em qual shard cada registro fica; dele depende a carga se espalhar ou virar um hot spot. Capítulo 23.

SLA (*acordo de nível de serviço*): sigla de Service Level Agreement, a promessa contratual feita ao cliente, como 99,9% de disponibilidade no mês, em geral com compensação se for descumprida. Capítulo 48.

SLI (*indicador de nível de serviço*): sigla de Service Level Indicator, a métrica medida de fato, como a porcentagem de requisições bem-sucedidas. Capítulo 48.

SLO (*objetivo de nível de serviço*): sigla de Service Level Objective, a meta interna para um SLI, normalmente mais rígida que o SLA. Capítulo 48.

Span: trecho de um trace que representa uma operação, como uma chamada ou uma consulta, com início, duração e referência ao span pai. Capítulo 47.

Split brain: situação em que dois nós se consideram líderes ao mesmo tempo, normalmente depois de uma partição, e aceitam escritas conflitantes. Capítulo 38.

SSE: sigla de Server-Sent Events, canal de mão única sobre HTTP pelo qual o servidor envia eventos ao cliente. Capítulo 20.

Stateful: serviço que guarda o contexto do usuário entre requisições, geralmente na memória da própria instância. Capítulo 8.

Stateless: serviço que trata cada requisição como nova e guarda o estado fora, em banco ou cache, de modo que qualquer instância atende qualquer requisição. Capítulo 8.

Sticky session (*afinidade de sessão*): manter o mesmo cliente sempre no mesmo servidor, por cookie ou hash de IP, quando o estado fica na instância. Capítulos 8 e 12.

T

TCP: protocolo de transporte orientado a conexão que entrega os bytes em ordem e sem perdas, usando confirmações e retransmissões. Capítulo 17.

Throughput (*vazão*): quantas requisições o sistema atende por segundo. Capítulo 5.

Thundering herd: muitos clientes ou processos atacando o mesmo recurso no mesmo instante, como novas tentativas sincronizadas depois de uma falha ou uma chave popular que expira no cache. Capítulo 35.

TLS: sigla de Transport Layer Security, protocolo que cifra e autentica a comunicação entre cliente e servidor; sucessor do SSL, é o que protege o HTTPS. Capítulo 50.

Token bucket: algoritmo de rate limiting em que fichas são repostas a uma taxa fixa e cada requisição gasta uma, o que permite rajadas até o tamanho do balde. Capítulo 32.

Tracing distribuído (*distributed tracing*): seguir uma requisição por todos os serviços que ela atravessa, com um trace ID propagado de chamada em chamada e um span por operação. Capítulo 47.

Transação compensatória (*compensating transaction*): ação que desfaz o efeito de um passo já confirmado de uma saga, como estornar um pagamento. Capítulo 39.

TTL (*tempo de vida*): sigla de time to live, o prazo de validade de um item em cache ou de um registro DNS, depois do qual ele expira. Capítulos 16 e 41.

Two-phase commit (*commit em duas fases*): protocolo (2PC) em que um coordenador pergunta a todos os participantes se podem confirmar e só depois manda todos confirmarem ou desfazerem. Capítulo 40.

U

UDP: protocolo de transporte sem conexão, sem garantia de entrega nem de ordem, mais leve e com menos latência que o TCP. Capítulo 17.

W

WAL: sigla de write-ahead log, o registro sequencial em que cada mudança é gravada antes de ser aplicada aos dados, o que permite recuperar o banco depois de uma queda. Capítulo 27.

WebSocket: protocolo de conexão persistente e full-duplex, em que cliente e servidor trocam mensagens a qualquer momento. Capítulo 20.

Write-back: estratégia em que a escrita vai para o cache e só depois chega ao banco; é rápida, mas perde dados se o cache cair antes disso. Capítulo 42.

Write-through: estratégia em que cada escrita vai para o cache e para o banco na mesma operação, mantendo os dois sincronizados. Capítulo 42.

Z

Zero Trust (*confiança zero*): modelo de segurança em que estar dentro da rede não dá confiança: toda requisição é autenticada, autorizada e cifrada, com o acesso mínimo necessário. Capítulo 51.

APÊNDICE E

Simulado

São 40 questões de múltipla escolha, na ordem das seis partes do livro, e cada uma tem uma única alternativa correta. A maioria é um cenário de entrevista: leia o enunciado até o fim, porque o detalhe que decide a resposta costuma estar nele. Responda tudo antes de conferir; na versão HTML, você marca as alternativas e recebe a nota no fim. Cada explicação aponta o capítulo para revisar. Acertar 32 ou mais (80%) é um bom sinal de que a base está firme.

1 **Você vai escalar horizontalmente uma aplicação web que guarda a sessão de cada usuário na memória do próprio servidor. O que precisa mudar primeiro?**

a. Trocar as máquinas por instâncias com mais memória, para caber mais sessões

b. Levar as sessões para um armazenamento compartilhado, como Redis ou banco

c. Desligar os health checks do load balancer, para não derrubar sessões ativas

d. Usar Round Robin no load balancer, para distribuir as sessões por igual

2 **Um colega afirma que o banco distribuído da empresa "é CA". Qual é o problema dessa afirmação?**

a. CA só é possível quando o cluster tem pelo menos três réplicas em cada região

b. CA exige abrir mão da durabilidade das escritas que já foram confirmadas

c. CA só vale para bancos relacionais que oferecem transações ACID completas

d. Partições são inevitáveis, então a escolha real é entre C e A quando uma acontece

3 **Seu banco replica cada escrita de forma síncrona entre São Paulo e a Virgínia, nos Estados Unidos, para que toda leitura veja o dado mais recente. Mesmo sem nenhuma falha de rede, que preço você paga, segundo o PACELC?**

a. Latência maior, porque cada escrita espera a confirmação da outra região

b. Disponibilidade menor, porque o CAP proíbe consistência forte sem partição

c. Durabilidade menor, porque a escrita fica só na memória até sincronizar

d. Nenhum, porque sem partição não há trade-off entre latência e consistência

4 **Você está projetando uma fintech com transferências entre contas e um feed de atividades com curtidas. Qual combinação faz mais sentido?**

a. BASE para tudo, porque escala melhor e o saldo se acerta sozinho depois

b. ACID para tudo, porque consistência eventual é sempre um defeito de projeto

c. ACID para as transferências e BASE para o feed e as curtidas

d. BASE para as transferências e ACID para o feed e as curtidas

5 **Um serviço passou a juntar as gravações em lotes de 100 antes de enviá-las ao banco. As gravações por segundo subiram muito, mas os usuários dizem que cada operação ficou mais lenta. O que aconteceu?**

a. A latência caiu, mas o throughput caiu junto por causa do tamanho dos lotes

b. O banco virou gargalo, e o throughput do sistema como um todo diminuiu

c. O throughput subiu, mas a latência aumentou, porque cada item espera o lote encher

d. Nada mudou de fato, porque lotes não afetam nem o throughput nem a latência

6 **Você dobrou os servidores de aplicação de 10 para 20, mas o throughput quase não mudou. Toda requisição grava no mesmo banco primário, que está com a CPU no limite. O que explica o resultado?**

a. O primário é uma parte que não se paraleliza e limita o ganho do todo

b. Faltou trocar o algoritmo do load balancer para Least Connections

c. Servidores stateless não se beneficiam de escalabilidade horizontal

d. O ganho só aparece depois que o cache dos servidores novos esquenta

7 **Você está começando um produto novo com um time de quatro pessoas, e as regras de negócio ainda mudam toda semana. Qual arquitetura é a escolha mais sensata?**

a. Um microsserviço por tabela do banco, para escalar cada parte separadamente

b. Microsserviços que compartilham o mesmo banco e fazem deploy todos juntos

c. Um serviço por desenvolvedor, cada um com sua linguagem e seu banco

d. Um monólito bem modularizado, extraindo serviços quando a dor for real

8 **Você está avaliando serverless para quatro cargas de trabalho. Em qual ele se encaixa melhor?**

a. Um servidor de jogo que mantém conexões e estado em memória por horas

b. Um webhook que recebe rajadas imprevisíveis e fica parado boa parte do dia

c. Uma conversão de vídeo que roda sem parar por seis horas seguidas

d. Uma API de tráfego alto e constante em que cada milissegundo conta

9 **Seu serviço roda em servidores idênticos e recebe requisições que levam de 5 ms a 30 s (relatórios pesados). Com Round Robin, alguns servidores ficam sobrecarregados enquanto outros ficam ociosos. Qual algoritmo resolve melhor?**

a. Least Connections

b. IP Hash

c. Round Robin ponderado pelo hardware de cada servidor

d. Aleatório (random)

10 **O time começou a colocar regras de negócio, cálculo de frete e junção de respostas de vários serviços dentro do API Gateway. Qual é o risco?**

a. O gateway deixa de conseguir aplicar rate limiting e autenticação

b. Os clientes passam a precisar do endereço de cada microsserviço

c. O gateway vira gargalo e um mini-monólito que todo time precisa alterar

d. O gateway deixa de ser um proxy reverso e vira um proxy direto

11 **Usuários na Ásia reclamam que as imagens e o JavaScript do seu site, hospedado em São Paulo, demoram para carregar. Qual é a solução mais direta?**

a. Aumentar a CPU e a memória do servidor de origem em São Paulo

b. Servir os arquivos estáticos por uma CDN, com cópias perto dos usuários

c. Reduzir o TTL do DNS do domínio para as respostas chegarem antes

d. Trocar HTTP por WebSocket para entregar as imagens e os scripts

12 **Você vai migrar o site para um novo endereço IP na semana que vem, e o registro DNS atual tem TTL de 24 horas. O que fazer para a troca se propagar rápido?**

a. Aumentar o TTL para 48 horas, para os resolvedores guardarem o IP novo por mais tempo

b. Trocar o registro A por um CNAME que aponte direto para o novo endereço IP

c. Fazer a troca e pedir aos usuários que limpem o cache do navegador

d. Baixar o TTL para poucos minutos pelo menos 24 horas antes da troca

13 **Você está projetando o áudio de uma chamada de voz em tempo real. Um pacote que chega 300 ms atrasado já não serve para nada. Qual protocolo de transporte combina mais?**

a. TCP, porque garante que todo pacote chegue, ainda que atrasado

b. TCP, porque retransmite pacotes perdidos mais rápido que o UDP

c. UDP, porque garante que os pacotes cheguem na ordem em que saíram

d. UDP, porque não segura os pacotes novos esperando um que se perdeu

14 **Qual problema o HTTP/3 resolve e o HTTP/2 não?**

a. Um pacote TCP perdido travar todos os streams da mesma conexão

b. A falta de multiplexação de várias requisições numa conexão

c. A falta de suporte a criptografia com TLS entre cliente e servidor

d. A ausência de compressão dos cabeçalhos das requisições

15 **Você está projetando a comunicação interna entre dezenas de microsserviços com muito tráfego e quer contratos tipados e mensagens pequenas. Qual é a melhor escolha?**

a. REST com JSON, porque o JSON fica menor na rede do que o protobuf

b. WebSocket com texto puro, porque o protocolo define contratos tipados

c. gRPC com Protocol Buffers, que gera código tipado e usa mensagens binárias

d. Server-Sent Events, porque permitem streaming nos dois sentidos

16 **Você está projetando o placar ao vivo de um campeonato: só o servidor envia atualizações, e o navegador não manda nada depois de abrir a página. Qual é a opção mais simples que atende bem?**

a. WebSocket, porque o SSE não funciona sobre HTTP comum
b. Server-Sent Events, um canal de mão única do servidor para o cliente
c. Polling a cada 100 ms, que gasta menos do que manter uma conexão aberta
d. UDP direto do navegador, porque é o transporte mais rápido

17 **Você está projetando um cluster de mil nós que entram e saem o tempo todo, e cada nó precisa saber quais outros estão vivos sem depender de um servidor central. Qual é a melhor escolha?**

a. Gossip: cada nó troca periodicamente o que sabe com alguns nós aleatórios
b. Um coordenador que recebe heartbeats de todos e publica a lista de vivos
c. Two-phase commit entre todos os nós a cada entrada ou saída de um nó
d. Consultar o DNS a cada segundo para ver quais IPs estão respondendo

18 **Você vai fazer sharding por intervalo da tabela de eventos de um app de analytics, usando a data e hora do evento como shard key. Qual é o problema?**

a. Consultas por período ficam impossíveis, porque cada dia cai num shard aleatório
b. Cada evento precisa ser gravado em todos os shards ao mesmo tempo
c. O resharding fica impossível depois que os dados estão distribuídos
d. Toda escrita nova cai no shard do intervalo mais recente, que vira hot spot

19 **Um usuário troca a foto do perfil, a página recarrega e mostra a foto antiga. O sistema grava no primário e lê de réplicas com replicação assíncrona. Qual é a correção mais direcionada?**

a. Adicionar mais réplicas de leitura para dividir melhor a carga
b. Ler do primário os dados do próprio usuário logo depois que ele escreve
c. Trocar a replicação por multi-primário, com escrita em qualquer nó
d. Aumentar o TTL do cache do navegador para a página de perfil

20 **Um cluster de cache com 10 nós usa hashing consistente com nós virtuais, e a carga está bem distribuída. Você adiciona o 11º nó. Aproximadamente que fração das chaves muda de nó?**

a. Quase todas, como acontece com hash(chave) mod N

b. Cerca de metade, divididas entre o nó novo e os vizinhos

c. Cerca de 1/11, só as que passam a pertencer ao nó novo

d. Nenhuma, porque só as chaves criadas depois vão para o nó novo

21 **Você está projetando o armazenamento de telemetria de milhões de sensores: centenas de milhares de escritas por segundo e poucas leituras. Qual estrutura favorece essa carga?**

a. B-Tree, porque atualiza as páginas no lugar e não precisa de compaction

b. Bloom filter, porque guarda os dados ocupando pouquíssima memória

c. LSM-Tree, porque acumula as escritas em memória e grava em lote no disco

d. B-Tree com vários índices secundários, porque índices aceleram inserções

22 **O feed do seu app mostra cada post com o nome e a foto do autor e é lido milhões de vezes por minuto. O join com a tabela de usuários virou o gargalo. Qual técnica atende?**

a. Desnormalizar, copiando o nome e a foto do autor para cada post

b. Normalizar mais, separando o nome e a foto em tabelas próprias

c. Remover os índices da tabela de usuários para acelerar o join

d. Ler cada post dentro de uma transação 2PC com a tabela de usuários

23 **Antes de ir ao disco, o banco consulta um Bloom filter, que responde "não está" para uma chave. O que você pode concluir?**

a. A chave provavelmente não está, mas pode ser um falso negativo

b. A chave está, mas foi apagada há pouco e ainda não foi compactada

c. Nada, porque as respostas negativas do filtro são aleatórias

d. A chave com certeza não está, e a leitura no disco pode ser pulada

24 **Você está projetando a busca de um assistente de IA que precisa achar os trechos da documentação mais parecidos em significado com a pergunta, mesmo sem palavras em comum. Qual é a melhor escolha?**

a. Um índice B-Tree sobre o texto, buscando por igualdade exata

b. Embeddings num banco vetorial, buscando os vizinhos mais próximos

c. Um Bloom filter com as palavras de cada trecho da documentação

d. Consultas com LIKE '%termo%' sobre o texto no banco relacional

25 **Sua API pública precisa aceitar rajadas curtas (um app que sincroniza 20 itens de uma vez ao abrir), mas limitar cada chave a uma média de 10 requisições por segundo. Qual configuração atende?**

a. Janela fixa de 1 segundo com limite de 10 requisições

b. Token bucket com capacidade de 20 fichas e reposição de 10 por segundo

c. Janela deslizante de 1 segundo com limite de 10 requisições

d. Circuit breaker que abre quando a chave passa de 10 requisições por segundo

26 **O que um circuit breaker faz quando está aberto?**

a. Repete cada chamada com backoff exponencial até o serviço voltar a responder

b. Deixa as chamadas passarem normalmente e só conta quantas falharam

c. Separa as chamadas em pools de threads isolados por funcionalidade

d. Falha as chamadas na hora, sem acionar o serviço, até o tempo de espera passar

27 **Você quer garantir que, se o serviço de relatórios travar, ele não ocupe todas as conexões do pool que o checkout também usa. O que fazer?**

a. Dar pools de conexões separados ao checkout e aos relatórios

b. Aplicar retry com backoff exponencial nas chamadas de relatório

c. Aumentar o pool compartilhado para caber os dois com folga

d. Exigir uma idempotency key em toda chamada de relatório

28 **Depois de uma queda de 30 s no banco, 50 mil clientes que falharam juntos tentam de novo após 1 s, 2 s, 4 s..., todos nos mesmos instantes, e o banco cai de novo ao voltar. O que falta na política de retry?**

a. Backoff exponencial entre as tentativas

b. Mais tentativas por cliente antes de desistir

c. Jitter, um componente aleatório no tempo de espera

d. Um intervalo fixo de 1 segundo entre as tentativas

29 **O app envia um pagamento, a conexão cai antes da resposta e o app reenvia a mesma requisição. Como evitar a cobrança dupla?**

a. Reenviar com a mesma idempotency key, e o servidor devolver o resultado guardado

b. Reenviar com uma idempotency key nova, para o servidor distinguir as tentativas

c. Trocar o POST por um GET, que pelo protocolo HTTP já é idempotente

d. Colocar um circuit breaker entre o app e o serviço de pagamentos

30 **Num cluster Raft de 5 nós, uma partição isola o líder atual com apenas mais um nó, e os outros 3 elegem um novo líder. O que impede o líder antigo de confirmar novas escritas?**

a. O heartbeat do novo líder chega até ele e o desliga na hora

b. Ele não consegue a confirmação da maioria, 3 de 5 nós

c. Ele apaga o próprio log assim que percebe a partição

d. Os clientes passam a usar two-phase commit com ele

31 **Você está projetando o checkout de um e-commerce com serviços de pedidos, pagamento e estoque, cada um com seu banco, em alto volume. Você não quer locks distribuídos nem um coordenador que trave tudo se cair. Qual abordagem usar?**

a. Two-phase commit, com um coordenador segurando locks nos três bancos até decidir

b. Gravar nos três serviços em paralelo e ignorar as falhas parciais

c. Repetir cada passo indefinidamente até os três serviços confirmarem

d. Uma saga, com transações locais em sequência e ações compensatórias

32 **Você está projetando o cache do catálogo de produtos: muita leitura, poucas escritas, e o sistema precisa continuar funcionando se o cache cair. Qual estratégia é a escolha padrão?**

a. Write-back, gravando só no cache e levando ao banco depois

b. O cache como única fonte dos dados, sem banco por trás

c. Cache-aside, buscando no banco e preenchendo o cache a cada miss

d. Nenhum cache, porque o banco é mais rápido em cargas de leitura

33 **Um cache com espaço para 3 itens usa LRU e começa vazio. Chegam os acessos A, B, C, A, D. Qual item é removido quando D entra?**

a. A, porque foi o primeiro a entrar no cache

b. C, porque foi o último a entrar antes de D

c. Nenhum, porque A foi acessado duas vezes

d. B, porque é o que está há mais tempo sem acesso

34 **No seu cadastro, o e-mail de boas-vindas é enviado com uma chamada síncrona ao provedor de e-mail; quando o provedor fica lento, o cadastro inteiro fica lento. Qual é a mudança indicada?**

a. Colocar o pedido de envio numa fila e deixar um worker mandar o e-mail

b. Aumentar o timeout da chamada ao provedor de e-mail

c. Adicionar retries síncronos com backoff na chamada ao provedor

d. Esperar a confirmação de entrega do e-mail antes de responder

35 **Quando um pedido é criado, os serviços de estoque, e-mail e analytics precisam reagir, cada um do seu jeito. Qual modelo de mensageria é o indicado?**

a. Uma fila ponto a ponto da qual os três serviços consomem

b. Pub/Sub, com um tópico de pedidos e cada serviço como assinante

c. Uma dead letter queue compartilhada pelos três serviços

d. Chamadas encadeadas: pedidos chama estoque, que chama e-mail

36 **Uma mensagem com um campo malformado faz o consumidor falhar sempre: ela volta para a fila, é reprocessada sem parar e atrasa todas as outras. O que resolve?**

a. Aumentar o número máximo de tentativas para ela ter mais chances

b. Apagar a fila inteira e reiniciar os consumidores do zero

c. Limitar as tentativas e mover a mensagem para uma dead letter queue

d. Trocar a fila por pub/sub para cada mensagem ter vários consumidores

37 **Um checkout passa por 12 serviços e às vezes leva 8 s. Você precisa ver, numa requisição lenta específica, quanto tempo cada serviço gastou. O que usar?**

a. Logs em nível DEBUG em todos os serviços, lidos um serviço por vez

b. Um heartbeat em cada serviço, para saber quais deles estão vivos

c. Tracing distribuído, com trace ID propagado e um span por operação

d. A latência média de cada serviço, num painel de métricas por hora

38 **Qual frase usa SLI, SLO e SLA corretamente?**

a. O SLO medido foi 99,95%; o SLI interno é 99,9%; o SLA é a meta da equipe

b. O SLI medido foi 99,95%; o SLO interno é 99,9%; o SLA no contrato é 99,5%

c. O SLA medido foi 99,95%; o SLO no contrato é 99,5%; o SLI é a meta interna

d. O SLI no contrato é 99,5%; o SLA interno é 99,9%; o SLO medido foi 99,95%

39 **Você está construindo um app com "Entrar com Google" e precisa saber quem é o usuário (nome e e-mail). O que usar?**

a. OAuth 2.0 puro, tratando o access token como prova de quem é o usuário

b. Pedir a senha do Google ao usuário e guardá-la criptografada no banco

c. Um certificado TLS de cliente emitido para cada usuário do app

d. OIDC, que soma ao OAuth 2.0 um ID token com a identidade do usuário

40 **Você está projetando a comunicação entre microsserviços numa arquitetura Zero Trust. O que se espera?**

a. Toda chamada é autenticada, autorizada e cifrada (por exemplo, com mTLS), mesmo dentro da VPC

b. Chamadas dentro da VPC dispensam autenticação, porque o firewall já barra o tráfego externo

c. Basta ter TLS no load balancer de borda; o tráfego interno pode seguir em texto puro

d. Um serviço aceita qualquer requisição que venha de um endereço IP da rede interna

Gabarito comentado

1	b	Com o estado fora da instância, qualquer servidor atende qualquer requisição, e adicionar ou remover máquinas não faz ninguém perder a sessão. Sticky session até funciona como remendo, mas a sessão some quando aquela máquina cai. Capítulos 1 e 8.
2	d	Num sistema distribuído, tolerar partições não é opcional: cabos, switches e datacenters falham. O CAP pergunta o que o sistema faz quando a rede se divide: recusar para não errar (CP) ou responder com o que tem (AP). Capítulo 2.
3	a	O "ELC" do PACELC diz que, mesmo sem partição, você troca latência por consistência: a ida e volta entre os continentes, de mais de 100 ms, entra em toda escrita. Bancos que confirmam só na réplica local respondem mais rápido, mas podem servir dado velho na outra região. Capítulo 3.
4	c	Dinheiro não pode sumir nem duplicar, então transferências pedem transações atômicas e isoladas. Um contador de curtidas atrasado alguns segundos não prejudica ninguém, e com BASE o feed ganha disponibilidade e escala. Capítulo 4.
5	c	Lotes diluem o custo fixo de cada ida ao banco e aumentam a vazão, mas cada gravação espera as outras chegarem. Throughput e latência medem coisas diferentes, e melhorar um pode piorar o outro. Capítulo 5.
6	a	É a Lei de Amdahl: a parte que roda em série limita o ganho; se 20% do trabalho for sequencial, nem infinitas máquinas deixam o sistema mais de 5 vezes mais rápido. Antes de adicionar servidores, ataque o gargalo, que aqui é o banco. Capítulo 6.
7	d	Microsserviços resolvem problemas de times grandes e de partes com escala muito diferente, e cobram em rede, observabilidade e consistência de dados. Com time pequeno e domínio instável, as fronteiras ainda vão mudar, e mudar uma fronteira dentro de um monólito é barato. Capítulo 9.
8	b	Serverless brilha em cargas disparadas por eventos e com picos: escala sozinho e não cobra pelo tempo ocioso. Tarefas longas esbarram no limite de duração das funções, não há garantia de que o estado em memória sobreviva entre chamadas e, com tráfego alto e constante, servidores próprios costumam sair mais baratos e ter latência mais previsível. Capítulo 10.
9	a	Least Connections olha quantas conexões cada servidor tem abertas, então quem está preso num relatório longo recebe menos trabalho novo. Round Robin, ponderado ou não, e o aleatório ignoram a carga do momento, e o IP Hash prende cada cliente a um servidor, por mais ocupado que ele esteja. Capítulo 12.

10	c	O API Gateway é um proxy reverso que deve ficar fino: roteamento, autenticação, rate limiting e logs. Com lógica de negócio dentro, toda mudança de regra passa a depender de um deploy dele, que concentra acoplamento e carga. Capítulo 14.
11	b	O problema é a distância: cada ida e volta até São Paulo custa centenas de milissegundos para quem está na Ásia, e uma máquina maior não encurta o caminho. A CDN guarda cópias em servidores próximos dos usuários e ainda tira carga da origem. Capítulo 15.
12	d	Resolvedores guardam cada resposta pelo TTL que receberam: se você só baixar o TTL na hora da troca, quem já tem o IP antigo em cache fica com ele por até 24 horas. Baixando antes, todos os caches já estarão com TTL curto quando o IP mudar. Capítulo 16.
13	d	No TCP, um pacote perdido faz os seguintes esperarem a retransmissão, e o áudio trava. No UDP, a aplicação segue em frente: perder um pedacinho de som é melhor do que atrasar a conversa. Capítulo 17.
14	a	O HTTP/2 já multiplexa streams e comprime cabeçalhos, mas todos os streams dividem uma conexão TCP, que entrega os bytes em ordem: uma perda segura todo mundo (head-of-line blocking no transporte). O QUIC, base do HTTP/3, roda sobre UDP com streams independentes, então só o stream afetado espera. Capítulo 18.
15	c	O gRPC parte de um contrato (.proto) que gera cliente e servidor tipados, e o protobuf binário é menor e mais rápido de processar que JSON. REST continua ótimo para APIs públicas, em que legibilidade e compatibilidade pesam mais. Capítulo 19.
16	b	Quando só o servidor fala, SSE resolve sobre HTTP comum, e o navegador ainda reconecta sozinho se a conexão cair. WebSocket também funcionaria, mas é full-duplex e traz mais peças para um problema de mão única. Capítulo 20.
17	a	Com gossip, a informação se espalha como boato: em poucas rodadas, da ordem de log N, o cluster todo fica sabendo, sem ponto central de falha. O Cassandra e o Consul usam gossip para saber quem está no cluster e detectar falhas. Capítulo 22.
18	d	Com uma chave sempre crescente, como data e hora, o shard “de agora” recebe todas as gravações enquanto os outros só guardam histórico. Uma saída comum é compor a chave com um hash, por exemplo do ID do dispositivo, para espalhar as escritas. Capítulo 23.
19	b	O sintoma é replication lag: a réplica ainda não recebeu a escrita. Garantir read-your-writes, lendo do primário os dados do próprio usuário por alguns segundos depois que ele escreve, resolve sem tornar toda a replicação síncrona. Capítulo 24.

20	c	No anel, o nó novo assume só os trechos imediatamente antes de cada posição dele, e o resto das chaves fica onde estava. Com hash(chave) mod N, passar de 10 para 11 nós muda o destino de cerca de 10 em cada 11 chaves. Capítulo 25.
21	c	A LSM-Tree transforma escritas espalhadas em gravações sequenciais em lote, o que a deixa muito rápida para escrever; o preço aparece nas leituras e na compaction. Cassandra, ScyllaDB e RocksDB seguem essa linha. Capítulo 26.
22	a	Em caminhos de leitura pesada, duplicar alguns campos troca um join caro por uma leitura direta. O preço é manter as cópias em dia quando o autor muda o nome ou a foto, geralmente com um evento processado em segundo plano. Capítulo 28.
23	d	Se algum dos bits da chave estiver zerado, ela nunca foi inserida: Bloom filter não tem falso negativo. Só o "talvez esteja" pode errar (falso positivo), e nesse caso o banco faz a leitura normal. Capítulo 30.
24	b	Embeddings colocam textos de significado parecido perto uns dos outros, então "como cancelo minha assinatura?" encontra "encerrar o plano". O banco vetorial acha os mais próximos por uma medida de distância, como a similaridade de cosseno, em geral com índices aproximados para responder rápido. Capítulo 31.
25	b	O balde cheio libera uma rajada de até 20 requisições, e a reposição de 10 fichas por segundo segura a média no longo prazo. As janelas com limite de 10 por segundo recusariam parte da rajada, e circuit breaker protege contra falhas de dependências, não contra excesso de uso. Capítulo 32.
26	d	Aberto, o circuit breaker poupa o serviço com problema e também quem chama, que deixa de ficar preso em timeouts. Passado o tempo de espera, ele fica meio-aberto, deixa algumas chamadas de teste passarem e fecha de novo se elas derem certo. Capítulo 33.
27	a	É o bulkhead: os relatórios podem esgotar o próprio pool sem tocar no do checkout, como um compartimento alagado que não afunda o navio. Aumentar o pool compartilhado só adia o problema, porque um serviço travado acaba prendendo qualquer quantidade de conexões. Capítulo 34.
28	c	O backoff exponencial já espaça as tentativas de cada cliente, mas, sem jitter, clientes que falharam juntos voltam juntos: é o thundering herd. Sortear parte da espera espalha as novas tentativas e dá ao banco tempo para se recuperar. Capítulo 35.
29	a	O servidor guarda a chave com o resultado da primeira execução; quando a mesma chave chega de novo, ele devolve aquele resultado em vez de cobrar outra vez. Uma chave nova a cada tentativa faria o servidor tratar a repetição como outro pagamento. Capítulo 36.

30	**b**	No Raft, uma escrita só é confirmada quando a maioria do cluster a grava no log, e do lado com 2 nós isso nunca acontece. Quando a rede volta, o líder antigo vê o mandato maior do novo líder, vira seguidor, e as entradas que ele não confirmou são substituídas. Capítulo 38.
31	**d**	Na saga, cada serviço confirma a própria parte e, se o pagamento for recusado, compensações desfazem o que já foi feito, como liberar o estoque reservado. O 2PC daria atomicidade, mas com locks durante a votação e risco de bloqueio se o coordenador cair, exatamente o que o enunciado quer evitar. Capítulos 39 e 40.
32	**c**	No cache-aside, o banco continua sendo a fonte da verdade: se o cache cair, a aplicação lê direto do banco, mais devagar, mas sem perder dados. Com write-back, uma queda do cache levaria embora escritas que ainda não chegaram ao banco. Capítulos 41 e 42.
33	**d**	O segundo acesso a A o torna o mais recente, então a ordem do menos para o mais recente fica B, C, A, e o LRU remove B. Remover A seria FIFO, que olha só a ordem de entrada. Capítulo 43.
34	**a**	A fila desacopla o cadastro do envio: o usuário recebe a resposta na hora, e o worker manda o e-mail no ritmo do provedor, tentando de novo se ele falhar. As outras opções mantêm o provedor lento dentro da requisição do usuário. Capítulo 44.
35	**b**	No pub/sub, cada assinante recebe sua própria cópia do evento, e o serviço de pedidos nem precisa saber quem está ouvindo. Numa fila ponto a ponto, cada mensagem vai para um único consumidor, então só um dos três veria cada pedido. Capítulo 45.
36	**c**	Uma poison message nunca vai dar certo sozinha, então tentar de novo só desperdiça trabalho. Na DLQ ela sai do caminho das outras e fica guardada para alguém investigar, corrigir e reprocessar. Capítulo 46.
37	**c**	O trace ID amarra todos os spans da mesma requisição, e cada span registra início e duração, então o trace mostra exatamente onde os 8 s foram gastos. Médias por serviço escondem a requisição lenta, e logs soltos não dizem quais linhas pertencem a ela. Capítulo 47.
38	**b**	SLI é o placar (o que se mede), SLO é a meta interna e SLA é o contrato com o cliente. O SLO fica mais rígido que o SLA para a equipe perceber o problema antes de violar o contrato. Capítulo 48.
39	**d**	O OAuth 2.0 trata de autorização: o access token diz o que o app pode acessar e é destinado à API, não serve como prova de identidade. O OIDC acrescenta a autenticação, com um ID token assinado (um JWT) que o app valida para saber quem fez login. Capítulo 49.

40	a	Zero Trust parte do princípio de que o atacante pode já estar dentro: estar na rede não dá confiança. Com mTLS, cada serviço prova sua identidade com um certificado, e cada chamada ainda passa por uma checagem de autorização. Capítulos 50 e 51.

50 Conceitos de System Design, edição de estudo em português. Conteúdo original de Arslan Ahmad (DesignGurus), traduzido, expandido e ilustrado para estudo pessoal.

Escrito em Markdown e montado com pandoc. Diagramas desenhados com roughjs na fonte Excalifont. Texto em Literata, títulos em Bricolage Grotesque, código em JetBrains Mono.